

DSA27 · CS2101 · IS122

Lab Manual

Data Structures and Algorithms · Weeks 1 to 15

Students' edition

Dr. Haitham A. El-Ghareeb

Faculty of Computers and Information Sciences, Mansoura University

Fall 2026

Contents

Before the first lab	6
The labs	7
Lab 01 — Lab 01 — The Interpreter, Numbers, Strings and Lists	8
Part 0 — Set up (before the lab)	8
Part 1 — Three ways to run Python	9
Part 2 — Numbers	11
Part 3 — Strings	14
Part 4 — Lists	20
Part 5 — First steps towards programming	23
Part 6 — Functions and tests, just enough to start	25
Part 7 — Exercises	26
Part 8 — Take-home practice (not graded)	27
Summary	28
Lab 02 — Lab 02 — Control Flow, Functions, Errors and Modules	29
Part 1 — if statements	29
Part 2 — for loops and range	31
Part 3 — break, continue, else, pass	33
Part 4 — match statements	35
Part 5 — Defining functions	36
Part 6 — Errors and exceptions	41
Part 7 — Modules	45
Part 8 — Counting and timing: a first look at complexity	48
Part 9 — Exercises	49
Part 10 — Take-home practice (not graded)	50
Summary	51
Lab 03 — Lab 03 — Data Structures, Classes and Generators	52
Part 1 — More on lists	52
Part 2 — Comprehensions	56
Part 3 — Tuples	58
Part 4 — Sets	59
Part 5 — Dictionaries	60
Part 6 — Looping techniques	62
Part 7 — Classes	63
Part 8 — Iterators and generators	69
Part 9 — Exercises	71

Part 10 — Bridge to this week's lecture: recursion	72
Part 11 — Take-home practice (not graded)	74
Summary	74
Lab 04 — Lab 04 — Building a Dynamic Array	76
Part 0 — Before you start	76
Part 1 — Growing: <code>_resize</code> and <code>append</code>	78
Part 2 — Reading and writing: <code>__getitem__</code> and <code>__setitem__</code>	81
Part 3 — Inserting in the middle: <code>insert_at</code>	82
Part 4 — Removing: <code>pop</code>	85
Part 5 — The growth factor experiment	87
Part 6 — Measure it	89
Part 7 — Exercises at a glance	90
Part 8 — Take-home practice (not graded)	90
Part 9 — Bridge to Lecture 05: linked lists	91
Summary	92
Lab 05 — Lab 05 — Building a Linked List	93
Part 0 — Before you start	93
Part 1 — Nodes by hand	95
Part 2 — <code>push_front</code> : $O(1)$ at the head	96
Part 3 — <code>append</code> : $O(n)$ without a tail	98
Part 4 — Walking: <code>find</code> and <code>__getitem__</code>	99
Part 5 — <code>insert_at</code> : walk, then splice	100
Part 6 — Removing: <code>pop_front</code> and <code>remove</code>	102
Part 7 — <code>reverse</code> : three references	103
Part 8 — Measure it: the $O(n^2)$ trap	105
Part 9 — Challenge: the <code>_tail</code> reference (not graded)	107
Part 10 — Exercises at a glance	108
Part 11 — Take-home practice (not graded)	109
Part 12 — Bridge to Lecture 06: stacks	109
Summary	110
Lab 06 — Lab 06 — Stacks, Brackets and Postfix	112
Part 0 — Before you start	113
Part 1 — <code>push</code> , <code>pop</code> and <code>peek</code> : the top at the end	115
Part 2 — Measure it: the dishonest stack	117
Part 3 — <code>is_balanced(text)</code>	119
Part 4 — <code>evaluate_postfix(tokens)</code> , in <code>dsa/translation.py</code>	121
Part 5 — <code>infix_to_postfix(tokens)</code> : the shunting-yard algorithm	123
Part 6 — The whole calculator	125
Part 7 — Exercises at a glance	126
Part 8 — Take-home practice (not graded)	127
Part 9 — Bridge to next week's lecture: queues	128
Summary	128

Lab 07 — Lab 07 — Queues and the Ring Buffer	130
Part 0 — Before you start	131
Part 1 — SlowQueue: the obvious design	133
Part 2 — The ring on paper	134
Part 3 — CircularQueue.enqueue	136
Part 4 — CircularQueue.dequeue	138
Part 5 — Full or empty?	139
Part 6 — Challenge: grow when full	140
Part 7 — Measure it	142
Part 8 — Exercises at a glance	144
Part 9 — Take-home practice (not graded)	145
Part 10 — Bridge to next week's lecture: searching	145
Summary	146
Lab 08 — Lab 08 — Eight Ways to Search	148
Part 0 — Before you start	149
Part 1 — linear_search: the baseline	150
Part 2 — binary_search: halve what is left	150
Part 3 — Break it: lo < hi	153
Part 4 — binary_search_recursive	154
Part 5 — lower_bound and upper_bound: one loop, one comparison	155
Part 6 — Jump, exponential and interpolation search	157
Part 7 — Count the reads	160
Part 8 — Check your bounds against bisect	162
Part 9 — Measure it	162
Part 10 — Exercises at a glance	163
Part 11 — Take-home practice (not graded)	164
Part 12 — Bridge to Lecture 09: sorting	165
Summary	165
Lab 09 — Lab 09 — Sorts You Can Watch	167
Part 0 — Before you start	168
Part 1 — The pattern: one generator, two uses	169
Part 2 — bubble_sort_steps and bubble_sort	170
Part 3 — selection_sort_steps and selection_sort	172
Part 4 — insertion_sort_steps and insertion_sort	173
Part 5 — counting_sort	175
Part 6 — Count the comparisons	176
Part 7 — The price of watching, and the fix	177
Part 8 — Measure the three sorts	178
Part 9 — Exercises at a glance	178
Part 10 — Take-home practice (not graded)	179
Part 11 — Bridge to Lecture 10: divide, then merge	180
Summary	180
Lab 10 — Lab 10 — Divide, Conquer, and a Pivot	182

Part 0 — Before you start	183
Part 1 — <code>_merge</code> : two sorted runs become one	184
Part 2 — <code>merge_sort_steps</code> and <code>merge_sort</code>	185
Part 3 — <code>_partition</code> : Lomuto's one pass	187
Part 4 — <code>quick_sort_steps</code> : pivots and the call stack	187
Part 5 — <code>heap_sort_steps</code> : the array is the tree	189
Part 6 — Count the comparisons	191
Part 7 — Stability, and the test that cannot see it	192
Part 8 — Measure the time: what are you timing?	193
Part 9 — Exercises at a glance	193
Part 10 — Take-home practice (not graded)	194
Part 11 — Bridge to Lecture 11: trees	195
Summary	195
Lab 11 — Lab 11 — Trees and the Four Walks	197
Part 0 — Before you start	198
Part 1 — <code>insert</code> and <code>contains</code> : one path down	199
Part 2 — See your tree; <code>min</code> and <code>max</code>	201
Part 3 — <code>height</code> and <code>size</code> : recursion on trees	202
Part 4 — <code>is_valid</code> : the whole subtree, not just the parent	203
Part 5 — Three walks, one line apart	204
Part 6 — <code>level_order</code> : a queue, not a call stack	205
Part 7 — <code>delete</code> : three cases	207
Part 8 — Measure it	209
Part 9 — Exercises at a glance	210
Part 10 — Take-home practice (not graded)	211
Part 11 — Bridge to Lecture 12: the tree in an array	211
Summary	212
Lab 12 — Lab 12 — Heaps and Priority Queues	213
Part 0 — Before you start	214
Part 1 — The array is the tree	215
Part 2 — <code>is_valid</code> and <code>peek</code>	216
Part 3 — <code>_sift_up</code> and <code>push</code>	218
Part 4 — <code>_sift_down</code> and <code>pop</code>	219
Part 5 — <code>heapify</code>	221
Part 6 — Break it: <code>MaxHeap</code> for free	223
Part 7 — <code>PriorityQueue</code> : a heap of triples	223
Part 8 — Measure it: <code>heapify</code> against <code>n</code> pushes	225
Part 9 — Check your heap against <code>heapq</code>	227
Part 10 — Exercises at a glance	228
Part 11 — Take-home practice (not graded)	228
Part 12 — Bridge to Lecture 13: hash tables	229
Summary	229
Lab 13 — Lab 13 — Hash Tables, Twice	231

Part 0 — Before you start	232
Part 1 — Hashing by hand	233
Part 2 — Chaining: get and put	234
Part 3 — Chaining: delete	235
Part 4 — Chaining: _resize	237
Part 5 — Open addressing: _probe, put, get	238
Part 6 — Tombstones, and breaking them on purpose	239
Part 7 — Open addressing: _resize	241
Part 8 — Measure it	242
Part 9 — Check against dict, the oracle	244
Part 10 — Exercises at a glance	245
Part 11 — Take-home practice (not graded)	245
Part 12 — Bridge to Lecture 14: graphs	246
Summary	247
Lab 14 — Lab 14 — Graphs and the Two Searches	248
Part 0 — Before you start	249
Part 1 — Graph: the adjacency list	250
Part 2 — MatrixGraph: the adjacency matrix	252
Part 3 — bfs: breadth first, with your queue	253
Part 4 — shortest_path_unweighted: BFS with parents	254
Part 5 — dfs and dfs_iterative	255
Part 6 — connected_components and has_cycle	256
Part 7 — topological_sort: Kahn's algorithm	258
Part 8 — Measure it	258
Part 9 — Draw it, and walk a real graph	259
Part 10 — Exercises at a glance	260
Part 11 — Take-home practice (not graded)	261
Part 12 — Bridge to Lecture 15: a tree is a graph you walk depth first	261
Summary	262
Lab 15 — Lab 15 — A Calculator, Stage by Stage	264
Part 0 — Before you start	265
Part 1 — tokenize: characters into tokens	266
Part 2 — The grammar on paper	268
Part 3 — The parser: parse, expr, term, factor	269
Part 4 — Break it: two grammars that look right	272
Part 5 — evaluate and calculate	272
Part 6 — See it	274
Part 7 — Measure it	275
Part 8 — Exercises at a glance	277
Part 9 — Take-home practice (not graded)	277
Part 10 — Course wrap-up and exam preparation	278
Summary	279

Before the first lab

Two hours a week, fifteen weeks. This manual is what you work from in the lab.

Weeks 1–3 bring everyone in the room to the same Python: the interpreter, numbers, strings and lists; control flow, functions and errors; data structures, classes and generators. **From Week 4** each lab walks you through building that week's structure in the course repository, one method at a time, with drawings, predictions and the exact tests to run.

Before Lab 01. Install Python 3.10 or newer and Graphviz, then:

```
git clone https://github.com/helghareeb/DSA27.git
cd DSA27
py -3.13 -m venv .venv          # macOS / Linux: python3 -m venv .venv
.\.venv\Scripts\activate        # macOS / Linux: source
↪ .venv/bin/activate
pip install -r requirements.txt
pytest -m "not challenge"       # every one of these must pass
```

How every lab works.

- **Read the lecture section** a part names before you start it.
- **Draw before you code.** Each part has a “Draw it” step. A structure you cannot draw, you cannot write.
- **Predict at every Checkpoint** — write your prediction down, then run the code. The prediction is the exercise; running it is only the check.
- **Run the part's tests before moving on.** tests/ defines what “correct” means. When a test fails, read its output aloud before you change anything.
- **Check-off.** At the end of the session your TA asks you to run the week's tests and to explain one line of your code that they choose.

Attendance is recorded every session and counts toward the 75% rule: below it you are not allowed into the final exam.

About this edition. This is the students' edition: it has no answers to the checkpoints and no solutions. When a lab mentions a worked solution, it means the repository's solutions/ folder — open it only after you have tried, and after your tests pass, to compare. The struggle is where the learning happens: a solution read before you have tried teaches you almost nothing.

The labs

Lab 01 — Lab 01 — The Interpreter, Numbers, Strings and Lists

How to use this lab. Work through it from top to bottom with the Python interpreter open. Type every example yourself — do not copy and paste. When you see **Try it**, stop and do it. When you see **Checkpoint**, write down your prediction before you run the code. Then do the exercises in `labs/lab01.py` until `pytest tests/test_lab01.py` passes.

Duration	One 2-hour lab session, plus about 2 hours at home
You will write	<code>labs/lab01.py</code> — 10 small functions
Graded by	<code>tests/test_lab01.py</code>
Before you come	Finish the setup in Part 0. The lab is not the time to install Python.

What you will be able to do

By the end of this lab you can:

1. start Python in three ways, and know when to use each;
2. use Python as a calculator, and say what `/`, `//`, `%` and `**` do — including with negative numbers;
3. make, index, slice and format strings, and use their common methods;
4. make, index, slice and change lists, and explain why `b = a` does **not** copy a list;
5. write a `while` loop, and a first `if`;
6. run a test file with `pytest` and read what it tells you.

Point 4 matters more than it looks. This entire course is about how data is arranged in memory, and Python lists are the first place you meet the idea that a name *refers to* an object rather than *containing* it.

Part 0 — Set up (before the lab)

You need **Python 3.10 or newer**. The course uses 3.13.

Windows. Install from <https://www.python.org/downloads/>. On the first screen of the installer tick **Add python.exe to PATH**. Then, in a new PowerShell window:

```
py --version
```

If python opens the Microsoft Store instead of running, turn off the Store aliases: *Settings* → *Apps* → *Advanced app settings* → *App execution aliases*, switch off python.exe and python3.exe.

macOS / Linux. Use python3 wherever this manual says py or python.

Now get the course repository and create its virtual environment:

```
git clone https://github.com/helghareeb/DSA27.git
cd DSA27
py -3.13 -m venv .venv
.\.venv\Scripts\activate
pip install -r requirements.txt
pytest -m "not challenge"
```

On macOS or Linux the two middle lines are python3 -m venv .venv and source .venv/bin/activate.

Every test in that last command must pass. They check your machine, not your code.

What is a virtual environment? A folder (.venv) holding a private copy of Python and its installed packages for *this project only*. activate makes the current terminal use it — your prompt changes to start with (.venv). Every time you open a new terminal to work on the course, cd into DSA27 and activate again. Forgetting to activate is the most common reason a package “is installed but cannot be found”.

Editor. Use VS Code with the *Python* extension from Microsoft. Open the DSA27 folder (*File* → *Open Folder*), then choose the interpreter .venv\Scripts\python.exe when VS Code asks (or *Ctrl+Shift+P* → *Python: Select Interpreter*).

Part 1 — Three ways to run Python

1.1 The interactive interpreter (the REPL)

Type python (or py) in an activated terminal:

```
(.venv) PS C:\...\DSA27> python
Python 3.13.x ... on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

>>> is the **primary prompt**: Python is waiting for a statement. It **R**eads what you type, **E**valuates it, **P**rints the result, and **L**oops — hence *REPL*. When a statement continues over several lines you get the **secondary prompt** . . . :

```
>>> the_world_is_flat = True
>>> if the_world_is_flat:
...     print("Be careful not to fall off!")
...
Be careful not to fall off!
```

The empty `...` line (just press Enter) tells Python the block is finished.

To leave: type `exit()` or `quit()`, or press **Ctrl+Z then Enter** on Windows (**Ctrl+D** on macOS/Linux).

Two conveniences:

- In the REPL, the value of the last expression is stored in the variable `_` (underscore). Useful when you are using Python as a calculator.
- `help(len)` shows the documentation of anything. `help()` alone starts an interactive help system; `q` leaves a help page.

In this manual, code that starts with `>>>` is typed into the REPL. Lines with no prompt are what Python prints back.

1.2 Scripts

A script is a text file ending in `.py`. Create `hello.py` in the DSA27 folder:

```
# hello.py – my first script
name = "DSA27"
print("Hello,", name)
```

Run it from the terminal:

```
python hello.py
```

A script does **not** print the value of each expression the way the REPL does; only `print()` produces output. That surprises everyone once.

Everything after `#` on a line is a **comment** and is ignored. A `#` inside a string is just a character:

```
spam = 1          # a comment
text = "# this is not a comment"
```

Python source files are UTF-8, so Arabic works in strings and comments. Try `len('مرحبا')` in the REPL — it is 5, one per letter, not one per byte.

1.3 Notebooks

A Jupyter notebook mixes code cells, output and notes. The course has one per lecture in `notebooks/`. Start Jupyter with `jupyter lab`, or open an `.ipynb` file directly in VS Code and choose the `.venv` interpreter as the kernel.

Which one when? The REPL to try one line. A notebook to explore and draw. A .py file for anything you want to keep, test, or submit — which is every exercise in this course.

Try it. Start the REPL, type `import this`, and read The Zen of Python. (This is also part of Homework 1.) Then leave the REPL with `exit()`.

Part 2 — Numbers

2.1 Python as a calculator

```
>>> 2 + 2
4
>>> 50 - 5*6
20
>>> (50 - 5*6) / 4
5.0
>>> 8 / 5          # division ALWAYS gives a float
1.6
```

Numbers without a decimal point, like 2 and 20, are of type **int**. Numbers with one, like 5.0 and 1.6, are **float**. Ask Python with `type()`:

```
>>> type(20)
<class 'int'>
>>> type(5.0)
<class 'float'>
```

The arithmetic operators:

Operator	Meaning	Example	Result
+ - *	add, subtract, multiply	7 * 3	21
/	true division — always float	17 / 3	5.666666666666667
//	floor division — round down	17 // 3	5
%	remainder (modulo)	17 % 3	2
**	power	2 ** 10	1024

// and % go together: for any a and non-zero b,

```
a == (a // b) * b + a % b          # always True
```

```
>>> 17 // 3      # how many whole 3s fit in 17
5
>>> 17 % 3      # what is left over
2
>>> 5 * 3 + 2    # and back again
17
>>> divmod(17, 3) # both at once
(5, 2)
```

Floor means down, not towards zero. This matters with negative numbers and catches C and Java programmers:

```
>>> -7 // 2      # -3.5 rounded DOWN is -4
-4
>>> -7 % 2      # so the remainder must be 1: -4*2 + 1 == -7
1
```

Precedence. `**` binds tighter than unary minus, and both bind tighter than `*` and `/`, which bind tighter than `+` and `-`. When unsure, use parentheses.

```
>>> -3 ** 2      # means -(3 ** 2)
-9
>>> (-3) ** 2
9
>>> 2 ** 3 ** 2  # ** groups right to left: 2 ** 9
512
```

2.2 Integers never overflow

In C an `int` is usually 32 bits and silently wraps around. A Python `int` grows as large as memory allows:

```
>>> 2 ** 100
1267650600228229401496703205376
>>> 2 ** 64 - 1    # the largest unsigned 64-bit integer in C
18446744073709551615
```

This is convenient, and it has a cost: arithmetic on huge integers is not $O(1)$. Adding two 1000-digit numbers takes longer than adding two small ones. Keep that in mind in Week 2.

2.3 Floats are approximations

A float is a 64-bit binary fraction. Most decimal fractions cannot be stored exactly:

```
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
```

```
False
>>> round(0.1 + 0.2, 2) == 0.3
True
```

This is not a Python bug; C, Java and JavaScript do exactly the same. Two rules follow:

1. **Never compare floats with ==** after arithmetic. Round, or check that the difference is tiny: `abs(x - y) < 1e-9`.
2. **Never store money as a float.** Store piasters (or cents) as an `int`.

Mixing `int` and `float` gives a `float`:

```
>>> 4 * 3.75 - 1
14.0
```

`round(x, n)` rounds to `n` decimal places; `round(x)` gives the nearest `int`. Python rounds exact halves to the nearest **even** number ("banker's rounding"), so `round(2.5)` is 2 and `round(3.5)` is 4.

2.4 Variables and assignment

`=` binds a **name** to a value. It prints nothing.

```
>>> width = 20
>>> height = 5 * 9
>>> width * height
900
```

Using a name that was never assigned is an error:

```
>>> n
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'n' is not defined
```

Read error messages **from the bottom up**. The last line says what went wrong (`NameError`) and why (name `'n'` is not defined). Lab 02 is about errors in detail.

Multiple assignment. The right-hand side is evaluated completely *first*, then the names on the left are bound left to right:

```
>>> a, b = 0, 1
>>> a, b = b, a + b      # uses the OLD a and b on the right
>>> a, b
(1, 1)
```

That is also the Python way to swap two variables — no temporary needed:

```
>>> x, y = 3, 7
>>> x, y = y, x
>>> x, y
(7, 3)
```

Augmented assignment. `x += 1` means `x = x + 1`. Likewise `-=`, `*=`, `/=`, `//=`, `%=`. Python has **no** `++` or `--` operator.

Names. Letters, digits and `_`, not starting with a digit, case-sensitive. Use `lower_case_with_underscores` for variables and functions — this is the Python convention (PEP 8), and the course code follows it.

2.5 Converting between types

```
>>> int("42") + 1
43
>>> float("3.5")
3.5
>>> int(3.99)           # int() truncates towards zero; it does not round
3
>>> int(-3.99)
-3
>>> str(42) + "!"
'42!'
>>> int("4.5")
Traceback (most recent call last):
...
ValueError: invalid literal for int() with base 10: '4.5'
```

`bool` is the type of `True` and `False`. They behave like 1 and 0 in arithmetic, which is occasionally handy: `True + True` is 2.

Checkpoint 1. Predict each result, then check it in the REPL.

- (a) `7 / 7` (b) `7 // 2` (c) `-7 // 2` (d) `7 % -2` (e) `2 ** -1`
(f) `10 - 2 * 3 ** 2` (g) `int(" 12 ")` (h) `round(0.5) + round(1.5)`
-

Part 3 — Strings

3.1 Writing strings

A string (`str`) is a sequence of characters. Single and double quotes are equivalent; pick the one that saves you escaping:

```
>>> 'spam eggs'
'spam eggs'
>>> "doesn't"
"doesn't"
>>> 'doesn\'t'          # or escape the quote with a backslash
"doesn't"
>>> '"Yes," they said.'
'"Yes," they said.'
```

The REPL shows a string *as you would type it* (with quotes). `print()` shows the text itself:

```
>>> s = 'First line.\nSecond line.'      # \n is a newline
>>> s
'First line.\nSecond line.'
>>> print(s)
First line.
Second line.
```

Common escape sequences: `\n` newline, `\t` tab, `\\` a backslash, `\'` and `\"` quotes.

Raw strings switch escapes off. Use them for Windows paths and, later, regular expressions:

```
>>> print('C:\some\name')          # \n became a newline!
C:\some
ame
>>> print(r'C:\some\name')
C:\some\name
```

Python 3.12 and later also print `SyntaxWarning: invalid escape sequence '\s'` for the first line. That warning is a gift: it tells you a backslash in your string is not doing what you think. The fix is the `r` prefix.

Triple quotes (`"""..."""` or `'''...'''`) make a string that spans several lines. Every docstring in `dsa/` and `labs/` is one.

```
print("""\
Usage: thingy [OPTIONS]
      -h                Display this usage message
      -H hostname       Hostname to connect to
""")
```

The `\` at the end of the first line stops a newline being included there.

3.2 Joining and repeating

```
>>> 3 * 'un' + 'ium'
```

```
'unununium'
>>> 'Py' 'thon'           # two literals side by side are joined
'Python'
>>> prefix = 'Py'
>>> prefix + 'thon'       # joining a variable needs +
'Python'
>>> 'Lab ' + 1
Traceback (most recent call last):
...
TypeError: can only concatenate str (not "int") to str
>>> 'Lab ' + str(1)
'Lab 1'
```

3.3 Indexing

Characters are numbered from **0**. Negative indices count from the end, starting at **-1**:

```
+---+---+---+---+---+
| P | y | t | h | o | n |
+---+---+---+---+---+
  0   1   2   3   4   5
-6  -5  -4  -3  -2  -1
```

```
>>> word = 'Python'
>>> word[0]
'p'
>>> word[5]
'n'
>>> word[-1]           # last character
'n'
>>> word[-2]
'o'
>>> word[42]
Traceback (most recent call last):
...
IndexError: string index out of range
```

There is no separate character type: `word[0]` is simply a string of length 1.

3.4 Slicing

`s[i:j]` is the part from index `i` **up to but not including** `j`.

```
>>> word[0:2]
'Py'
>>> word[2:5]
```

```
'tho'
>>> word[:2]           # omitted start means 0
'Py'
>>> word[4:]           # omitted end means len(word)
'on'
>>> word[-2:]          # the last two characters
'on'
>>> word[:2] + word[2:] # always equal to word
'Python'
```

The best way to remember slices: the indices point **between** characters, as in the diagram above. `word[2:5]` is everything between edge 2 and edge 5.

The length of `s[i:j]` is `j - i` when both are in range. That “up to but not including” rule is why `range(n)` in Lab 02 stops at `n - 1`, and why so many algorithms in this course use *half-open* intervals `[lo, hi)`.

Out-of-range **indices** are errors, but out-of-range **slices** are quietly trimmed:

```
>>> word[4:42]
'on'
>>> word[42:]
''
```

A third number is the **step**:

```
>>> '0123456789'[:2]    # every second character
'02468'
>>> word[::-1]          # step -1: the string reversed
'nohtyP'
```

3.5 Strings are immutable

You cannot change a string in place:

```
>>> word[0] = 'J'
Traceback (most recent call last):
...
TypeError: 'str' object does not support item assignment
```

Build a new one instead:

```
>>> 'J' + word[1:]
'Jython'
```

Every “changing” string method below returns a **new** string and leaves the original alone.

3.6 Useful string operations

```
>>> s = '  Data Structures  '
>>> len(s)
19
>>> s.strip()                # remove whitespace at both ends
'Data Structures'
>>> s.strip().lower()
'data structures'
>>> s.upper()
'  DATA STRUCTURES  '
>>> 'Structures' in s        # substring test
True
>>> s.strip().split()        # split on runs of whitespace
['Data', 'Structures']
>>> 'a,b,,c'.split(',')      # split on an exact separator
['a', 'b', '', 'c']
>>> '-'.join(['2026', '09', '23'])
'2026-09-23'
>>> 'banana'.count('a')
3
>>> 'banana'.find('n')       # first index, or -1 if absent
2
>>> 'banana'.index('n')     # first index, or ValueError if absent
2
>>> 'banana'.replace('a', 'o')
'bonono'
>>> 'lab01.py'.endswith('.py')
True
>>> '42'.isdigit()
True
>>> 'x..!'.strip('!.')      # strip the given characters instead
'x'
```

You do not need to memorise these. You need to know they exist, and how to find them: `help(str)` or `dir(str)`.

3.7 Printing and f-strings

`print()` takes any number of arguments, separates them with a space, and ends with a newline. Both can be changed:

```
>>> print('a', 'b', 'c')
a b c
>>> print('a', 'b', 'c', sep='-')
a-b-c
```

```
>>> print('no newline', end=' | ')
no newline | >>>
```

An **f-string** — a string with `f` before the quote — evaluates expressions in `{ }` and inserts them:

```
>>> name, n = 'Ada', 3
>>> f'{name} solved {n} problems'
'Ada solved 3 problems'
>>> f'{n * 2 + 1}'
'7'
```

After a `:` you can say how to format the value:

```
>>> pi = 3.14159265
>>> f'{pi:.2f}'           # 2 digits after the decimal point
'3.14'
>>> f'{7:03d}'           # an int, at least 3 wide, padded with zeros
'007'
>>> f'[{ "hi":>6}]'      # right-align in 6 characters
'[    hi]'
>>> f'[{ "hi":<6}]'      # left-align
'[hi    ]'
>>> f'{1234567:,}'       # thousands separator
'1,234,567'
>>> f'{n = }'           # show the expression and its value – great
↪ for debugging
'n = 3'
```

A small table, with columns lined up:

```
>>> for name, mark in [('Ali', 91.5), ('Mona', 88), ('Yousef', 100)]:
...     print(f'{name:<8}{mark:>7.1f}')
...
Ali          91.5
Mona         88.0
Yousef       100.0
```

(for is in Lab 02. Here just enjoy the alignment.)

3.8 Reading input

`input(prompt)` prints the prompt, waits for a line of text and returns it — always as a **string**:

```
>>> age = input('Age? ')
Age? 20
```

```
>>> age + 1
Traceback (most recent call last):
...
TypeError: can only concatenate str (not "int") to str
>>> int(age) + 1
21
```

Exercises in this course never use `input()`: a function that asks the keyboard cannot be tested automatically. Functions take **parameters** and **return** results. Keep `input()` for small scripts you run by hand.

Checkpoint 2. With `s = 'Mansoura'`, predict:

- (a) `s[1]` (b) `s[-3:]` (c) `s[2:5]` (d) `s[:3]` (e) `s[5:2]`
 (f) `s.lower().count('a')` (g) `s[:3] * 2` (h) `f'{s:*^12}'`

Part 4 — Lists

4.1 Making and reading lists

A list is an **ordered, changeable** sequence of values, written in square brackets:

```
>>> squares = [1, 4, 9, 16, 25]
>>> squares[0]
1
>>> squares[-1]
25
>>> squares[-3:]          # slicing works exactly as for strings
[9, 16, 25]
>>> len(squares)
5
>>> squares + [36, 49]    # + makes a NEW list
[1, 4, 9, 16, 25, 36, 49]
>>> [0] * 4
[0, 0, 0, 0]
```

A list can hold values of different types, including other lists:

```
>>> mixed = [42, 'hello', 3.14, [1, 2]]
>>> mixed[3][0]          # first element of the inner list
1
```

4.2 Lists are mutable

Unlike strings, lists can be changed in place:

```
>>> cubes = [1, 8, 27, 65, 125]    # something is wrong here
>>> 4 ** 3
64
>>> cubes[3] = 64                  # replace the wrong value
>>> cubes
[1, 8, 27, 64, 125]
>>> cubes.append(216)              # add to the end
>>> cubes.append(7 ** 3)
>>> cubes
[1, 8, 27, 64, 125, 216, 343]
```

You can even assign to a **slice**, which can change the length of the list:

```
>>> letters = ['a', 'b', 'c', 'd', 'e', 'f', 'g']
>>> letters[2:5] = ['C', 'D', 'E']    # replace some
>>> letters
['a', 'b', 'C', 'D', 'E', 'f', 'g']
>>> letters[2:5] = []                # remove them
>>> letters
['a', 'b', 'f', 'g']
>>> letters[:] = []                  # empty the whole list
>>> letters
[]
```

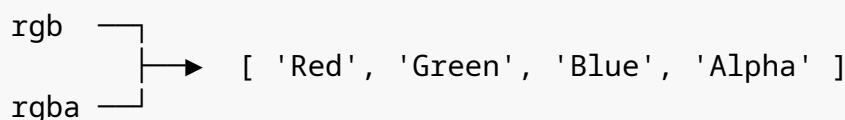
Lab 03 covers all the list methods (insert, pop, remove, sort, ...) and what each one costs.

4.3 Names refer to objects — the most important idea in this lab

This is the idea from Lecture 01: a Python variable is **a name bound to an object**, not a box holding a value. Assignment never copies.

```
>>> rgb = ['Red', 'Green', 'Blue']
>>> rgba = rgb                      # a second NAME for the SAME list
>>> rgba.append('Alpha')
>>> rgb                             # rgb "changed" too
['Red', 'Green', 'Blue', 'Alpha']
>>> rgba is rgb                     # `is` asks: the same object?
True
>>> id(rgb) == id(rgba)             # id() is the object's identity
True
```

Draw it:



To get a **separate** list, copy it. A full slice `[ : ]` returns a new list:

```
>>> correct_rgba = rgb[:]          # or rgb.copy() or list(rgb)
>>> correct_rgba.append('X')
>>> rgb
['Red', 'Green', 'Blue', 'Alpha']
>>> correct_rgba is rgb
False
>>> correct_rgba == rgb[:4] + ['X']  # == asks: equal VALUES?
True
```

`==` compares **values**; `is` compares **identity**. Two different lists can be `==` without being `is`.

The copy is shallow. `rgb[ : ]` is a new outer list, but it holds references to the *same* inner objects. With nested lists this bites:

```
>>> grid = [[0, 0], [0, 0]]
>>> copy = grid[:]
>>> copy[0][0] = 9
>>> grid
[[9, 0], [0, 0]]
```

And the classic trap — making a grid with `*`:

```
>>> board = [[0] * 3] * 3          # three references to ONE inner list
>>> board[0][0] = 'X'
>>> board
[['X', 0, 0], ['X', 0, 0], ['X', 0, 0]]
```

Lab 03 shows the right way (a list comprehension).

Why this matters for the course: when you build a linked list in Week 5, each node's `next` is exactly this kind of reference. If `b = a` surprises you now, `node.next = other` will surprise you then.

Checkpoint 3. Predict the final value of `a`, `b` and `c`.

```
a = [1, 2, 3]
b = a
c = a[:]
b[0] = 100
c[1] = 200
a = a + [4]
b.append(5)
```

Part 5 — First steps towards programming

5.1 A while loop

Here is the first few Fibonacci numbers, where each is the sum of the two before:

```
>>> a, b = 0, 1
>>> while a < 10:
...     print(a)
...     a, b = b, a + b
...
0
1
1
2
3
5
8
```

Several ideas at once:

- **Multiple assignment** on the first and last lines, as in Part 2.4. The last line is the whole algorithm: *the new a is the old b; the new b is the old sum.*
- The **while loop** runs its body as long as the condition `a < 10` is true. In Python, any non-zero number and any non-empty string or list count as true; `0`, `' '` and `[]` count as false.
- **Comparisons:** `<`, `>`, `==` (equal), `!=` (not equal), `<=`, `>=`.
- **Indentation is syntax.** The body of the loop is the indented lines. There are no braces. Use **4 spaces** per level — VS Code does this when you press Tab. Mixing tabs and spaces is an error.

`print(a, end=', ')` keeps the output on one line:

```
>>> a, b = 0, 1
>>> while a < 1000:
...     print(a, end=', ')
...     a, b = b, a + b
...
0,1,1,2,3,5,8,13,21,34,55,89,144,233,377,610,987,
```

5.2 A first if

`if` runs a block only when a condition is true. `else` runs otherwise. Lab 02 covers it properly; for now this is enough:

```
>>> n = 7
>>> if n % 2 == 0:
...     print('even')
```

```
... else:
...     print('odd')
...
odd
```

Combine conditions with and, or and not:

```
>>> x = 15
>>> x > 10 and x % 5 == 0
True
```

5.3 A loop that builds a list

The pattern you will write most often: start empty, append in a loop.

```
>>> powers = []
>>> p = 1
>>> while p < 100:
...     powers.append(p)
...     p = p * 2
...
>>> powers
[1, 2, 4, 8, 16, 32, 64]
```

And the same with an index, walking through an existing list:

```
>>> names = ['ali', 'mona', 'omar']
>>> i = 0
>>> while i < len(names):
...     print(i, names[i].upper())
...     i += 1
...
0 ALI
1 MONA
2 OMAR
```

Forget the `i += 1` and the loop never ends. Press **Ctrl+C** to stop a runaway program — you will need it at least once this term.

Checkpoint 4. *How many times does the body of this loop run, and what is printed?*

```
n, steps = 100, 0
while n > 1:
    n = n // 2
    steps += 1
print(n, steps)
```

This loop halves its input every time. Remember the count — it is the reason binary search, in Week 8, is so fast.

Part 6 — Functions and tests, just enough to start

The exercises are written as **functions**. Lab 02 teaches functions properly; here is the minimum.

```
def area(width, height):
    """Area of a rectangle."""
    return width * height
```

- `def` starts a function definition. `width` and `height` are **parameters**.
- The indented lines are the body.
- The string right after the `def` line is the **docstring** — the function's documentation and, in this course, its contract.
- `return` sends a value back to the caller and ends the function.

```
>>> area(3, 4)
12
>>> result = area(2, 5) + 1
>>> result
11
```

A function can return several values separated by commas. The caller gets them together as a **tuple** (more in Lab 03) and can unpack them:

```
def min_max(a, b):
    if a < b:
        return a, b
    return b, a

>>> min_max(9, 2)
(2, 9)
>>> low, high = min_max(9, 2)
```

print is not return. A function that prints its answer and returns nothing returns `None`, and every test will fail:

```
def bad_area(width, height):
    print(width * height)      # shows 12 on the screen ...

>>> x = bad_area(3, 4)
12
>>> print(x)                  # ... but gives nothing back
None
```

6.1 How the exercises work

Open `labs/lab01.py`. Every function looks like this:

```
def celsius_to_fahrenheit(celsius):
    """Convert Celsius to Fahrenheit, rounded to one decimal place.
    ...
    celsius_to_fahrenheit(100)    -> 212.0
    """
    raise NotImplementedError
```

Replace `raise NotImplementedError` with your code. Then run the tests from the DSA27 folder, with the virtual environment active:

```
pytest tests/test_lab01.py -v          # every Lab 01 test, one per line
pytest tests/test_lab01.py -x          # stop at the first failure
pytest tests/test_lab01.py -k fib      # only tests with "fib" in their
↳ name
```

A failing test looks like this:

```
def test_celsius_to_fahrenheit(c, f):
>     assert celsius_to_fahrenheit(c) == f
E     assert 97.88000000000001 == 97.9
```

Read it as: *the test called your function with `c = 36.6`; you returned `97.88000000000001`; it expected `97.9`.* You forgot to round. The test file is the specification — open `tests/test_lab01.py` and read it whenever you are unsure what is wanted.

You can also try a function by hand in the REPL:

```
>>> from labs.lab01 import celsius_to_fahrenheit
>>> celsius_to_fahrenheit(100)
212.0
```

After you change the file, the REPL still has the **old** version. Leave with `exit()` and start again, or use the tests — they always load the current file.

Part 7 — Exercises

All in `labs/lab01.py`. Do them in order. Each docstring gives examples. Use only what this lab taught: no `import`, no `for` (that is next week).

#	Function	Practises
1	<code>seconds_to_hms(seconds)</code>	<code>//</code> , <code>%</code> , f-string padding
2	<code>celsius_to_fahrenheit(celsius)</code>	arithmetic, round

#	Function	Practises
3	<code>split_evenly(total, people)</code>	// and % together, returning two values
4	<code>initials(full_name)</code>	split, indexing, while, building a string
5	<code>is_palindrome_word(word)</code>	strip, lower, slicing with step -1
6	<code>mask_email(email)</code>	index, slicing, string repetition, if
7	<code>middle(values)</code>	slice arithmetic with //
8	<code>rotate_left(values, k)</code>	slicing, %, not changing the input
9	<code>fib_list(n)</code>	while, multiple assignment, append
10	<code>collatz_steps(n)</code>	while, if/else, counting

Hints, only if you are stuck for more than ten minutes:

- **1.** Hours are seconds // 3600. What is left is seconds % 3600; do the same again with 60. Then `f'{h}:{m:02d}:{s:02d}'`.
- **4.** `' a b '.split()` is `['a', 'b']` — the spaces problem disappears. Start with `result = ''` and add `word[0].upper() + '.'` for each word.
- **6.** `at = email.index('@')`. The name is `email[:at]`; the domain, including the @, is `email[at:]`.
- **7.** Work out by hand which indices you want for lengths 0 to 5, and look for the pattern. `(n - 1) // 2` and `n // 2` are worth trying.
- **8.** An empty list needs its own if, because `k % 0` is an error. Otherwise, after `k = k % len(values)`, the answer is two slices joined.
- **9.** Stop when the list has n numbers: `while len(result) < n:`

When all ten pass:

```
tests/test_lab01.py
↪ ..... [100%]
```

show the TA.

Part 8 — Take-home practice (not graded)

Do these in a scratch file, for example `practice/week01.py`.

1. Write a script that prints the multiplication table from 1×1 to 9×9 in nine right-aligned rows, using two while loops and an f-string.

- Given `s = 'Data Structures and Algorithms'`, print, each with one expression: the number of words; the string with its words in reverse order; the acronym DSaA; the string with every vowel replaced by `*`.
- Without running it, write down what `x` is after `x = [1, [2, 3]]`; `y = x[:]`; `y[1].append(4)`; `y[0] = 9`. Then run it. Explain the result in one sentence using the words *shallow copy*.
- Write `digits(n)` that returns the list of decimal digits of a non-negative integer, most significant first, using only `%`, `//` and a `while` loop. Why does `digits(0)` need thought?
- Time something. In a script:

```
import time
start = time.perf_counter()
total, i = 0, 0
while i < 10_000_000:
    total += i
    i += 1
print(time.perf_counter() - start, 'seconds')
```

Run it with 1, 2, 4 and 8 million. What happens to the time when the input doubles? That question is all of Week 2.

Summary

Idea	The one line to keep
REPL vs script	The REPL prints every value; a script prints only what you print.
/ vs // vs %	/ is always float; // rounds down ; <code>a == (a//b)*b + a%b</code> .
int vs float	int never overflows; float is approximate — never <code>==</code> after arithmetic.
Strings	Immutable. Index from 0; -1 is last; <code>s[i:j]</code> stops <i>before</i> <code>j</code> .
Lists	Mutable. Same indexing and slicing as strings.
Names	A name refers to an object. <code>b = a</code> copies nothing . <code>a[:]</code> is a shallow copy.
<code>==</code> vs <code>is</code>	Equal values vs the same object.
Indentation	Is syntax. Four spaces.
<code>print</code> vs <code>return</code>	Tests see only what you return.

Lab 02 — Lab 02 — Control Flow, Functions, Errors and Modules

How to use this lab. As in Lab 01: type every example, predict before you run at each **Checkpoint**, then make `pytest tests/test_lab02.py` pass. This lab assumes Lab 01 — if slicing or `b = a` still surprises you, go back first.

Duration	One 2-hour lab session, plus about 3 hours at home
You will write	<code>labs/lab02.py</code> — 13 functions
Graded by	<code>tests/test_lab02.py</code>
Connects to	Lecture 02 — Complexity. Part 7 counts and times loops.

What you will be able to do

1. choose between `if/elif/else` and `match`, and write both;
 2. loop with `for` over any sequence, use `range`, `enumerate` and `zip`, and control a loop with `break`, `continue` and `else`;
 3. define functions with default, keyword, `*args` and `**kwargs` parameters, and avoid the mutable-default trap;
 4. explain what happens to a list when you pass it to a function;
 5. read a traceback, catch an exception with `try/except`, and raise your own;
 6. write a module, import it, and use the `if __name__ == "__main__":` guard;
 7. count the steps a loop takes and time it — the start of Week 2.
-

Part 1 — if statements

```
x = int(input("Please enter an integer: "))
if x < 0:
    x = 0
    print('Negative changed to zero')
elif x == 0:
    print('Zero')
elif x == 1:
    print('Single')
else:
```

```
print('More')
```

There can be any number of `elif` (“else if”) parts, and the `else` is optional. Python checks the conditions **top to bottom** and runs the **first** block whose condition is true — then skips the rest. So order matters:

```
score = 95
if score >= 50:
    grade = 'pass'
elif score >= 90:          # never reached: 95 already matched above
    grade = 'excellent'
```

Put the most specific condition first.

1.1 What counts as true

Any value can be used as a condition. These are **false**:

False, None, 0, 0.0, '' (empty string), [], (), {}, set(), range(0)

Everything else is **true**. So the idiomatic way to test “is this list empty?” is:

```
if not values:
    print('empty')
```

rather than `if len(values) == 0:`.

1.2 Comparisons and Boolean operators

- Comparisons **chain**: `0 <= x < 10` means `0 <= x` and `x < 10`.
- and, or, not. not binds tightest, then and, then or: `a or b and not c` means `a or (b and (not c))`.
- and and or **short-circuit**: they stop as soon as the answer is known, and they return the **last value they looked at**, not necessarily True or False:

```
>>> 0 or 'default'
'default'
>>> 'x' and 'y'
'y'
>>> [] and 1/0          # 1/0 is never evaluated
[]
```

Short-circuiting is how you write a safe test:

```
if i < len(values) and values[i] == target:    # never an IndexError
    ...
```

- in and not in test membership: `'a' in 'banana'`, `3 in [1, 2, 3]`.
- is tests identity. Use it for None, and **only** for None (and True/False):

```
if result is None:          # correct
if result == None:          # works, but not the Python way
```

Checkpoint 1. What does each expression give?

- (a) `1 < 2 < 3 > 2` (b) `' '` or `[]` or `0` (c) `not []`
(b) `5 and 0 and 1/0` (e) `[0]` and `'yes'`
-

Part 2 — for loops and range

2.1 for iterates over the items

Python's `for` is not C's counting loop. It walks through **the items** of any sequence, in order:

```
>>> words = ['cat', 'window', 'defenestrate']
>>> for w in words:
...     print(w, len(w))
...
cat 3
window 6
defenestrate 12
```

It works on anything iterable — strings, lists, tuples, dictionaries, files, ranges:

```
>>> for ch in 'DSA':
...     print(ch, end=' ')
...
D S A
```

Compare with the Lab 01 while version. There is no index to forget to increment, so there is no bug to write.

2.2 Do not change a collection while you loop over it

Removing items from a list while a `for` loop walks through it skips elements. Loop over a **copy**, or — better — build a new list:

```
users = ['ali', 'mona', 'guest', 'guest2', 'omar']

# wrong: skips 'guest2', because the list shifts under the loop
for u in users:
    if u.startswith('guest'):
        users.remove(u)

# right: loop over a copy
```

```

for u in users[:]:
    if u.startswith('guest'):
        users.remove(u)

# best: build a new list
active = []
for u in users:
    if not u.startswith('guest'):
        active.append(u)

```

2.3 range

To loop over numbers, use range:

```

>>> for i in range(5):
...     print(i, end=' ')
...
0 1 2 3 4

```

range(stop) gives 0, 1, ..., stop - 1 — “up to but not including”, exactly like slicing. range(n) has exactly n numbers in it. There are two more forms:

```

>>> list(range(5, 10))           # start, stop
[5, 6, 7, 8, 9]
>>> list(range(0, 10, 3))        # start, stop, step
[0, 3, 6, 9]
>>> list(range(-10, -100, -30)) # counting down
[-10, -40, -70]
>>> list(range(10, 0, -1))
[10, 9, 8, 7, 6, 5, 4, 3, 2, 1]
>>> sum(range(4))                # 0 + 1 + 2 + 3
6

```

range(10) is **not a list**. It is a small object that produces the numbers on demand, so range(10**12) costs no memory. list(range(...)) makes the list when you really want one.

2.4 enumerate and zip

When you need the index **and** the item, do not write for i in range(len(values)). Use enumerate:

```

>>> for i, name in enumerate(['ali', 'mona', 'omar']):
...     print(i, name)
...
0 ali

```

```
1 mona
2 omar
>>> for rank, name in enumerate(['ali', 'mona'], start=1):
...     print(f'{rank}. {name}')
...
1. ali
2. mona
```

To walk two sequences side by side, use `zip`. It stops at the shorter one:

```
>>> questions = ['name', 'quest', 'favourite colour']
>>> answers = ['Lancelot', 'the holy grail', 'blue']
>>> for q, a in zip(questions, answers):
...     print(f'What is your {q}? It is {a}.')
...
What is your name? It is Lancelot.
What is your quest? It is the holy grail.
What is your favourite colour? It is blue.
```

`range(len(values))` still has its place — when you need to compare `values[i]` with `values[i + 1]`, or write into `values[i]`. Sorting algorithms in Week 9 are full of it.

2.5 Nested loops

A loop inside a loop runs its inner body `outer × inner` times:

```
>>> for i in range(1, 4):
...     for j in range(1, 4):
...         print(f'{i * j:3}', end='')
...     print()
...
1  2  3
2  4  6
3  6  9
```

Remember that multiplication. It is where $O(n^2)$ comes from.

Part 3 — break, continue, else, pass

break leaves the innermost loop immediately. **continue** skips the rest of this iteration and goes to the next one.

```
>>> for num in range(2, 10):
...     if num % 2 == 0:
...         print(f'Found an even number {num}')
```

```
...         continue
...     print(f'Found an odd number {num}')
...
Found an even number 2
Found an odd number 3
Found an even number 4
Found an odd number 5
Found an even number 6
Found an odd number 7
Found an even number 8
Found an odd number 9
```

else on a loop runs when the loop finishes **without** hitting break. Think of it as “no break”. It is exactly what a search needs:

```
>>> for n in range(2, 10):
...     for x in range(2, n):
...         if n % x == 0:
...             print(n, 'equals', x, '*', n // x)
...             break
...         else:
...             # the inner loop found no factor
...             print(n, 'is a prime number')
...
2 is a prime number
3 is a prime number
4 equals 2 * 2
5 is a prime number
6 equals 2 * 3
7 is a prime number
8 equals 2 * 4
9 equals 3 * 3
```

Look carefully: the else belongs to the inner **for**, not to the if.

pass does nothing. Use it where Python needs a statement and you have none yet:

```
def todo():
    pass          # write this later

class EmptyForNow:
    pass
```

(raise `NotImplementedError`, as in the exercise files, is better for a function that must not be used before it is written — it fails loudly instead of silently returning `None`.)

Part 4 — match statements

match (Python 3.10+) compares a value against **patterns** and runs the first one that fits. At its simplest it replaces a chain of `elif ==`:

```
def http_error(status):
    match status:
        case 400:
            return "Bad request"
        case 404:
            return "Not found"
        case 418:
            return "I'm a teapot"
        case 401 | 403:          # | means "or"
            return "Not allowed"
        case _:                # _ matches anything: the default
            return "Something's wrong with the internet"
```

Its real power is **destructuring**: a pattern can look like the data and pull it apart. Here the value is a list of words:

```
def run(command):
    match command.split():
        case ['quit']:
            return 'bye'
        case ['go', direction]:          # exactly two words,
        ↪ binds direction
            return f'going {direction}'
        case ['drop', *objects]:         # 'drop' then any
        ↪ number of words
            return f'dropping {len(objects)} things'
        case ['go', ('north' | 'south')]: # never reached — see
        ↪ below
            return 'unreachable'
        case _:
            return f'unknown command: {command!r}'
```

```
>>> run('go north')
'going north'
>>> run('drop sword shield')
'dropping 2 things'
>>> run('dance')
'unknown command: 'dance''
```

- A plain name in a pattern (direction) **captures** whatever is there.
- `*objects` captures “the rest” as a list, possibly empty.
- Cases are tried **in order**; the fourth case above can never match because the second

catches every two-word go.

- A **guard** adds a condition: `case ['go', d] if d in exits:`.

`{command!r}` in the f-string means “use `repr()`”, which shows quotes around a string — useful in error messages.

Part 5 — Defining functions

5.1 The basics

```
def fib(n):
    """Print the Fibonacci numbers less than n."""
    a, b = 0, 1
    while a < n:
        print(a, end=' ')
        a, b = b, a + b
    print()

>>> fib(2000)
0 1 1 2 3 5 8 13 21 34 55 89 144 233 377 610 987 1597
```

- The first statement may be a **docstring**. Tools show it: try `help(fib)`. Write one for every function — in this course the docstring is the contract.
- A function is an object like any other. `f = fib` gives it a second name; `f(100)` then works.
- A function with no return, or a bare `return`, returns **None**:

```
>>> print(fib(0))
```

None

Better: **return** the values and let the caller decide what to do with them.

```
def fib2(n):
    """Return a list of the Fibonacci numbers less than n."""
    result = []
    a, b = 0, 1
    while a < n:
        result.append(a)
        a, b = b, a + b
    return result

>>> fib2(100)
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

5.2 Scope: local and global

Variables assigned inside a function are **local**: they exist only while the function runs, and they hide any global variable with the same name.

```
count = 0

def increment():
    count = count + 1      # UnboundLocalError: count is local here

def increment_ok():
    global count           # possible, but almost always a bad design
    count = count + 1
```

When a name is **read**, Python looks in this order: **L**ocal, **E**nclosing function, **G**lobal (the module), **B**uilt-in — the *LEGB* rule. Assigning to a name anywhere in a function makes it local for the whole function.

The rule for this course: a function gets what it needs from its **parameters** and gives results back with **return**. No globals.

5.3 What happens when you pass a list

Arguments are passed **by object reference** — the parameter becomes a new *name* for the *same object*. (This is the `b = a` rule from Lab 01 again.)

```
def add_item(values):
    values.append('new')    # changes the caller's list

def replace(values):
    values = ['brand', 'new'] # rebinds the LOCAL name only

>>> data = ['old']
>>> add_item(data)
>>> data
['old', 'new']
>>> replace(data)
>>> data
['old', 'new']
```

A function can **mutate** an object it is given, but it cannot make the caller's *name* point somewhere else. Numbers and strings are immutable, so a function can never change the caller's number or string.

This is why the exercises say “return a NEW list” or “the input must not change”. A test checks it.

5.4 Default argument values

```
def ask_ok(prompt, retries=4, reminder='Please try again!'):
    ...

ask_ok('Really quit?')
ask_ok('OK to overwrite?', 2)
ask_ok('OK to overwrite?', 2, 'Come on, only yes or no!')
```

The trap. A default value is evaluated **once**, when the `def` runs — not on every call. With a mutable default, every call shares one object:

```
def f(a, L=[]):
    L.append(a)
    return L

>>> f(1)
[1]
>>> f(2)
[1, 2]          # the SAME list as last time
>>> f(3)
[1, 2, 3]
```

The fix is to default to `None` and create the object inside:

```
def f(a, L=None):
    if L is None:
        L = []
    L.append(a)
    return L
```

This is exercise 11.

5.5 Keyword arguments

Arguments can be passed **by name**, in any order, after the positional ones:

```
def parrot(voltage, state='a stiff', action='vroom', type='Norwegian
↪ Blue'):
    print(f"-- This parrot wouldn't {action} if you put {voltage}
↪ volts through it.")
    print(f"-- Lovely plumage, the {type}. It's {state}!")

parrot(1000)                # 1 positional
parrot(voltage=1000)        # 1 keyword
parrot(action='VOOOOOM', voltage=1000000) # 2 keywords, any order
parrot('a million', 'bereft of life', 'jump') # 3 positional
```

```

parrot(voltage=5.0, 'dead')           # SyntaxError:
↪ positional after keyword
parrot(110, voltage=220)              # TypeError: two values
↪ for voltage

```

Keyword arguments make calls readable: `sorted(names, reverse=True)` says what it means; `sorted(names, None, True)` would not.

Special markers in the parameter list control how arguments may be passed:

```

def f(pos_only, /, normal, *, kw_only):
    ...

f(1, 2, kw_only=3)           # fine
f(1, normal=2, kw_only=3)    # fine
f(pos_only=1, normal=2, kw_only=3) # TypeError
f(1, 2, 3)                  # TypeError: kw_only must be named

```

Everything before `/` is positional-only; everything after `*` is keyword-only. You will read these in library documentation more often than you write them.

5.6 Any number of arguments: `*args` and `**kwargs`

A parameter written `*name` collects **extra positional arguments** into a tuple. A parameter written `**name` collects **extra keyword arguments** into a dictionary.

```

def report(title, *values, **options):
    print(title, values, options)

>>> report('marks', 90, 75, 88, sep=';', rounded=True)
marks (90, 75, 88) {'sep': ';', 'rounded': True}
>>> report('none')
none () {}

```

The reverse — **unpacking** a list or dictionary into arguments at a call — uses the same stars:

```

>>> args = [3, 6]
>>> list(range(*args))           # same as range(3, 6)
[3, 4, 5]
>>> opts = {'sep': '-', 'end': '!\n'}
>>> print('a', 'b', **opts)     # same as print('a', 'b', sep='-',
↪ end='!\n')
a-b!

```

Exercise 12, `stats(*numbers)`, uses this.

5.7 lambda: small anonymous functions

`lambda args: expression` makes a function in one line. It is limited to a single expression, which it returns.

```
>>> square = lambda x: x * x
>>> square(7)
49
>>> pairs = [(1, 'one'), (2, 'two'), (3, 'three'), (4, 'four')]
>>> pairs.sort(key=lambda pair: pair[1])      # sort by the word
>>> pairs
[(4, 'four'), (1, 'one'), (3, 'three'), (2, 'two')]
```

`key=` functions are where you will use `lambda` most — for sorting, `min`, `max`.

Functions that return functions. A function defined inside another remembers the variables around it, even after the outer function has returned. That is called a *closure*:

```
def make_incrementor(n):
    return lambda x: x + n

>>> add42 = make_incrementor(42)
>>> add42(0)
42
>>> add42(1)
43
```

Exercises 12 and 13 practise passing and returning functions.

5.8 Docstrings, annotations and style

Docstring conventions. First line: a short summary sentence. If there is more, a blank line, then detail. Every `dsa/` skeleton follows this shape.

Annotations are optional hints about types. Python does not enforce them; editors and tools use them:

```
def greet(name: str, times: int = 1) -> str:
    return ('Hello ' + name + '! ') * times
```

Style (PEP 8), the parts that matter most:

- 4 spaces per indentation level, never tabs.
- Lines no longer than about 79–88 characters (the repo sets a ruler at 88).
- Blank lines between functions; blank lines inside functions sparingly.
- Comments on their own line when possible; a space after `#`.
- Spaces around operators and after commas: `a = f(1, 2) + g(3, 4)`.
- `lower_case_with_underscores` for functions and variables; `UpperCamelCase` for classes; `self` as the first method parameter.

- ASCII or UTF-8 only, and English names for code identifiers.

Readable code is not decoration. You will read your code far more often than you write it — and so will the TA marking it.

Checkpoint 2. *What is printed?*

```
def g(x, items=[]):
    items.append(x)
    x = x * 10
    return len(items)

n = 1
print(g(n), g(n), n)
```

Part 6 — Errors and exceptions

6.1 Two kinds of error

A **syntax error** means Python could not even read your code. Nothing runs:

```
>>> while True print('Hello world')
File "<stdin>", line 1
    while True print('Hello world')
            ^^^^^
SyntaxError: invalid syntax
```

The caret points at where Python noticed the problem. The real mistake is often **just before** it — here, the missing `:`.

An **exception** happens while the code runs:

```
>>> 10 * (1/0)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
ZeroDivisionError: division by zero
>>> 4 + spam*3
NameError: name 'spam' is not defined
>>> '2' + 2
TypeError: can only concatenate str (not "int") to str
```

6.2 Reading a traceback

Put this in a file `trace_demo.py` and run it:

```
def average(values):
    return total(values) / len(values)

def total(values):
    result = 0
    for v in values:
        result += v
    return result

print(average([1, 2, 3]))
print(average([]))
```

2.0

Traceback (most recent call last):

```
File "trace_demo.py", line 11, in <module>
    print(average([]))
    ~~~~~^
```

```
File "trace_demo.py", line 2, in average
    return total(values) / len(values)
    ~~~~~^
```

ZeroDivisionError: division by zero

Read it **from the bottom up**:

1. **Last line:** the exception type and message — *what* went wrong.
2. **The frame above it:** the line that raised it — *where*, file and line.
3. **Further up:** who called that function, and who called that — *how we got there*. The top is where the program started.

That list of frames is the **call stack**: each function call pushes a frame, each return pops one. You will study the call stack properly in Week 3, when functions call *themselves*, and again in Week 6, when you build a stack yourself.

The exceptions you will meet this term:

Exception	Typical cause
NameError	a misspelt variable, or used before assignment
TypeError	wrong type: '2' + 2, calling a non-function, wrong number of arguments
ValueError	right type, wrong value: int('abc'), [1, 2].index(9)
IndexError	list or string index out of range
KeyError	dictionary key not present
AttributeError	None.append(1) — usually a function that returned None

Exception	Typical cause
ZeroDivisionError	$x / 0, x // 0, x \% 0$
RecursionError	a recursive function with no working base case (Week 3)
NotImplementedError	every skeleton in dsa/ and labs/ — until you write it
StopIteration	an iterator has no more items (Lab 03)

6.3 Handling exceptions: try / except

```
while True:
    try:
        x = int(input('Please enter a number: '))
        break
    except ValueError:
        print('Oops! That was no valid number. Try again...')
```

How it runs:

1. The try block runs.
2. If no exception happens, the except block is skipped.
3. If an exception happens, the rest of the try block is skipped. If its type matches the except, that block runs and the program carries on after the whole statement.
4. If it does not match, it travels up to the caller — and if nobody catches it, the program stops with a traceback.

The full form:

```
def read_ratio(a, b):
    try:
        result = int(a) / int(b)
    except ValueError as err:                # bind the exception object
        print('not a number:', err)
        return None
    except ZeroDivisionError:
        print('cannot divide by zero')
        return None
    else:                                    # runs only if the try did
        ↪ NOT raise
        print('ok')
        return result
    finally:                                # ALWAYS runs, even after return
        print('done')
```

```
>>> read_ratio('6', '3')
ok
done
2.0
>>> read_ratio('6', '0')
cannot divide by zero
done
```

- Several types in one clause: `except (TypeError, ValueError):`.
- `else` holds the code that should run only on success — keeping it out of the `try` means you do not accidentally catch its errors too.
- `finally` is for clean-up that must happen no matter what: closing a file, releasing a lock.

Catch narrowly. A bare `except:` or `except Exception:` catches *everything*, including your own bugs and misspelt names, and hides them. Catch the specific exception you expect, around the smallest block of code that can raise it.

6.4 Raising exceptions

A function that is given input it cannot handle should **say so**, loudly:

```
def factorial(n):
    if n < 0:
        raise ValueError(f'factorial is undefined for negative n, got
        ↪ {n}')
    result = 1
    for k in range(2, n + 1):
        result *= k
    return result
```

```
>>> factorial(-1)
Traceback (most recent call last):
...
ValueError: factorial is undefined for negative n, got -1
```

Many skeletons in `dsa/` say “Raises `ValueError` for ...”. The tests check it like this:

```
def test_factorial_rejects_negative():
    with pytest.raises(ValueError):
        factorial(-1)
```

Returning `None`, `-1` or an error string instead would **fail** that test — and, more importantly, would let a wrong value travel silently into the rest of a program.

A bare `raise` inside an `except` block re-raises the exception you caught, after you have done something with it (logged it, for example).

6.5 assert

`assert condition, message` raises `AssertionError` if the condition is false. It states something you believe must be true at that point:

```
def mean(values):  
    assert len(values) > 0, 'mean of an empty list'  
    return sum(values) / len(values)
```

Use `assert` to catch **your own** bugs while developing, not to validate user input (Python can be run with assertions switched off). `pytest` tests are written as plain `assert` statements — that is all a test is.

Checkpoint 3. What is printed?

```
def f(x):  
    try:  
        print('A')  
        y = 10 // x  
        print('B')  
    except ZeroDivisionError:  
        print('C')  
        return -1  
    else:  
        print('D')  
        return y  
    finally:  
        print('E')  
  
print(f(0))  
print(f(5))
```

Part 7 — Modules

7.1 A file is a module

Any `.py` file is a **module**; its name is the file name without `.py`. Create `fibonacci.py`:

```
"""Fibonacci numbers module."""  
  
def fib(n):  
    """Print the Fibonacci series up to n."""  
    a, b = 0, 1  
    while a < n:  
        print(a, end=' ')  
        a, b = b, a + b
```

```

print()

def fib2(n):
    """Return the Fibonacci series up to n."""
    result = []
    a, b = 0, 1
    while a < n:
        result.append(a)
        a, b = b, a + b
    return result

```

In the REPL, started in the same folder:

```

>>> import fibo
>>> fibo.fib(100)
0 1 1 2 3 5 8 13 21 34 55 89
>>> fibo.fib2(10)
[0, 1, 1, 2, 3, 5, 8]
>>> fibo.__name__
'fibo'

```

The forms of import:

import fibo	# use as fibo.fib(...)
import fibo as fb	# use as fb.fib(...)
from fibo import fib, fib2	# use as fib(...) directly
from fibo import fib as f	# rename on import
from fibo import *	# everything – avoid: you can no longer
↪ tell	# where a name came from

A module's top-level code runs **once**, the first time it is imported in a program. If you edit `fibo.py`, a running REPL keeps the old version — restart it.

7.2 Scripts and modules at once: `__name__`

When a file is **run** (`python fibo.py`), Python sets its `__name__` to `"__main__"`. When it is **imported**, `__name__` is the module name. So this block runs only when the file is run directly:

```

if __name__ == "__main__":
    import sys
    fib(int(sys.argv[1]))

```

```

python fibo.py 50
0 1 1 2 3 5 8 13 21 34

```

Importing `fibonacci` does not run it. Use this guard for demos and quick checks at the bottom of your own files.

7.3 Packages, and how `from dsa.recursion import factorial` works

A **package** is a folder of modules with an `__init__.py` file in it. The course repository has three: `dsa/`, `viz/` and `labs/`. A dotted name walks into the folder:

```
from labs.lab02 import is_prime      # labs/lab02.py, function
↳ is_prime
from dsa.recursion import factorial  # dsa/recursion.py
from viz.draw import draw_array     # viz/draw.py
```

Python finds `labs` because it searches the folders in `sys.path`, and the first of those is the folder you started Python in (or the script's folder). That is why you run everything **from the DSA27 folder**. `tests/conftest.py` adds the repository root to `sys.path` so `pytest` can always find the packages.

`dir(module)` lists the names a module defines — useful for exploring:

```
>>> import math
>>> dir(math)[:6]
['__doc__', '__loader__', '__name__', '__package__', '__spec__',
↳ 'acos']
```

7.4 The standard library: three modules you need now

Python comes with hundreds of modules. Three are enough for this month:

```
>>> import math
>>> math.sqrt(16), math.floor(-2.5), math.ceil(2.1), math.pi
(4.0, -3, 3, 3.141592653589793)
>>> math.log2(1024), math.isqrt(17)      # isqrt: integer square root
(10.0, 4)
>>> math.inf > 10**100                    # infinity, handy as a
↳ starting minimum
True

>>> import random
>>> random.seed(27)                        # same seed -> same "random" numbers
↳ every run
>>> random.randint(1, 6)                  # an int from 1 to 6 inclusive
>>> random.choice(['a', 'b', 'c'])
>>> random.sample(range(100), 5)          # 5 distinct values
>>> values = list(range(10)); random.shuffle(values) # in place

>>> import time
```

```
>>> start = time.perf_counter()          # a high-resolution clock, in
↳ seconds
>>> # ... the work you want to time ...
>>> elapsed = time.perf_counter() - start
```

Part 8 — Counting and timing: a first look at complexity

Lecture 02 says an algorithm's cost is how its work **grows** with the input. You can see that growth with two tools from this lab: a counter and a clock.

Count. Add a counter to the body of a loop and see how it depends on n :

```
def count_single(n):
    steps = 0
    for i in range(n):
        steps += 1
    return steps

def count_nested(n):
    steps = 0
    for i in range(n):
        for j in range(n):
            steps += 1
    return steps

for n in [10, 100, 1000]:
    print(n, count_single(n), count_nested(n))
```

```
10 10 100
100 100 10000
1000 1000 1000000
```

Multiply n by 10: the single loop does 10 times the work; the nested loop 100 times. That is $O(n)$ against $O(n^2)$, counted rather than claimed.

Time. Put this in `timing.py`:

```
import time

def is_prime_slow(n):
    """Try every divisor from 2 to  $n - 1$ ."""
    if n < 2:
        return False
    for d in range(2, n):
        if n % d == 0:
```

```

        return False
    return True

for n in [1_000_003, 2_000_003, 4_000_037, 8_000_009]:    # all prime
    start = time.perf_counter()
    is_prime_slow(n)
    print(f'{n:>10,}    {time.perf_counter() - start:.3f} s')
```

Each n is about double the one before. The time should roughly double too — the signature of $O(n)$. Now replace `range(2, n)` with a loop that stops when $d * d > n$ (exercise 5), and run it again. The times collapse, because the loop now runs about $\sqrt{n}$ times instead of n . For $n = 8,000,009$ that is under 3,000 iterations instead of 8 million.

Two warnings about timing, which Week 2 develops:

- A single measurement is noisy. Run it several times and keep the **smallest** (the other runs were interrupted by something else on your machine). `viz/complexity.py` does exactly this for you.
- Timing tells you about *this* machine and *this* input. Counting tells you the shape. You want both.

Part 9 — Exercises

All in `labs/lab02.py`. Run `pytest tests/test_lab02.py -v`.

#	Function	Practises
1	<code>classify_triangle(a, b, c)</code>	<code>if/elif</code> , <code>chained ==</code> , <code>raise ValueError</code>
2	<code>course_result(coursework, final, attendance)</code>	order of conditions, range checks
3	<code>fizzbuzz(n)</code>	<code>for</code> , <code>range</code> , <code>%</code> , building a list
4	<code>is_prime(n)</code>	early return, stopping at $\sqrt{n}$
5	<code>primes_below(n)</code>	calling your own function
6	<code>first_repeated(values)</code>	nested loops, early exit
7	<code>count_pairs(n)</code>	counting a nested loop, then a formula
8	<code>calculator(command)</code>	match with sequence patterns
9	<code>parse_int(text, default=None)</code>	<code>try/except</code> <code>ValueError</code>
10	<code>stats(*numbers)</code>	<code>*args</code> , a loop that tracks min and max

#	Function	Practises
11	<code>append_to(item, target=None)</code>	the mutable-default trap, is None
12	<code>make_multiplier(k)</code>	returning a function
13	<code>apply_n(f, x, n)</code>	passing a function

Exercise 2 applies the real rules of this course, from the course guide: you need at least 75% attendance to sit the final, at least 30% of the final, and at least 60% overall.

Hints:

- **1.** Validate first (positive sides, then the triangle inequality), classify second. `a == b == c` is a chained comparison.
 - **4.** `d * d <= n` avoids floats; `math.isqrt(n)` is the other clean way. Check `n < 2` before anything else.
 - **6.** For each position `i`, look for values `[i]` among the elements *before* it. The first `i` where you find it gives the answer.
 - **7.** For `n = 4` the pairs are (0,1) (0,2) (0,3) (1,2) (1,3) (2,3). How many pairs can you make from `n` things?
 - **8.** `match` command `split()`: with cases like `case ['add', a, b]:`. The captured `a` and `b` are **strings**; convert with `int()`. What happens to `int('x')`? Is that the exception the docstring asks for?
 - **10.** Start smallest and largest at `numbers[0]`, not at `0` — why not `0`?
 - **12.** `return lambda x: x * k`, or an inner `def`.
-

Part 10 — Take-home practice (not graded)

1. Write `print_calendar(first_weekday, days)` that prints a month like a wall calendar, seven right-aligned columns, starting on the given weekday (0 = Saturday). Use `range`, `end=' '` and `%`.
 2. The *digital root* of 942 is $9 + 4 + 2 = 15 \rightarrow 1 + 5 = 6$. Write `digital_root(n)` with loops only. Then find a one-line formula using `%` and check it against your loop for every `n` below 10,000.
 3. Write `binary(n)` that returns the binary digits of a non-negative integer as a string, using `//` and `%`. Compare with `bin(n)`. How many loop iterations does it take for `n = 1,000,000`? Why that number?
 4. Write a script `guess.py` that picks a random number from 1 to 100 and lets the user guess, saying “higher” or “lower” and handling non-numbers with `try/except`. With the best strategy, what is the largest number of guesses ever needed? (You have met this number twice now.)
 5. Extend Part 8: plot, on paper, the times of `is_prime_slow` against `n`. Is it a straight line?
-

Summary

Idea	The one line to keep
if/elif Truthiness	First true condition wins. Most specific first. 0, '', [], {}, None are false. if not values:
for	Iterates over items. enumerate for the index, zip for pairs.
range(n) Loop else match	0 .. n-1, lazy, exactly n numbers. Runs when the loop ends without break. Patterns that look like the data; _ is the default.
Arguments	Passed by object reference: mutations show, rebinding does not.
Defaults	Evaluated once. Never a mutable default: use None.
Exceptions	Read tracebacks bottom-up. Catch narrowly. raise on bad input.
Modules	A file is a module; a folder with <code>__init__.py</code> is a package.
Cost	Nested loops multiply. Count it, then time it.

Lab 03 — Lab 03 — Data Structures, Classes and Generators

How to use this lab. Same routine: type every example, predict at each **Checkpoint**, then make `pytest tests/test_lab03.py` pass. This is the longest of the three labs and the most important: from Week 4 on, every structure you build is a **class**, and the lists and dictionaries of this lab are exactly what you will learn to build yourself — inside `dsa/` you may not use them for storage (Lecture 02).

Duration	One 2-hour lab session, plus about 4 hours at home
You will write	<code>labs/lab03.py</code> — 9 functions and one class, <code>Bag</code>
Graded by	<code>tests/test_lab03.py</code>
Connects to	Lecture 03 — Recursion (Part 11), and every week after it

What you will be able to do

1. use every list method, and say which ones are $O(1)$ and which are $O(n)$;
2. use a list as a stack, and know why it makes a bad queue;
3. write list, set and dictionary comprehensions;
4. choose between a list, a tuple, a set and a dictionary for a job — and justify it by cost;
5. write a class with `__init__`, methods and special methods such as `__len__`, `__iter__`, `__eq__` and `__repr__`;
6. write a generator with `yield`, and explain what `for` does behind the scenes;
7. trace a recursive function on the call stack.

Part 1 — More on lists

1.1 All the list methods

Method	What it does	Cost
<code>a.append(x)</code>	add <code>x</code> at the end	$O(1)^*$
<code>a.extend(iterable)</code>	add every item of <code>iterable</code> at the end	$O(k)$

Method	What it does	Cost
<code>a.insert(i, x)</code>	insert <code>x</code> before position <code>i</code>	$O(n)$
<code>a.remove(x)</code>	remove the first item equal to <code>x</code> ; <code>ValueError</code> if none	$O(n)$
<code>a.pop()</code>	remove and return the last item	$O(1)$
<code>a.pop(i)</code>	remove and return the item at <code>i</code>	$O(n)$
<code>a.clear()</code>	remove everything	$O(n)$
<code>a.index(x)</code>	position of the first <code>x</code> ; <code>ValueError</code> if none	$O(n)$
<code>a.count(x)</code>	how many times <code>x</code> appears	$O(n)$
<code>a.sort()</code>	sort in place	$O(n \log n)$
<code>a.reverse()</code>	reverse in place	$O(n)$
<code>a.copy()</code>	a shallow copy, same as <code>a[:]</code>	$O(n)$
<code>x in a</code>	membership test	$O(n)$
<code>a[i], a[i] = x, len(a)</code>	index, assign, length	$O(1)$

* *amortised* — usually instant, occasionally slow when the list has to grow. You will build this yourself in Week 4 and see why the average stays $O(1)$.

Where do those costs come from? A Python list is an **array of references** stored side by side in memory. Reaching `a[i]` is one address calculation — $O(1)$. But inserting at the front means shifting every other reference one place to the right — $O(n)$. Looking for a value means checking items one by one — $O(n)$. Keep that picture; the whole of Week 4 is built on it.

```
>>> fruits = ['orange', 'apple', 'pear', 'banana', 'kiwi', 'apple',
↪ 'banana']
>>> fruits.count('apple')
2
>>> fruits.index('banana')
3
>>> fruits.index('banana', 4)      # search starting from position 4
6
>>> fruits.reverse()
>>> fruits
['banana', 'apple', 'kiwi', 'banana', 'pear', 'apple', 'orange']
>>> fruits.append('grape')
>>> fruits.sort()
>>> fruits
['apple', 'apple', 'banana', 'banana', 'grape', 'kiwi', 'orange',
↪ 'pear']
>>> fruits.pop()
```

```
'pear'
```

1.2 sort() versus sorted()

`a.sort()` changes the list and returns **None**. `sorted(a)` leaves `a` alone and returns a **new** list. `sorted` works on any iterable; `sort` only on lists.

```
>>> marks = [72, 95, 60]
>>> result = marks.sort()
>>> print(result)          # the classic mistake
None
>>> sorted('python')
['h', 'n', 'o', 'p', 't', 'y']
```

Both take `key=` (a function applied to each item, whose result is compared) and `reverse=True`:

```
>>> words = ['banana', 'Kiwi', 'apple', 'Cherry']
>>> sorted(words)          # capitals sort before lower
↪ case
['Cherry', 'Kiwi', 'apple', 'banana']
>>> sorted(words, key=str.lower)
['apple', 'banana', 'Cherry', 'Kiwi']
>>> sorted(words, key=len, reverse=True)
['banana', 'Cherry', 'apple', 'Kiwi']
```

Sorting by several things — the key returns a *tuple*, and tuples compare item by item (Part 5):

```
>>> students = [('mona', 88), ('ali', 95), ('omar', 88)]
>>> sorted(students, key=lambda s: (-s[1], s[0])) # mark high->low,
↪ then name
[('ali', 95), ('mona', 88), ('omar', 88)]
```

Python's sort is **stable**: items that compare equal keep their original order. You will see why that property matters when you write sorts in Weeks 9–10.

Items must be comparable with each other: `sorted([3, 'a'])` is a `TypeError`.

1.3 Lists as stacks

A **stack** is last-in, first-out (LIFO) — like a pile of plates. With a list, push is `append` and pop is `pop()`, both $O(1)$ at the **end**:

```
>>> stack = [3, 4, 5]
>>> stack.append(6)
>>> stack.append(7)
```

```
>>> stack
[3, 4, 5, 6, 7]
>>> stack.pop()
7
>>> stack.pop()
6
>>> stack
[3, 4, 5]
```

In Week 6 you write a `Stack` class with exactly this behaviour — on top of your own `DynamicArray`, not a list.

1.4 Lists as queues — and why not

A **queue** is first-in, first-out (FIFO). You *can* use a list — append to join, `pop(0)` to leave — but `pop(0)` shifts every remaining item: $O(n)$ per operation. For a real queue use `collections.deque`, which is $O(1)$ at both ends:

```
>>> from collections import deque
>>> queue = deque(['Eric', 'John', 'Michael'])
>>> queue.append('Terry')           # Terry arrives
>>> queue.append('Graham')         # Graham arrives
>>> queue.popleft()                # the first to arrive leaves
'Eric'
>>> queue.popleft()
'John'
>>> queue
deque(['Michael', 'Terry', 'Graham'])
```

In Week 7 you will **measure** the difference between these two, and then build the $O(1)$ version yourself as a ring buffer.

1.5 The `del` statement

`del` removes by **position** or **slice** (where `remove` removes by value):

```
>>> a = [-1, 1, 66.25, 333, 333, 1234.5]
>>> del a[0]
>>> a
[1, 66.25, 333, 333, 1234.5]
>>> del a[2:4]
>>> a
[1, 66.25, 1234.5]
>>> del a[:]
>>> a
[]
```

```
>>> del a          # deletes the NAME; using `a` now is a NameError
```

Part 2 — Comprehensions

2.1 List comprehensions

The loop-and-append pattern is so common that Python has a short form for it:

```
squares = []
for x in range(10):
    squares.append(x ** 2)

squares = [x ** 2 for x in range(10)]          # same result
```

A comprehension is brackets containing an expression, then a for, then any number of further for or if clauses. The clauses read in the same order as the nested loops they replace:

```
>>> [x for x in range(20) if x % 3 == 0]
[0, 3, 6, 9, 12, 15, 18]
>>> [(x, y) for x in [1, 2, 3] for y in [3, 1, 4] if x != y]
[(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

That second one is exactly:

```
combs = []
for x in [1, 2, 3]:
    for y in [3, 1, 4]:
        if x != y:
            combs.append((x, y))
```

More examples:

```
>>> vec = [-4, -2, 0, 2, 4]
>>> [x * 2 for x in vec]
[-8, -4, 0, 4, 8]
>>> [abs(x) for x in vec]
[4, 2, 0, 2, 4]
>>> fresh = [' banana', ' loganberry ', 'passion fruit ']
>>> [w.strip() for w in fresh]
['banana', 'loganberry', 'passion fruit']
>>> [(x, x ** 2) for x in range(4)]          # a tuple needs its
↪ parentheses
[(0, 0), (1, 1), (2, 4), (3, 9)]
>>> vec2 = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
```

```
>>> [num for elem in vec2 for num in elem] # flatten one level
[1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> ['even' if x % 2 == 0 else 'odd' for x in range(4)] # if-else
↪ BEFORE the for
['even', 'odd', 'even', 'odd']
```

Note the two different ifs: ... for x in xs if cond **filters** items; a if cond else b for x in xs **chooses** a value for every item.

2.2 Nested comprehensions

The expression at the front can itself be a comprehension. This 3×4 matrix is a list of three rows:

```
>>> matrix = [
...     [1, 2, 3, 4],
...     [5, 6, 7, 8],
...     [9, 10, 11, 12],
... ]
>>> [[row[i] for row in matrix] for i in range(4)] # transpose
[[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

Read it from the outside in: *for each column index i, build the list of row[i] for every row.* (The built-in way is `list(zip(*matrix))`, which gives tuples.)

This is also the correct way to make the grid that went wrong in Lab 01:

```
>>> board = [[0] * 3 for _ in range(3)] # three SEPARATE inner lists
>>> board[0][0] = 'X'
>>> board
[['X', 0, 0], [0, 0, 0], [0, 0, 0]]
```

`_` is the conventional name for a loop variable you do not use.

When not to. A comprehension should fit on a line or two and be read at a glance. If it needs three conditions and a nested loop, write the loop.

Checkpoint 1. Predict:

- (a) `[c.upper() for c in 'abc' if c != 'b']`
- (b) `[i * j for i in range(1, 3) for j in range(1, 3)]`
- (c) `[[i * j for j in range(1, 3)] for i in range(1, 3)]`
- (d) `[len(w) for w in 'to be or not'.split() if len(w) > 2]`

Part 3 — Tuples

A **tuple** is a sequence of values separated by commas, usually in parentheses. It indexes and slices like a list, but it is **immutable**:

```
>>> t = 12345, 54321, 'hello!'
>>> t[0]
12345
>>> t
(12345, 54321, 'hello!')
>>> u = t, (1, 2, 3)           # tuples can nest
>>> u
((12345, 54321, 'hello!'), (1, 2, 3))
>>> t[0] = 88888
Traceback (most recent call last):
...
TypeError: 'tuple' object does not support item assignment
```

A tuple cannot change, but an object *inside* it can:

```
>>> v = ([1, 2, 3], [3, 2, 1])
>>> v[0].append(4)
>>> v
([1, 2, 3, 4], [3, 2, 1])
```

Empty and one-item tuples have awkward syntax — it is the comma that makes a tuple, not the parentheses:

```
>>> empty = ()
>>> singleton = 'hello',      # note the trailing comma
>>> len(empty), len(singleton)
(0, 1)
>>> type(('hello'))           # just a string in parentheses
<class 'str'>
```

Packing and unpacking. `t = 1, 2, 3` *packs* three values into a tuple. `x, y, z = t` *unpacks* them — the number of names must match. This is what the multiple assignment in Lab 01 was all along, and what happens when a function “returns two values”. A starred name takes the rest:

```
>>> first, *rest = [1, 2, 3, 4]
>>> first, rest
(1, [2, 3, 4])
>>> for name, mark in [('ali', 95), ('mona', 88)]: # unpacking in a
↪     for
...     print(name, mark)
```

List or tuple? Use a **list** for a variable-length collection of similar things (the students in a

class). Use a **tuple** for a fixed group of related values, often of different types (one student's (name, id, mark)). And only immutable things can be dictionary keys or set members, so a tuple can be a key where a list cannot.

Part 4 — Sets

A **set** is an unordered collection with **no duplicates**. Its superpower is membership testing in **O(1) on average** — against O(n) for a list — because it is built on a hash table, which you will build yourself in Week 13.

```
>>> basket = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}
>>> basket                                     # duplicates gone; order not guaranteed
{'orange', 'banana', 'pear', 'apple'}
>>> 'orange' in basket                         # fast
True
>>> 'crabgrass' in basket
False
```

`{}` is an **empty dictionary**, not an empty set. Use `set()`:

```
>>> type({}), type(set())
(<class 'dict'>, <class 'set'>)
```

Set algebra:

```
>>> a = set('abracadabra')
>>> b = set('alacazam')
>>> a                                     # unique letters in a
{'a', 'r', 'b', 'c', 'd'}
>>> a - b                                 # in a but not in b
{'r', 'd', 'b'}
>>> a | b                                 # in a or b or both
{'a', 'c', 'r', 'd', 'b', 'm', 'z', 'l'}
>>> a & b                                 # in both
{'a', 'c'}
>>> a ^ b                                 # in a or b but not both
{'r', 'd', 'b', 'm', 'z', 'l'}
```

(The order the REPL prints a set in may differ on your machine. That is the point: a set has no order.)

Methods: `s.add(x)`, `s.remove(x)` (KeyError if absent), `s.discard(x)` (no error), `s.pop()`, `len(s)`. Set comprehensions work too:

```
>>> {x for x in 'abracadabra' if x not in 'abc'}
{'r', 'd'}
```

Set members must be **hashable** — in practice, immutable: numbers, strings, tuples of those. A list cannot go into a set.

The pattern you will use constantly — “have I seen this before?”:

```
seen = set()
for item in items:
    if item in seen:
        print('duplicate:', item)
    seen.add(item)
```

With a list for seen, that loop is $O(n^2)$. With a set it is $O(n)$. Exercise 1 has a test that only an $O(n)$ solution passes in reasonable time.

Part 5 — Dictionaries

A **dictionary** maps **keys** to **values**. Keys are unique and hashable; values can be anything. Looking up, adding, changing and removing by key are all $O(1)$ on average — dictionaries are hash tables too.

```
>>> tel = {'jack': 4098, 'sape': 4139}
>>> tel['guido'] = 4127          # add
>>> tel
{'jack': 4098, 'sape': 4139, 'guido': 4127}
>>> tel['jack']                  # look up
4098
>>> del tel['sape']              # remove
>>> tel['irv'] = 4127
>>> tel
{'jack': 4098, 'guido': 4127, 'irv': 4127}
>>> list(tel)                    # the keys, in insertion order
['jack', 'guido', 'irv']
>>> sorted(tel)
['guido', 'irv', 'jack']
>>> 'guido' in tel               # `in` checks KEYS
True
>>> 'jack' not in tel
False
>>> tel['nobody']
Traceback (most recent call last):
...
KeyError: 'nobody'
```

Dictionaries **remember insertion order** (guaranteed since Python 3.7). Assigning to an existing key replaces its value and keeps its position.

Other ways to build one:

```
>>> dict([('sape', 4139), ('guido', 4127), ('jack', 4098)])
{'sape': 4139, 'guido': 4127, 'jack': 4098}
>>> dict(sape=4139, guido=4127, jack=4098)      # keys that are simple
↪ names
{'sape': 4139, 'guido': 4127, 'jack': 4098}
>>> {x: x ** 2 for x in (2, 4, 6)}              # dict comprehension
{2: 4, 4: 16, 6: 36}
>>> dict(zip(['a', 'b'], [1, 2]))
{'a': 1, 'b': 2}
```

5.1 Reading safely

```
>>> tel.get('nobody')                          # None instead of KeyError
>>> tel.get('nobody', 0)                       # or a default of your choice
0
>>> tel.keys(), tel.values(), tel.items()
(dict_keys(['jack', 'guido', 'irv']), dict_values([4098, 4127, 4127]),
 dict_items([('jack', 4098), ('guido', 4127), ('irv', 4127)]))
>>> tel.pop('irv')                             # remove and return
4127
```

5.2 The counting pattern

Counting things is the most common dictionary job, and it has three idioms:

```
words = 'the cat and the hat and the bat'.split()

# 1. get with a default
counts = {}
for w in words:
    counts[w] = counts.get(w, 0) + 1

# 2. setdefault – for grouping, when the value is a list
by_letter = {}
for w in words:
    by_letter.setdefault(w[0], []).append(w)

# 3. the library does it for you
from collections import Counter
Counter(words).most_common(2)                # [('the', 3), ('and', 2)]
```

```
>>> counts
{'the': 3, 'cat': 1, 'and': 2, 'hat': 1, 'bat': 1}
```

```
>>> by_letter
{'t': ['the', 'the', 'the'], 'c': ['cat'], 'a': ['and', 'and'], 'h':
↳ ['hat'], 'b': ['bat']}
```

setdefault(key, default) returns the value for key, first storing default there if the key was missing. collections.defaultdict(list) does the same job automatically. Know all three; in the exercises, write the first two yourself.

Part 6 — Looping techniques

```
>>> knights = {'gallahad': 'the pure', 'robin': 'the brave'}
>>> for k, v in knights.items():           # key and value together
...     print(k, v)
...
gallahad the pure
robin the brave
>>> for i, v in enumerate(['tic', 'tac', 'toe']):
...     print(i, v)
...
0 tic
1 tac
2 toe
>>> for i in reversed(range(1, 10, 2)):    # backwards
...     print(i, end=' ')
...
9 7 5 3 1
>>> basket = ['apple', 'orange', 'apple', 'pear', 'orange', 'banana']
>>> for f in sorted(set(basket)):          # unique, in order
...     print(f, end=' ')
...
apple banana orange pear
```

Changing a dictionary while looping over it is an error (RuntimeError: dictionary changed size during iteration). Loop over list(d) — a copy of the keys — or build a new dictionary:

```
>>> raw = [56.2, float('nan'), 51.7, 55.3, 52.5, float('nan'), 47.8]
>>> import math
>>> [value for value in raw if not math.isnan(value)]
[56.2, 51.7, 55.3, 52.5, 47.8]
```

6.1 Comparing sequences

Sequences of the same type compare **lexicographically** — like words in a dictionary: the first items are compared; if equal, the second; and so on. The shorter sequence is smaller if it runs out first:

```
(1, 2, 3) < (1, 2, 4) # True
[1, 2, 3] < [1, 2, 4] # True
'ABC' < 'C' < 'Pascal' < 'Python' # True
(1, 2, 3, 4) < (1, 2, 4) # True
(1, 2) < (1, 2, -1) # True
(1, 2, 3) == (1.0, 2.0, 3.0) # True
(1, 2, ('aa', 'ab')) < (1, 2, ('abc', 'a'), 4) # True
```

That is why a sort key that returns a tuple sorts by several fields at once. Strings compare by character code, so all capitals come before all lower-case letters: 'Z' < 'a' is True.

6.2 Which structure?

Need	Use	Why
Ordered items, add at the end, index by position	list	O(1) index and append
A fixed record of a few values	tuple	immutable, can be a key
"Is x in here?", no duplicates	set	O(1) average membership
Look up a value by a key	dict	O(1) average lookup
Add and remove at both ends	collections.deque	O(1) at both ends
Count things	dict or Counter	

This table is the course in miniature: the same data, a different structure, a different cost. From Week 4 you stop *using* these and start *building* them.

Checkpoint 2. Predict:

```
d = {'a': 1, 'b': 2}
d['c'] = d.get('a', 0) + d.get('z', 10)
d['a'] = 5
print(list(d.items()))
print(len({1, 1.0, True, 'one'}))
print(sorted({'b': 1, 'a': 2}, key=lambda k: -len(k)))
```

Part 7 — Classes

7.1 Why classes, in this course

Lecture 01 separated the **ADT** — the contract: which operations exist and what they cost — from the **data structure** that keeps the contract. A class is how Python lets you write both in one place: the methods are the contract; the attributes are the structure; and code outside uses only the methods.

`dsa/stack.py` is a class. So are the dynamic array, the linked list, the queues, the tree, the heap, the hash map and the graph. Every one of them has the shape you learn in this part.

7.2 A first class

```
class Dog:
    """A dog with a name and a list of tricks."""

    kind = 'canine'                # CLASS variable: shared by every
    ↪ Dog

    def __init__(self, name):
        self.name = name          # INSTANCE variables: one per Dog
        self.tricks = []

    def add_trick(self, trick):
        self.tricks.append(trick)

    def describe(self):
        return f'{self.name} knows {len(self.tricks)} tricks'

>>> d = Dog('Fido')              # create an instance: calls __init__
>>> e = Dog('Buddy')
>>> d.add_trick('roll over')
>>> e.add_trick('play dead')
>>> d.tricks
['roll over']
>>> d.describe()
'Fido knows 1 tricks'
>>> d.kind, e.kind                # shared
('canine', 'canine')
```

What each part means:

- `class Dog`: creates a new type. Its name is UpperCamelCase.
- `__init__` is the *initialiser*. Calling `Dog('Fido')` creates an empty object and immediately calls `__init__(that_object, 'Fido')`. Its job is to set up the object's attributes. It returns nothing.
- `self` is the object the method was called on. `d.add_trick('roll over')` is exactly `Dog.add_trick(d, 'roll over')`. The name `self` is only a convention — but

never use another.

- **Attributes** are created by assigning to `self.something`. Each object has its own.
- A **method** is a function defined in the class. It always takes `self` first.

The shared-list trap. A mutable **class** variable is shared by every instance — the same mistake as a mutable default argument:

```
class BadDog:
    tricks = []                # ONE list for ALL dogs

    def __init__(self, name):
        self.name = name

    def add_trick(self, trick):
        self.tricks.append(trick)

>>> d, e = BadDog('Fido'), BadDog('Buddy')
>>> d.add_trick('roll over')
>>> e.tricks                    # Buddy "knows" Fido's trick
['roll over']
```

Rule: **create per-object data in `__init__`**. Class variables are for constants.

7.3 Encapsulation by convention

Python has no truly private attributes. By convention, a name starting with one underscore — `self._items` — means *internal: do not touch from outside*. The `dsa/` skeletons use this everywhere:

```
class Stack:
    def __init__(self):
        self._items = []        # the data structure: private

    def push(self, value):       # the ADT: public
        self._items.append(value)
```

Code outside the class should call `push`, never `stack._items.append`. Then you can change how the stack is stored — say, to a linked list — without breaking anyone. That freedom is the whole point of separating ADT from implementation.

(This example uses a Python list only to show the idea. Inside `dsa/` a list is **not allowed** as storage: from Lecture 02, every structure keeps its data in the course `Array`, in nodes, or in a structure you built — and `dsa/stack.py` is built on your own `DynamicArray`.)

7.4 Special methods — making your class feel built in

Methods whose names start and end with double underscores (“dunder” methods) are called by Python itself when you use built-in syntax on your object:

You write	Python calls
<code>len(obj)</code>	<code>obj.__len__()</code>
<code>x in obj</code>	<code>obj.__contains__(x)</code>
<code>for x in obj</code>	<code>obj.__iter__()</code>
<code>obj[i]</code>	<code>obj.__getitem__(i)</code>
<code>obj[i] = v</code>	<code>obj.__setitem__(i, v)</code>
<code>obj == other</code>	<code>obj.__eq__(other)</code>
<code>obj < other</code>	<code>obj.__lt__(other)</code>
<code>obj + other</code>	<code>obj.__add__(other)</code>
<code>repr(obj)</code> , the REPL	<code>obj.__repr__()</code>
<code>str(obj)</code> , <code>print(obj)</code>	<code>obj.__str__()</code> (falls back to <code>__repr__</code>)
<code>if obj:</code>	<code>obj.__bool__()</code> , else <code>obj.__len__()</code> <code>!= 0</code>

A complete worked example — a fraction that always stays in lowest terms:

```
import math

class Fraction:
    """An exact rational number numerator/denominator, in lowest
    ↪ terms."""

    def __init__(self, numerator, denominator=1):
        if denominator == 0:
            raise ZeroDivisionError('denominator must not be zero')
        if denominator < 0:
            # keep the sign on top
            numerator, denominator = -numerator, -denominator
        g = math.gcd(numerator, denominator)
        self._num = numerator // g
        self._den = denominator // g

    def __repr__(self):
        return f'Fraction({self._num}, {self._den})'

    def __str__(self):
        return f'{self._num}/{self._den}'

    def __eq__(self, other):
        if not isinstance(other, Fraction):
            return NotImplemented
        return self._num == other._num and self._den == other._den

    def __lt__(self, other):
        return self._num * other._den < other._num * self._den
```

```

def __add__(self, other):
    return Fraction(self._num * other._den + other._num * self._den,
                    self._den * other._den)

def __mul__(self, other):
    return Fraction(self._num * other._num, self._den * other._den)

```

```

>>> half = Fraction(1, 2)
>>> third = Fraction(2, 6)           # stored as 1/3
>>> third
Fraction(1, 3)
>>> print(half + third)
5/6
>>> half * third
Fraction(1, 6)
>>> Fraction(3, 6) == half
True
>>> sorted([half, third, Fraction(3, 4)]) # works because of __lt__
[Fraction(1, 3), Fraction(1, 2), Fraction(3, 4)]
>>> Fraction(1, 10) + Fraction(2, 10) == Fraction(3, 10) # exact,
↪ unlike 0.1 + 0.2
True

```

Points to notice:

- **__repr__ should be unambiguous**, ideally code that recreates the object. It is what the REPL, the debugger and a failing pytest message show you. A good `__repr__` saves hours. `__str__` is the friendly version for users.
- **__eq__ returns NotImplemented** (a special value, not the exception) when it does not know how to compare with the other type. Python then tries the other side, and finally falls back to `is`.
- **Validate in __init__**: an object that is created invalid stays invalid.
- Without `__eq__`, `==` means `is` — two separately created equal fractions would be “different”.
- Python’s standard library has a full version of this class: `from fractions import Fraction`. Building it yourself is the point.

7.5 Inheritance, briefly

A class can **extend** another, inheriting its methods and overriding some:

```

class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):

```

```

        return '...'

    def introduce(self):
        return f'I am {self.name}. {self.speak()}'

class Cat(Animal):
    # Cat is-an Animal
    def speak(self):
        # override
        return 'Meow'

class Lion(Cat):
    def __init__(self, name, pride):
        super().__init__(name)
        self.pride = pride
        # run the parent's __init__ first

    def speak(self):
        return super().speak().upper() + '!'

```

```

>>> Lion('Simba', 'Pride Rock').introduce()
'I am Simba. MEOW!'
>>> isinstance(Lion('x', 'y'), Animal)
True
>>> issubclass(Cat, Lion)
False

```

introduce is defined once, in Animal, but calls whichever speak belongs to the actual object. In this course inheritance is used mainly for one thing — **your own exception types**:

```

class EmptyStackError(Exception):
    """Raised when popping or peeking an empty stack."""

def pop(stack):
    if not stack:
        raise EmptyStackError('pop from an empty stack')
    return stack.pop()

```

except EmptyStackError: then catches exactly that problem and nothing else. Since it inherits from Exception, except Exception: would also catch it.

Checkpoint 3. What is printed?

```
class Counter:
    total = 0
    def __init__(self):
        self.count = 0
    def tick(self):
        self.count += 1
        Counter.total += 1
    def __len__(self):
        return self.count

a, b = Counter(), Counter()
a.tick(); a.tick(); b.tick()
print(len(a), len(b), Counter.total, a.total, bool(Counter()))
```

Part 8 — Iterators and generators

8.1 What for really does

for x in thing: works on lists, strings, dictionaries, files, ranges. How? It asks thing for an **iterator** with `iter()`, then calls `next()` on it until the iterator raises `StopIteration`:

```
>>> it = iter('abc')
>>> next(it)
'a'
>>> next(it)
'b'
>>> next(it)
'c'
>>> next(it)
Traceback (most recent call last):
...
StopIteration
```

for catches that `StopIteration` for you and ends the loop. So to make *your* class work in a for loop, `list()`, `sorted()`, `sum()` and in, you give it an `__iter__` method that returns an iterator. The easy way to write one is a generator.

8.2 Generators

A **generator** is a function that uses `yield` instead of `return`. Calling it does not run it — it returns a generator object. Each `next()` runs the body until the next `yield`, hands that value out, and **pauses** there, keeping all its local variables until it is resumed:

```
def reverse(data):
    for index in range(len(data) - 1, -1, -1):
```

```
yield data[index]
```

```
>>> for char in reverse('golf'):
...     print(char, end=' ')
...
f l o g
>>> g = reverse('ab')
>>> g
<generator object reverse at 0x...>
>>> next(g), next(g)
('b', 'a')
```

When the function body ends, the generator raises `StopIteration` for you.

Why generators matter:

- **Laziness.** Values are produced one at a time, when asked for. A generator over a billion items uses almost no memory — like `range`.
- **Easy iterators.** `__iter__` written as a generator is three lines:

```
class Countdown:
    def __init__(self, start):
        self.start = start

    def __iter__(self):
        n = self.start
        while n > 0:
            yield n
            n -= 1

>>> list(Countdown(3))
[3, 2, 1]
>>> sum(Countdown(100))
5050
```

- **Pausing an algorithm.** The course's animation tools use this. Write a sort that yields the list after every comparison, and `viz.animate` can play it frame by frame (see the repository `README.md`). You will do exactly that in Week 9.

8.3 Generator expressions

A comprehension in **parentheses** instead of brackets is a generator expression: it produces values lazily instead of building a list. Use it when the values are consumed straight away by a function:

```
>>> sum(i * i for i in range(10))           # no list is ever built
285
```

```
>>> xvec, yvec = [10, 20, 30], [7, 5, 3]
>>> sum(x * y for x, y in zip(xvec, yvec)) # dot product
260
>>> max(len(w) for w in ['to', 'be', 'or', 'not'])
3
>>> any(n % 7 == 0 for n in [3, 14, 5]) # stops at the first True
True
```

Part 9 — Exercises

All in `labs/lab03.py`. Run `pytest tests/test_lab03.py -v`.

#	Name	Practises
1	<code>unique_in_order(values)</code>	a set for “seen”, a list for order — $O(n)$
2	<code>word_frequencies(text)</code>	<code>split</code> , <code>strip</code> , the counting pattern
3	<code>top_k(frequencies, k)</code>	<code>items()</code> , sorted with a tuple key, slicing
4	<code>transpose(matrix)</code>	nested list comprehension
5	<code>invert(mapping)</code>	grouping with <code>setdefault</code> , then sorting
6	<code>common_elements(first, second)</code>	set intersection
7	<code>group_by_length(words)</code>	grouping into lists, order preserved
8	<code>Bag</code>	a class with a dict inside and ten methods
9	<code>countdown(n)</code>	a generator
10	<code>chunks(values, size)</code>	a generator with slicing

About Bag. A *bag* (or *multiset*) is an ADT: like a set, but it counts. `Bag(['a', 'b', 'a'])` holds two 'a's and one 'b'. The docstring is the full contract. Design notes:

- Keep a dictionary `self._counts` mapping each item to how many there are. Do not keep a list of all items — `count` and `in` would be $O(n)$.
- `__len__` must be $O(1)$. Recounting all the values every time is $O(k)$; keep a running total in another attribute and update it in `add` and `remove`.
- When the last copy of an item is removed, delete its key — otherwise `in`, `distinct()` and `==` will all be wrong. The test `test_bag_equality_ignores_order` checks this.
- `__iter__` yields each item as many times as it occurs: a generator with two nested loops.

- `__eq__`: two bags are equal when their count dictionaries are equal. Return `NotImplemented` if other is not a `Bag`.
- `__repr__`: `sorted(self)` uses your `__iter__`; then an f-string with `!r` gives the quotes: `f'Bag({sorted(self)!r})'`.
- `__init__(self, items=())` — a tuple is immutable, so this default is safe. Initialise the attributes, then call `self.add` for each item.

Other hints:

- **2.** `word.strip('.,;:!?')` removes those characters from both ends only.
- **3.** `sorted(frequencies.items(), key=lambda pair: (-pair[1], pair[0]))` gives the order; then slice. Negating the count sorts high to low while the word still sorts A to Z.
- **4.** The empty matrix needs its own check: `matrix[0]` does not exist.
- **10.** `range(0, len(values), size)` gives the start of each chunk. Where does the `ValueError` get raised — when `chunks` is called, or when the first value is asked for? Read the test.

Part 10 — Bridge to this week's lecture: recursion

Lecture 03 is recursion, and the course exercises for it are in `dsa/recursion.py`, graded by `tests/test_recursion.py`. Everything here uses only what you already know — functions, `if`, and the call stack from Lab 02.

10.1 A function that calls itself

```
def countdown_rec(n):
    if n == 0:                                # base case: stop
        print('lift-off')
        return
    print(n)
    countdown_rec(n - 1)                      # recursive case: a SMALLER problem
```

```
>>> countdown_rec(3)
3
2
1
lift-off
```

Every recursive function needs both parts. The **base case** answers a problem small enough to answer directly. The **recursive case** makes the problem smaller and trusts the function to solve the smaller version.

10.2 Recursion that returns a value

The sum of a list: *the sum of an empty list is 0; otherwise it is the first item plus the sum of the rest.*

```
def total(values):
    if not values:
        return 0
    return values[0] + total(values[1:])
```

Trace `total([4, 7, 1])`. Each call waits, on the **call stack**, for the one it made:

```
total([4, 7, 1])
= 4 + total([7, 1])
    = 7 + total([1])
        = 1 + total([])
            = 0                                <- base case: the stack stops growing
        = 1 + 0 = 1                            <- and unwinds, top frame first
    = 7 + 1 = 8
= 4 + 8 = 12
```

Four frames were on the stack at the deepest point. For a list of length n , $n + 1$ frames: $O(n)$ stack space. (And, since `values[1:]` copies the list each time, $O(n^2)$ time — a better version passes an index instead of slicing. Noticing that is exactly the Week 2 habit.)

10.3 When it goes wrong

Forget the base case, or make a recursive case that does not get smaller, and Python stops you:

```
def forever(n):
    return forever(n - 1)

>>> forever(5)
Traceback (most recent call last):
...
RecursionError: maximum recursion depth exceeded
>>> import sys
>>> sys.getrecursionlimit()
1000
```

Raising the limit is almost never the fix. A wrong base case is.

10.4 Watch it in the debugger

Put `total` in a file, set a breakpoint on its first line (click in the gutter in VS Code), press **F5**, and step with **F11** (step into). Open the **Call Stack** panel: you will see one `total` frame per call, each with its own values. That picture is Lecture 03.

Then open `dsa/recursion.py` and start with `factorial`, `sum_digits` and `gcd`:

```
pytest tests/test_recursion.py -v -k "factorial or sum_digits or gcd"
```

Part 11 — Take-home practice (not graded)

1. Write `is_anagram(a, b)` three ways: sorting, a dictionary of counts, and your Bag. Which is $O(n \log n)$ and which $O(n)$?
2. Write a class `Matrix` with `__init__(rows)`, `__repr__`, `__eq__`, `__add__`, `__mul__` (matrix product) and a `transpose()` method. Raise `ValueError` for incompatible shapes.
3. Write a generator `fib_gen()` that yields Fibonacci numbers **forever**. Use it with `zip(range(10), fib_gen())` to print the first ten. Why is there no infinite loop?
4. Give your Bag an `__add__` that returns a new bag with the counts of both, and a `most_common(k)` method. Compare the results with `collections.Counter`.
5. Write `total(values, i=0)` from Part 10.2 again, without slicing. Time both versions for lists of 500 and 900 items. What did the slice cost?

Summary

Idea	The one line to keep
List costs	End is cheap (<code>append</code> , <code>pop()</code>), front is $O(n)$ (<code>insert(0)</code> , <code>pop(0)</code>), <code>in</code> is $O(n)$.
sort vs sorted	<code>sort()</code> changes the list and returns <code>None</code> ; <code>sorted()</code> returns a new list.
Stack / queue	A list is a fine stack and a bad queue. <code>deque</code> is the queue.
Comprehensions	<code>[expr for x in xs if cond]</code> — a loop that builds a list, in one line.
Tuples	Immutable; a comma makes one; unpack with <code>a, b = t</code> .
Sets	No duplicates; $O(1)$ average in. <code>set()</code> is empty, <code>{}</code> is a dict.
Dicts	Key → value, $O(1)$ average; insertion-ordered; <code>get</code> , <code>setdefault</code> , <code>items</code> .
Classes	Data in <code>__init__</code> via <code>self</code> ; methods take <code>self</code> ; never a mutable class variable.
Special methods	<code>__len__</code> , <code>__iter__</code> , <code>__eq__</code> , <code>__repr__</code> make your class feel built in.

Idea	The one line to keep
Generators	<code>yield</code> pauses the function; lazy; the easiest way to write <code>__iter__</code> .
Recursion	A base case, and a recursive case that gets smaller.

Lab 04 — Lab 04 — Building a Dynamic Array

How to use this lab. From this week the lab is where you **build** the week's structure. For every part: read the Lecture 04 section it names, **draw before you code** (on paper, or with `viz.draw`), predict at each **Checkpoint**, then write one method and run the tests named for it before moving on. Build in the order given — each part stands on the one before. The worked solution is in `solutions/dsa/dynamic_array.py`, for after you have tried.

Duration	One 2-hour lab session, plus about 3 hours at home
You will write	<code>dsa/dynamic_array.py</code> — <code>_resize</code> , <code>append</code> , <code>__getitem__</code> , <code>__setitem__</code> , <code>insert_at</code> , <code>pop</code>
Graded by	<code>tests/test_dynamic_array.py</code> (19 tests)
Connects to	Lecture 04 — Dynamic Arrays; used again by Stack (Week 6) and SlowQueue (Week 7)

What you will be able to do

1. explain the difference between a dynamic array's **size** and its **capacity**, and draw both;
 2. grow a fixed-size Array by the three steps *allocate*, *copy*, *switch*;
 3. write `append` so that it is amortised $O(1)$, and say why its worst case is still $O(n)$;
 4. translate negative indices and check bounds against the size, not the capacity;
 5. insert and remove in the middle by shifting — in the right direction;
 6. measure, with `resize_count` and with a timer, why growing by a **factor** beats growing by a **constant**;
 7. explain why `pop(0)` is $O(n)$, and what that will cost a queue in Week 7.
-

Part 0 — Before you start

0.1 Check your environment

From the DSA27 folder, with the virtual environment active:

```
pytest -m "not challenge" -q          # the environment check: must pass
pytest tests/test_array.py -q        # the course Array you build on:
↪ given, must pass
```

```
pytest tests/test_dynamic_array.py -q # your exercise: 18 failed, 1
↳ passed
```

The one test that passes before you write anything is `test_empty`: an empty `DynamicArray()` never calls a method of yours. Every other test calls one of your methods — most of them `append`, directly or through the constructor — and so hits `NotImplementedError`.

0.2 You have written half of this already

Week 2's `dsa/array_ops.py` had `insert_at(arr, size, index, value)` and `remove_at(arr, size, index)`: a **partly filled** `Array` plus a separate `size` saying how many slots are in use. A dynamic array is exactly that pair — the `Array` and its `size` — kept together inside one object, plus one new trick: when the `Array` is full, replace it with a bigger one.

If your Week 2 functions pass,

```
pytest tests/test_array_ops.py -k "insert_at or remove_at" -q # 9
↳ tests
```

then the shifting loops of Parts 3 and 4 below are already in your hands. Open `dsa/array_ops.py` next to `dsa/dynamic_array.py`. (Lecture 04, “The contract with costs”: `insert_at` and `pop(i)` are “Lecture 02’s `insert_at` and `remove_at` ... moved inside the class”.)

0.3 Read the skeleton

Open `dsa/dynamic_array.py`. The docstrings are the contract; the test file is the full specification.

Given — do not change	Yours
<code>make_block(capacity)</code> — returns <code>Array(capacity)</code>	<code>_resize(capacity)</code>
<code>__init__(values=(), growth=2)</code>	<code>append(value)</code>
<code>__len__</code> — returns <code>self._size</code>	<code>__getitem__(index)</code> , <code>__setitem__(index, value)</code>
<code>__iter__</code> — yields slots <code>0 .. size-1</code>	<code>insert_at(index, value)</code>
<code>__repr__</code> — <code>DynamicArray([...])</code>	<code>pop(index=-1)</code>

Read `__init__` closely — it tells you the four attributes you work with:

Attribute	Meaning
<code>self._block</code>	the <code>Array</code> underneath; <code>len(self._block)</code> is the capacity

Attribute	Meaning
<code>self._capacity</code>	how many slots <code>_block</code> has — keep it equal to <code>len(self._block)</code>
<code>self._size</code>	how many slots, from index 0, hold data — what <code>len()</code> returns
<code>self.growth</code>	the growth factor , 2 by default
<code>self.resize_count</code>	how many times <code>_resize</code> has run

Notice two things. First, a new array starts with **capacity 1**, not 0 — so the very first `append` always has room. Second, the constructor builds the array by calling `self.append` for each value, so until `append` works, even `DynamicArray([1, 2, 3])` fails.

0.4 The storage rule, this week

A `dsa/` structure stores its data only in the course `Array`, in node objects, or in another `dsa/` structure (Lecture 02). Here that means: **`_block` is an `Array`, and there is no Python list anywhere in the class** — not as the storage, not as scratch space inside `_resize`, not “just for a moment” in `insert_at`. You move every element yourself, one index at a time. That is how you see what each operation costs.

0.5 A picture to draw with

Throughout this lab you will want to see capacity and size at the same time. Put this helper in the scratch cell of notebooks/04-dynamic-array.ipynb (or a scratch file — **not** in `dsa/`):

```
from dsa.dynamic_array import DynamicArray
from viz.draw import draw_array

def show(arr):
    """Draw every slot of the block; the used ones (0 .. size-1) are
    ↪ marked."""
    slots = list(arr._block)          # peeking inside: for drawing only
    return draw_array(slots, done=range(len(arr)),
                      title=f"size {len(arr)}, capacity {arr._capacity}")
```

Reading `_block` from outside the class breaks encapsulation, which is fine in a notebook for learning and never fine in real code. The unused slots draw as `None`: they exist in the `Array`, but hold nothing the user put there.

Part 1 — Growing: `_resize` and `append`

1.1 The idea

Lecture 04, “Allocate, copy, switch”. A full Array cannot grow in place. So `_resize(capacity)`:

1. **allocates** a new block with `make_block(capacity)`;
2. **copies** the size elements across, index by index — $O(\text{size})$;
3. **switches** `self._block` to the new block, sets `self._capacity`, and adds one to `self.resize_count`.

The old block is no longer referred to by anything, so Python’s garbage collector reclaims it.

The lecture prints `_resize` in full because it is mechanical. The real exercise is *append*: *when* to call it, and *with what capacity*. The docstring tells you: when full, call `self._resize(self._capacity * self.growth)`.

1.2 Draw it

On paper, starting from `DynamicArray()` (capacity 1, size 0), draw the block after each of five appends of 'a' ... 'e'. For each resize, draw the old block, the new block, and an arrow for each copied element. Count the arrows.

Then check your drawing once Part 1 passes:

```
arr = DynamicArray()
for ch in 'abcde':
    arr.append(ch)
    show(arr)
```

Checkpoint 1. After the five appends above, give `len(arr)`, `arr._capacity`, `arr.resize_count`, `list(arr._block)`, and the total number of elements copied by all the resizes.

1.3 Write it

`_resize(self, capacity)` — the three steps of 1.1. Copy `range(self._size)`, not the capacity: only the used slots hold data.

`append(self, value)`, in words:

1. if the array is full (size equals capacity), resize to `capacity * growth`;
2. write `value` into slot `size` — the first free slot;
3. add one to `size`.

The order matters: **resize before you write**. Slot `size` of a full block does not exist.

1.4 Test it

```
pytest tests/test_dynamic_array.py -v -k "constructor or grows_beyond
↳ or doubling"
```

This selects three tests: `test_constructor_takes_an_iterable`, `test_grows_beyond_initial_capacity` (1,000 appends, every value survives every resize) and `test_doubling_keeps_resizes_logarithmic` (after 1,000 appends, `resize_count` must be more than 0 and fewer than 20). `test_append_and_index` also appends, but it reads with `arr[0]` — it waits for Part 2. The filter says `grows_beyond`, not just `grows`, on purpose: `-k grows` would also pick up `test_insert_at_grows_when_full`, which waits for Part 3.

1.5 When it fails

Writing before checking for room. Every test that appends twice fails with

```
IndexError: Array index 1 out of range for length 1 (valid: 0..0; no
↳ negative indices)
```

The message comes from the course `Array`, not from your class: the second append wrote to slot 1 of a one-slot block, *then* noticed it was full. Move the “if full, resize” to the top of append.

The same message appears if `_resize` builds and fills the new block but **forgets to switch** `self._block` to it — the writes still go to the old, one-slot block. Read `_resize` line by line: allocate, copy, switch.

Forgetting to update `self._capacity` in `_resize`.

```
IndexError: Array index 2 out of range for length 2 (valid: 0..1; no
↳ negative indices)
```

The first resize makes a block of 2, but `_capacity` still says 1. The third append compares size 2 with capacity 1, sees “not full”, and writes slot 2 of a two-slot block.

Growing by a constant. `self._capacity + self.growth` instead of `self._capacity * self.growth` passes every other test and fails one, `test_doubling_keeps_resizes_logarithmic`.

```
AssertionError: 500 reallocations for 1000 appends – are you growing
↳ by a constant instead of doubling?
assert 500 < 20
```

With `+ 2` the capacity goes 1, 3, 5, 7, ..., so there is a resize every second append. The array still *works* — every value is kept — which is exactly why the test counts resizes rather than checking values. Part 5 measures what this costs.

Forgetting `self.resize_count += 1`. The array works, so only `test_doubling_keeps_resizes_logarithmic` fails:

```
AssertionError: resize_count never went up – increment it in _resize
assert 0 > 0
```

The counter is not decoration: Part 5 and the notebook read it to show the doubling as a staircase. Look at the last line of `_resize`.

Part 2 — Reading and writing: `__getitem__` and `__setitem__`

2.1 The idea

Lecture 04, “Reading and writing are still $O(1)$ ”. Two rules, both about the difference between size and capacity:

- the valid indices are **0 to size – 1**. The slots from size to capacity – 1 exist in the Array, but reading one must raise `IndexError`, exactly as with a Python list;
- a negative index counts from the **last used** slot: add size, not capacity. The course Array has no negative indices, so your class provides them — in $O(1)$, by one addition.

The lecture prints `__getitem__` in full; it is five lines. Read it there, then write `__setitem__` yourself — the same translation and the same bounds check, then a write instead of a read.

2.2 Draw it

Draw a block of capacity 8 holding 10, 20, 30, 40, 50. Under each slot write its positive index; under the five used slots also write the negative index a user would type (–5 ... –1). Mark the slots a user may never reach.

```
a = DynamicArray([10, 20, 30, 40, 50])
show(a)
```

Checkpoint 2. For `a = DynamicArray([10, 20, 30, 40, 50])`, what does each line give — a value or an `IndexError`?

- (a) `a[-1]` (b) `a[-5]` (c) `a[5]` (d) `a[-6]`
 (e) `a._block[5]` (f) `a._block[-1]`

2.3 Write it

```
__setitem__(self, index, value):
```

1. if index is negative, add `self._size`;
2. if the result is not in `0 .. size – 1`, raise `IndexError`;
3. write value into `self._block[index]`.

Do not skip step 2 because “the Array will raise anyway”. It will not: slot 6 of an eight-slot block is perfectly valid to the Array, so `a[6] = 'x'` would silently write into a slot the user

cannotsee. `test_setitem_out_of_range_raises` checks exactly this: `arr[3] = 99` on `DynamicArray([1, 2, 3])` must raise.

2.4 Test it

```
pytest tests/test_dynamic_array.py -v -k "index or setitem"
```

Seven tests: `test_append_and_index`, `test_negative_indexing`, `test_index_out_of_range_raises` three times (for indices 3, -4 and 99 on a three-element array), `test_setitem` and `test_setitem_out_of_range_raises`.

2.5 When it fails

Checking against the capacity. `if not 0 <= index < self._capacity:`

```
FAILED tests/test_dynamic_array.py::test_index_out_of_range_raises[3]
E       Failed: DID NOT RAISE IndexError
```

`DynamicArray([1, 2, 3])` has capacity 4, so slot 3 exists and holds `None`; your method returned `None` instead of raising. Compare with `self._size`.

No negative translation. `test_negative_indexing` fails with

```
IndexError: Array index -1 out of range for length 4 (valid: 0..3; no
↪ negative indices)
```

The -1 went straight through to the course `Array`, which — like C — has no negative indices. Translate first.

No bounds check in `__setitem__`.

```
FAILED tests/test_dynamic_array.py::test_setitem_out_of_range_raises
E       Failed: DID NOT RAISE IndexError
```

The write went into slot 3 of the four-slot block: the `Array` saw nothing wrong, and `list(arr)` still shows `[1, 2, 3]` because slot 3 is past the size. Use the same check as `__getitem__`.

Adding the capacity. `index += self._capacity` fails two tests: `test_negative_indexing` with `IndexError: 3` (for `[1, 2, 3]`, -1 became 3, past the size) and `test_index_out_of_range_raises` with `DID NOT RAISE` (-4 became 0, a valid slot). -1 means the last **used** slot.

Part 3 — Inserting in the middle: `insert_at`

3.1 The idea

Lecture 04, “The contract with costs”. To insert at index, every element from index to size – 1 moves one place right, then the new value goes into the gap. That is Week 2’s `insert_at` with two differences:

- Week 2 raised `OverflowError` when the array was full. Here a full array simply **re-sizes first** — that is the point of a dynamic array;
- Week 2 returned the new size. Here the size is an attribute: update `self._size`.

The index rule has not changed: $0 \leq \text{index} < \text{size}$, or `IndexError`.

Cost: $O(\text{size} - \text{index})$ shifts, so $O(n)$ at the front and $O(1)$ at the end.

3.2 Draw it

Draw `[1, 2, 3, 4]` (capacity 4, full) and `insert_at(0, 0)`. First the resize (capacity 8), then the shifts — draw each one as an arrow, numbered in the order you do them. Which element must move first?

Checkpoint 3. A student shifts from the **front**:

```
for i in range(index, self._size):
    self._block[i + 1] = self._block[i]
```

- What is `list(arr)` after `arr = DynamicArray([1, 2, 3])` and `arr.insert_at(1, 99)`?
- And after `arr = DynamicArray([1, 2, 3, 4])` and `arr.insert_at(0, 0)`?

3.3 Write it

- Check the index **first**, before anything changes: raise `IndexError` unless $0 \leq \text{index} < \text{len}(\text{self})$. The docstring says so. `index == len(self)` is allowed and appends; a negative index is **not** translated here. That is the rule of Week 2’s `insert_at` and of the linked list in Week 5. (Python’s own `list.insert` quietly clamps a bad index instead. This course’s structures do not.)
- If the array is full, resize (by the growth factor, as in `append`).
- Shift slots `index .. size - 1` one place right, **starting from the end**: slot `size` gets slot `size - 1`, then slot `size - 1` gets slot `size - 2`, and so on down to `index + 1`. `range(self._size, index, -1)` walks exactly those destinations.
- Write value into slot `index`; add one to `size`.

3.4 Test it

```
pytest tests/test_dynamic_array.py -v -k insert
```

Five tests:

- `test_insert_at_shifts_right`: `[1, 2, 3]` with 99 inserted at 1;

- `test_insert_at_out_of_range_raises`, three times: indices -1, 6 and 7 on `DynamicArray([1, 2, 3, 4, 5])` must raise `IndexError`. That array has 5 values in 8 slots, so slots 6 and 7 **exist** in the block: only your own check can reject them;
- `test_insert_at_grows_when_full`: 100 inserts at the front of an empty array. It resizes seven times on the way, and the result must be `[99, 98, ..., 0]`.

Then try the edges by hand, watching with `show(arr)`:

```
arr = DynamicArray([1, 2, 3, 4])      # full: capacity 4
arr.insert_at(0, 0)                  # must resize first
arr.insert_at(len(arr), 5)           # at the end: no shifts
print(arr, arr._capacity)            # DynamicArray([0, 1, 2, 3, 4,
↪ 5]) 8
```

3.5 When it fails

Shifting from the front — Checkpoint 3:

```
assert [1, 99, 2, 2] == [1, 99, 2, 3]
```

The 3 was overwritten by the 2 before it was moved. Walk from the end. `test_insert_at_grows_when_full` fails too, with `assert [99, 98, 98, 98, 98, 98, ...] == [99, 98, 97, 96, 95, 94, ...]`.

Forgetting to update the size.

```
assert [1, 99, 2] == [1, 99, 2, 3]
```

Everything moved correctly, but size still says 3, so `__iter__` stops one early.

Forgetting to resize when full. `test_insert_at_shifts_right` still passes: `DynamicArray([1, 2, 3])` has capacity 4, so it is not full. But `test_insert_at_grows_when_full` fails on its second insert:

```
IndexError: Array index 1 out of range for length 1 (valid: 0..0; no
↪ negative indices)
```

The first shift wrote slot 1 of the one-slot block the array started with. Resize first, exactly as in `append`.

Clamping the index like `list.insert`. All three out-of-range tests fail:

```
FAILED
↪ tests/test_dynamic_array.py::test_insert_at_out_of_range_raises[-
↪ 1]
FAILED
↪ tests/test_dynamic_array.py::test_insert_at_out_of_range_raises[6]
FAILED
↪ tests/test_dynamic_array.py::test_insert_at_out_of_range_raises[7]
E      Failed: DID NOT RAISE IndexError
```

No index check at all. Two of them fail:

FAILED

```
↪ tests/test_dynamic_array.py::test_insert_at_out_of_range_raises[6]
```

FAILED

```
↪ tests/test_dynamic_array.py::test_insert_at_out_of_range_raises[7]
```

E Failed: DID NOT RAISE IndexError

With spare room the course Array sees nothing wrong: `insert_at(7, 99)` returns quietly, the array still reads `[1, 2, 3, 4, 5, None]`, and the 99 is hidden in slot 7. Index `-1` does raise — but only from the Array, in the middle of the shifting loop, *after* the values have already moved: the array is left as `[1, 1, 2, 3, 4]`. Check the index yourself, **before** anything moves. A check placed after the loop fails `test_insert_at_out_of_range_raises[-1]` with `AssertionError: check the index BEFORE shifting anything, showing that [1, 1, 2, 3, 4]`.

Part 4 — Removing: pop

4.1 The idea

`pop(index=-1)` removes and returns the value at index. It is Week 2's `remove_at`, inside the class:

- the default `-1` means the last element — nothing shifts: **$O(1)$** ;
- any other index shifts slots `index + 1 .. size - 1` one place **left**: **$O(\text{size} - \text{index})$** .
`pop(0)` moves every other element.

That last line is the most important cost of the week. In Week 7, `SlowQueue` dequeues with `DynamicArray.pop(0)`, and you will measure what it costs (Lecture 04, “What it is built for”).

4.2 Draw it

Draw `[10, 20, 30, 40]` (capacity 4). Then `pop(0)`: circle the value returned, draw the three left shifts as numbered arrows, and write what is left in slot 3.

Checkpoint 4. Starting from `arr = DynamicArray([10, 20, 30, 40])`, give the value returned, `len(arr)` and `list(arr._block)` after each step:

- (a) `arr.pop()` (b) `arr.pop(0)` (c) `arr.pop(-1)`
 (b) `arr.pop(5)` (e) `arr.pop()` (f) `arr.pop()`

4.3 Write it

1. If the array is empty, raise `IndexError` (the docstring and `test_pop_on_empty_raises` require it).
2. Translate a negative index by adding `size`; if the result is not in `0 .. size - 1`, raise `IndexError`.
3. Keep the value at `index` — you return it at the end.
4. Shift slots `index + 1 .. size - 1` one place left, **starting from the front**: slot `index` gets slot `index + 1`, and so on. (Left shifts go front-to-back; right shifts in Part 3 went back-to-front. Draw it if unsure.)
5. Subtract one from `size`, then set the slot that just fell out of use to `None`.
6. Return the value.

Why step 5's `None`? The old last slot still refers to an object. Nobody can reach it through your class any more, but the garbage collector can see the reference, so the object is never freed. Lecture 04's table says to "clear the freed slot", and `test_pop_clears_the_freed_slot` checks it. Question bank W4-B4 is this bug.

4.4 Test it

```
pytest tests/test_dynamic_array.py -v -k pop
```

Three tests: `test_pop_from_end_and_middle` (`pop()` then `pop(0)` on `[1, 2, 3]`), `test_pop_clears_the_freed_slot` (the same two pops on `['a', 'b', 'c']`, after which slots 1 and 2 of the block must be `None`) and `test_pop_on_empty_raises`. Then run the whole file — all 19 must pass:

```
pytest tests/test_dynamic_array.py -v
```

4.5 When it fails

No negative translation. `pop()` passes `-1` to the `Array`:

```
IndexError: Array index -1 out of range for length 4 (valid: 0..3; no
↪ negative indices)
```

in `test_pop_from_end_and_middle` and `test_pop_clears_the_freed_slot`. Same fix as Part 2 — the default argument *is* a negative index.

Forgetting to clear the freed slot. Only `test_pop_clears_the_freed_slot` fails:

```
AssertionError: assert 'b' is None
```

After `pop()` and `pop(0)` on `['a', 'b', 'c']`, the size is 1, but slot 1 still holds `'b'`: it was shifted into slot 0 and never cleared behind it.

Shifting the wrong way (`self._block[i + 1] = self._block[i]`):

```
assert [1] == [2]
```

The first element was copied rightwards over its neighbours instead of the neighbours moving left.

Passing by accident. Leave out the empty check and translate -1 anyway: `DynamicArray().pop()` still raises — but it is the Array complaining, `Array index -1 out of range for length 1 (valid: 0..0; no negative indices)`, not your class. The test only asks for *an* `IndexError`, so it passes. Raise your own, with a message that tells the user what happened.

A bug no test catches. A shift loop that runs one step too far, `range(index, self._size)`, reads slot size. That works while there is spare capacity, and every test pops from an array that has some, so all 19 pass. On a full array it crashes: `DynamicArray([1, 2, 3, 4]).pop(0)` gives `IndexError: Array index 4 out of range for length 4 (valid: 0..3; no negative indices)`. Try it by hand with `show(arr)`.

Part 5 — The growth factor experiment

5.1 What the skeleton lets you change

`growth` is a constructor argument, so you can try other **factors** without touching your class:

```
for g in (2, 3, 10):
    arr = DynamicArray(growth=g)
    for i in range(1000):
        arr.append(i)
    print(g, arr.resize_count, arr._capacity)
```

It must be a whole number greater than 1. `make_block` builds an Array, and an Array needs a whole number of slots — so `DynamicArray(growth=1.5)` fails at its first resize with `TypeError: 'float' object cannot be interpreted as an integer`. (Lecture 04, “Any constant factor works”, shows the rounding you would need.)

Checkpoint 5. (a) Predict the three lines the loop above prints. (b) `DynamicArray([1], growth=1)` works. What does `DynamicArray([1, 2], growth=1)` do, and why?

5.2 Growing by a constant instead

The skeleton only multiplies. To grow by a constant **step**, write a subclass in your notebook that ignores the capacity it is asked for:

```
class StepArray(DynamicArray):
    """Grows by a constant 10 slots instead of by a factor."""

    def _resize(self, capacity):
        super()._resize(self._capacity + 10)
```

Your append still calls `_resize` when full; the subclass just changes the answer to “how big?”. Keep it out of `dsa/dynamic_array.py`, so the tests keep checking the real class (Homework 4, item 2).

Now count. To see the copying itself, not just the number of resizes, add a counter in the same way:

```
class Counting(DynamicArray):
    def __init__(self, values=(), growth=2):
        self.copies = 0                # before super().__init__,
        ↪ which appends
        super().__init__(values, growth)

    def _resize(self, capacity):
        self.copies += len(self)      # a resize copies every element
        super()._resize(capacity)

class CountingStep(Counting):
    def _resize(self, capacity):
        super()._resize(self._capacity + 10)

for n in (1000, 2000, 4000, 8000):
    d, s = Counting(range(n)), CountingStep(range(n))
    print(n, d.resize_count, d.copies, s.resize_count, s.copies)
```

You should get:

n	× 2: resizes	× 2: copies	+ 10: resizes	+ 10: copies
1,000	10	1,023	100	49,600
2,000	11	2,047	200	199,200
4,000	12	4,095	400	798,400
8,000	13	8,191	800	3,196,800

Read the columns as n doubles. With doubling, resizes go up by **one** (they are $\log_2 n$) and copies **double**, staying under $2n$ — linear. With a constant step, resizes double and copies go up **four times** — quadratic, roughly $n^2 / (2 \times 10)$. That is Lecture 04’s “The three side by side”, produced by your own class.

Part 6 — Measure it

Counting is exact; timing is what users feel. Both should tell the same story. Use `viz.complexity.measure` (it times the best of three runs and does not time building the input):

```
from viz.complexity import measure, plot_growth

def fill(arr_and_n):
    arr, n = arr_and_n
    for i in range(n):
        arr.append(i)

sizes = [500, 1000, 2000, 4000]
doubling = measure(fill, sizes, lambda n: (DynamicArray(), n))
step = measure(fill, sizes, lambda n: (StepArray(), n))
plot_growth({"x 2 (doubling)": doubling, "+ 10 (constant step)": step},
            reference=["n", "n^2"])
```

Expect the doubling curve to be a straight line close to the $O(n)$ reference, and the constant-step curve to bend upwards like $O(n^2)$. Your seconds will differ from your neighbour's; the **shapes** will not. Keep the sizes modest. Every element you move goes through the course Array's bounds check, so it is far slower than a Python list, and `measure` runs each size four times (one warm-up, then the best of three). Doubling n to 8,000 makes the constant-step run about four times longer, not twice.

Now the cost that sets up Week 7 — removing from the front against removing from the back:

```
def drain_front(arr):
    while len(arr):
        arr.pop(0)

def drain_back(arr):
    while len(arr):
        arr.pop()

sizes = [250, 500, 1000, 2000] # small on purpose: see below
front = measure(drain_front, sizes, lambda n: DynamicArray(range(n)))
back = measure(drain_back, sizes, lambda n: DynamicArray(range(n)))
plot_growth({"pop(0) until empty": front, "pop() until empty": back},
            reference=["n", "n^2"])
```

This takes a few seconds, almost all of it in `drain_front`. Do not be tempted to add 8,000: emptying 8,000 elements with `pop(0)` moves 32 million elements, and four runs of that take minutes. That slowness is the result.

Checkpoint 6. Each time n doubles, by roughly what factor should each of these times grow: (a) filling a doubling array, (b) filling a StepArray, (c) `drain_front`, (d) `drain_back`?

Emptying an array with `pop(0)` shifts $n - 1$ elements, then $n - 2$, and so on: the triangular sum from Lecture 02, $\Theta(n^2)$ in total — $O(n)$ per removal. A queue that removes from the front of a dynamic array, one customer at a time, pays exactly that. In Week 7 you will build this `SlowQueue`, measure it, and then fix it with a ring buffer.

Part 7 — Exercises at a glance

Method	Target cost	The trap	Tests (-k)
<code>_resize(capacity)</code>	$O(\text{size})$	forgetting to switch <code>_block</code> , update <code>_capacity</code> or count <code>resize_count</code>	"constructor or grows_beyond or doubling" (3)
<code>append(value)</code>	amortised $O(1)$, worst $O(n)$	writing before the "is it full?" check; + instead of *	"constructor or grows_beyond or doubling" (3)
<code>__getitem__(index)</code>	$O(1)$	bounds against capacity; adding capacity to a negative index	"index or setitem" (7)
<code>__setitem__(index, value)</code>	$O(1)$	no bounds check — silently writes an unused slot	"index or setitem" (7)
<code>insert_at(index, value)</code>	$O(n)$	shifting from the front; no resize when full; clamping instead of raising <code>IndexError</code>	<code>insert</code> (5)
<code>pop(index=-1)</code>	$O(1)$ at the end, $O(n)$ elsewhere	no negative translation; not clearing the freed slot	<code>pop</code> (3)

Part 8 — Take-home practice (not graded)

1. **Question bank, Week 4, Part C** (`docs/question-bank/week04-questions.md`). Do W4-T1 and W4-T2 (doubling: which appends `resize`, total work, average per ap-

pend) and W4-T3 (the same with a step of + 3), by hand, then check them with the Counting class of Part 5.

2. **Part E, W4-B1 to W4-B4.** Four buggy methods. You have now seen three of those bugs in your own code; say which.
3. **Part F, W4-C1 to W4-C3,** in `practice/week04.py`:

```
pytest tests/test_practice_week04.py -v
```

`capacity_after` and `copies_for` count without building anything. `ShrinkingArray` is a small dynamic array of your own that also halves when a pop leaves it a quarter full (Lecture 04, “Shrinking”). It is Homework 4’s item 4 in a form the tests can check.

4. **Homework 4, item 2** (Lecture 04, “This Week”): your Part 5 and Part 6 measurements, with `resize_count` plotted against `n` for both growth rules.

Answers to the question bank are in `docs/question-bank/week04-answers.md`, and the worked solutions in `solutions/dsa/dynamic_array.py` and `solutions/practice/week04.py` — for after you have tried.

Part 9 — Bridge to Lecture 05: linked lists

Everything expensive this week came from one fact: the elements sit **side by side**. That makes `a[i]` a single address calculation — and it makes `insert_at(0, x)` move every element, and growing copy them all.

Lecture 05 gives up contiguity. Each value lives in its own small **node**, anywhere in memory, holding a reference to the next one. Node is already given in `dsa/linked_list.py`; try it now:

```
>>> from dsa.linked_list import Node
>>> head = Node(3, Node(7, Node(1)))      # 3 -> 7 -> 1 -> None
>>> head = Node(0, head)                  # insert at the front
>>> head.value, head.next.value, head.next.next.value
(0, 3, 1)
```

Inserting at the front was one new node and one reference: nothing shifted, nothing was copied, however long the chain. Draw it:

```
from viz.draw import draw_linked_list
draw_linked_list([0, 3, 7, 1], highlight=0, title="after inserting 0
↪ at the front")
```

The price is the other side of this week’s table: to reach the fourth value you must follow three references, so indexing becomes $O(n)$. Lecture 04’s last slide calls it “the opposite trade”. Bring this week’s cost table to the lecture and fill in the second column there.

Summary

Idea	The one line to keep
Size and capacity	Capacity is how many slots the Array has; size is how many hold data. $0 \leq \text{size} \leq \text{capacity}$.
Growing	Allocate a bigger Array, copy size elements, switch — $O(n)$, and count it.
append	Resize before writing; grow by capacity * growth.
Factor vs constant	$\times 2$: fewer than $2n$ copies for n appends. + c : about $n^2/2c$.
Amortised $O(1)$	Any n appends cost fewer than $3n$; one append can still cost $O(n)$.
Indexing	Bounds are $0 \dots \text{size} - 1$; a negative index adds size, not capacity.
insert_at	IndexError unless $0 \leq \text{index} \leq \text{size}$; resize if full; shift right from the end . $O(n)$.
pop	Shift left from the front , clear the freed slot. $O(1)$ at the end, $O(n)$ at the front.
pop(0)	$O(n)$ — which is why Week 7's SlowQueue is slow.

Lab 05 — Lab 05 — Building a Linked List

How to use this lab. Before each part, read the Lecture 05 section it names. Then **draw before you code**: boxes for nodes, arrows for references, on paper, before and after the operation. Predict at each **Checkpoint**, then write the method, then run the tests the part names. Build the class one method at a time, in the order below. Almost every linked-list bug lives in one of four places: the empty list, a one-node list, the first node and the last node. Test those by hand.

Duration	One 2-hour lab session, plus about 4 hours at home
You will write	<code>dsa/linked_list.py</code> — the <code>LinkedList</code> methods <code>push_front</code> , <code>append</code> , <code>insert_at</code> , <code>pop_front</code> , <code>remove</code> , <code>find</code> , <code>reverse</code> and <code>__getitem__</code>
Graded by	<code>tests/test_linked_list.py</code> (24 tests)
Connects to	Lecture 05 — Linked Lists; and Lecture 06 — Stacks

What you will be able to do

1. build a chain of `Node` objects by hand at the REPL, and say what every reference points at;
 2. write the traversal loop without thinking, and know when to stop **on** the last node and when to stop **after** it;
 3. insert at the front, in the middle and at the end, changing references in an order that never loses the rest of the list;
 4. remove from the front and by value, including the head special case;
 5. reverse a list in place with three references, and trace it on paper;
 6. keep `_size` correct in every method, so that `len` stays $O(1)$;
 7. measure why `for i in range(len(ll)): ll[i]` is $O(n^2)$, and write the $O(n)$ loop instead;
 8. say what a tail reference buys and what it costs.
-

Part 0 — Before you start

0.1 Environment check

From the DSA27 folder, with the virtual environment active (the prompt starts with `(.venv)`):

```
git pull
pytest -m "not challenge" -q
```

The second line must end with passed. If it does not, fix that first — Lab 01, Part 0, and the TA guide's table of environment problems.

0.2 What this lab stands on

The linked list is the one structure of the term that does not stand on another dsa/ structure: it stores its data in **node objects**, the second of the three things the storage rule allows (Lecture 02). So there is no prerequisite test file to pass first. What you do need from earlier weeks:

- **Lab 01** — a name refers to an object; `b = a` copies nothing. `node.next` is exactly that: one more name bound to one more object.
- **Lab 03** — classes, `self`, special methods, and generators. The given `__iter__` is a generator.
- **Lecture 04** — `DynamicArray`. This week's costs are the mirror image of last week's, and you will compare them.

0.3 Read the skeleton

Open `dsa/linked_list.py`. Read every docstring: each one is part of the contract, and states the target cost as well as the behaviour.

Given — do not change	Yours
<code>Node(value, next=None)</code> , with <code>__slots__</code> and <code>__repr__</code>	<code>push_front(value)</code>
<code>LinkedList.__init__(values=())</code> — note that it calls <code>self.append</code>	<code>append(value)</code>
<code>__len__</code> — returns <code>self._size</code>	<code>insert_at(index, value)</code>
<code>__iter__</code> — the traversal loop, as a generator	<code>pop_front()</code>
<code>__repr__</code> — <code>LinkedList([1, 2, 3])</code>	<code>remove(value)</code>
	<code>find(value)</code>
	<code>reverse()</code>
	<code>__getitem__(index)</code>

Two things follow from the given code.

- **`__init__` calls `append`.** Until `append` works, `LinkedList([1, 2, 3])` raises `NotImplementedError`. Most tests build their list that way, so `append` comes early

in the build order.

- **`__len__` returns `_size` and never counts.** So every method you write that adds or removes a node must update `self._size`. Nothing else will.

The object holds exactly two attributes: `self.head` (the first Node, or None when the list is empty) and `self._size`. The tests check `ll.head` directly, so there is **no sentinel node** in this exercise (Lecture 05, “Doubly linked, and sentinels”): head is the first *real* node.

The storage rule, here. Your methods may create Node objects and change references. They may not keep a Python list, dict, set or deque inside the LinkedList, and reverse in particular may not copy the values out into a list. A list as a *temporary* in your own REPL experiments is fine.

Run the tests once now, to see the starting point:

```
pytest tests/test_linked_list.py -q
```

Expect 22 failed, 2 passed. The two that pass test only the given code: `test_node_holds_a_value_and_a_link` and `test_empty_list`.

Part 1 — Nodes by hand

Read Lecture 05, “A node: a value and a link” and “Build one by hand”.

A linked list is nothing more than a chain of nodes, each holding a value and a reference to the next, with None after the last. Before you write a single method, build one yourself at the REPL. Everything in this lab is only a question of **which reference you change, and in what order**.

```
>>> from dsa.linked_list import Node
>>> c = Node(1)
>>> b = Node(7, c)
>>> a = Node(3, b)
>>> a.value, a.next.value, a.next.next.value
(3, 7, 1)
>>> a.next.next.next is None
True
>>> a.next is b
True
```

Notice the order you had to build it in: from the **end** backwards, because a node’s next must already exist when you create it.

Node has `__slots__ = ("value", "next")`. As well as saving memory (Lecture 05, “Memory: a node is not free”), this catches typing mistakes:

```
>>> a.nxt = Node(9)
AttributeError: 'Node' object has no attribute 'nxt' and no __dict__
↪ for setting new attributes
```

Without `__slots__`, that line would silently create a new attribute, and your list would never change.

Draw it

On paper, draw the three nodes above as boxes with arrows, and label each arrow with the expression that reaches it (`a.next`, `a.next.next`, ...). Then let the course's drawing helper draw the same chain. It takes plain values, head first:

```
from viz.draw import draw_linked_list

draw_linked_list([3, 7, 1], highlight=1, title="a.next is the node
↪ holding 7")
```

In the notebook `notebooks/05-linked-list.ipynb` the figure appears under the cell. In a script, add `import matplotlib.pyplot as plt` and `plt.show()` at the end. `highlight=i` colours the node at position `i` — use it to show where a pointer is while you walk.

Checkpoint 1. *Predict each line:*

```
c = Node("c"); b = Node("b", c); a = Node("a", b)
print(repr(a.next.next))
a.next = c
print((a.next.value, b.next.value))
d = Node("d", a)
print(d.next.next.value, d.next.next.next)
```

After `a.next = c`, what still refers to the node holding "b"?

Part 2 — `push_front`: $O(1)$ at the head

Read Lecture 05, "At the front: $O(1)$ ".

A new first node needs two references changed, whatever the length of the list: the new node must point at the old first node, and `head` must point at the new node. Then `_size` goes up by one. Nothing is shifted, nothing is copied.

Draw it

Draw `head → 3 → 7 → None`. Add a box 5 above it. Draw the two arrows that change, numbered **1** and **2** in the order you will set them. Now try the other order on a second drawing, and follow the arrows from `head`.

Checkpoint 2. A student writes `push_front` with the two assignments the wrong way round:

```
def push_front(self, value):
    new = Node(value)
    self.head = new
    new.next = self.head
    self._size += 1
```

After `ll = LinkedList(); ll.push_front(3); ll.push_front(7)`, predict `len(ll)`, `ll.head.value` and `ll.head.next` is `ll.head`. What does `list(itertools.islice(ll, 5))` return — and what would plain `list(ll)` do?

Write it

1. Create the new node, linked to the **current** head.
2. Make head refer to the new node.
3. Add one to `_size`.

The rule from the lecture: **link the new node in before you let go of the old one.** Steps 1 and 2 also fit in a single assignment, because Python evaluates the whole right-hand side — including the old `self.head` — before it assigns: `Node(value, self.head)` already holds the old head when `self.head` changes.

The empty list needs no special case: when head is `None`, the new node's next is `None`, which is exactly right for a one-node list.

Test it

```
pytest tests/test_linked_list.py -v -k "node_holds or empty_list or
↪ push_front"
```

That selects 3 tests: `test_node_holds_a_value_and_a_link`, `test_empty_list` and `test_push_front_reverses_insertion_order`. (A plain `-k empty` would also pick up `test_pop_front_on_empty_raises`, which you have not written yet.)

When it fails

- **The test never finishes.** No dot, no F, nothing. You set head before linking the new node, so the new node points at itself, and the given `__iter__` walks round that one-node loop for ever — exactly Checkpoint 2 and question W5-S2. Press Ctrl+C; the traceback ends inside `__iter__`. Fix the order.
- **`assert 0 == 3`**, with where `0 = len(LinkedList([3, 2, 1]))`. The values are right and the length is wrong: you forgot `self._size += 1`. The repr in the message is the list; the number is `len`.

Part 3 — append: $O(n)$ without a tail

Read Lecture 05, “At the end: $O(n)$ — unless you keep a tail”.

To append, you must find the **last** node and point its `next` at the new node. The list holds no reference to the last node, so you walk from head — $O(n)$. This is the one walk in the lab that must stop **on** the last node rather than after it, so its loop condition is `while node.next is not None`.

The empty list has no last node at all. That is the special case: the new node simply becomes the head.

Draw it

Draw `head → 1 → 2 → 3 → None`. Put your finger on head and move it along one arrow at a time, stopping on the node whose next is None. Count your moves. Then do the same with `draw_linked_list([1, 2, 3], highlight=i)` for $i = 0, 1, 2$.

Checkpoint 3. With a correct `push_front` and `append`:

```
ll = LinkedList()
ll.push_front(2)
ll.append(3)
ll.push_front(1)
ll.append(4)
```

- What is `ll`?
- How many times does the walking step (`node = node.next`) run inside the last `append`?
- `LinkedList(range(5))` calls `append` five times. How many walking steps in total? And for `LinkedList(range(1000))`?

Write it

- Create the new node (its `next` is None — it will be last).
- If the list is empty, make it the head.
- Otherwise start at head and step along while the current node has a next; then link the current node — the last one — to the new node.
- Add one to `_size`, in **both** branches.

Keep the docstring’s challenge — a `_tail` reference — for later (Part 9). Get the $O(n)$ version passing first.

Test it

```
pytest tests/test_linked_list.py -v -k "append or constructor"
```

2 tests: `test_append_preserves_insertion_order` and `test_constructor_takes_an_iterable`. Once these pass, run the whole file: many tests that failed with `NotImplementedError` at `LinkedList([...])` now get as far as the method they are really testing.

When it fails

- **AttributeError: 'NoneType' object has no attribute 'next'** — in 19 of the 24 tests, because the constructor uses `append`. You walked without handling the empty list: `self.head` is `None`, and `None.next` does not exist. Add the empty case first.
- **AttributeError: 'NoneType' object has no attribute 'next' and no `__dict__` for setting new attributes** — note the second half: this is an *assignment* to `None.next`. You walked with `while node is not None`, which runs one step too far — off the end — so `node` is `None` when you try to link it. Stop **on** the last node: `while node.next is not None`.
- **Both tests pass, but later parts fail on lengths** — `assert 1 == 3` with where `1 = len(LinkedList([1, 2, 3]))`, or an `IndexError: 2 from ll[2]` on a three-value list. You added `_size += 1` in only one branch (here, only the empty one). The two `append` tests never check `len`; the later ones do.

Part 4 — Walking: `find` and `__getitem__`

Read Lecture 05, “The traversal loop” and “Indexing is $O(n)$ — that is the lesson”.

Every linked-list algorithm is the traversal loop plus a little bookkeeping:

```
node = self.head
while node is not None:
    # ... use node.value ...
    node = node.next
```

The given `__iter__` is this loop with `yield node.value` in the middle. `find` is this loop with an index counter and an early return. `__getitem__` is this loop stopped after index steps. Both walk the whole list in the worst case: $O(n)$. Here the loop stops **after** the last node (`node is not None`), because every node must be checked.

Draw it

`draw_linked_list(list(ll), highlight=i)` for `i = 0, 1, 2, ...` is exactly what `__getitem__` does in its head: to reach position 3 it must visit 0, 1 and 2 first. There is no arithmetic that jumps ahead — compare `DynamicArray[i]`, which is one address calculation.

Checkpoint 4. With `ll = LinkedList(["a", "b", "c", "b"])`, predict `ll.find("b")`, `ll.find("z")`, `ll[3]` and `ll[4]`. How many `node = node.next`

steps does `ll[3]` take?

Write it

find(value) — walk with a node reference and an index that starts at 0. At each node, if its value equals `value`, return the index; otherwise step both on. If the loop ends, return -1. The **first** match wins.

__getitem__(index) —

1. If `index` is out of range (not $0 \leq \text{index} < \text{self}._\text{size}$), raise `IndexError`. `_size` makes this check $O(1)$: you know before walking.
2. Start at head and take exactly `index` steps.
3. Return that node's value.

The docstring, the tests and the lecture ask only for $0 \leq \text{index} < \text{len}(\text{self})$. Supporting negative indices as well (`ll[-1]`, by adding `self._size` first) is a reasonable extra, and it is what the worked solution does.

Test it

```
pytest tests/test_linked_list.py -v -k "find or getitem"
```

3 tests: `test_find`, `test_getitem` and `test_getitem_out_of_range_raises`.

When it fails

- **assert -1 == 2**, with where `-1 = find('c')` on `LinkedList(['a', 'b', 'c'])`. Your `find` loop is `while node.next is not None`: it stops **on** the last node without checking it, so the last value is never found (question W5-B1). Use `while node is not None`.
- **AttributeError: 'NoneType' object has no attribute 'next'** in `test_getitem_out_of_range_raises`. No range check: walking five steps along a one-node list runs off the end. The test expects `IndexError` — check first, walk second.

Part 5 — `insert_at`: walk, then splice

Read Lecture 05, “In the middle: walk, then splice”.

A singly linked list can only be changed **from the node before** the change, because only that node holds the reference that must be redirected. To put a new node at index `i`, stop at index `i - 1` — call it `prev` — and splice in two assignments, in the safe order: the new node takes `prev`'s link first, then `prev` points at the new node.

Index 0 has no `prev`. It is `push_front` — so call `push_front`.

Draw it

Draw `head → 10 → 20 → 30 → None` and insert 15 at index 1. Mark `prev`. Number the two arrows that change. Then insert 40 at index 3 — the end — and check that the same two steps work when `prev.next` is `None`.

Checkpoint 5. *Predict what is printed, and what the last line does:*

```
ll = LinkedList([10, 20, 30])
ll.insert_at(3, 40)
ll.insert_at(1, 15)
ll.insert_at(0, 5)
print(ll, len(ll))
ll.insert_at(7, 99)
```

Write it

1. Validate: unless `0 <= index <= self._size`, raise `IndexError`. Note the `<=` at the top end: `index == len(self)` is legal and means “at the end”.
2. If `index == 0`, call `push_front(value)` and return. (It already updates `_size` — do not add one twice.)
3. Otherwise start `prev` at `head` and step `index - 1` times.
4. Splice the new node in after `prev`, taking `prev`’s old `next` as its own.
5. Add one to `_size`.

Test it

```
pytest tests/test_linked_list.py -v -k insert_at
```

5 tests: `test_insert_at` with index 0, 1 and 3, and `test_insert_at_rejects_out_of_range` with -1 and 5.

When it fails

- **assert [1, 2, 'x', 3] == [1, 'x', 2, 3]** for index 1, and **AttributeError: 'NoneType' object has no attribute 'next'** for index 3. You walked `index` steps instead of `index - 1`: you stopped **on** the node that should come after the new one, so the new node lands one place too far right — and at the end there is no node to stop on. Walk to the node **before** the gap.
- **Failed: DID NOT RAISE IndexError** for index -1, and an `AttributeError` for index 5. No validation. `range(-2)` is empty, so -1 quietly inserts after the head; 5 walks off the end. Check the range first, with `<=`.
- **assert 3 == 4**, with where `3 = len(LinkedList(['x', 1, 2, 3]))`, for index 0 only. Your `push_front` never updated `_size`; `insert_at(0, ...)` inherited the bug. Fix it there, not here.

Part 6 — Removing: `pop_front` and `remove`

Read Lecture 05, “From the front: $O(1)$ ” and “By value: skip over it”.

`pop_front` moves head one node on and returns the old head’s value. The old first node is no longer referenced by anything, so Python’s garbage collector reclaims it — you never free a node yourself. An empty list has no head to pop: the contract says `IndexError`.

`remove(value)` finds the node **before** the one holding value and bypasses it: `prev.next = prev.next.next`. The **head** is the special case — it has no node before it. (A sentinel node would remove the special case; this exercise does not use one, so you write it.)

Draw it

Draw `head → 3 → 7 → 1 → 9 → None` and remove 7 (question W5-S3). Mark `prev` on the node holding 3 and cross out the arrow that changes. Then remove 3 from the original list: where is `prev` now? There is none — which is exactly why the head needs its own branch.

Checkpoint 6. Predict the four values printed on the first line, then the second line, then what the last line does:

```
ll = LinkedList([4, 8, 4, 2])
print(ll.remove(4), ll.remove(5), ll.pop_front(), ll.remove(2))
print(ll, len(ll))
ll.pop_front()
ll.pop_front()
```

(The arguments of `print` are evaluated left to right.)

Write it

`pop_front()`

1. If the list is empty, raise `IndexError` (the solution’s message is “pop from empty list”).
2. Remember the head’s value.
3. Move head to the second node (which may be `None`).
4. Subtract one from `_size`; return the remembered value.

`remove(value)`

1. If the list is empty, return `False`.
2. If the head holds value, move head on, subtract one from `_size`, return `True`.
3. Otherwise start `prev` at the head. While `prev.next` exists: if `prev.next.value == value`, bypass it, subtract one from `_size`, return `True`; else step `prev` on.
4. Nothing matched: return `False`.

Notice that the loop looks one node **ahead** (`prev.next.value`) — that is how it stays on the node before the match. Return exactly `True` or `False`: the tests use `is True`.

Test it

```
pytest tests/test_linked_list.py -v -k "pop_front or remove"
```

6 tests: test_pop_front, test_pop_front_on_empty_raises, test_remove_first_match_only, test_remove_updates_the_length, test_remove_head and test_remove_absent_value.

When it fails

- **AttributeError: 'NoneType' object has no attribute 'value'** in test_pop_front_on_empty_raises. You read `self.head.value` before checking for an empty list. The contract promises `IndexError`, not a crash.
 - **assert False is True**, with where `False = remove(1)` on `LinkedList([1, 2, 3])`, in test_remove_head — and, in test_remove_updates_the_length, `assert 2 == 1` with where `2 = len(LinkedList([1, 3]))`. Your loop only ever looks at `prev.next`, so it never checks the head itself (question W5-B2): the second `remove(1)` found nothing. Add the head branch before the loop.
 - **assert 3 == 1**, with where `3 = len(LinkedList([3]))`, in test_remove_updates_the_length only. The values are right and the length is not: you forgot `_size -= 1` in both branches, the head one and the loop one. Forget it in only one of them and the message is `assert 2 == 1` (still `LinkedList([3])`): the test removes once through each branch, so it catches either.
-

Part 7 — reverse: three references

Read Lecture 05, “Three references”.

Reversal re-links the **same** nodes: every arrow is turned round, and the old last node becomes the head. You keep two references — `prev`, the head of the part already reversed, and `node`, the head of the part still to do — plus a third, temporary one to remember what comes after `node`. One step moves one node across the line:

1. remember `node.next` — you are about to overwrite it;
2. point `node.next` back at `prev`;
3. advance: `prev` becomes `node`, `node` becomes the remembered next.

Start with `prev = None` (nothing reversed yet) and `node = self.head`. Stop when `node` is `None`. Then `prev` is the new head. $O(n)$ time, $O(1)$ extra space.

Draw it — on paper, before any code

This is homework item 4 of the lecture; do it now. Draw `1 → 2 → 3 → 4 → None` four times, one row per step, and fill in this table:

After step	prev	node	Reversed part	Part still to do
start	None	1	(empty)	1 → 2 → 3 → 4
1				
2				
3				
4				

In the middle of the reversal there are **two** chains, not one list, so draw each chain separately. To check your drawing at the REPL, a small helper that collects the values of a chain is enough (it is a REPL aid, not part of your class):

```
def chain(node):
    values = []
    while node is not None:
        values.append(node.value)
        node = node.next
    return values

draw_linked_list(chain(prev), title="reversed so far")
draw_linked_list(chain(node), title="still to do")
```

Checkpoint 7. (a) On `LinkedList([1, 2, 3, 4])`, after **two** steps of the loop, what are `prev.value`, `node.value`, `chain(prev)` and `chain(node)`?

(b) A student's loop is right but they forget the final `self.head = prev`. After `ll = LinkedList([1, 2, 3, 4]); ll.reverse()`, what do `print(ll)` and `len(ll)` show? Where are the other three nodes?

Write it

The three moves above, in a `while node is not None` loop, then set head. Step 1 is the one people forget: once `node.next = prev` has run, the only reference to the rest of the list is gone — unless you saved it first. The empty list and the one-node list need no special case; check that by running your loop on paper for both.

Test it

```
pytest tests/test_linked_list.py -v -k "reverse and not push_front"
```

5 tests: `test_reverse`, `test_reverse_handles_short_lists` for `[]`, `[1]` and `[1, 2]`, and `test_reverse_relinks_rather_than_rebuilding`. A plain `-k reverse` selects 6, because `test_push_front_reverses_insertion_order` contains the word too.

When it fails

- **assert [1] == [4, 3, 2, 1]** (and `assert [1] == [2, 1]`). Either you forgot `self.head = prev` — head is still the old first node, which now points at None — or you stepped with `node = node.next` **after** turning the link round, so the step went back to prev (question W5-B4). Save the next node first; set head last.
- **test_reverse_relinks_rather_than_rebuilding fails** with two sets of ids that differ (Extra items in the left set: ...), while the other reverse tests pass. You built a new list of new nodes — for example with `push_front` for every value. The result looks right, but it is $O(n)$ extra space and not what was asked. Re-link the nodes you have.
- **The same test fails with the old last node must become the head: relink, do not copy values**, then `assert Node(3) is Node(1)` and where `Node(3) = LinkedList([3, 2, 1]).head`. You copied the values into a Python list and wrote them back into the same nodes in reverse order. The values look reversed, but the head is still the old **first** node object — it now holds 3, which is why its repr says `Node(3)`; the `Node(1)` on the right is the old last node, which still holds 1. That version uses $O(n)$ extra space and a Python list, against the docstring and the storage rule, and turns no link round at all. Re-link.

When all five pass, run the whole file:

```
pytest tests/test_linked_list.py -v
```

24 passed is the goal. The worked solution is in `solutions/dsa/linked_list.py` — open it **after** you have passed, or after an honest hour stuck, and compare it line by line with yours.

Part 8 — Measure it: the $O(n^2)$ trap

Read Lecture 05, “Indexing is $O(n)$ — that is the lesson” and “Measured, not asserted”.

`ll[i]` looks exactly like array indexing, and Python lets you write it because you wrote `__getitem__`. But every `ll[i]` starts again at head. So this innocent loop

```
for i in range(len(ll)):
    total += ll[i]
```

does $0 + 1 + 2 + \dots + (n - 1)$ steps: the triangular sum, $O(n^2)$. The loop that uses `__iter__` walks the list once: $O(n)$.

Checkpoint 8. For a list of $n = 1000$ nodes, how many `node = node.next` steps does the indexing loop above take in total? If n doubles, by roughly what factor should its time grow — and the time of `for value in ll`?

Put this in a script (or a notebook cell) and run it once all your tests pass. It builds each test list with `push_front` backwards, which is $O(n)$; building it with `LinkedList(range(n))`

would itself be $O(n^2)$ — see the take-home question W5-K1.

```
from dsa.linked_list import LinkedList
from viz.complexity import measure, plot_growth

def build(n):
    ll = LinkedList()
    for value in reversed(range(n)):
        ll.push_front(value)
    return ll

def sum_by_index(ll):
    total = 0
    for i in range(len(ll)):
        total += ll[i]          # each ll[i] walks from the head
    return total

def sum_by_iteration(ll):
    total = 0
    for value in ll:           # one walk in total
        total += value
    return total

sizes = [500, 1000, 2000, 4000]
indexed = measure(sum_by_index, sizes, build)
iterated = measure(sum_by_iteration, sizes, build)
plot_growth({"for i in range(len(ll)): ll[i]": indexed,
            "for value in ll": iterated},
            reference=["n", "n^2"], loglog=True)
```

(In a script, finish with `import matplotlib.pyplot as plt; plt.show()`.)

What you should see. On the log-log axes both lines are straight, and the indexing line is about **twice as steep**: its slope matches the dashed $O(n^2)$ reference, the iteration line's matches $O(n)$. Print the timings: each time n doubles, the iteration time roughly doubles and the indexing time roughly quadruples (often a little more, as the list outgrows the processor's cache). At $n = 4000$ the indexing loop is hundreds to thousands of times slower. Your exact numbers will differ; the **shapes** will not.

A second measurement in the same style: replace the two functions with building a list of n values by `append` (`LinkedList(range(n))`) and by `push_front` (`build(n)`), with `make_input=lambda n: n`. Which one bends?

The lesson for your own code, every week from now on: an interface does not tell you the

cost. **Iterate a linked list; never index it in a loop.**

Part 9 — Challenge: the `_tail` reference (not graded)

Read Lecture 05, “At the end: $O(n)$ — unless you keep a tail” and “The costs, side by side”.

The append docstring sets a challenge: keep a second reference, `_tail`, to the last node, and append becomes $O(1)$ — link after `_tail`, then move `_tail`. The price is that **every** method that can change the last node must keep `_tail` correct. Do this in a copy, or on a branch, once all 24 tests pass — the tests do not require it, and they must still pass afterwards.

Audit every method. For each, ask: *can this change which node is last, or make the list empty, or make it non-empty?*

Method	What <code>_tail</code> needs	Cost after the change
<code>__init__</code>	start as <code>None</code> — before the loop that appends	—
<code>append</code>	empty list: head and tail are the new node; else link after <code>_tail</code> , move <code>_tail</code>	$O(1)$ (was $O(n)$)
<code>push_front</code>	on an empty list the new node is also the tail	$O(1)$
<code>insert_at</code>	at <code>index == len(self)</code> the new node is the new tail — or just call <code>append</code>	$O(n)$
<code>pop_front</code>	popping the only node must clear <code>_tail</code>	$O(1)$
<code>remove</code>	removing the last node: <code>_tail</code> moves back to <code>prev</code> — which you hold only because you walked to it	$O(n)$
<code>reverse</code>	the old head becomes the tail	$O(n)$

The row that matters is `remove`. Removing the last node is still $O(n)$ in a singly linked list, tail or no tail: the new last node is the one *before* it, and only a walk from the head finds it (question W5-M16). A **doubly** linked list fixes that, for the price of a second reference in every node.

Checkpoint 9. A student adds `_tail` and updates `__init__`, `append`, `push_front` and `pop_front`, but forgets `remove`. Predict:

```
ll = LinkedList([1, 2, 3])
print(ll.remove(3))
ll.append(4)
print(list(ll), len(ll))
```

That is the general lesson of the lecture: every extra reference buys speed somewhere and costs care everywhere. In Week 7 you will need exactly this structure — head for one end, tail for the other — for a linked queue.

Part 10 — Exercises at a glance

Method	Target cost	The trap	Tests (-k)
push_front(value)	O(1)	link the new node before moving head; wrong order is a self-loop	push_front (1)
append(value)	O(n); O(1) with <code>_tail</code>	the empty list has no last node; stop on the last node	append or constructor (2)
find(value)	O(n)	stop after the last node, or the last value is never found; -1 when absent	find (1)
<code>__getitem__</code> (index)	O(n)	IndexError before walking; never index in a loop	getitem (2)
insert_at(index, value)	O(n)	index 0 is push_front; stop at index - 1; index == len is legal	insert_at (5)
pop_front()	O(1)	IndexError when empty; <code>_size -= 1</code>	pop_front (2)
remove(value)	O(n)	the head has no prev; <code>_size -= 1</code> in both branches; return True/False	remove (4)
reverse()	O(n) time, O(1) space	save next before overwriting it; set head last; re-link, do not rebuild	reverse and not push_front (5)

Every method that adds or removes a node: **update_size**.

Part 11 — Take-home practice (not graded)

The Week 5 question bank is `docs/question-bank/week05-questions.md`, with answers in `week05-answers.md` — try first, then check.

Coding problems W5-C1 to W5-C5 are in `practice/week05.py`. Each takes the first Node of a chain and must walk and re-link, never copy into a Python list:

```
pytest tests/test_practice_week05.py -v
```

- **W5-C1** `middle_value` and **W5-C5** `kth_from_end` — two references moving at different speeds, or a fixed distance apart.
- **W5-C2** `has_cycle` — detect exactly the self-loop of Checkpoint 2, in $O(1)$ extra space.
- **W5-C3** `merge_sorted_chains` and **W5-C4** `remove_duplicates_sorted` — re-linking, the reverse skill again.

Trace and state questions to do on paper, with the drawings:

- **W5-S1** — a sequence of every operation you wrote; give the list after each line, `len` and a `find`.
- **W5-S2** — the wrong-order `push_front`, on a list that was not empty.
- **W5-T4** — `reverse()` on $1 \rightarrow 2 \rightarrow 3$, step by step.
- **W5-K1** — why building a list with `append` in a loop is $\Theta(n^2)$, and two ways to make it $\Theta(n)$. (You measured one of them in Part 8.)

Worked solutions: `solutions/practice/week05.py`, and `pytest --solutions tests/test_practice_week05.py` runs the tests on them. As always, they are for **after** you have tried.

Part 12 — Bridge to Lecture 06: stacks

Lecture 06 is the **stack**: last in, first out, with `push`, `pop` and `peek`, all $O(1)$. Every stack operation touches only one end, the **top**. So the question is: which end of which structure is $O(1)$?

You have just built the answer twice over. On your `DynamicArray` (Week 4) the cheap end is the **end**. On your `LinkedList` the cheap end is the **head**: `push_front` and `pop_front` are $O(1)$, worst case, with no resizing. A stack is a linked list with the top at the head.

Try it with the class you have just finished:

```
>>> from dsa.linked_list import LinkedList
>>> stack = LinkedList()
>>> for token in ["(", "[", "{"]:
```

```

...     stack.push_front(token)           # push
...
>>> stack
LinkedList(['{', '[', '('])
>>> stack.head.value                       # peek: the last one pushed
'{'
>>> stack.pop_front(), stack.pop_front() # pop, pop
('{', '[')
>>> len(stack)
1

```

The last bracket opened is the first one out — which is exactly what checking "(a[b]{c})" needs, and the first application in Lecture 06.

Before the lecture, think about the other choice: put the top at the **tail** of your linked list instead. Which of push and pop stays $O(1)$ — even with a `_tail` — and which becomes $O(n)$? (Part 9 has the answer.)

`dsa/stack.py` itself is built on your `DynamicArray`, with the top at the end; Lecture 06 compares the two honest designs, and one dishonest one, by measurement.

Summary

Idea	The one line to keep
Structure	Nodes (a value and a next), a head, and <code>_size</code> . No sentinel here.
Traversal	<code>while node is not None: ... node = node.next</code> — stop after the last node to visit all; while <code>node.next</code> is not <code>None</code> stops on it, for append.
<code>push_front</code>	$O(1)$. Link the new node to the old head before moving head.
<code>append</code>	$O(n)$ — walk to the last node; the empty list is the special case. $O(1)$ with <code>_tail</code> .
<code>insert_at</code>	$O(n)$. Index 0 is <code>push_front</code> ; otherwise stop at <code>index - 1</code> and splice.
<code>pop_front</code>	$O(1)$. <code>IndexError</code> when empty.
<code>remove</code>	$O(n)$. Change it from the node before ; the head is the special case.
<code>reverse</code>	Three references; save next first; <code>head = prev last</code> . $O(n)$ time, $O(1)$ space.
<code>__getitem__</code>	$O(n)$ by contract. Indexing in a loop is $O(n^2)$: iterate instead.

Idea	The one line to keep
<code>_size</code>	Every method that adds or removes a node updates it — <code>len</code> depends on nothing else.
<code>_tail</code>	Buys $O(1)$ append; costs care in every method; removing the last node stays $O(n)$.

Lab 06 — Lab 06 — Stacks, Brackets and Postfix

How to use this lab. The routine from Week 4 on: read the section of Lecture 06 that each part names, **draw before you code** — on paper, with the stack as a column and the top at the end — and predict at each **Checkpoint** before you run anything. Then build one method at a time and run the named tests after each part. The stack itself is the smallest class you will write this term; the work of this lab is the three algorithms that stand on it.

Duration	One 2-hour lab session, plus about 4 hours at home
You will write	<code>dsa/stack.py</code> — <code>Stack.push</code> , <code>Stack.pop</code> , <code>Stack.peak</code> , <code>is_balanced</code> , <code>infix_to_postfix</code> ; and <code>evaluate_postfix</code> in <code>dsa/translation.py</code>
Graded by	<code>tests/test_stack_queue.py</code> -k "lifo or peek or empty_behaviour or balanced or infix" (12 tests) and <code>tests/test_translation.py</code> -k <code>evaluate_postfix</code> (11 tests)
Connects to	Lecture 06 — Stacks; Lecture 04 (your <code>DynamicArray</code>); Lecture 07 — Queues

What you will be able to do

1. implement the Stack ADT on your own `DynamicArray`, with every operation $O(1)$, and say why the top must be the **end** of the array;
2. measure a stack whose top is index 0, and explain why it passes every test and is still wrong;
3. check balanced brackets with a stack, and name the three ways a text can be unbalanced;
4. evaluate a postfix expression with one stack, getting the operand order right;
5. trace the shunting-yard algorithm on paper, with an output list and an operator stack, and then implement it;
6. chain the two into a calculator: infix, to postfix, to a value;
7. read a failing stack test and go straight to the line that caused it.

Part 0 — Before you start

0.1 Environment and prerequisites

From the DSA27 folder, with the virtual environment active (the prompt starts with `(.venv)`):

```
pytest -m "not challenge" -q          # the environment check — must
↪ pass
pytest tests/test_dynamic_array.py -q # your Week 4 work — must pass:
↪ 19 tests
```

The second line is not optional. Your Stack keeps its items in **your own** `DynamicArray` from Week 4 — the storage rule of Lecture 02 at work — so a bug in last week's `append`, `pop` or `__getitem__` shows up this week as a stack bug. If `DynamicArray.append` still raises `NotImplementedError`, every stack test that pushes fails with `NotImplementedError`, and the last line of the traceback is in `dsa/dynamic_array.py`, not in `dsa/stack.py`. Fix Week 4 first.

0.2 The right `-k` filter

`tests/test_stack_queue.py` holds the tests for this week **and** next week's queues. `pytest -k` matches the words you give it against each test's name **and the name of its file** — and the file is called `test_stack_queue.py`. So `-k stack` selects all 18 tests, queues included — and so does any filter with `stack` or `queue` in it. The six queue tests then fail with `NotImplementedError` until Week 7, which is noise, not your bug. Use the names of the tests themselves, as Lecture 06 does:

```
pytest tests/test_stack_queue.py -v -k "lifo or peek or
↪ empty_behaviour or balanced or infix"
```

That selects exactly the 12 stack tests (6 deselected):

Filter word	Tests it selects
<code>lifo</code>	<code>test_stack_is_lifo</code>
<code>peek</code>	<code>test_stack_peek_does_not_remove</code>
<code>empty_behaviour</code>	<code>test_stack_empty_behaviour</code>
<code>balanced</code>	<code>test_is_balanced</code> — 7 cases
<code>infix</code>	<code>test_infix_to_postfix</code> and <code>test_infix_to_postfix_is_left_associative</code>

The same trap waits in `tests/test_translation.py`; Part 4 deals with it.

0.3 Read the skeleton

Open `dsa/stack.py`. Read the module docstring first: it tells you where the top goes before you write a line.

Given to you	Yours
<code>__init__(values=())</code> — makes <code>self._items = DynamicArray()</code> and pushes each value	<code>push(value)</code> — $O(1)$ amortised
<code>is_empty()</code> , <code>__len__()</code> — both ask <code>self._items</code>	<code>pop()</code> — $O(1)$; <code>IndexError</code> when empty
<code>__repr__()</code> — prints the items bottom to top, and the top	<code>peek()</code> — $O(1)$; <code>IndexError</code> when empty
	<code>is_balanced(text)</code> — $O(n)$
	<code>infix_to_postfix(tokens)</code> — $O(n)$

And in `dsa/translation.py`, **only** `evaluate_postfix(tokens)` is this week's. Everything else in that file — `tokenize`, `Parser`, `evaluate`, `calculate` — is Week 15. Leave it alone.

Look closely at `__repr__`:

```
return f"Stack({list(self._items)!r}) # top =
↪ {self._items[len(self._items) - 1]!r}"
```

It shows the items from bottom to top and names the **last** one as the top. That line is a contract: the top of this stack is the end of the array.

Checkpoint 1. On the untouched skeleton — before you have written anything — which of these lines work, and what do they give? Which raise, and why?

```
from dsa.stack import Stack
Stack()
len(Stack())
Stack().is_empty()
Stack([1, 2])
```

0.4 The storage rule, this week

- `Stack` stores its items in `self._items`, a `DynamicArray`. Not a Python list.
- `is_balanced`, `infix_to_postfix` and `evaluate_postfix` use **your Stack** as their working storage. A Python list used as a stack (append and pop) would pass the tests and break the rule — Lecture 06, “This Week”.
- Inputs and results are lists: `infix_to_postfix` takes a list of token strings and returns a list of token strings. That is the interface the tests use.

Part 1 — push, pop and peek: the top at the end

1.1 The idea

Lecture 06, “Two Honest Implementations”: the top goes where the $O(1)$ end is. On a dynamic array that is the **end**. `DynamicArray.append` is amortised $O(1)$ and `DynamicArray.pop()` with no argument removes the last item without moving anything else. So:

- **push** is an append;
- **pop** is a pop from the end — after checking the stack is not empty;
- **peek** reads the last item — after the same check — and removes nothing.

Every operation touches one slot. Nothing shifts.

1.2 Draw it

On paper, draw an array of capacity 4 and run `push(3)`, `push(1)`, `push(4)`, `pop()`, `push(5)`. After each step write down the size and which cell is the top. Then check your last picture with the course drawing helper:

```
from viz.draw import draw_array
draw_array([3, 1, 5], highlight=2, title="a stack on an array: the top
↪ is the last cell")
```

Notice what never happens in your drawing: no item ever moves to a new cell.

Checkpoint 2. What is printed?

```
s = Stack()
s.push('a'); s.push('b'); s.push('c')
print(s.pop())
s.push('d')
print(s.peek())
print(len(s))
print(s)
```

1.3 Write it

1. **push(value)**: append value to `self._items`. One line.
2. **pop()**: if the stack is empty, raise `IndexError('pop from empty stack')`. Otherwise remove the last item of `self._items` and return it.
3. **peek()**: if the stack is empty, raise `IndexError('peek at empty stack')`. Otherwise return the item at index `len(self._items) - 1`. Do not remove it.

Hints:

- Use `self.is_empty()` or `len(self._items) == 0` for the check — both are given and both are $O(1)$.

- Why raise your **own** `IndexError`, when `DynamicArray.pop` already raises one on an empty array? Checkpoint 3 answers that.
- Your methods talk to `self._items` only through its public methods: `append`, `pop`, `len`, indexing. Never reach into `self._items._block`.

Checkpoint 3. A student writes `peek` with no empty check at all:

```
def peek(self):
    return self._items[len(self._items) - 1]
```

What does `Stack().peek()` do with a correct Week 4 `DynamicArray`? Does `test_stack_empty_behaviour` pass? Why is it still worth writing the check?

1.4 Test it

```
pytest tests/test_stack_queue.py -v -k "lifo or peek or
↳ empty_behaviour"
```

Three tests: `test_stack_is_lifo` (push 1, 2, 3; pop gives 3 then 2; length 1 left), `test_stack_peek_does_not_remove` (peek on `Stack([1, 2])` is 2, length still 2) and `test_stack_empty_behaviour` (`IndexError` from both pop and peek on an empty stack).

1.5 When it fails

The stack is a queue. push appends but pop calls `self._items.pop(0)`:

```
E assert 1 == 3
E + where 1 = pop()
E + where pop = Stack([2, 3]) # top = 3.pop
```

`test_stack_is_lifo` pushed 1, 2, 3 and got 1 back — the **first** in. The `__repr__` in the message says the top is 3, because the given `__repr__` trusts the contract that the top is the end. Push and pop must use the same end.

peek removes. peek written as `return self.pop()`:

```
E assert 1 == 2
E + where 1 = len(Stack([1]) # top = 1)
```

The value was right (2); the length after it was not. Peek reads; it never changes the stack.

peek reads the wrong end. `return self._items[0]` gives the bottom:

```
E assert 1 == 2
E + where 1 = peek()
E + where peek = Stack([1, 2]) # top = 2.peek
```

Off by one. `self._items[len(self._items)]` is one past the last item:

```
E   IndexError: 2
```

The last used index is `len - 1`. (If your `DynamicArray.__getitem__` supports negative indices, as Week 4 required, `self._items[-1]` also works.)

None instead of an error. `if self.is_empty(): return None:`

```
E   Failed: DID NOT RAISE IndexError
```

Lecture 06, “The Stack ADT”: a `None` can travel a long way through a program before anyone notices. The contract says `IndexError`.

Part 2 — Measure it: the dishonest stack

2.1 The idea

Lecture 06, “...and one dishonest one”: put the top at **index 0** of the array and every push shifts the whole stack one place right, every pop shifts it one place left. Each operation is $O(n)$, so n pushes followed by n pops are $O(n^2)$. The stack still behaves perfectly — same values, same order, same errors. Only the cost is wrong.

2.2 Draw it

Redraw Part 1.2 with the top at index 0: `push(3)`, `push(1)`, `push(4)`. Count how many items move at each push (0, then 1, then 2). With n items already on the stack, how many move for one more push?

```
draw_array([5, 1, 3], highlight=0, title="top at index 0: every push
↳ shifts the rest")
```

2.3 Build the dishonest stack and time both

Put this in a scratch file (not in `dsa/`) and run it once your `Stack` passes Part 1. It subclasses your `Stack` and changes only where the top is:

```
from dsa.stack import Stack
from viz.complexity import measure, plot_growth

class FrontStack(Stack):
    """The dishonest design: the top is index 0 of the DynamicArray."""

    def push(self, value):
        self._items.insert_at(0, value)          # shifts every item
        ↳ right
```

```

def pop(self):
    if self.is_empty():
        raise IndexError('pop from empty stack')
    return self._items.pop(0)           # shifts every item left

def peek(self):
    if self.is_empty():
        raise IndexError('peek at empty stack')
    return self._items[0]

def push_then_pop(cls):
    def work(n):
        s = cls()
        for i in range(n):
            s.push(i)
        while not s.is_empty():
            s.pop()
    return work

sizes = [200, 400, 800, 1600]
end = measure(push_then_pop(Stack), sizes, lambda n: n, repeat=1)
front = measure(push_then_pop(FrontStack), sizes, lambda n: n,
    ↪ repeat=1)
for n, a, b in zip(sizes, end[1], front[1]):
    print(f'{n:5d}    top at end {a * 1000:8.1f} ms    top at index 0 {b
    ↪ * 1000:8.1f} ms')
plot_growth({'top at index 0': front, 'top at the end': end},
    reference=['n^2'], loglog=True)

```

It takes around ten seconds, almost all of it in FrontStack. Do not add bigger sizes casually: at $n = 4,000$ the dishonest stack needs about half a minute per run on a typical laptop, while the honest one needs a few hundredths of a second.

Checkpoint 4. Before you run it: (a) does FrontStack pass the three stack tests of Part 1? (b) When n doubles, by roughly how much should each column of times grow?

2.4 What you should see

- **Top at the end:** each doubling of n roughly doubles the time — linear in total, $O(1)$ per operation. At small sizes the numbers are noisy; look at the trend.
- **Top at index 0:** each doubling roughly **quadruples** the time. On the log-log plot its line runs parallel to the dashed $O(n^2)$ reference, and sits far above the honest line.
- The gap at $n = 1,600$ is a factor of a hundred or more, and it grows with n .

Your numbers will differ from your neighbour's; the shapes will not. You can check that `FrontStack` passes the tests by temporarily pasting its three methods over yours in `dsa/stack.py` and running Part 1's command — then undo it. That is the lesson of the lecture: **the tests check what your stack does. Only you can check how.** A TA will ask you what push costs.

Part 3 — `is_balanced(text)`

3.1 The idea

Lecture 06, “Application 1: Balanced Brackets”. The rule “the most recently opened bracket must close first” is last in, first out — a stack. Read the text once, left to right:

- an **opener** `(`, `[` or `{`: push it;
- a **closer** `)`, `]` or `}`: the stack must be **non-empty**, and the opener you pop must be its **partner**;
- **anything else**: ignore it;
- at the **end of the text**: the stack must be **empty**.

3.2 Three checks, three ways to fail

Each check catches one way a text can be unbalanced. Leave any one out and six of the seven test cases still pass.

Input	What happens	The check that catches it
<code>([])</code>	<code>)</code> pops <code>[</code> — not its partner	the partner check
<code>)</code>	a closer arrives with nothing on the stack	the non-empty check
<code>(</code>	the text ends with <code>(</code> still on the stack	the empty-at-the-end check

3.3 Draw it

Trace `(a[b]{c})` on paper. After every character, write the stack (bottom on the left). You should find the letters change nothing and the stack is never taller than two:

```
(  ['(']
a  ['(']
[  ['(', '[']
b  ['(', '[']
]  ['(']
{  ['(', '{']
c  ['(', '{']
}  ['(']
```

```
)    []                <- empty at the end: balanced
```

Checkpoint 5. Predict `is_balanced` for each, and for each `False` name the check that decides it: `"a)(b"`, `"[]"`, `"((a)"`, `"{[()]}"`, `"{}x"`, `"{}"`.

3.4 Write it

1. Build a small dictionary mapping each closer to its opener: `)` to `(`, `]` to `[`, `}` to `{`. (A dictionary as a **lookup table** is fine; the storage rule is about the structure's working storage, which is the Stack.)
2. Make an empty Stack for the openers.
3. For each character: if it is an opener, push it. If it is a closer: if the stack is empty, return `False`; pop, and if what you popped is not the partner, return `False`.
4. After the loop, return whether the stack is empty.

The test compares with `is True` and `is False`, so return real booleans: `openers.is_empty()` is one.

Cost: every character is looked at once, and each push and pop is $O(1)$: $O(n)$ time, and $O(n)$ extra space in the worst case `(( (( (...)`.

3.5 Test it

```
pytest tests/test_stack_queue.py -v -k balanced
```

Seven cases of `test_is_balanced`: `"", "()", "(a[b]{c})"` and `"no brackets at all"` are balanced; `"([)]"`, `"("` and `)"` are not — one for each row of the table in 3.2.

3.6 When it fails

No partner check — any closer just pops:

```
E  AssertionError: assert True is False
E    +  where True = is_balanced('([)]')
```

No non-empty check — the closer pops an empty stack, and your own pop raises:

```
E  IndexError: pop from empty stack
```

The failing case is `test_is_balanced()-False`. An `IndexError` from inside `is_balanced` is never the right answer to “is this balanced?”.

No check at the end — return `True` after the loop:

```
E  AssertionError: assert True is False
E    +  where True = is_balanced('(')
```

Read the test ID in the FAILED line: the input in square brackets tells you which of the three checks is missing.

Part 4 — `evaluate_postfix(tokens)`, in `dsa/translation.py`

4.1 The idea

Lecture 06, “Application 2: Postfix”. In postfix each operator comes **after** its two operands, so there are no parentheses and no precedence rules. One stack of values evaluates it:

- a **number**: push it;
- an **operator**: pop **two** values, apply the operator, push the result;
- at the **end**: exactly one value is left, and it is the answer.

Postfix comes before shunting-yard in this lab, as in the lecture, because it is the easier of the two and because, once it works, it can check every answer your `infix_to_postfix` gives.

4.2 The operand-order trap

For `8 3 -`, the first value popped is 3 and the second is 8, and the answer is `8 - 3`: **the second value popped is the left operand**. Written with names:

```
right = values.pop()      # popped first
left  = values.pop()      # popped second
```

Get it the other way round and `+` and `*` still work, because they commute; `-` and `/` silently give wrong answers. Name your variables `left` and `right`, not `a` and `b`, and the bug has nowhere to hide.

4.3 Draw it

Lecture 06 traced `3 4 2 * +`. Trace `8 3 - 2 -` the same way on paper, as a table: token, action, stack (top on the right).

Checkpoint 6. (a) What is `8 3 - 2 -`? (b) What would a version with the operands swapped give for the same tokens? (c) What does `evaluate_postfix(['8', '2', '/'])` return — and why does the test's `== 4` still pass?

4.4 Write it

1. Add `from dsa.stack import Stack` to `dsa/translation.py` (at the top, or inside the function). `evaluate_postfix` uses your `Stack`, not a list.
2. Make an empty `Stack` of values.
3. For each token:

- if it is one of + - * /: if fewer than two values are on the stack, raise `ValueError`; otherwise pop right, pop left, compute left <op> right, and push the result;
 - otherwise it is a number: convert the string and push it. Use `int` unless the token contains a decimal point, then `float` — so `['3']` gives 3.
4. After the loop, if the stack does not hold **exactly one** value, raise `ValueError`. Otherwise pop it and return it.

Hints:

- Division is `/`, true division: `7 / 2` is 3.5. Do **not** use `//`, which gives 3 — `test_evaluate_postfix_divides_exactly` is there to catch it.
- You do not need to handle division by zero yourself: Python's `/` raises `ZeroDivisionError`, which is exactly what the docstring asks for.
- Check the count **before** popping. Otherwise an operator with too few operands raises your stack's `IndexError`, not `ValueError`.

4.5 Test it

```
pytest tests/test_translation.py -v -k evaluate_postfix
```

That runs the 11 tests of this week: `test_evaluate_postfix` (4 cases), `test_evaluate_postfix_opera`, `test_evaluate_postfix_divides_exactly`, `test_evaluate_postfix_rejects_malformed_inp` (4 cases: `[], ['3', '4'], ['+'], ['3', '+']`) and `test_evaluate_postfix_division_by_zero`. It is the command Lecture 06's Homework 6 gives.

Do not shorten it to `-k postfix`. That selects **12** tests: the extra one, `test_postfix_pairs_with_the`, calls `tokenize` — Week 15 — and fails with `NotImplementedError` until then, even with a perfect `evaluate_postfix`. Part 6 does the same check with `split()` instead.

4.6 When it fails

Operands swapped — three tests fail, not one:

```
E AssertionError: assert -2 == 2
E + where -2 = evaluate_postfix(['5', '3', '-'])
```

```
E AssertionError: assert 0.2857142857142857 == 3.5
E + where 0.2857142857142857 = evaluate_postfix(['7', '2', '/'])
```

and, in `test_evaluate_postfix_division_by_zero`, `1 0 /` computes `0 / 1`:

```
E Failed: DID NOT RAISE ZeroDivisionError
```

No “fewer than two” check — `['+']` and `['3', '+']` pop an empty stack:

```
E   IndexError: pop from empty stack
```

The test wanted `ValueError`: malformed input is the caller's mistake, and the docstring says which error reports it.

No “exactly one at the end” check — `['3', '4']` returns 4:

```
E   Failed: DID NOT RAISE ValueError
```

and `[]` pops an empty stack at the end: `IndexError: pop from empty stack`.

Floor division — `//` instead of `/`:

```
E   AssertionError: assert 3 == 3.5
E       + where 3 = evaluate_postfix(['7', '2', '/'])
```

Part 5 — `infix_to_postfix(tokens)`: the shunting-yard algorithm

5.1 The idea

Lecture 06, “Application 3: Infix to Postfix”. Read the infix tokens left to right, with an **output list** and an **operator stack**:

1. **Number** → output.
2. **Operator** → first pop to the output every operator on the stack with **higher or equal precedence**, stopping at a `(`; then push this one.
3. `(` → push.
4. `)` → pop to the output until the `(`; discard the `(`.
5. **End** → pop everything left to the output.

Precedence: `*` and `/` above `+` and `-`. Numbers go straight through; operators wait on the siding until a later operator of lower or equal precedence, a `)` or the end of the input lets them out.

Read rule 2 to the end: “stopping at a `(`”. A `(` on the stack is **not an operator** — it has no precedence, and it is a barrier. Operators inside a pair of parentheses must never leave past the `(`; only rule 4 removes it.

5.2 Trace before you code

Do these on paper **before** you open the editor. Use three columns — token, output, stack (top on the right) — exactly like the two traces in the lecture.

8 - 3 - 2 checks the “equal precedence pops” rule:

Token	Output	Stack
8	8	
-	8	-
3	8 3	-
-	8 3 -	- ← the first - has equal precedence: it leaves first
2	8 3 - 2	-
end	8 3 - 2 -	

Evaluate it: 8 3 - is 5, then 5 2 - is 3 — (8 - 3) - 2, left associative, as arithmetic requires.

2 * 3 + 4 checks that a lower-precedence operator empties the higher ones first:

Token	Output	Stack
2	2	
*	2	*
3	2 3	*
+	2 3 *	+ ← * is higher: it leaves, then + goes on
4	2 3 * 4	+
end	2 3 * 4 +	

Checkpoint 7. Trace (1 + 2) * (3 - 4) / 5 as a table. What is the postfix, and what value does your evaluate_postfix give for it?

5.3 Write it

1. A precedence table: a small dictionary {'+': 1, '-': 1, '*': 2, '/': 2}.
2. output is an empty Python list (it is the result); operators is an empty Stack.
3. For each token:
 - **operator**: while the stack is not empty, **and** its top is an operator (not (, **and** the top's precedence is >= this token's, pop the top to output. Then push the token.
 - (: push it.
 -): while the top is not (, pop to output. Then pop the (and throw it away.
 - **anything else** is a number: append it to output.
4. At the end, pop everything left on the stack to output, and return output.

Every token is pushed at most once and popped at most once: $O(n)$.

Checkpoint 8. A student writes > instead of >= in step 3. What does their function return for 8 - 3 - 2? What does that postfix evaluate to? Would test_infix_to_postfix alone catch it?

5.4 Test it

```
pytest tests/test_stack_queue.py -v -k infix
```

Two tests. `test_infix_to_postfix` has the two cases traced in the lecture, $3 + 4 * 2$ and $(3 + 4) * 2$. `test_infix_to_postfix_is_left_associative` has $8 - 3 - 2$ and $8 / 4 / 2$ — the “equal precedence pops” rule, which the first test cannot see (Checkpoint 8). Check your trace of Checkpoint 7 at the REPL too; no test has two pairs of parentheses.

5.5 When it fails

The (is treated as an operator — the “is it an operator?” test is missing from the while condition, so the precedence table is asked about (:

```
E   KeyError: '('
```

It happens in the second case, when `+` arrives with `(` on top of the stack.

> **instead of `>=`** — equal precedence stays on the stack, and the operators group from the right:

```
E   AssertionError: assert ['8', '3', '2', '-', '-'] == ['8', '3',  
↪  '-', '2', '-']  
E       At index 2 diff: '2' != '-'
```

The failing test is `test_infix_to_postfix_is_left_associative`; the other one still passes.

Nothing flushed at the end — the operators are still on the siding, and both tests fail. The first says:

```
E   AssertionError: assert ['3', '4', '2'] == ['3', '4', '2', '*', '+']
```

The (is not discarded — rule 4 pops to the `(` but leaves it there, and the end-of-input flush puts it into the output:

```
E   AssertionError: assert ['3', '4', '+', '2', '*', '('] == ['3',  
↪  '4', '+', '2', '*']  
E       Left contains one more item: '('
```

Postfix never contains a parenthesis. If one appears in your output, rule 4 is wrong.

Part 6 — The whole calculator

Lecture 06: “Infix → (shunting-yard) → postfix → (evaluation) → a value: two passes, two stacks, $O(n)$, no tree and no recursion.” You have both halves. Join them:

```

from dsa.stack import infix_to_postfix
from dsa.translation import evaluate_postfix

def calc(expression):
    tokens = expression.split()          # tokens must be separated by
    ↪ spaces
    postfix = infix_to_postfix(tokens)
    return postfix, evaluate_postfix(postfix)

for e in ['3 + 4 * 2', '( 3 + 4 ) * 2', '8 - 3 - 2',
          '2 * ( 3 + ( 4 - 1 ) )', '10 / 4']:
    print(e, '->', *calc(e))

```

```

3 + 4 * 2 -> ['3', '4', '2', '*', '+'] 11
( 3 + 4 ) * 2 -> ['3', '4', '+', '2', '*'] 14
8 - 3 - 2 -> ['8', '3', '-', '2', '-'] 3
2 * ( 3 + ( 4 - 1 ) ) -> ['2', '3', '4', '1', '-', '+', '*'] 12
10 / 4 -> ['10', '4', '/'] 2.5

```

This is what `test_postfix_pairs_with_the_stack_exercise` checks, with `split()` standing in for the Week 15 `tokenize`.

Checkpoint 9. Predict: (a) `calc('7 / 2 * 2')`; (b) `calc('3+4')` — written without spaces.

Checkpoint 9(b) is why Week 15 begins with a **tokenizer**: turning `'3+4'` or `'12*(3+4)'` into tokens is a job of its own. Week 15 then builds the other route from infix to a value — a recursive parser and an expression tree — and a test there checks that both routes agree.

Part 7 — Exercises at a glance

Method	Target cost	The trap	Tests
<code>Stack.push(value)</code>	$O(1)$ amortised	the top at the end of <code>self._items</code> — index 0 passes the tests and is $O(n)$	-k <code>lifo</code>
<code>Stack.pop()</code>	$O(1)$	pop the same end you push; <code>IndexError</code> when empty, not <code>None</code>	-k <code>"lifo or empty_behaviour"</code>

Method	Target cost	The trap	Tests
<code>Stack.peak()</code>	$O(1)$	read, do not remove; index <code>len - 1</code> , not <code>len</code> ; <code>IndexError</code> when empty	-k "peek or empty_behaviour"
<code>is_balanced(text)</code>	$O(n)$	three checks: partner, non-empty, empty at the end	-k balanced (7)
<code>evaluate_postfix(tokens)</code>	$O(n)$	second popped is the left operand; <code>ValueError</code> for malformed input; / not //	tests/test_translation.py -k evaluate_postfix (11)
<code>infix_to_postfix(tokens)</code>	$O(n)$	pop on higher or equal ; (stops the popping; discard the (; flush at the end	-k infix (2)

All 23 together:

```
pytest tests/test_stack_queue.py -v -k "lifo or peek or
↳ empty_behaviour or balanced or infix"
pytest tests/test_translation.py -v -k evaluate_postfix
```

Worked solutions are in `solutions/dsa/stack.py` and `solutions/dsa/translation.py` — for **after** your tests pass, to compare, as `solutions/README.md` explains. `pytest --solutions ...` runs the same tests on them if you are unsure whether a failure is in your code or in your reading of the test.

Part 8 — Take-home practice (not graded)

The question bank for this week is `docs/question-bank/week06-questions.md`, with answers in `week06-answers.md`.

1. **Write the code** — `practice/week06.py`, problems W6-C1 to W6-C5, checked by `pytest tests/test_practice_week06.py -v`. Use your Stack as the working storage in each. Start with **W6-C1** `reverse_words` (a warm-up), then **W6-C4** `tags_balanced` — this lab's Part 3 with tags instead of single characters — and **W6-C2** `MinStack`, whose hint is a second stack. **W6-C3** `next_greater` is the hardest and the most instructive: a loop inside a loop that is still $O(n)$ in total, because each index is pushed once and popped at most once — the same argument as shunting-yard.
2. **Trace** — on paper, then check at the REPL:

- **W6-T1:** a sequence of push, pop, peek and len — the Checkpoint 2 kind of question, as the exam asks it;
 - **W6-T2:** evaluate $5\ 1\ 2\ +\ 4\ *\ +\ 3\ -$, showing the stack after every token;
 - **W6-T5:** summing a nested list with a stack instead of recursion — in what order are the numbers printed, and why?
3. **Lecture 06, Homework 6, item 4:** sketch undo and redo for a text editor with two stacks. What happens to the redo stack when the user types after undoing?

Solutions: `solutions/practice/week06.py`, again for after.

Part 9 — Bridge to next week's lecture: queues

A stack touches **one** end, which is why either of your structures could keep its contract in $O(1)$. Lecture 07's queue is first in, first out: items join at the back and leave from the **front**. It needs **both** ends — and on your `DynamicArray`, the front is exactly where `FrontStack` lost its race in Part 2. `pop(0)` shifts everything.

Here is one small idea to try now, with nothing but this week's `Stack`. Push three people on one stack, then pour it, item by item, into a second:

```
from dsa.stack import Stack

inbox, outbox = Stack(), Stack()
for person in ['Ali', 'Mona', 'Omar']:           # arrive in this order
    inbox.push(person)
while not inbox.is_empty():                       # pour: pop one, push
    ↪ the other
    outbox.push(inbox.pop())
print(outbox)
print(outbox.pop(), outbox.pop(), outbox.pop())
```

```
Stack(['Omar', 'Mona', 'Ali']) # top = 'Ali'
Ali Mona Omar
```

Pouring reversed the order, and LIFO reversed is FIFO: they leave in the order they arrived. Lecture 07 turns this into a whole queue — and asks what it costs when people keep arriving while others leave. Before the lecture, think about one question: in your `DynamicArray`, what would it take to remove from the front **without** shifting anything?

Summary

Idea	The one line to keep
The Stack ADT	LIFO: push, pop, peek act on the top only, all $O(1)$.
Empty stack	pop and peek raise <code>IndexError</code> — your own, with a stack message.
Where the top goes	The end of the <code>DynamicArray</code> : append and pop there move nothing.
The dishonest stack	Top at index 0: same values, same tests pass, $O(n)$ per operation — measured quadratic.
Balanced brackets	Push openers; each closer needs a non-empty stack and its partner on top; empty at the end.
Postfix	Push numbers; an operator pops two — the second popped is the left operand .
Shunting-yard	Numbers to the output; operators wait on a stack and leave on higher or equal precedence, at <code>)</code> , or at the end.
The calculator	Infix → postfix → value: two passes, two stacks, $O(n)$.
The -k filter	-k also matches the file name: use the test names, not stack.

Lab 07 — Lab 07 — Queues and the Ring Buffer

How to use this lab. Read the matching section of Lecture 07 before each part. **Draw before you code:** every ring in this lab fits on a scrap of paper, and a ring you have traced by hand is one you can write. At each **Checkpoint**, write your prediction down first. Build `dsa/queue.py` one method at a time, and run the named tests after each part — not all of them at the end.

Duration	One 2-hour lab session, plus about 3 hours at home
You will write	<code>dsa/queue.py</code> — <code>SlowQueue.enqueue</code> , <code>SlowQueue.dequeue</code> , <code>CircularQueue.enqueue</code> , <code>CircularQueue.dequeue</code> , and the challenge: growing a full ring
Graded by	the queue half of <code>tests/test_stack_queue.py: pytest tests/test_stack_queue.py -v -k "fifo or dequeue or circular" (6 tests)</code>
Connects to	Lecture 07 — Queues; Lecture 04 (your <code>DynamicArray</code>); Lecture 08 — Searching

What you will be able to do

1. write a FIFO queue on your own `DynamicArray`, and say exactly which of its two operations is $O(n)$ and why;
2. trace a ring buffer on paper — block, head and size after every operation — and check the trace against the real object;
3. compute the back of a ring as $(\text{head} + \text{size}) \% \text{capacity}$, and wrap head with the same %;
4. explain why `head == tail` cannot tell a full ring from an empty one, and how `_size` settles it;
5. grow a full ring without scrambling it, by copying in **queue order**;
6. write your own test for behaviour the provided tests never reach;
7. measure the cost of one dequeue at several sizes, and read $O(n)$ against $O(1)$ off the plot.

Part 0 — Before you start

0.1 Environment and prerequisite

From the DSA27 folder, with the virtual environment active (the prompt starts with `(.venv)`):

```
pytest -m "not challenge" -q          # the environment check: must pass
pytest tests/test_dynamic_array.py -q # your Week 4 DynamicArray: 19
↪ passed
```

SlowQueue stands on **your** DynamicArray: its enqueue is your append and its dequeue is your `pop(0)`. If `test_dynamic_array.py` fails, fix that first — a queue built on a broken array fails in ways that look like queue bugs. In particular, `test_pop_on_empty_raises` must pass: your `pop` raises `IndexError` on an empty array, and this lab relies on it.

CircularQueue does **not** use DynamicArray. It stands directly on the course Array (`dsa/array.py`) — the fixed block of slots from Lecture 02, which cannot grow and has no negative indices.

0.2 Read the skeleton: what is given, what is yours

Open `dsa/queue.py`. Read all of it before you write anything.

Class	Given to you	Yours
SlowQueue	<code>__init__(values=())</code> — builds <code>self._items</code> , a DynamicArray; <code>__len__</code>	enqueue, dequeue
CircularQueue	<code>__init__(capacity=8)</code> — <code>_block</code> , <code>_capacity</code> , <code>_head</code> , <code>_size</code> ; <code>is_empty</code> , <code>is_full</code> , <code>__len__</code> , <code>__iter__</code> , <code>__repr__</code>	enqueue (and the growth challenge in its docstring), dequeue

Read `CircularQueue.__iter__` twice. It already contains the ring arithmetic you need:

```
def __iter__(self):
    for offset in range(self._size):
        yield self._block[(self._head + offset) % self._capacity]
```

The i -th item of the queue, counting from the front, lives in slot $(\text{head} + i) \% \text{capacity}$. Everything in this lab is that one line, used in three places.

Notice also what is **not** stored: there is no `_tail`. As the class docstring says, the slot the next enqueue writes to — the tail, $(\text{head} + \text{size}) \% \text{capacity}$ — is computed each time, not stored (Lecture 07, “Idea 2: wrap around”).

0.3 The storage rule, this week

A `dsa/` structure stores its data only in the course `Array`, in node objects, or in another `dsa/` structure — never in a Python `list`, `dict`, `set` or `collections.deque` (Lecture 02). Here:

- `SlowQueue` keeps everything in `self._items`, your `DynamicArray`. Do not add a list beside it.
- `CircularQueue` keeps everything in `self._block`, an `Array`. When it grows, the new block is another `Array` — not a list you convert later.
- Lists are still fine as **inputs and outputs**: `list(q)` in a test, or `SlowQueue([5, 6])`, is not storage.

0.4 The tests, and one warning about -k

The queue tests live in the same file as Week 6's stack tests. Four test functions cover the queues; two are run once for each class, so `pytest` reports **six** tests:

Test	Checks
<code>test_queue_is_fifo[SlowQueue], [CircularQueue]</code>	enqueue 1, 2, 3; dequeue gives 1 then 2; len is 1
<code>test_dequeue_on_empty_raises[SlowQueue], [CircularQueue]</code>	dequeueing a new, empty queue raises <code>IndexError</code>
<code>test_circular_queue_wraps_around</code>	capacity 3: fill, dequeue once, enqueue into the freed slot
<code>test_circular_queue_interleaved_operations</code>	capacity 4: ten enqueues and eight dequeues, wrapping twice

The command that runs exactly these six is:

```
pytest tests/test_stack_queue.py -v -k "fifo or dequeue or circular"
```

It is the same command as Lecture 07's "Exercises" slide. Why such a long filter, and not simply `-k queue`? `-k` matches against the file name as well as the test name, and the file is `test_stack_queue.py`. So a filter that contains `queue` (or `stack`) matches **every** test in the file: `-k queue` selects all 18, stack tests included. That is harmless if your Week 6 Stack passes, but it is not "the queue half". The smaller filters used in this lab avoid both words:

Filter	Selects
<code>-k SlowQueue</code>	the 2 <code>SlowQueue</code> tests
<code>-k circular</code>	the 4 <code>CircularQueue</code> tests
<code>-k wraps</code>	<code>test_circular_queue_wraps_around</code> only
<code>-k interleaved</code>	<code>test_circular_queue_interleaved_operations</code> only

Right now all six fail with `NotImplementedError`. That is the starting line.

Part 1 — SlowQueue: the obvious design

1.1 The idea

Lecture 07, “The obvious design”: the back of the queue is the end of the `DynamicArray` and the front is index 0. Enqueue is `append` — amortised $O(1)$. Dequeue is `pop(0)` — every remaining element shifts one slot left, $O(n)$. The class is named for its flaw, because in Part 7 you will measure it.

Could you swap the ends? Then dequeue would be `pop()` at the end, $O(1)$, but enqueue would be `insert_at(0, x)`, $O(n)$. On a plain array one of the two operations is always at the front. That is the whole difference from Week 6: a stack only ever touches one end.

1.2 Draw it

Draw a `DynamicArray` holding 5 6 7 in slots 0, 1, 2. Dequeue once. Draw the block again, with an arrow for every element that moved. How many arrows? For a queue of 1,000 items, how many?

1.3 Checkpoint

Checkpoint 1. Once your `SlowQueue` works, what does this print? And how many elements does that one dequeue move?

```
from dsa.queue import SlowQueue
q = SlowQueue([5, 6])
q.enqueue(7)
print(q.dequeue(), len(q))
print(q._items)
```

1.4 Write it

enqueue(value) — one call on `self._items`.

dequeue():

1. If the queue is empty, raise `IndexError` — the docstring’s last line, “Raises `IndexError` when empty”, and `test_dequeue_on_empty_raises` checks it.
2. Remove the item at index 0 and **return** it.

Hints:

- Your `DynamicArray.pop` already raises `IndexError` on an empty array, so the test passes even without step 1. Write the check anyway, with a message that names the queue — `IndexError("dequeue from empty queue")` — so that an error from your queue does not look like an error from the array underneath.

- Do not reach into `self._items._block`. `SlowQueue` uses the `DynamicArray` through its public methods only — that is the ADT boundary from Lecture 01.

1.5 Test it

```
pytest tests/test_stack_queue.py -v -k SlowQueue
```

Two tests: `test_queue_is_fifo[SlowQueue]` and `test_dequeue_on_empty_raises[SlowQueue]`.

1.6 When it fails

`SlowQueue` has no `__repr__`, so pytest shows it as `<dsa.queue.SlowQueue object at 0x...>`. Read the assert line, not the object.

What you see	Cause	Fix
<code>assert 3 == 1 in</code> <code>test_queue_is_fifo[SlowQueue]</code>	<code>self._items.pop()</code> — the new item leaves first: a stack, not a queue (W7-B1)	<code>pop(0)</code>
<code>assert None == 1</code>	you called <code>pop(0)</code> but forgot return	return what pop gives you
Failed: DID NOT RAISE <code>IndexError</code> in <code>test_dequeue_on_empty_raises[SlowQueue]</code>	if <code>len(self._items) == 0</code> : return <code>None</code> — a quiet <code>None</code> is not an error	<code>raise IndexError(...);</code> <code>None</code> could be a real value in the queue

Part 2 — The ring on paper

Do this part with a pencil before you touch `CircularQueue`.

2.1 The idea

Lecture 07, “Idea 1” and “Idea 2”. Instead of shifting everything left, keep a head index and move **it**. On its own that leaves dead slots behind the front; the ring fixes it by treating the slot after the last one as slot 0. Three numbers describe the whole state: head (the slot of the front item), size (how many items), and capacity (how many slots). The back is computed:

```
tail = (head + size) % capacity      # where the NEXT enqueue writes
```

2.2 Draw it: the lecture’s trace

Copy this table from Lecture 07, “Trace: capacity 4”, and check every row with the formula. Before “enqueue E”: `tail = (2 + 2) % 4 = 0` — the write wraps round to slot 0.

Operation	Block	head	size	Returns
start	— — — —	0	0	
enqueue A, B, C	A B C —	0	3	
dequeue	— B C —	1	2	A
dequeue	— — C —	2	1	B
enqueue D	— — C D	2	2	
enqueue E	E — C D	2	3	
enqueue F	E F C D	2	4	
dequeue	E F — D	3	3	C

Read the last block as a ring from head: D (slot 3), E (slot 0), F (slot 1). No element ever moved.

2.3 Checkpoint

Checkpoint 2. Trace `q = CircularQueue(capacity=4)` through the operations below. After **each** one, write the block, head and size. What does each dequeue return, and what are `list(q)`, `q._block`, `q._head` and `q._size` at the end?

```
enqueue 1, enqueue 2, enqueue 3, enqueue 4,
dequeue, dequeue, dequeue,
enqueue 5, enqueue 6,
dequeue
```

This trace makes **both** indices wrap: the back wraps on “enqueue 5”, and head wraps on the last dequeue.

2.4 Check a trace against the real object

Once Parts 3 and 4 are done, you can check any paper trace at the REPL. The fields are private by convention, not by force — reading them to debug is fine; writing to them from outside the class is not.

```
>>> from dsa.queue import CircularQueue
>>> q = CircularQueue(capacity=4)
>>> for v in 'ABC':
...     q.enqueue(v)
...
>>> q.dequeue(), q.dequeue()
('A', 'B')
>>> q._block, q._head, q._size
(Array([None, None, 'C', None]), 2, 1)
>>> list(q)
['C']
```

To **see** the ring, draw its block with `viz.draw.draw_array`. The front slot is highlighted, and the rest of the queue is shown in green:

```
import matplotlib.pyplot as plt
from viz.draw import draw_array

def show(q):
    cells = ['' if v is None else v for v in q._block]
    rest = [(q._head + i) % q._capacity for i in range(1, len(q))]
    tail = (q._head + len(q)) % q._capacity
    draw_array(cells, highlight=q._head if len(q) else None, done=rest,
               title=f'head={q._head} size={len(q)} tail={tail}')
    plt.show()
```

After “enqueue E” in the lecture’s trace, `show(q)` draws E _ C D with slot 2 highlighted, slots 3 and 0 green, and the title `head=2 size=3 tail=1`. Put `show` in the 07-queues notebook and call it after every step of Checkpoint 2.

Part 3 — CircularQueue.enqueue

3.1 The idea

Lecture 07, “The two operations”:

1. full? — decide what to do (below);
2. write value at $(\text{head} + \text{size}) \% \text{capacity}$;
3. `size += 1`.

Nothing moves, and head does not change: enqueue only touches the back.

What to do when full. Start with the simple, honest choice: `raise IndexError("queue is full")`. The provided tests never overfill a ring, so this passes them. Part 6 replaces it with growth.

3.2 Draw it

A ring of capacity 5 holds d in slot 3 and e in slot 4. Draw it. Mark head. Now mark the slot the next enqueue writes to.

3.3 Checkpoint

Checkpoint 3. A `CircularQueue` has capacity 5, `head = 3` and `size = 2`.

- (a) Which slots hold the queue, front first?
- (b) Which slot does the next enqueue write to?
- (c) A student writes `self._block[self._head + self._size] = value`, with no

% What happens on that enqueue?

3.4 Write it

Write the three steps above. Hints:

- Test fullness with `self._size == self._capacity` — or call the given `is_full()`. **Not** with `tail == head` (Part 5 explains why).
- Compute the slot **after** any full check, from the current fields.
- Keep it $O(1)$: no loop in enqueue.

3.5 Test it

`dequeue` is not written yet, so every `CircularQueue` test still fails at its first `dequeue`. Check `enqueue` at the REPL instead — this is exactly what the given `__iter__` is for:

```
>>> q = CircularQueue(capacity=3)
>>> for v in (1, 2, 3):
...     q.enqueue(v)
...
>>> list(q), q.is_full(), q._block
([1, 2, 3], True, Array([1, 2, 3]))
>>> q.enqueue(4)
Traceback (most recent call last):
...
IndexError: queue is full
```

3.6 When it fails

These are the symptoms once `dequeue` is written too, when you run `pytest tests/test_stack_queue -v -k circular`:

What you see	Cause	Fix
<code>IndexError: Array index 3 out of range for length 3 (valid: 0..2; no negative indices) in test_circular_queue_wraps_around, raised from dsa/array.py</code>	the slot is <code>head + size</code> with no <code>% capacity</code> : after one <code>dequeue</code> , the fourth value is written to slot 3 of a 3-slot block	<code>(self._head + self._size) % self._capacity</code>
<code>IndexError: queue is full on the first enqueue, in three tests</code>	fullness tested as <code>tail == head</code> — also true for an empty ring (W7-B3)	test <code>self._size == self._capacity</code>

What you see	Cause	Fix
your own empty-queue IndexError (for example dequeue from empty queue) in test_queue_is_fifo[CircularQueue], although three values were enqueued	forgot <code>self._size += 1</code> : every enqueue writes to the same slot, and the queue stays “empty”	increment <code>_size</code> after the write
assert 3 == 1 in test_queue_is_fifo[CircularQueue], instead of 1	writing at <code>self._head</code> instead of the tail: each enqueue overwrites the front	write at <code>(head + size) % capacity</code>

Part 4 — CircularQueue.dequeue

4.1 The idea

Lecture 07, “The two operations”:

1. empty? — raise `IndexError`;
2. read the value at head; clear that slot;
3. `head = (head + 1) % capacity`; `size -= 1`.

Then return the value.

Why clear the slot? The queue is correct without it: the slot is outside the live part of the ring and will be overwritten later. But until then the block keeps a reference to the dequeued object, and Python cannot free it. It is the same care as your `DynamicArray.pop` setting the vacated slot to `None`. No test checks it; the TA may ask about it.

4.2 Draw it

Take the lecture’s trace at “enqueue F” (E F C D, head 2, size 4). Dequeue four times on paper. Write head after each: it should go 3, 0, 1, 2 — and the block should end empty.

4.3 Checkpoint

Checkpoint 4. A student writes step 3 as `self._head += 1`, with no %, and everything else correctly. Of the four `CircularQueue` tests selected by `-k circular`, which fail, and with what message? Why does `test_circular_queue_wraps_around` not catch it?

4.4 Write it

Write the steps above. Hints:

- Read the value into a local variable **before** you clear the slot.
- Check emptiness with `self._size == 0` (or `is_empty()`), first thing.
- No loop. If you find yourself writing `for` in `dequeue`, stop and redraw.

4.5 Test it

```
pytest tests/test_stack_queue.py -v -k circular
```

Four tests: `test_queue_is_fifo[CircularQueue]`, `test_dequeue_on_empty_raises[CircularQueue]`, `test_circular_queue_wraps_around` and `test_circular_queue_interleaved_operations`. Then the whole queue half:

```
pytest tests/test_stack_queue.py -v -k "fifo or dequeue or circular"
```

Six passed. Now go back to Checkpoint 2 and check your paper trace against the real object, as in Part 2.4.

4.6 When it fails

What you see	Cause	Fix
IndexError: Array index 4 out of range for length 4 (valid: 0..3; no negative indices) in test_circular_queue_interleaved_operations only, raised by __getitem__	head never wraps (Checkpoint 4, W7-B2)	<code>self._head = (self._head + 1) % self._capacity</code>
assert None == 1 in three tests Failed: DID NOT RAISE IndexError in test_dequeue_on_empty_raises assert 3 == 1 where 3 = len(CircularQueue([3, None, None]))	the slot was cleared before it was read no emptiness check: an empty ring returns the None instead of CircularQueue._1 forgot <code>self._size -= 1</code> : the queue never shrinks, and <code>__iter__</code> now reads cleared slots	read into a local first, then clear <code>if self._size == 0: raise IndexError(...)</code> <code>decrement_size</code>

Part 5 — Full or empty?

5.1 The idea

Lecture 07, “Full or empty?”. Compute the tail at the start of a queue’s life and again when it is full. In both, `tail == head`: the next write would go to the front’s slot. A ring that stored only head and tail could not tell the two apart. `dsa/queue.py` stores `_size` instead of a tail, which is why `is_empty(size == 0)` and `is_full(size == capacity)` are one-liners. The other two fixes — leave one slot unused, or keep a flag — also work; having none is the bug.

5.2 Checkpoint

Checkpoint 5. For each ring, give head, the computed tail, `is_empty()` and `is_full()`:

```
a = CircularQueue(3)                                # new

b = CircularQueue(3)
for v in (1, 2, 3):
    b.enqueue(v)
b.dequeue()
b.enqueue(4)
```

What is `b._block`? What would `head == tail` say about each?

This is also the exam’s favourite bug (W7-B3): an enqueue that tests `tail == head` for fullness refuses the very first value.

Part 6 — Challenge: grow when full

6.1 Why bother

The docstring of `CircularQueue.enqueue` calls it a challenge, and the tests do not require it. Do it anyway: Week 11’s `level_order` and Week 14’s `bfs` use your `CircularQueue`, and neither knows in advance how many items it will hold.

The plan is Lecture 04’s: when full, allocate a bigger Array — double the capacity — copy, and switch. Doubling keeps enqueue **amortised** $O(1)$: over n enqueues the copies total less than $2n$.

6.2 The trap: the naive copy

Lecture 07, “Growing: the trap”. The obvious copy is the one you wrote in `DynamicArray._resize`: slot i of the old block to slot i of the new. On a ring that has **wrapped**, that is wrong.

Take the full ring from the lecture’s trace after “enqueue F”: block `E F C D`, head 2, size 4. The naive copy gives the block `E F C D _ _ _ _`, with head still 2 and capacity now 8. Reading from head: `C, D`, then two empty slots. `E` and `F` are “behind” the front and lost.

Checkpoint 6. A student grows with the naive copy (slot i to slot i , head unchanged, capacity updated to 8), then enqueues G into the grown ring above. Where is G written, and what is `list(q)`?

If the ring has **not** wrapped yet (`head = 0`), the naive copy happens to work. That is exactly why this bug survives casual testing — and why the test you write in 6.4 wraps first.

6.3 Write it: unroll in queue order

Write a helper `_grow(self, capacity)` and call it from `enqueue` when the ring is full, instead of raising.

1. Make a new Array of the new capacity.
2. For i from 0 to `size - 1`: the i -th item of the queue lives at $(\text{head} + i) \% \text{old_capacity}$ — the same formula as `__iter__`. Put it in slot i of the new block.
3. Switch: the block becomes the new one, `head` becomes **0**, and `capacity` becomes the new capacity. Three fields, all three.

Hints:

- **Grow first, then compute the slot.** In `enqueue`, the full check and the growth come before $(\text{head} + \text{size}) \% \text{capacity}$. Compute the slot from the old capacity and you write into the wrong place in the new block.
- **Capacity 0.** `CircularQueue(0)` is legal. Doubling 0 gives 0, and the next % is `ZeroDivisionError: integer modulo by zero`. Grow to $\max(1, 2 * \text{capacity})$.
- `size` does not change during growth: the same items, in a bigger block.

6.4 Test it: write your own test

The provided tests cannot check this — they never overfill a ring. So write the test yourself. Create `test_my_queue.py` in the DSA27 folder (**not** in `tests/`, which stays exactly as it was given):

```
from dsa.queue import CircularQueue

def test_grows_after_wrapping():
    q = CircularQueue(capacity=4)
    for value in "ABC":
        q.enqueue(value)
    q.dequeue()
    q.dequeue()                # head is now 2
    for value in "DEF":
        q.enqueue(value)      # E and F wrap round to slots 0 and 1
    assert q.is_full()
    q.enqueue("G")            # must grow, not raise
    assert list(q) == ["C", "D", "E", "F", "G"]
    assert [q.dequeue() for _ in range(5)] == ["C", "D", "E", "F", "G"]
```

```

    assert q.is_empty()

def test_grows_many_times_in_order():
    q = CircularQueue(capacity=2)
    expected = []
    ↪ test
    for value in range(100):
        q.enqueue(value)
        expected.append(value)
        if value % 3 == 0:
            assert q.dequeue() == expected.pop(0)
    assert list(q) == expected

```

```

pytest test_my_queue.py -v
pytest tests/test_stack_queue.py -v -k "fifo or dequeue or circular"

```

The first test is the lecture's trace plus one enqueue; the second grows a ring six times, from 2 to 128 slots, with dequeues in between — so head is never 0 when the ring fills, and every one of those growths happens on a wrapped ring. Run the provided tests again afterwards: growth must not break anything that worked.

6.5 When it fails

What you see in test_grows_after_wrapping	Cause
<pre> IndexError: queue is full assert ['C', 'D', None, None, 'G'] == ['C', 'D', 'E', 'F', 'G'] assert ['E', 'F', None, None, 'G'] == [...] assert ['G', 'D', 'E', 'F', 'G'] == [...] assert ['C', 'D', 'G', 'F', None] == [...] </pre>	enqueue still raises instead of calling <code>_grow</code> the naive copy: slot <code>i</code> to slot <code>i</code>, head kept copied in queue order, but forgot <code>head = 0</code> forgot to update capacity: every % still wraps at 4, so G overwrote slot 0 of the new block the slot was computed before growing, with the old capacity

Part 7 — Measure it

7.1 What the lecture claims

Lecture 07, “Measured”: one `SlowQueue` dequeue costs $O(n)$ — a straight line of slope 1 on a log-log plot — while one `CircularQueue` dequeue costs $O(1)$, a flat line. You can now check that on your own code.

7.2 The snippet

Run this in notebooks/07-queues.ipynb (the notebook’s “Try it” cell imports `measure` and `plot_growth` for you):

```
from dsa.queue import CircularQueue, SlowQueue
from viz.complexity import measure, plot_growth

OPS = 200                                # dequeues timed per run

def make_slow(n):
    return SlowQueue(range(n))            # n items, built before the
    ↪ clock starts

def make_ring(n):
    q = CircularQueue(capacity=n)
    for i in range(n):
        q.enqueue(i)
    return q

def churn(q):
    for _ in range(OPS):                  # size stays n: one out, one in
        q.enqueue(q.dequeue())

sizes = [250, 500, 1000, 2000, 4000]
slow = measure(churn, sizes, make_slow)
ring = measure(churn, sizes, make_ring)

def per_op(result):
    ns, secs = result
    return ns, [s / OPS for s in secs]

plot_growth({"SlowQueue": per_op(slow), "CircularQueue":
    ↪ per_op(ring)},
            reference=["n", "1"], loglog=True, title="One dequeue,
    ↪ queue of n items")
```

Points to notice:

- `measure` builds each input with `make_slow` or `make_ring` **outside** the timing, so only the 200 dequeue-and-enqueue pairs are timed.

- One dequeue on the ring is far too quick to time on its own, so churn does 200 and `per_op` divides. Each pair enqueues what it dequeued, so the queue holds n items throughout — the ring never needs to grow.
- Sizes from 250 to 4,000 keep the `SlowQueue` runs short: measure runs each size four times (one warm-up, then three timed), and the whole cell takes well under a minute. Every doubling of n doubles the `SlowQueue` part of that wait, so add larger sizes one at a time.

7.3 What to expect

On a log-log plot, the `SlowQueue` points lie close to the dashed $O(n)$ line, rising from a fraction of a millisecond to several milliseconds per dequeue. The `CircularQueue` points sit at a few microseconds — about a hundred times lower at $n = 250$, and over a thousand times lower at $n = 4,000$ — and do not climb as n grows. Timings vary with your machine and with what else is running; single points wobble, the **shape** does not.

Checkpoint 7. Without running anything: when n doubles from 2,000 to 4,000, by roughly what factor should the time per `SlowQueue` dequeue grow? Per `CircularQueue` dequeue? And if one `SlowQueue` dequeue at $n = 4,000$ took 5 ms, roughly how long would one take at $n = 1,000,000$?

Homework item 2 of Lecture 07 asks for the **total** cost — n enqueues followed by n dequeues — where the `SlowQueue` is $O(n^2)$ and the ring $O(n)$. That version is much slower for the `SlowQueue`; keep n at 4,000 or below there too.

Part 8 — Exercises at a glance

Method	Target cost	The trap	Tests
<code>SlowQueue.enqueue</code>	amortised $O(1)$	none — one append	-k <code>SlowQueue</code>
<code>SlowQueue.dequeue</code>	$O(n)$, on purpose	<code>pop()</code> instead of <code>pop(0)</code> ; returning <code>None</code> when empty	-k <code>SlowQueue</code>
<code>CircularQueue.enqueue</code>	$O(1)$	the % on the tail; testing full by <code>tail == head</code> ; forgetting <code>size += 1</code>	-k <code>circular</code>
<code>CircularQueue.dequeue</code>	$O(1)$	head must wrap; read before clearing; <code>IndexError</code> when empty	-k <code>circular</code>
<i>challenge:</i> grow when full	amortised $O(1)$	copy in queue order; then <code>head = 0</code> and the new capacity	your <code>test_my_queue.py</code>

All six provided tests:

```
pytest tests/test_stack_queue.py -v -k "fifo or dequeue or circular"
```

Part 9 — Take-home practice (not graded)

The Week 7 question bank is `docs/question-bank/week07-questions.md`, with answers in `week07-answers.md`. Do these after the lab:

1. **W7-S1** — draw the block, head and size of a `CircularQueue(capacity=5)` through ten operations. Check your drawing with `show(q)` from Part 2.4.
2. **W7-T1** — a capacity-4 trace that prints `is_full()` and `list(q)`. Predict, then run it.
3. **W7-S2** — draw what the missing % of Checkpoint 3 does inside `test_circular_queue_wraps_around`.
4. The coding problems **W7-C1 to W7-C5** in `practice/week07.py` — a queue from two stacks, Josephus, moving averages on a `CircularQueue(k)`, reversing the first `k` values, and the binary numbers generated breadth-first. They use your `CircularQueue` and your Week 6 Stack as working storage:

```
pytest tests/test_practice_week07.py -v
```

Worked solutions are in `solutions/dsa/queue.py` and `solutions/practice/week07.py` — for **after** you have tried (`solutions/README.md` says how to use them without wasting them). `pytest --solutions tests/test_stack_queue.py -v` runs the tests on the reference, if you are unsure whether a failure is your code or your reading of the test.

Part 10 — Bridge to next week's lecture: searching

Lecture 08 is searching, and its exercises are in `dsa/searching.py`, graded by `tests/test_searching.py`. This week you made a structure fast by changing how it is **stored**. Next week you make a question fast — “is `x` in here?” — by changing how the data is **ordered**.

10.1 Halving

Linear search looks at every value: $O(n)$. Binary search looks at the middle of a **sorted** array and throws away the half the target cannot be in. How many halvings take a million values down to one?

```
>>> n, halvings = 1_000_000, 0
>>> while n > 1:
...     n //= 2
...     halvings += 1
... 
```

```
>>> halvings
19
```

Nineteen halvings, and one more comparison to settle the last value: at most 20 comparisons for a million items (Lecture 08, “How many comparisons?”). The `SlowQueue` of Part 7 did a million moves for **one** dequeue at that size.

10.2 The precondition

Binary search works **only** because the values are sorted. Nothing checks this for you — checking would itself cost $O(n)$. Python’s `bisect` module does the halving; watch what happens when the promise is broken:

```
>>> from bisect import bisect_left
>>> bisect_left([3, 8, 12, 19, 24, 31], 19)      # sorted: 19 is at
↪ index 3
3
>>> unsorted = [24, 3, 31, 8, 19, 12]
```

Checkpoint 8. Predict `bisect_left(unsorted, 19)`. (It returns the index where 19 would be inserted to keep the list sorted — assuming it is sorted.)

No error, no warning — just a wrong answer. In `dsa/searching.py`, write `linear_search` first, then `binary_search` from the steps in Lecture 08, “The algorithm, and the promise it keeps”:

```
pytest tests/test_searching.py -v -k "unsorted or (binary_search and
↪ not recursive)"
```

That filter selects 12 of the file’s 56 tests: the linear-search test on unsorted data and the eleven that check the loop form of `binary_search`. (A plainer `-k "linear or binary"` also picks up jump, exponential and interpolation search, through `test_agrees_with_linear_search_on_random_data`.)

Summary

Idea	The one line to keep
Queue	FIFO: enqueue at the back, dequeue at the front; empty dequeue raises <code>IndexError</code> .
<code>SlowQueue</code>	<code>append</code> and <code>pop(0)</code> on your <code>DynamicArray</code> : correct, and $O(n)$ per dequeue.

Idea	The one line to keep
The ring	head moves, nothing shifts; the i -th item is at $(\text{head} + i) \% \text{capacity}$.
Enqueue	write at $(\text{head} + \text{size}) \% \text{capacity}$, then $\text{size} += 1$. $O(1)$.
Dequeue	read at head, clear it, $\text{head} = (\text{head} + 1) \% \text{capacity}$, $\text{size} -= 1$. $O(1)$.
Full or empty	$\text{head} == \text{tail}$ means either; size tells them apart.
Growing	copy in queue order, then $\text{head} = 0$ and the new capacity; double, for amortised $O(1)$.
Your own test	the provided tests never overflow a ring — a test that wraps first is yours to write.
Measured	per dequeue: <code>SlowQueue</code> a line of slope 1, <code>CircularQueue</code> flat.
Next week	binary search: $O(\log n)$, but only on sorted data.

Lab 08 — Lab 08 — Eight Ways to Search

How to use this lab. The routine from Week 4: read the lecture section named at the top of each part, **draw before you code** — here that means a table of `lo`, `hi` and `mid` on paper — and predict at each **Checkpoint**. Then write one function, run the tests named in its part, and only then move on. This is the midterm week, so the lab is kept tight: eight short functions, and one idea — a sorted array can be searched without looking at most of it. Parts 1–5 are the session; Parts 6–9 are for home.

Duration	One 2-hour lab session, plus about 3 hours at home
You will write	<code>dsa/searching.py</code> — <code>linear_search</code> , <code>binary_search</code> , <code>binary_search_recursive</code> , <code>lower_bound</code> , <code>upper_bound</code> , <code>jump_search</code> , <code>exponential_search</code> , <code>interpolation_search</code> ; and, for yourself, a small <code>CountingReads</code> class
Graded by	<code>tests/test_searching.py</code> (56 tests)
Connects to	Lecture 08 — Searching; and Lecture 09, where sorting makes these searches possible

What you will be able to do

1. trace a binary search as a table of `lo`, `hi` and `mid`, for a hit and for a miss, and say where `lo` ends up;
2. write `binary_search` from its invariant, and test it by hand on sizes 0, 1 and 2 before running `pytest`;
3. explain exactly which tests a `lo < hi` bug fails, and why the others pass;
4. write the recursive version without slicing, and say what a slice costs;
5. write `lower_bound` and `upper_bound` as one half-open loop that differs in a single comparison, and use them to count duplicates;
6. write `jump`, `exponential` and `interpolation search`, including their edge cases;
7. count the reads each search makes with a sequence class of your own, and show interpolation search collapsing on skewed data.

Part 0 — Before you start

0.1 Environment

From the DSA27 folder, with the virtual environment active (the prompt starts with `(.venv)`):

```
git pull
pytest -m "not challenge" -q          # the environment check: must pass
pytest tests/test_searching.py -q    # 56 failures: nothing written
↪ yet
```

The 56 failures are all `NotImplementedError` — that is the starting point.

0.2 Prerequisites

None from `dsa/`. Unlike the stack and the queues, this week's functions stand on nothing you built: they take an ordinary Python list as **input**, read it, and return an index. You need while loops with two changing variables (Lab 01), integer division `//` (Lab 01), and recursion with a default argument (Lecture 03).

0.3 Read the skeleton

Open `dsa/searching.py`. There is no class this week, so there is nothing **given** except the module docstring, `__all__`, and the eight docstrings. All eight function bodies are **yours**. Read every docstring before you write a line: each one states the contract and the target cost.

Function	Returns	Input must be
<code>linear_search</code>	index of the first match, or <code>-1</code>	anything
<code>binary_search</code>	an index of target, or <code>-1</code>	sorted
<code>binary_search_recursive</code>	the same; takes <code>lo=0</code> , <code>hi=None</code> too	sorted
<code>lower_bound</code>	first <code>i</code> with <code>values[i] >=</code> target; may be <code>len(values)</code>	sorted
<code>upper_bound</code>	first <code>i</code> with <code>values[i] ></code> target; may be <code>len(values)</code>	sorted
<code>jump_search</code> , <code>exponential_search</code> , <code>interpolation_search</code>	index, or <code>-1</code>	sorted

0.4 The storage rule, this week

The storage rule says a `dsa/` **structure** keeps its data only in the course Array, in nodes, or in another `dsa/` structure. These are functions, not structures: they store nothing, and a list as input is allowed (Lecture 08, “This Week”). Two rules follow from the point of the week instead:

- **Do not copy the input.** No `values[a:b]`, no `sorted(values)`, no `list(values)`. A copy is $O(n)$ and destroys the $O(\log n)$ you are writing.
 - **Do not call the library to do the search.** No `in`, `values.index`, or `bisect` inside `dsa/searching.py`. You will use `bisect` in Part 8 — as a check, after your own bounds work.
-

Part 1 — `linear_search`: the baseline

Lecture 08, “Linear Search”. Look at every element in order; return the index of the **first** one equal to target; if the loop ends, return `-1`. It is the only function this week that must work on unsorted data.

Write it. A `for i in range(len(values))` loop, one comparison, and the `return -1` **after** the loop. Read each element as `values[i]`: every search this week touches its input only through `len(values)` and `values[i]`, and that is exactly what Part 7 will count.

Test it.

```
pytest tests/test_searching.py -v -k "linear_search and not agrees"
```

This runs two tests: `test_linear_search_works_on_unsorted_data` and `test_linear_search_returns_the_first_match` which searches `[4, 7, 7, 1, 7]` for 7 and expects 1. Do **not** use plain `-k linear`: it selects 8 tests, because `test_agrees_with_linear_search_on_random_data` (five of them) and `test_binary_search_does_not_scan_linearly` contain the word too.

When it fails.

- `assert -1 == 2` for `[5, 2, 9]` and 9 (and `assert -1 == 1` in the first-match test): the `return -1` is **inside** the loop, so the function gives up after looking at one element. Dedent it.
 - `assert 4 == 1` in `test_linear_search_returns_the_first_match`: the loop remembers `i` and keeps going, so it returns the **last** match. Return as soon as you find one.
-

Part 2 — `binary_search`: halve what is left

Lecture 08, “The algorithm, and the promise it keeps”. The lecture gives the steps; your job is to make every `+ 1` and `<=` agree with one decision: **lo and hi are both inclusive**, so

the part still in play is `values[lo..hi]`. The docstring of `binary_search` names the four classic bugs from the lecture; Part 2.5 shows what each one looks like when you run the tests.

Invariant: if target is anywhere in values, it is in values[lo..hi].

2.1 Draw it

On paper, before any code. Use the lecture's table format — one row per turn of the loop, and a last row showing $lo > hi$ for a miss:

```
lo    hi    mid    values[mid]    decision
```

You can also draw each step. highlight marks mid; done marks what has been discarded (paste into the notebook `notebooks/08-searching.ipynb`):

```
from viz.draw import draw_array
A = [4, 9, 13, 20, 26, 31, 37, 44]
draw_array(A, highlight=3, title="lo=0 hi=7 mid=3")
draw_array(A, highlight=5, done=[0, 1, 2, 3], title="lo=4 hi=7 mid=5")
```

Checkpoint 1. With $A = [4, 9, 13, 20, 26, 31, 37, 44]$, trace `binary_search(A, 37)` and `binary_search(A, 10)` as tables of `lo`, `hi`, `mid` and values `[mid]`. How many comparisons does each make? For the miss, what is `lo` when the loop ends, and what does that number mean?

2.2 Test sizes 0, 1 and 2 by hand

Lecture 08, “Easy to state, hard to get right”: nearly every binary-search bug shows up on a list of length 0, 1 or 2. Do these **before** you run pytest — they take two minutes and they find the bug while you still remember why you wrote the line.

Checkpoint 2. For a correct `binary_search`, give the return value and the number of times the loop body runs:

- (a) `binary_search([], 5)`
 (b) `binary_search([7], 7)`
 (c) `binary_search([7], 3)`
 (d) `binary_search([3, 8], 3)`
 (e) `binary_search([3, 8], 8)`
 (f) `binary_search([3, 8], 5)`

Then: if the loop condition were `lo < hi` instead of `lo <= hi`, which of the six answers would change?

2.3 Write it

1. Set `lo` to the first index and `hi` to the **last** index (not the length).
2. While the range values `[lo..hi]` is non-empty:

- compute mid with integer division;
- read values[mid] **once** into a local variable;
- equal: return mid;
- too small: everything up to and including mid is ruled out — move lo past it;
- too large: everything from mid on is ruled out — move hi below it.

3. The range is empty: return -1.

Hints:

- Both updates must **move past** mid. If an update can leave lo and hi unchanged, the loop never ends.
- Write `mid = lo + (hi - lo) // 2` or `(lo + hi) // 2`. In Python both are correct; in Java and C only the first is safe (Lecture 08, the Bloch story). Say which you chose, and why, when the TA asks.
- Reading values[mid] once per turn is not only tidy: in Part 7 you will count reads, and a function that reads it twice will look twice as expensive.

2.4 Test it

```
pytest tests/test_searching.py -v -k "binary_search and not recursive"
```

This selects 11 tests: the four cases of `test_finds_target_in_sorted_list` (targets 1, 5, 9 and the missing 4 in [1, 3, 5, 7, 9]), `test_handles_empty_and_single_element`, the three cases of `test_target_outside_the_values` (0, 10 and 99 in the same list — below the first value, just past the last, and far past it), `test_finds_first_and_last_element`, `test_agrees_with_linear_search_on_random_data` — all with `[binary_search]` in the name — and `test_binary_search_does_not_scan_linearly`, which searches a million integers and is slow if your code scans. The `and not recursive` part matters: `binary_search_recursive` contains `binary_search`.

2.5 When it fails

Bug	What you see	Fix
<code>mid = (lo + hi) / 2</code>	every test fails with TypeError: list indices must be integers or slices, not float	<code>//</code> — an index is an int
no return -1 after the loop	assert None == -1 in 6 tests, for example on ([], 1) — the function fell off the end	add the final return -1

Bug	What you see	Fix
<code>hi = len(values)</code> with <code>lo <= hi</code>	<code>IndexError: list index out of range</code> in 4 tests: empty/single, the outside targets 10 and 99, and random data	inclusive <code>hi</code> is the last index, <code>len(values) - 1</code>
<code>lo = mid</code> or <code>hi = mid</code>	no failure message at all: the run stops at one test and never finishes — for <code>lo = mid</code> , the case that searches for 9	press Ctrl+C ; the traceback shows the while line. Use <code>mid + 1</code> and <code>mid - 1</code>

The last one is worth tracing once. With `lo = mid`, searching `[3, 8]` for 8: `lo = 0`, `hi = 1`, `mid = 0`, `3 < 8`, so `lo = mid = 0` — and nothing has changed. The next turn is identical, for ever. With `hi = mid`, searching `[42]` for 7 sticks at `lo = hi = 0` in the same way.

Part 3 — Break it: `lo < hi`

Homework 8, item 4, done in the lab where you can ask about it. Your `binary_search` passes its 11 tests. Now change **one character**: `lo <= hi` becomes `lo < hi`. Leave everything else alone.

Checkpoint 3. Before you run anything:

- (a) With `lo < hi`, the loop stops as soon as the range has **one** element left, without looking at it. Which **positions** of a list can the broken search never find? Is there one it misses for every $n \geq 1$?
- (b) Of the 11 tests selected in Part 2.4, which fail? Write down their names.
- (c) The random-data test and the three outside-the-values tests pass. Why do they not catch this bug?

Now run it and compare with your prediction:

```
pytest tests/test_searching.py -v -k "binary_search and not recursive"
```

Read each failing assertion aloud. `assert -1 == 4` means your function returned `-1` where the test expected 4: it **missed** a value that was there. A bug that turns hits into misses is the worst kind — no exception, no crash, just a wrong answer.

Then **put the = back** and run the command again: 11 passed. If you are not sure you restored exactly what you had, `git diff dsa/searching.py` shows every line you have changed since your last commit.

What to take from this: a test suite is a set of examples, and the bug passes every example that never needs the last one-element range. The empty list, the single element, and the first and last positions are in the tests for exactly this reason (Lecture 08, “Easy to state, hard to get right”).

Part 4 — `binary_search_recursive`

Lecture 08, “Recursive binary search”. Same contract, same inclusive range — but instead of a loop that moves `lo` or `hi`, a **call** on the half that can still hold the target.

4.1 Draw it

Draw the calls as a column of frames, one per call, each with its own `lo` and `hi` — the call-stack picture from Lecture 03. The deepest frame is the one that returns first.

Checkpoint 4. A student writes this “elegant” version:

```
def bsr_slice(values, target):
    if not values:
        return -1
    mid = len(values) // 2
    if values[mid] == target:
        return mid
    if values[mid] < target:
        return bsr_slice(values[mid + 1:], target)
    return bsr_slice(values[:mid], target)
```

- (a) What do `bsr_slice([1, 3, 5, 7, 9], 9)` and `bsr_slice([1, 3, 5, 7, 9], 1)` return? Which half is safe to slice, and which is not?
- (b) For your correct `binary_search_recursive(list(range(1000)), -1)`, how many calls are on the stack at the deepest point, counting the last call whose range is empty?

4.2 Write it

1. `hi=None` is the usual way to write a default that depends on another argument. If `hi` is `None`, set it to the last index. Write `if hi is None`, not `if not hi` — see the bugs below.
2. Base case: the range is empty (`lo > hi`) — return `-1`.
3. Compute `mid` and read `values[mid]` once.
4. Equal: return `mid`. Otherwise **return** the result of a call on `lo..mid - 1` or `mid + 1..hi`, passing the same values — never a slice.

4.3 Test it

```
pytest tests/test_searching.py -v -k recursive
```

Ten tests: the four found/missing cases, empty and single, the three targets outside the values, first and last, and random data — each with `[binary_search_recursive]` in its name.

4.4 When it fails

Bug	What you see	Fix
slicing, as in Checkpoint 4	<code>assert 1 == 4</code> for <code>([1, 3, 5, 7, 9], 9)</code> and <code>assert 1 == (50 - 1)</code> in the first/last test	pass <code>lo</code> and <code>hi</code> ; never slice
a recursive call without return in front	<code>assert None == 0</code> , <code>assert None == 4, ...</code> — 9 of the 10 tests fail	the result of the inner call must be returned by every outer call
<code>if not hi:</code> instead of <code>if hi is None:</code>	only the random-data test fails, with <code>RecursionError: maximum recursion depth exceeded</code>	<code>hi = 0</code> is a real range end, and <code>not 0</code> is <code>True</code> : the range silently resets to the whole list. Test for <code>None</code> with <code>is</code>

The third bug is the nasty one: searching `[1, 3, 5, 7, 9]` for 1 or 0 works, but for 2 the search narrows to `lo = 1`, `hi = 0`, which is empty and correct — except that `hi = 0` is falsy, so the call starts again from `hi = 4`, and recurses until Python stops it.

Part 5 — `lower_bound` and `upper_bound`: one loop, one comparison

Lecture 08, “Bounds: Searching for Positions”. A bound answers “**where**”, and always with a position: the insertion point, or the start or end of a run of duplicates. Neither function ever returns `-1`; both may return `len(values)`.

5.1 The half-open range

The lecture writes both with a **half-open** range `[lo, hi)`: `lo` is a candidate, `hi` is **not**. Start with `lo = 0`, `hi = len(values)` — so `len(values)` is a possible answer — and loop while `lo < hi`, that is, while the range still has a candidate.

Notice what changed from Part 2. There, $lo < hi$ was a bug; here it is correct, because hi means something different. The loop condition follows from what lo and hi mean — it is not a style choice.

The invariant for `lower_bound`: everything **left of** lo is $< \text{target}$, and everything **from** hi on is $\geq \text{target}$. When the range closes, $lo == hi$ is the boundary between the two — the answer.

For `upper_bound`, replace $< \text{target}$ by $\leq \text{target}$ and $\geq \text{target}$ by $> \text{target}$. That is the only difference between the two functions.

5.2 Draw it

Take $v = [1, 2, 2, 2, 5]$ and target 2. Mark each element **L** (“goes left of the answer”) or **R**, for each function:

index	0	1	2	3	4	
v	1	2	2	2	5	
<code>lower_bound: $v[i] < 2$?</code>			L	R	R	R → first R at 1
<code>upper_bound: $v[i] \leq 2$?</code>			L	L	L	R → first R at 4

Both functions are the same binary search for the **first R**. Only the test that decides L or R differs.

Checkpoint 5. $v = [3, 3, 5, 8, 8, 8, 9]$. Give:

- `lower_bound(v, 8)` and `upper_bound(v, 8)`, and how many 8s there are;
- `lower_bound(v, 4)` and `upper_bound(v, 4)`;
- `lower_bound(v, 10)` and `upper_bound(v, 0)`;
- the number of 3s, computed with the two bounds.

5.3 Write it

Write `lower_bound` first:

- lo, hi = the half-open range of the whole list.
- While the range is not empty: take mid . If $v[mid]$ is an **L**, the answer is to its right — move lo past mid . Otherwise mid is an **R**, so the answer is mid or something left of it: hi becomes mid , not $mid - 1$. (Everything from hi on is R — that is the invariant — and mid has just been proved R.)
- Return lo . Check it by hand on $[], [2]$ and $[1, 2]$ with target 2: the answers are 0, 0 and 1.

Then copy it to `upper_bound` and change **one comparison**. If you find yourself changing anything else, stop and re-read 5.1.

Why $hi = mid$ is safe here when it hung the loop in Part 2: with a half-open range, mid is always strictly less than hi , so $hi = mid$ always shrinks the range. Try $lo = 0, hi = 1: mid = 0$, and either update makes the range empty.

5.4 Test it

```
pytest tests/test_searching.py -v -k bound
```

Three tests: `test_lower_bound`, `test_upper_bound` and `test_bounds_count_duplicates`, which checks that `upper_bound(v, 2) - lower_bound(v, 2)` is 4 on `[1, 2, 2, 2, 2, 5]`.

5.5 When it fails

Bug	What you see	Fix
<code>hi = len(values) - 1</code> (inclusive habit)	<code>assert 4 == 5</code> for <code>lower_bound([1, 2, 2, 2, 5], 6)</code> — the answer <code>len(values)</code> is unreachable	<code>hi = len(values)</code>
return -1 when the answer is <code>len(values)</code> <code>upper_bound</code> with < instead of <=	<code>assert -1 == 5</code> <code>assert 1 == 4</code> in the upper-bound test, and <code>assert 0 == 4</code> in the duplicates test	a bound is a position; <code>len(values)</code> is a valid one this is <code>lower_bound</code> again: a 2 must count as L for <code>upper_bound</code>
<code>while lo <= hi</code> with <code>hi = mid</code>	the run stops and never finishes	half-open means <code>lo < hi</code>

Part 6 — Jump, exponential and interpolation search

Lecture 08, “Three More Searches”. Each of these is a short function with one idea and two or three edge cases. The same ten tests run on each of them, so a working `binary_search` is your model for what “correct” looks like.

6.1 Draw it

Use one list for all three: `B = [0, 5, 10, 15, 20, 25, 30, 35, 40, 45]` (`n = 10`), target 30, which is at index 6.

Checkpoint 6. For each search, list the **indices** it reads, in order, until it returns 6:

- (a) `jump_search(B, 30)` — the step is `math.isqrt(10)`;
- (b) `exponential_search(B, 30)` — slot 0 first, then the bounds 1, 2, 4, ..., then a binary search between the last two bounds;
- (c) `interpolation_search(B, 30)` — it reads `B[lo]`, `B[hi]`, then the guess.

6.2 jump_search

1. Empty list: return `-1`. Otherwise `step = math.isqrt(n)`, at least 1.
2. Jump block by block: while there is a **next** block and the **last** element of the current block is smaller than the target, move to the next block.
3. Walk forward inside the block you stopped in — but not past `n`: the last block may be shorter than `step`. Return the index if you find the target; return `-1` as soon as you pass a value larger than it, or when the block ends.

```
pytest tests/test_searching.py -v -k jump
```

Bug	What you see	Fix
<code>step = n ** 0.5</code>	<code>TypeError: list indices must be integers or slices, not float</code>	<code>math.isqrt(n)</code> returns an <code>int</code>
the walk runs to <code>prev + step</code> with no <code>min(..., n)</code>	<code>IndexError: list index out of range</code> in <code>test_target_outside_the_values</code> for 10 and 99 — the only 2 of the 10 tests that walk off the end of the short last block	cap the walk at <code>n</code>

6.3 exponential_search

1. Empty list: `-1`. If slot 0 holds the target, return 0 — the doubling starts at 1.
2. `bound = 1`; while `bound < n` and `values[bound] < target`, double it.
3. The target, if present, is between `bound // 2` and `bound` — but `bound` may be past the end, so cap it at `n - 1`. Binary search that range.

For step 3 you may reuse your own code: `binary_search_recursive` already takes `lo` and `hi`, so one call does it. That is a good reason it takes them.

```
pytest tests/test_searching.py -v -k exponential
```

Bug	What you see	Fix
<code>bound = 0</code> to start	the run stops at the case that searches for 5, and never finishes: <code>0 * 2</code> is 0	start at 1; check slot 0 separately
no <code>bound < n</code> in the doubling loop	<code>IndexError: list index out of range</code> in 5 tests, including [42] searched for 7 and the outside targets 10 and 99	test <code>bound < n</code> first : and stops before the read

Bug	What you see	Fix
hi = bound, not capped	IndexError: list index out of range in 4 tests: the outside targets 10 and 99, first/last, and random data	min(bound, n - 1)

6.4 interpolation_search

1. lo, hi inclusive, as in Part 2. While lo <= hi: read values[lo] and values[hi] once each.
2. **Trap 1 — outside the range.** If the target is below values[lo] or above values[hi], it is not there: return -1.
3. **Trap 2 — equal ends.** If values[hi] == values[lo], the formula would divide by zero. Every value in the range is the same: compare one of them with the target and return.
4. **Trap 3 — integer division.** Compute the guess with the lecture's formula, using //, so that pos is an int.
5. Then as in binary search, with pos in place of mid.

```
pytest tests/test_searching.py -v -k interpolation
```

Bug	What you see	Fix
no equal-ends check	ZeroDivisionError: integer division or modulo by zero in the empty/single test — a one-element range has equal ends	trap 2
no out-of-range check	the run stops at the missing-target case (4 in [1, 3, 5, 7, 9]) and never finishes	trap 1 — see below
/ in the formula	TypeError: list indices must be integers or slices, not float in 6 of the 10 tests	trap 3

The missing range check does not crash; it hangs. Searching [1, 3, 5, 7, 9] for 4: the first guess is 1 (value 3), so lo = 2. Now the range is [5, 7, 9], the target 4 is below it, and the formula gives $2 + (4 - 5) * 2 // 4 = 1$ — a guess **left of lo**. Value 3 is too small, so lo = pos + 1 = 2 again. Nothing changes, for ever. The check in trap 1 is what keeps every guess inside [lo, hi].

Part 7 — Count the reads

Lecture 08, “Measured”, right-hand panel. A timing depends on your machine; a **count** of how many elements a search reads does not. You will count them with a class of your own that looks like a list to the search.

7.1 Write CountingReads

This is not graded, and it does not go in `dsa/`: write it in the scratch space at the end of `notebooks/08-searching.ipynb`, or in a file of your own outside `dsa/`. The contract:

- `CountingReads(values)` keeps the list and a counter `reads`, starting at 0;
- `len(c)` is the length of the list;
- `c[i]` adds 1 to `reads` and returns `values[i]`.

Three methods: `__init__`, `__len__` and `__getitem__` (Lab 03, “Special methods”). Your searches use only `len(values)` and `values[i]`, so they cannot tell a `CountingReads` from a list. Check it first:

```
>>> c = CountingReads([10, 20, 30])
>>> len(c), c[1], c.reads
(3, 20, 1)
```

(`tools/figures_l08.py` has the course’s version, also called `CountingReads`. Look at it after writing yours.)

Checkpoint 7. With `V = list(range(1024))` and a fresh `CountingReads(V)` for each call, predict reads after:

- `linear_search(c, -1)`
- `binary_search(c, -1)`
- `binary_search(c, 1024)`
- `interpolation_search(c, 500)`
- and `interpolation_search` for 500 on the **skewed** list `list(range(1023)) + [10 ** 9]` — 1,023 evenly spread values and one huge one.

7.2 All the searches, side by side

```
import random
from dsa import searching

SEARCHES = [searching.linear_search, searching.binary_search,
             searching.binary_search_recursive, searching.jump_search,
             searching.exponential_search,
             ↪ searching.interpolation_search]
```

```
def average_reads(search, values, targets):
    counted = CountingReads(values)
    for t in targets:
        assert search(counted, t) == values.index(t)
    return counted.reads / len(targets)

rng = random.Random(8)
for n in [1024, 65536]:
    values = sorted(rng.sample(range(10 * n), n))
    targets = [rng.choice(values) for _ in range(100)]
    print(f'n = {n}')
    for search in SEARCHES:
        reads = average_reads(search, values, targets)
        print(f' {search.__name__:24} {reads:10.1f}')
```

With the reference solutions, the averages come out at about:

Search	n = 1,024	n = 65,536	Shape
linear	522	33,000	$n/2$
binary (loop or recursive)	9.0	14.6	$\log_2 n$
jump	34	243	about $2\sqrt{n}$ at worst, $\sqrt{n}$ on average
exponential	17.5	30	about twice binary
interpolation	8.5	11.6	nearly flat: $\log \log n$, three reads a step

Yours may differ by a small constant — a binary search that reads `values[mid]` twice per turn shows twice the reads — but the **shape** must be the same: multiply n by 64 and linear grows 64 times, jump about 8 times ($\sqrt{64}$), and binary adds about 6 reads ($\log_2 64$).

7.3 Interpolation on skewed data

```
n = 1024
even = list(range(n))
skewed = list(range(n - 1)) + [10 ** 9]
targets = random.Random(1).sample(range(n - 1), 50)
for name, values in [('even', even), ('skewed', skewed)]:
    for search in (searching.binary_search,
        ⇨ searching.interpolation_search):
        reads = average_reads(search, values, targets)
        print(f'{name:7} {search.__name__:22} {reads:8.1f}')
```

On the even list interpolation needs 3 reads — one guess, straight to the answer — against about 9 for binary search. On the skewed list binary search does not notice anything (about 9 again), while interpolation needs about 1,480: **worse than linear search**, because each of its one-slot steps reads three values. The one huge value at the end tilts the straight line, so every guess lands at the far left (Lecture 08, “Interpolation search: guess, don’t halve”). An average case that hides a bad worst case — this is it.

Part 8 — Check your bounds against bisect

Only now, with your own bounds passing their tests, meet the library versions (Lecture 08, “lower_bound and upper_bound”): `bisect.bisect_left` is `lower_bound` and `bisect.bisect_right` is `upper_bound`. Use them as an **oracle** — a second opinion — on many random lists **with duplicates**, which the graded tests only touch once:

```
import bisect, random
from dsa.searching import lower_bound, upper_bound

rng = random.Random(1)
for trial in range(10_000):
    values = sorted(rng.choices(range(20), k=rng.randint(0, 15)))
    target = rng.randint(-2, 22)
    case = (values, target)
    assert lower_bound(values, target) == bisect.bisect_left(values,
        ↪ target), case
    assert upper_bound(values, target) == bisect.bisect_right(values,
        ↪ target), case
print('10,000 random lists: all agree')
```

The targets run from -2 to 22 on purpose, so that “before everything” and “after everything” are both tried, and `k` can be 0, so the empty list is too. If an assertion fails, the message is the smallest thing you need: the list and the target. Trace that one by hand.

This is also how you use `bisect` from now on: in your own programs, call the library; in `dsa/`, the point is that you can write it.

Part 9 — Measure it

Part 7 counted reads; this part times them, to see the lecture’s left-hand panel appear on your own machine. Run it in `notebooks/08-searching.ipynb`:

```
from viz.complexity import measure, plot_growth
from dsa.searching import linear_search, binary_search
```

```

sizes = [10_000, 20_000, 40_000, 80_000, 160_000, 320_000]
make = lambda n: (list(range(n)), -1)          # a miss: the worst case
↪ for both

linear = measure(lambda a: linear_search(*a), sizes, make)
binary = measure(lambda a: binary_search(*a), sizes, make)
plot_growth({'linear search': linear, 'binary search': binary},
            reference=['n', 'log n'])

```

measure builds each input outside the timing, and keeps the best of three runs. What to expect:

- **Linear** is a straight line through the origin: double n , double the time. It follows the dashed $O(n)$ reference.
- **Binary** looks flat — a few microseconds at every size. Its growth is real ($\log_2 320,000$ is about 18 against 13 for 10,000), but on this scale it is lost in timer noise, which is why Part 7 counted reads instead.

Add `loglog=True` to `plot_growth` and linear becomes a line of slope 1, while binary stays almost level. Your exact times will differ from anyone else's; the shapes will not.

Part 10 — Exercises at a glance

Function	Target cost	The trap	-k filter
<code>linear_search</code>	$O(n)$	return -1 inside the loop; must return the first match	see Part 1 (2)
<code>binary_search</code>	$O(\log n)$, $O(1)$ space	inclusive <code>hi = n - 1</code> ; <code>lo <= hi</code> ; <code>mid ± 1</code> ; //	see Part 2.4 (11)
<code>binary_search_recursive</code>	$O(\log n)$, $O(\log n)$ stack	pass <code>lo</code> , <code>hi</code> , never slice; <code>hi</code> is <code>None</code> ; return the call	recursive (10)
<code>lower_bound</code>	$O(\log n)$	half-open <code>[0, n)</code> ; <code>lo < hi</code> ; <code>hi = mid</code> ; may return <code>n</code>	bound (3)
<code>upper_bound</code>	$O(\log n)$	the same loop with <code><=</code> in place of <code><</code>	bound (3)
<code>jump_search</code>	$O(\sqrt{n})$	<code>math.isqrt</code> ; the last, partial block	jump (10)
<code>exponential_search</code>	$O(\log i)$	slot 0 first; bound starts at 1; cap at <code>n - 1</code>	exponential (10)

Function	Target cost	The trap	-k filter
interpolation_search	$O(\log \log n)$ average, $O(n)$ worst	target outside $[v[l_0], v[h_i]]$; equal ends; //	interpolation (10)

All 56 at once:

```
pytest tests/test_searching.py -v
```

Before you show the TA: `git diff --stat tests/` must print nothing. And check by eye that nothing in `dsa/searching.py` slices, sorts, or calls `in`, `index` or `bisect` — the tests cannot see all of those, but the TA will look.

Part 11 — Take-home practice (not graded)

The question bank for this week is `docs/question-bank/week08-questions.md`, with answers in `week08-answers.md`. Do the questions before opening the answers.

1. **Part F — write the code**, in `practice/week08.py`: `search_rotated`, `integer_sqrt`, `find_peak`, `closest_value` and `min_capacity` (W8-C1 to W8-C5). Every one is a binary search in disguise, and the tests count your reads, so an $O(n)$ answer fails:

```
pytest tests/test_practice_week08.py -v
```

Hint for `closest_value`: it is your `lower_bound` and one comparison.

2. **W8-T1** — the trace from Homework 8, item 2: `binary_search` on `[2, 5, 8, 12, 16, 23, 38, 56, 72, 91]` for 23 and for 60.
3. **W8-T3** — draw the recursive calls for `binary_search_recursive(V, 45)` and `(V, 7)`, and give the depth.
4. **W8-B2** — a binary search with `hi = mid`. You met it in Part 2.5; now find an input on which it hangs, without running it.
5. **W8-K3** — counting copies of `x` by finding one and walking outwards. What does it cost when every element is `x`? What do your bounds cost?

The worked solutions are in `solutions/dsa/searching.py` and `solutions/practice/week08.py` — for after you have tried. `pytest --solutions tests/test_practice_week08.py` runs the tests on them.

Part 12 — Bridge to Lecture 09: sorting

Every search in this lab except the first assumed a promise: **the input is sorted**. Lecture 09 is where sorted input comes from. Here is the first connection, using a function you already have.

`lower_bound(out, x)` is the position where `x` goes to keep `out` sorted. So inserting each item there, one at a time, **sorts** the items:

```
from dsa.searching import lower_bound

def sorted_by_inserting(items):
    out = []
    for x in items:
        out.insert(lower_bound(out, x), x)
        print(x, out)
    return out
```

```
>>> sorted_by_inserting([5, 2, 9, 1, 5, 6])
5 [5]
2 [2, 5]
9 [2, 5, 9]
1 [1, 2, 5, 9]
5 [1, 2, 5, 5, 9]
6 [1, 2, 5, 5, 6, 9]
[1, 2, 5, 5, 6, 9]
```

(`out` is an ordinary list, in a script of yours — not in `dsa/`.) Two questions to bring to Lecture 09:

- **What does it cost?** Finding the place is $O(\log n)$, but `insert` shifts every element after it — $O(n)$ (Lecture 02). For n items that is $O(n^2)$ in total, however clever the search. The search is not the bottleneck; the shifting is.
- **Where do equal items go?** With `lower_bound`, a new 5 goes **before** the 5s already there; with `upper_bound`, **after** them — so equal items keep their arrival order. That property is called **stability** (Lab 03, Part 1.2), and `tests/test_sorting.py` checks it: look for `test_insertion_sort_is_stable`.

This is the idea of **insertion sort**: grow a sorted prefix one item at a time. In `dsa/sorting.py` you will write `insertion_sort` by shifting elements yourself rather than calling `insert`, and its docstring already tells you the surprise: $O(n^2)$ in general, but $O(n)$ when the input is nearly sorted.

Summary

Idea	The one line to keep
Linear search	Needs no order; $O(n)$; return the first match.
Binary search	Sorted input; $O(\log n)$; $lo \dots hi$ inclusive, $lo \leq hi, mid \pm 1$.
Invariant	If the target is anywhere, it is in values $[lo \dots hi]$.
Test by hand	Sizes 0, 1 and 2 find nearly every boundary bug.
$lo < hi$ bug	Misses every position reached only as a one-element range — always the last.
Recursive	Pass lo and hi ; a slice costs $O(n)$ and loses the index.
Bounds	Half-open $[lo, hi)$, $lo < hi$, $hi = mid$; lower and upper differ in $<$ against $\leq$.
Counting	$upper_bound - lower_bound$ is the number of copies.
Jump / exponential	$O(\sqrt{n})$, forward only / $O(\log i)$, unknown length.
Interpolation	$O(\log \log n)$ on even data; $O(n)$ — worse than linear — on skewed data.
Measuring	Count reads with a class that has <code>__len__</code> and <code>__getitem__</code> .

Lab 09 — Lab 09 — Sorts You Can Watch

How to use this lab. The routine from Week 4: read the lecture section named at the top of each part, **draw before you code** — here that means the array after every pass, on paper — and predict at each **Checkpoint**. This week has one extra rule: write each sort **as a generator first**, the `_steps` form, and get the plain sort from it. Then you can watch your own code run, one comparison at a time, before you ask pytest whether it is right. Parts 1–5 are the session; Parts 6–8 are for home.

Duration	One 2-hour lab session, plus about 3 hours at home
You will write	<code>dsa/sorting.py</code> — <code>bubble_sort_steps</code> , <code>selection_sort_steps</code> , <code>insertion_sort_steps</code> , their three plain forms, and <code>counting_sort</code> ; and, for yourself, a small <code>Counted</code> class
Graded by	<code>tests/test_sorting.py -k "bubble or selection or insertion or counting or is_sorted"</code> (39 tests)
Connects to	Lecture 09 — Basic sorting; Lecture 08, whose searches need sorted input; and Lecture 10, which sorts the same file in $O(n \log n)$

What you will be able to do

1. draw the array after every pass of bubble, selection and insertion sort, and count the comparisons and the swaps or shifts;
 2. write a sort as a generator that yields a snapshot before each step, and watch it with `viz/animate.py`;
 3. get the plain sort from the generator without writing the algorithm twice;
 4. say which of the three sorts are stable, and show it with an input;
 5. write counting sort, and say when it is the right sort and when it is not;
 6. count comparisons with a class of your own, and explain the counts with best, worst and average case;
 7. measure what the snapshots cost, and fix it.
-

Part 0 — Before you start

0.1 Environment

From the DSA27 folder, with the virtual environment active (the prompt starts with `(.venv)`):

```
git pull
pytest -m "not challenge" -q          # the environment check: must pass
pytest tests/test_sorting.py -q `
    -k "bubble or selection or insertion or counting or is_sorted"
```

The last command shows **38 failed, 1 passed**. The one that passes is `test_is_sorted_helper`: `is_sorted` is given. The 38 failures are all `NotImplementedError`. Merge, quick and heap sort share the file and are Week 10's; the `-k` filter leaves out their 37 tests, and every command in this lab uses a filter.

0.2 Prerequisites

- **The course Array** (Lecture 02): `Array.from_values(values)` makes a full `Array` holding a copy of a list; `a[i]` and `a[i] = x` are $O(1)$; `list(a)` turns it back into a list. It has no negative indices and no slicing.
- **Generators** (Lab 03, Part 5): a function with `yield` in it returns a generator; each next runs it to the next `yield` and pauses there.
- **Special methods** (Lab 03): `__lt__` and `__gt__` are what `<` and `>` call. You need them in Part 6.

0.3 Read the skeleton

Open `dsa/sorting.py`. Given: the module docstring (read it — it shows the pattern this lab uses), `__all__`, and `is_sorted`. Yours this week:

Function	Returns	Contract
<code>bubble_sort_steps(values)</code>	a generator of (<code>list</code> , <code>highlight</code>) pairs	yield before each comparison; the last yield is sorted
<code>bubble_sort(values)</code>	a new sorted list	the input list is not changed
<code>selection_sort_steps</code> , <code>selection_sort</code>	the same two forms	
<code>insertion_sort_steps</code> , <code>insertion_sort</code>	the same two forms	stable
<code>counting_sort(values</code> , <code>max_value=None)</code>	a new sorted list	non-negative integers; $O(n + k)$

`highlight` is a tuple of the indices to colour in the animation — the pair about to be compared, for example — or `()` for none.

0.4 The storage rule, this week

A sort is a function, not a structure, but it does need somewhere to work. Copy the input into a course Array — `a = Array.from_values(values)` — sort the Array, and return `list(a)`. That gives you two things at once: the caller's list is never changed (`test_does_not_mutate_the_input`), and the working storage obeys the storage rule. Two more rules, from the point of the week:

- **Do not call a library sort.** No `sorted`, `list.sort`, `min` or `max` of the values inside the three basic sorts. (`counting_sort` may use `max` to find `k` — the lecture's step 1.)
- **Write each algorithm once.** The plain form must reuse the `_steps` form, not copy its loops (Part 1).

Part 1 — The pattern: one generator, two uses

Lecture 09, “The code, written as a generator” and “One yield, one frame”. The lecture shows `bubble_sort_steps` in full. Read it, close it, and write it from memory in Part 2; here you learn to **use** such a generator before you write one.

1.1 Watch one

Open `notebooks/09-sorting-basic.ipynb`. Its “Try it” cell holds a small `bubble_sort_steps` of its own (on a Python list, with no early exit). Run it: the slider steps through the sort one comparison at a time. Then try the other view:

```
from IPython.display import HTML
from viz.animate import animate_bars
HTML(animate_bars(bubble_sort_steps([5, 2, 9, 1, 7])).to_jshtml())
```

1.2 One generator, three consumers

A generator does nothing until something iterates it. Try, in the notebook:

```
frames = bubble_sort_steps([5, 2, 9, 1, 7])
print(frames)                # a generator object: nothing has run yet
first = next(frames)          # runs to the first yield, and pauses
print(first)
print(sum(1 for _ in frames)) # runs the rest, counting what is left
```

That is the whole trick. The **animation** iterates the generator and draws each frame; a **count** iterates it and adds 1 per frame; and the **plain sort** iterates it and keeps only the last frame. So the plain form is three lines, and the module docstring of `dsa/sorting.py` shows them. Write a small helper for it once, and use it in all three plain sorts:

```
def _finish(steps):
    """Run a _steps generator to the end and return its last state."""
```

It must work when the generator yields only once or twice (the empty list), and it must return a **list**.

Checkpoint 1. For the lecture's `bubble_sort_steps` — one yield before the loops, one before each comparison, one after — how many frames does each call produce?

- (a) `bubble_sort_steps([5, 2, 9, 1, 7])`
- (b) `bubble_sort_steps([1, 2, 3, 4, 5])`
- (c) `bubble_sort_steps([])`

And (d): the notebook's version yields the **same list object** every time. Why does `step_slider` still show different frames, while `frames = list(bubble_sort_steps([5, 2, 9, 1, 7]))`, with the notebook's version, shows the sorted list in every frame?

Part 2 — `bubble_sort_steps` and `bubble_sort`

Lecture 09, "Bubble sort". Compare neighbours; swap them when the left one is larger. Each pass carries the largest remaining value to the end, so each pass can stop one slot earlier. A pass with no swap means the array is sorted: stop.

2.1 Draw it

On paper, one row per pass: the array after the pass, and how many comparisons and swaps it made. Shade the slots that are in their final place.

You can check each row in the notebook:

```
from viz.draw import draw_array
draw_array([1, 3, 4, 7, 2, 9], done=[5], title="after pass 1")
```

Checkpoint 2. Trace bubble sort, with the early exit, on `[4, 1, 3, 9, 7, 2]`.

- (a) Write the array after each pass, with its comparisons and swaps.
- (b) How many passes run, and why does the last one not stop earlier?
- (c) Total comparisons and total swaps. Count the **inversions** of the input (pairs in the wrong order) and compare.

2.2 Write it

- Copy the input into an Array, and remember `n`.
- Yield the first state: `list(a), ()`.
- For each pass: the pass looks at `a[0..end]`, with end going from `n - 1` down to 1. Set `swapped = False` **at the start of each pass**.
- For each `j` from 0 while `j + 1 <= end`: **yield** `list(a), (j, j + 1)`, then compare and, if needed, swap and set `swapped`.

5. After the pass, if nothing was swapped, stop.
6. Yield the last state.

Then `bubble_sort` is one line, with your `_finish`.

Hints:

- Yield `list(a)`, never `a`. The animation keeps every frame; a frame must be a copy of the state **now**, not the `Array` that will change after the yield.
- `a[j], a[j + 1] = a[j + 1], a[j]` swaps in one line, and it works on the course `Array` as it does on a list.

2.3 Test it

```
pytest tests/test_sorting.py -v -k bubble
```

Twelve tests: the eight fixed cases of `test_sorts_correctly` (empty, one, two, `[5, 2, 9, 1, 7]`, three equal values, sorted, reversed, negatives), `test_does_not_mutate_the_input`, `test_sorts_random_lists` (20 random lists, fixed seed), and the two tests on the `_steps` form.

2.4 When it fails

Bug	What you see	Fix
<code>range(end + 1)</code> in the inner loop	<code>IndexError: Array index 5 out of range for length 5 (valid: 0..4; no negative indices)</code> — 10 of the 12 fail	the pass compares <code>a[j]</code> with <code>a[j + 1]</code> , so <code>j</code> stops at <code>end - 1</code>
the <code>if not swapped</code> : <code>break</code> indented inside the inner loop	<code>assert [2, 5, 1, 7, 9] == [1, 2, 5, 7, 9]</code> — 4 fail	the check belongs after the pass, not after one comparison
<code>a = values</code> instead of a copy	only <code>test_does_not_mutate_the_input</code> fails: sort a copy — callers do not expect their list changed	<code>Array.from_values(values)</code>
the last yield is <code>a, ()</code> instead of <code>list(a), ()</code>	<code>assert [Array([1, 2, 5, 7, 9]), ()] == [1, 2, 5, 7, 9]</code> in <code>test_step_form_is_a_generator_ending_sorted</code>	<code>yield list(a)</code>
<code>bubble_sort</code> returns <code>bubble_sort_steps(values)</code>	<code>assert <generator object ...> == [3, 3, 3]</code> — 9 fail	run the generator to the end: <code>_finish</code>

Bug	What you see	Fix
swapped = False before the outer loop	nothing fails	see below

The last bug is the dangerous one: all twelve tests pass. Once any pass has swapped, swapped stays True for ever, so the early exit never fires again and bubble sort always does all $n - 1$ passes. The output is right; the best case is gone. Only a count finds it — Part 6 will.

Part 3 — selection_sort_steps and selection_sort

Lecture 09, “Selection sort”. Round i finds the smallest value in $a[i..n-1]$ and swaps it into slot i .

3.1 Draw it

One row per round: the array after the round, the index of the minimum, and whether a swap happened.

Checkpoint 3. (a) Trace selection sort on $[4, 1, 3, 9, 7, 2]$: the array after each round, the comparisons per round, and the swaps. Compare the totals with Checkpoint 2.

(b) Sort the three cards 2a, 2b, 1c — key first, label second, compared by key only — with selection sort. Is the result stable? Which line of the algorithm is to blame?

3.2 Write it

1. Copy, yield the first state.
2. For each i from 0 to $n - 2$: `smallest = i`; for each j after i , **yield** `list(a), (smallest, j)` and then compare `a[j]` with `a[smallest]` — the best so far, not `a[i]`.
3. After the scan, swap `a[i]` and `a[smallest]` **only if** they differ, and yield the state after the swap.
4. Yield the last state. `selection_sort` is one line.

3.3 Test it

```
pytest tests/test_sorting.py -v -k selection
```

Twelve tests, the same set as bubble sort's.

3.4 When it fails

Bug	What you see	Fix
<code>if a[j] < a[i]:</code>	<code>assert [1, 2, 7, 5, 9]</code> <code>== [1, 2, 5, 7, 9] — 4</code> fail	compare with the smallest so far: <code>a[smallest]</code>
the swap inside the inner loop	nothing fails, but it swaps every time it finds a smaller value	one swap per round, after the scan: that is the point of selection sort
<code>for j in range(i, n)</code>	nothing fails	it compares <code>a[i]</code> with itself once per round: harmless, and wasted

The swap-inside-the-loop version is a different, worse algorithm that the tests cannot tell apart: it sorts, but it writes up to $n^2/2$ times instead of at most $n - 1$. Count its swaps once you have Part 6.

Part 4 — insertion_sort_steps and insertion_sort

Lecture 09, “Insertion sort” and “Insertion sort counts inversions”. Grow a sorted prefix: take `current = a[i]`, shift every larger value in `a[0..i-1]` one slot right, and drop `current` into the hole.

4.1 Draw it

One row per round: the array after inserting `a[i]`, and how many shifts it took.

Checkpoint 4. (a) Trace insertion sort on `[4, 1, 3, 9, 7, 2]`: the array after each round, the shifts, and the comparisons per round. Compare the total shifts with the inversions you counted in Checkpoint 2.

(b) How many comparisons does insertion sort make on `[1, 2, 3, 4, 5, 6]`? On `[6, 5, 4, 3, 2, 1]`?

(c) The loop condition is `j >= 0` and `a[j] > current`. A student writes `a[j] > current` and `j >= 0`. On a Python **list**, what would the test `a[j] > current` read when `j` is `-1`? What happens on the course Array?

4.2 Write it

1. Copy, yield the first state.
2. For each `i` from 1 to `n - 1`: `current = a[i]`, `j = i - 1`.
3. While `j` is a valid index **and** `a[j] > current`: move `a[j]` one slot right, move `j` left.
4. Put `current` at `j + 1`.

5. Yield after each shift, with a highlight of your choice, and yield the last state. `insertion_sort` is one line.

A choice to make: after a shift, the array holds the shifted value **twice** until current is dropped in. A frame taken then shows a duplicate. The reference writes current into the hole after every shift, so every frame is a permutation of the input; that costs one extra write per shift. Either is accepted — say which you chose, and why, when the TA asks.

4.3 Test it

```
pytest tests/test_sorting.py -v -k insertion
```

Thirteen tests: the twelve of the other two sorts, and `test_insertion_sort_is_stable`. That test sorts four Card objects that compare **by key only**, so that two cards with the same key are equal to the sort and still distinguishable by their label.

4.4 When it fails

Bug	What you see	Fix
<code>a[j] > current and j >= 0</code>	<code>IndexError: Array index -1 out of range for length 5 (valid: 0..4; no negative indices)</code> — 9 of 13 fail	<code>test j >= 0</code> first: and stops before the read
<code>a[j] >= current</code>	only <code>test_insertion_sort_is_stable</code> fails: <code>assert ['0d', '0b', '1c', '1a'] == ['0b', '0d', '1a', '1c']</code>	strictly greater: an equal key must stop the walk
no <code>a[j + 1] = current</code> after the loop	<code>assert [5, 5, 5, 9, 9] == [1, 2, 5, 7, 9]</code> — values duplicated, others lost; 7 fail	the shifted-over value is still there; current must be written back

The first bug is worth a moment. On a Python list, `a[-1]` is the **last** element, so the reversed condition reads a value that has nothing to do with the sort; then `j >= 0` is false and the loop stops anyway. The answer comes out right — by luck — and the bug stays in the code, waiting for a language where `a[-1]` is a crash or garbage (C). The course Array refuses negative indices, so you see the bug on the exact line, the first time it happens.

Part 5 — counting_sort

Lecture 09, “Counting sort”. Values are non-negative integers. Count how many times each value occurs; then write each value out, in increasing order, as many times as it was counted. No two values are ever compared.

5.1 Draw it

Draw three rows: the values, the counts array with its indices $0 \dots k - 1$ above it, and the output.

Checkpoint 5. (a) For `counting_sort([3, 0, 2, 3, 0, 3])`: what is k , what is counts, and what is the output?

(b) `counting_sort([2, 1_000_000])` sorts two numbers. How many counters does it make, and how much work does the write-out loop do? What does that say about when counting sort is a good idea?

(c) Why can counting sort not sort `[2.5, 1.0]` or `['b', 'a']` as written?

5.2 Write it

1. The empty list: return `[]` at once — `max([])` raises `ValueError`.
2. Check every value is a non-negative int; otherwise raise `ValueError`.
3. k is `max_value + 1` if `max_value` was given, else `max(values) + 1`.
4. `counts = Array(k, fill=0)`; one pass: `counts[v] += 1`.
5. An output `Array` of `len(values)` slots; for each v from 0 to $k - 1$, write v into the next free slot `counts[v]` times. Return it as a list.

5.3 Test it

```
pytest tests/test_sorting.py -v -k counting
```

One test, `test_counting_sort`, with two cases: `[3, 1, 4, 1, 5, 0]` and `[]`. That is thin, so test it yourself against sorted on random lists of small integers, as Part 8 of Lab 08 did with `bisect`:

```
import random
from dsa.sorting import counting_sort
rng = random.Random(1)
for trial in range(1000):
    values = [rng.randint(0, 20) for _ in range(rng.randint(0, 30))]
    assert counting_sort(values) == sorted(values), values
print('1,000 random lists: all agree')
```

5.4 When it fails

Bug	What you see	Fix
<code>k = max(values), without + 1</code>	<code>IndexError: Array index 5 out of range for length 5</code>	values run from 0 to max inclusive : max + 1 counters
no empty-list check	<code>ValueError: max() iterable argument is empty</code>	return <code>[]</code> before calling max

5.5 All 39 at once

```
pytest tests/test_sorting.py -v `
-k "bubble or selection or insertion or counting or is_sorted"
```

Part 6 — Count the comparisons

Lecture 09, “Counting comparisons and writes”. A timing depends on your machine; a **count** does not.

6.1 Write Counted

Not graded, and not in dsa/: write it at the end of the notebook. The contract:

- `Counted(v)` keeps a value `v`;
- `a < b` and `a > b` compare the two `vs`, and add 1 to a counter **shared by all** `Counted` objects — a class attribute, `Counted.compares`.

Your sorts only ever use `<` and `>` on the values, so they cannot tell a list of `Counted` from a list of numbers.

Checkpoint 6. Predict `Counted.compares` for $n = 100$, for each of your three sorts, on sorted input and on reversed input (six numbers).

6.2 All three sorts, four inputs

```
import random
from dsa import sorting

def comparisons(sort, values):
    Counted.compares = 0
    result = sort([Counted(v) for v in values])
    assert [c.v for c in result] == sorted(values)
    return Counted.compares
```

```

n = 1000
rng = random.Random(9)
nearly = list(range(n))
for _ in range(10):                                # swap 10 random neighbours
    i = rng.randrange(n - 1)
    nearly[i], nearly[i + 1] = nearly[i + 1], nearly[i]
inputs = {'sorted': list(range(n)), 'nearly': nearly,
          'random': rng.sample(range(n), n),
          'reversed': list(range(n, 0, -1))}
for sort in (sorting.bubble_sort, sorting.selection_sort,
             sorting.insertion_sort):
    counts = {name: comparisons(sort, v) for name, v in inputs.items()}
    print(f'{sort.__name__:15}', counts)

```

With the reference solution this prints:

	sorted	nearly	random	reversed
bubble_sort	999	1,997	491,874	499,500
selection_sort	499,500	499,500	499,500	499,500
insertion_sort	999	1,009	249,614	499,500

Yours should match exactly — the three algorithms are fully determined. If your `bubble_sort` shows 499,500 on sorted input, its early exit is broken: the swapped bug of Part 2.4. If your `insertion_sort` shows more than 999 on sorted input, its walk does not stop at the first value that is not greater.

Part 7 — The price of watching, and the fix

Lecture 09, “The price of watching”. Your plain sorts run the `_steps` generator to the end, and each yield makes a `list(a)` copy — $O(n)$ — before each of about $n^2/2$ comparisons.

7.1 Measure it

```

import random
from viz.complexity import measure, plot_growth
from dsa import sorting

sizes = [50, 100, 200, 400]
make = lambda n: random.Random(n).sample(range(n), n)
steps = measure(sorting.bubble_sort, sizes, make, repeat=1,
               ↪ warmup=False)

```

```
plot_growth({'bubble_sort': steps}, reference=['n^2', 'n^3'],
    ↪ loglog=True)
print([round(b / a, 1) for a, b in zip(steps[1], steps[1][1:])])
```

Checkpoint 7. Before you run it: if the plain sort is $O(n^2)$, by what factor should the time grow each time n doubles? And if it is $O(n^3)$? Which do you see, and why?

7.2 Fix it

Give each `_steps` function a parameter `snapshot=list`, and yield `snapshot(a)` instead of `list(a)`. The animation calls it with the default and gets copies, as before. The plain sort passes a function that returns the Array itself, with no copy at all; your `_finish` already turns the last state into a list. Run the tests again (all 39 must pass), then the measurement: the ratios drop from about 8 to about 4.

Part 8 — Measure the three sorts

```
sizes = [250, 500, 1000, 2000]
make = lambda n: random.Random(n).sample(range(n), n)
results = {sort.__name__: measure(sort, sizes, make, repeat=1,
    ↪ warmup=False)
    for sort in (sorting.bubble_sort, sorting.selection_sort,
                sorting.insertion_sort, sorting.counting_sort)}
plot_growth(results, reference=['n^2', 'n'], loglog=True)
```

It takes about half a minute. What to expect (Lecture 09, “Measured time”):

- the three $O(n^2)$ sorts are parallel lines of slope 2 on the log-log plot: double n , four times the time;
- selection sort is usually the fastest of the three **in Python**, although it compares the most — it writes almost nothing, and a swap costs more than a comparison here;
- counting sort is a line of slope 1, far below the others.

Then time `insertion_sort` alone on sorted, nearly sorted, random and reversed input of the same size (the four inputs of Part 6, at $n = 2000$). Nearly sorted is hundreds of times faster than random. That is why real library sorts finish small or nearly sorted pieces with insertion sort (Lecture 09, “Insertion sort on nearly sorted input”).

Part 9 — Exercises at a glance

Function	Target cost	The trap	-k filter
<code>bubble_sort_steps</code>	$O(n^2)$, $O(n)$ best	<code>range(end)</code> ; reset swapped each pass; <code>yield list(a)</code>	<code>bubble</code> (12)
<code>selection_sort_steps</code>	$O(n^2)$ always, at most $n - 1$ swaps	compare with <code>a[smallest]</code> ; swap after the scan	<code>selection</code> (12)
<code>insertion_sort_steps</code>	$O(n^2)$, $O(n)$ best	<code>j >= 0</code> first; <code>></code> not <code>>=</code> ; write current back	<code>insertion</code> (13)
the three plain sorts	as above	reuse the generator; no copy per step (Part 7)	(in the three above)
<code>counting_sort</code>	$O(n + k)$	<code>k = max + 1</code> ; the empty list	<code>counting</code> (1)

Before you show the TA: `git diff --stat tests/` must print nothing. And check by eye that nothing in the three basic sorts calls `sorted`, `sort`, `min` or `max`, and that the plain sorts do not repeat the loops of the `_steps` functions.

Part 10 — Take-home practice (not graded)

The question bank for this week is `docs/question-bank/week09-questions.md`, with answers in `week09-answers.md`. Do the questions before opening the answers.

1. **Part G — write the code**, in `practice/week09.py`: `count_inversions`, `sort_by_key`, `dutch_flag`, `sort_k_sorted` and `counting_sort_by_key` (W9-C1 to W9-C5). The tests count comparisons or reads for the last three, so an $O(n^2)$ answer to an $O(n)$ or $O(nk)$ problem fails:

```
pytest tests/test_practice_week09.py -v
```

Hint for `counting_sort_by_key`: turn the counts into **starting positions** with a running sum, then place the items in input order.

2. **W9-S1** — draw every pass of bubble sort on `[6, 5, 3, 1, 8, 7, 2, 4]`.
3. **W9-T4** — how many frames does each `_steps` generator yield on `[5, 4, 3, 2, 1]`? Predict, then check with `len(list(...))`.
4. **W9-B2** — a selection sort that swaps inside the inner loop. Count its swaps on reversed input of size 6, and compare with the real one.
5. **W9-E5** — the comparison lower bound in your own words, and why counting sort does not contradict it.

The worked solutions are in `solutions/dsa/sorting.py` and `solutions/practice/week09.py` — for after you have tried. `pytest --solutions tests/test_practice_week09.py` runs the tests on them.

Part 11 — Bridge to Lecture 10: divide, then merge

Insertion sort costs about $n^2/4$ comparisons on random input. What if you sorted two **halves** separately, and then combined them? Each half costs about $(n/2)^2/4 = n^2/16$, so both together cost $n^2/8$ — half as much. Combining two sorted lists is the `merge_sorted` you wrote in Week 2 (`dsa/array_ops.py`): one pass, at most $n - 1$ comparisons.

```
import random
from dsa import sorting
from dsa.array_ops import merge_sorted

values = random.Random(1).sample(range(1000), 1000)

Counted.compares = 0
sorting.insertion_sort([Counted(v) for v in values])
print('whole', Counted.compares)

Counted.compares = 0
left = sorting.insertion_sort([Counted(v) for v in values[:500]])
right = sorting.insertion_sort([Counted(v) for v in values[500:]])
merged = merge_sorted(left, right)
print('two halves, then merge', Counted.compares)
```

(`merge_sorted` compares with `<=`, so give `Counted` a `__le__` too, counting like the others.) With the reference solutions: **251,387** comparisons for the whole list, **125,992** for two halves and a merge. Split into four quarters and merge twice, and it drops to 65,733. Two questions to bring to Lecture 10:

- **Why stop at four?** Split the halves in half, and those in half again, until the pieces have one element — which is sorted already. What is left is only merging. That is **merge sort**, and it costs $O(n \log n)$.
- **Is it stable?** `merge_sorted` takes from the left list when the two front values are equal. Why does that one choice make the whole sort stable?

Summary

Idea	The one line to keep
<code>_steps</code> generator	Yield a snapshot before each step; the animation, a count and the plain sort all iterate it.
Plain form	Run the generator to the end and keep the last state — one algorithm, not two.

Idea	The one line to keep
Snapshots	<code>list(a)</code> per frame; but not in the plain sort, or $O(n^2)$ becomes $O(n^3)$.
Bubble sort	Neighbours swap; reset swapped each pass; best case $O(n)$.
Selection sort	$n(n - 1)/2$ comparisons always; at most $n - 1$ swaps; not stable.
Insertion sort	One shift per inversion; <code>j >= 0</code> first; <code>></code> keeps it stable.
Stability	Only visible with records that compare equal but are different.
Counting sort	$O(n + k)$, no comparisons; <code>k = max + 1</code> ; bad when <code>k</code> is huge.
Measuring	Count comparisons with a class whose <code><</code> and <code>></code> count.

Lab 10 — Lab 10 — Divide, Conquer, and a Pivot

How to use this lab. The routine from Week 4: read the lecture section named at the top of each part, **draw before you code** — this week that means a merge tree, a partition table and a heap drawn as a tree — and predict at each **Checkpoint**. Then write one function, run the tests named in its part, and only then move on. Three sorts, each built from one small helper: `_merge`, `_partition`, `_sift_down`. Get the helper right by hand and the sort is five lines. Parts 1–5 are the session; Parts 6–8 are for home.

Duration	One 2-hour lab session, plus about 3 hours at home
You will write	the second half of <code>dsa/sorting.py</code> — <code>merge_sort_steps</code> , <code>quick_sort_steps</code> , <code>heap_sort_steps</code> and their plain forms, with the helpers <code>_merge</code> , <code>_choose_pivot</code> , <code>_partition</code> and <code>_sift_down</code> ; and, for yourself, two small classes, <code>Counted</code> and <code>Card</code>
Graded by	<code>tests/test_sorting.py -k "merge or quick or heap"</code> (37 tests)
Connects to	Lecture 10 — Advanced sorting; Lab 09 (the <code>_steps</code> pattern); Lecture 03 (<code>merge_sorted</code>); and Week 12, where the heap returns as a priority queue

What you will be able to do

1. merge two sorted runs through a scratch Array by hand, and say which of two equal values goes first and why;
2. write merge sort as a recursive generator, and explain what `yield from` does and what happens without it;
3. trace Lomuto's partition as a table of `j`, boundary and the array, and state its invariant;
4. write quicksort with four pivot strategies and a call stack that stays $O(\log n)$ deep, and measure its $O(n^2)$ worst case yourself;
5. read an array as a tree, build a max-heap bottom-up, and write heap sort;
6. count the comparisons each sort makes, and check stability with values that compare by key only;
7. explain why timing your sorts can measure the snapshots instead of the sort — and

fix it.

Part 0 — Before you start

0.1 Environment

From the DSA27 folder, with the virtual environment active (the prompt starts with `(.venv)`):

```
git pull
pytest -m "not challenge" -q                                # the
↪ environment check: must pass
pytest tests/test_sorting.py -q -k "merge or quick or heap" # 37
↪ failures: nothing written yet
```

The 37 failures are all `NotImplementedError` — that is the starting point.

0.2 Prerequisites

- **Lab 09.** Your `bubble_sort_steps` and friends work, and you know the pattern: copy the input into an `Array`, yield a snapshot (`list(a)`, `highlight`) at each step, and write the plain form as “run the generator to the end, return the last state”. This week uses the same pattern, the same helper `_copy`, and the same `_finish` if you wrote one. If you did not, write the plain forms as in the docstring at the top of `dsa/sorting.py`.
- **Lecture 03.** Recursion, and `merge_sorted` in `dsa/recursion.py`. The merge step this week is the loop version of the same idea.
- **Generators (Lab 03).** `yield` makes a function a generator; calling it runs **nothing** until something iterates it. Part 2 depends on this.

0.3 Read the skeleton

Open `dsa/sorting.py` and read the second half: `merge_sort_steps`, `quick_sort_steps`, `heap_sort_steps`, and their plain forms. Each docstring gives the target cost. Two things to notice:

Function	Signature	Note
<code>merge_sort_steps</code>	<code>(values)</code>	$O(n \log n)$ always, $O(n)$ extra
<code>quick_sort_steps</code>	<code>(values,</code> <code>pivot="median3")</code>	pivot is "first", "last", "random" or "median3"
<code>heap_sort_steps</code>	<code>(values)</code>	$O(n \log n)$, $O(1)$ extra, not stable

The helpers — `_merge`, `_choose_pivot`, `_partition`, `_sift_down` — are not in the skeleton. You add them, above the functions that use them. A name that starts with `_` is private: the tests never call it, so you are free to shape it, but the lecture's shapes are the ones this lab traces.

0.4 The storage rule, this week

Copy the input into the course Array, sort the Array, return a list — as in Lab 09. Merge sort's scratch is an Array too. The median of three is computed with comparisons. Not allowed in `dsa/sorting.py`: `sorted()`, `list.sort()`, `heapq`, `bisect`, and slicing the Array (it raises `TypeError: Array does not support slicing` anyway). Python lists appear only as the snapshots you yield and the list you return.

Part 1 — `_merge`: two sorted runs become one

Lecture 10, "Merging two sorted runs" and "`_merge`: the code". You have written a merge twice already: in Week 2 (`merge_sorted` on two Arrays, `dsa/array_ops.py`) and in Week 3 (recursively, `dsa/recursion.py`). This one merges two **neighbouring** runs of the same Array, `a[lo:mid]` and `a[mid:hi]`, through a scratch Array, and copies the result back into `a`.

1.1 Draw it

Three indices: `i` walks the left run, `j` the right run, `k` the next free slot of scratch. Draw `a`, then scratch under it, and move the three arrows one step at a time.

Checkpoint 1. `a = [2, 5, 9, 1, 5, 6]`, `lo = 0`, `mid = 3`, `hi = 6`. Call the 5 at index 1 "5L" (left run) and the one at index 4 "5R".

- (a) In what order are the six values written to scratch? Does 5L or 5R go first?
- (b) How many comparisons `a[i] <= a[j]` are made?
- (c) Which of the two "leftover" loops runs, and what does it copy?
- (d) If the test were `a[i] < a[j]`, what would change?

1.2 Write it

`_merge(a, lo, mid, hi, scratch)`, above `merge_sort_steps`:

1. Start `i` at `lo`, `j` at `mid`, `k` at `lo` — `scratch[k]` lines up with `a[k]`, so the merged range goes back to the same place.
2. While both runs have elements: compare the fronts; copy the smaller to `scratch[k]`; advance that run's index and `k`. On a **tie, take from the left**: `<=`.
3. Copy whatever is left of the left run; then whatever is left of the right run. Only one of the two has anything.
4. Copy `scratch[lo:hi]` back into `a[lo:hi]` with a loop.

There is no test that calls `_merge` directly: its tests are merge sort's, in Part 2. So check it by hand now, in the notebook:

```
from dsa.array import Array
from dsa.sorting import _merge
a = Array.from_values([2, 5, 9, 1, 5, 6])
_merge(a, 0, 3, 6, Array(6))
print(list(a))          # [1, 2, 5, 5, 6, 9]
```

Part 2 — merge_sort_steps and merge_sort

Lecture 10, “merge_sort_steps: divide, conquer, combine”. One scratch Array, made once; an inner generator `sort(lo, hi)` that sorts `a[lo:hi]`; a snapshot after every merge.

2.1 Draw it

Draw the merge tree of the lecture's figure for your own input: split in the middle until each piece has one element, then merge neighbours on the way back up. Number the merges in the order the recursion does them — **left subtree completely first**.

Checkpoint 2. For `merge_sort_steps([6, 5, 3, 1, 8, 7, 2, 4])`:

- List the merged ranges `a[lo:hi]` in the order the merges happen.
- What is the array after the **third** merge? After the **sixth**?
- How many frames does the generator yield in total, counting the first and the last?
- How many comparisons does the whole sort make? (Count them merge by merge: a merge of two runs makes at most $\text{len} - 1$.)

2.2 Write it

- `a = _copy(values)` and `scratch = Array(len(a))`; yield the first frame.
- Inside, define `sort(lo, hi)`:
 - a range of 0 or 1 elements is sorted: return;
 - `mid = (lo + hi) // 2`;
 - sort the left half, sort the right half — each with **yield from**;
 - `_merge(a, lo, mid, hi, scratch)`, then yield `list(a)` with the merged range as the highlight, `tuple(range(lo, hi))`.
- `yield from sort(0, len(a))`, then yield the last frame.
- `merge_sort(values)`: run the generator to the end and return the last state.

Why `yield from` and not a plain call: `sort` contains `yield`, so `sort(lo, mid)` only **creates** a generator object and throws it away — nothing runs. `yield from sort(lo, mid)` runs it and passes its frames up.

2.3 Test it

```
pytest tests/test_sorting.py -v -k merge
```

Twelve tests: the eight fixed cases of `test_sorts_correctly`, “does not mutate the input”, 20 random lists, and the two generator tests — each with `merge_sort` in its name.

Then watch it, in `notebooks/10-sorting-advanced.ipynb`:

```
from viz.animate import step_slider
from dsa.sorting import merge_sort_steps
step_slider(merge_sort_steps([6, 5, 3, 1, 8, 7, 2, 4]))
```

2.4 When it fails

Bug	What you see	Fix
<code>sort(lo, mid)</code> without <code>yield from</code>	5 tests fail, e.g. <code>assert [5, 2, 9, 1, 7] == [1, 2, 5, 7, 9]</code> — only the top-level merge ever runs	<code>yield from sort(lo, mid)</code> , and the same for the right half
base case <code>if hi - lo <= 0</code>	11 tests fail with <code>RecursionError: maximum recursion depth exceeded — sort(lo, lo + 1) has mid = lo and calls itself on the same range for ever</code>	a one-element range is already sorted: <code>hi - lo <= 1</code>
no copy back from scratch	7 tests fail, including <code>AssertionError: frames never change — are you yielding the same list?</code>	the last loop of <code>_merge</code> : <code>a[k] = scratch[k]</code> for <code>k</code> in <code>lo..hi-1</code>
the leftover loop for the left run is missing	8 tests fail with <code>TypeError: '<=' not supported between instances of 'int' and 'NoneType' — the unfilled scratch slots hold None and were copied into a</code>	both leftover loops
<code><</code> instead of <code><=</code> in <code>_merge</code>	nothing fails	see Part 7 — the tests cannot see it

Part 3 — `_partition`: Lomuto's one pass

Lecture 10, "Lomuto's partition" and "`_partition`: the code". The pivot is `a[hi]`. One index `j` scans `lo..hi-1`; one index boundary marks the end of the "smaller than the pivot" region.

Invariant: `a[lo:boundary] < pivot <= a[boundary:j]`.

3.1 Draw it

A table, one row per value of `j`, as in the lecture's figure:

<code>j</code>	<code>a[j]</code>	<code>a[j] < pivot?</code>	<code>swap?</code>	array after the step	boundary
----------------	-------------------	-------------------------------	--------------------	----------------------	----------

and a last row for the final swap of the pivot into `a[boundary]`.

Checkpoint 3. Trace `_partition(a, 0, 7)` on `a = [5, 8, 1, 9, 3, 7, 2, 6]` (the pivot is 6).

- Fill in the table. Which values of `j` cause a swap?
- What is the array at the end, and what does the function return?
- How many comparisons with the pivot were made? Would that number change for a different order of the same eight values?

3.2 Write it

`_partition(a, lo, hi)` — both ends **inclusive**:

- `pivot = a[hi]; boundary = lo.`
- For `j` from `lo` to `hi - 1`: if `a[j] < pivot`, swap `a[boundary]` and `a[j]`, and add 1 to `boundary`.
- Swap `a[boundary]` and `a[hi]`. Return `boundary`.

The strict `<` sends values **equal** to the pivot to the right. That is what the invariant says; keep it.

Part 4 — `quick_sort_steps`: pivots and the call stack

Lecture 10, "Choosing the pivot" and "Stack depth: recurse on the smaller side".

4.1 `_choose_pivot(a, lo, hi, strategy, rng)`

Return the **index** of the pivot in `a[lo..hi]`:

- "first" $\rightarrow$ `lo`; "last" $\rightarrow$ `hi`;
- "random" $\rightarrow$ `rng.randint(lo, hi)` — `rng` is a `random.Random` made once in `quick_sort_steps`, with a fixed seed so the frames are the same every run;

- "median3" → whichever of `lo`, `mid` = $(lo + hi) // 2$ and `hi` holds the middle value of the three — with comparisons, no `sorted()`;
- anything else → `ValueError`.

The caller swaps the chosen index with `hi`, so `_partition` always finds the pivot at the end and never needs to know how it was chosen.

4.2 Draw it: two ways to recurse

Checkpoint 4. Run in your head `quick_sort(list(range(8)), pivot="first")` — sorted input, the first element as pivot.

- Where does each pivot end up, and how big are the two parts after each partition?
- How many partitions, and how many comparisons in total? Give the formula for n elements.
- Write the obvious version — partition, then **two** recursive calls. How deep does its call stack get on `list(range(n))`? At what n does Python stop it?
- With “recurse into the smaller part, loop on the larger”, how deep does the stack get on the same input?

4.3 Write it

- `a = _copy(values); rng = random.Random(27); yield the first frame.`
- The inner `sort(lo, hi)` — inclusive range — is a **while** `lo < hi`: loop:
 - choose the pivot, swap it to `hi`, partition: `q` is its final index;
 - yield a snapshot with `(q,)` highlighted;
 - if the left part `lo..q-1` is the smaller one: yield from `sort(lo, q - 1)`, then `lo = q + 1` and go round again for the right part;
 - otherwise the mirror image: recurse on `q + 1..hi`, then `hi = q - 1`.
- yield from `sort(0, len(a) - 1)`, and the last frame.
- `quick_sort(values, pivot="median3")`: pass pivot through.

4.4 Test it

```
pytest tests/test_sorting.py -v -k quick
```

Thirteen tests: the twelve kinds above, plus `test_quick_sort_survives_its_worst_case_input`, which sorts `list(range(200))`.

4.5 When it fails

Bug	What you see	Fix
no final swap in <code>_partition</code>	7 tests fail, e.g. <code>assert [1, 5, 2, 9, 7] == [1, 2, 5, 7, 9]</code>	the pivot must go to <code>a[boundary]</code> before you return boundary

Bug	What you see	Fix
if <code>lo < hi</code> : where the loop should be <code>while</code>	4 tests fail, e.g. <code>assert [5, 2, 1, 7, 9] == ...</code> — the larger part is never sorted	<code>while lo < hi</code> : — the loop is the second recursive call
the chosen pivot is never swapped to <code>hi</code>	nothing fails : every partition uses <code>a[hi]</code> , so all four strategies behave like "last"	count comparisons (Part 6): "median3" on sorted input is suddenly as slow as "first" — $n(n-1)/2$
two recursive calls instead of "smaller side, then loop"	nothing fails — 200 elements is under Python's limit — but <code>quick_sort(list(range(1000)), "first")</code> raises <code>RecursionError</code>	recurse into the smaller part only

The last two rows are why this part has a Checkpoint the tests cannot mark. Try the bad version once, on purpose, and see the `RecursionError` yourself.

Part 5 — `heap_sort_steps`: the array is the tree

Lecture 10, "The array is the tree", "Sift down" and "Build-heap in $O(n)$ ". The children of `a[i]` are `a[2i + 1]` and `a[2i + 2]`.

5.1 Draw it

Draw any list as a tree with the course's tool — no code of yours needed:

```
from viz.draw import draw_array_as_tree
draw_array_as_tree([2, 9, 4, 7, 1, 8, 5, 3])
```

Checkpoint 5. `v = [2, 9, 4, 7, 1, 8, 5, 3]`.

- Draw it as a tree. Which indices are leaves? Which is the last parent?
- Build-heap sifts down indices 3, 2, 1, 0 in that order. Give the array after each sift.
- Then heap sort's first step: swap `a[0]` with `a[7]` and sift down in `a[0:7]`. What is the array?
- Why does build-heap go from the last parent **down to 0**, and not from 0 up?

5.2 Write it

`_sift_down(a, index, size)` — the heap is `a[0:size]`:

- Loop: `largest = index`; compute left and right.

2. If `left < size` and `a[left] > a[largest]`, `largest = left`. The same for right, comparing with `a[largest]` — the **larger** child.
3. If `largest == index`, stop. Otherwise swap, and continue from `largest`.

`heap_sort_steps(values):`

1. `a = _copy(values)`, `n = len(a)`; yield the first frame.
2. Build-heap: sift down every index from `n // 2 - 1` **down to** 0, with `size = n`. Yield a frame — the heap.
3. For `end` from `n - 1` down to 1: swap `a[0]` and `a[end]`; sift down index 0 with `size = end`; yield with `(0, end)` highlighted.
4. The last frame.

5.3 Test it

```
pytest tests/test_sorting.py -v -k heap
```

Twelve tests. Then all 37:

```
pytest tests/test_sorting.py -v -k "merge or quick or heap"
```

5.4 When it fails

Bug	What you see	Fix
build-heap goes up , for index in <code>range(n // 2)</code>	only 2 tests fail: sorted input <code>[1, 2, 3, 4, 5]</code> gives <code>[1, 2, 4, 5, 3]</code> , and one random list	sifting a parent assumes its subtrees are already heaps: go from the last parent down to 0
build-heap stops at 1, <code>range(n // 2 - 1, 0, -1)</code>	5 tests fail, e.g. <code>[2, 3, 4, 5, 1]</code> — the root was never sifted	the stop value of range is excluded: -1
sift down with <code>size = n</code> in the sort loop	7 tests fail; the output is often reversed , e.g. <code>[5, 4, 3, 2, 1]</code>	the sorted tail is not part of the heap: <code>size = end</code>
only the left child compared	6 tests fail, e.g. <code>[1, 3, 2, 4, 5]</code>	compare with both children; move the larger up
the sort loop stops at 2, <code>range(n - 1, 1, -1)</code>	7 tests fail, e.g. <code>[2, 1]</code> stays <code>[2, 1]</code> — the last two are never ordered	<code>range(n - 1, 0, -1)</code>

The first row is the sneaky one: 35 of 37 tests pass. If your heap sort passes “almost everything”, check the direction of the build loop first.

Part 6 — Count the comparisons

Lecture 10, “Measured”. Lab 08 counted how many elements a search **reads** with a class that pretends to be a list. This week you count **comparisons** with a class that pretends to be a number. Not graded; write it in the notebook, not in dsa/.

6.1 Write Counted

```
class Counted:
    comparisons = 0                # one counter, shared by every
    ↪ Counted

    def __init__(self, value):
        self.value = value

    def __lt__(self, other):
        Counted.comparisons += 1
        return self.value < other.value
    # ... and the same for __le__, __gt__ and __ge__
```

Your sorts use <, <= and >; Python turns each into a call of the matching method, so the sorts cannot tell a Counted from a number. Then:

```
def comparisons(sort, values, *args):
    items = [Counted(v) for v in values]
    Counted.comparisons = 0
    result = sort(items, *args)
    assert [c.value for c in result] == sorted(values)
    return Counted.comparisons
```

(tools/figures_l10.py has the course’s version of both. Look at it after writing yours.)

6.2 The pivot experiment

Checkpoint 6. Predict, **then** run:

```
for n in [100, 200, 400]:
    print(n, [comparisons(quick_sort, list(range(n)), p)
              for p in ["first", "median3", "random"]])
```

- Give the exact numbers for "first". What happens to them each time n doubles?
- Roughly what size do you expect for "median3" — closer to n , $n \log_2 n$ or n^2 ?
- Replace quick_sort by merge_sort and by heap_sort on the same sorted lists. Which one does **fewer** comparisons than on random data, and why?

With the reference solution the "median3" counts are 606, 1,407 and 3,208, and "random" 642, 1,533 and 3,743 — a little over $n \log_2 n$ (664, 1,529, 3,458). Yours may differ by a few

percent if your median-of-three compares in a different order; the shape may not.

6.3 Build-heap is linear — count it

```
import random
from dsa.array import Array
from dsa.sorting import _sift_down

for n in [1000, 2000, 4000, 8000]:
    items = Array.from_values([Counted(x) for x in
    ↪ random.Random(3).sample(range(n), n)])
    Counted.comparisons = 0
    for index in range(n // 2 - 1, -1, -1):
        _sift_down(items, index, n)
    print(n, Counted.comparisons, round(Counted.comparisons / n, 2))
```

The reference gives 1,860, 3,712, 7,510 and 15,028 comparisons: about **1.87 n**, the same ratio at every size. An $O(n \log n)$ build would show the ratio growing. That is Lecture 10's sum $n(1/4 + 2/8 + 3/16 + \dots)$, measured.

Part 7 — Stability, and the test that cannot see it

Lecture 10, "Stability". A sort is stable if equal keys keep their input order. `tests/test_sorting.py` checks stability for insertion sort only (`test_insertion_sort_is_stable`), and none of the 37 tests of this lab checks it at all: they sort plain numbers, and two equal numbers cannot be told apart. Nor would **tuples** help — Python compares `(1, "a")` and `(1, "c")` by their second items, so any correct sort puts them in that order. To see stability, you need values that compare by key **only**, the way the insertion-sort test's `Card` class does.

7.1 Write Card

```
class Card:
    def __init__(self, key, tag):
        self.key, self.tag = key, tag
    def __lt__(self, other):
        return self.key < other.key
    # ... __le__, __gt__, __ge__ likewise — the tag is never compared
    def __repr__(self):
        return f"{self.key}{self.tag}"
```

Checkpoint 7. `cards = [Card(2, "a"), Card(1, "b"), Card(2, "c"), Card(1, "d")]`.

- (a) *Predict* `merge_sort(cards)`, `quick_sort(cards)` and `heap_sort(cards)`. Then run them.
- (b) Change the `<=` in your `_merge` to `<`. Which of the 37 tests fail? What does `merge_sort(cards)` give now? Put the `<=` back.
- (c) Write a test of your own — in a file outside `tests/`, or in the notebook — that fails for the `<` version and passes for the `<=` one.

This is the lesson of Lab 08's `lo < hi` bug again, from the other side: a test suite is a set of examples, and a property no example exercises is a property nobody has checked.

Part 8 — Measure the time: what are you timing?

Lecture 10, “Measured”, last paragraph. Time your `merge_sort` in the notebook:

```
import random, time
from dsa.sorting import merge_sort

for n in [1000, 2000, 4000]:
    values = [random.random() for _ in range(n)]
    start = time.perf_counter()
    merge_sort(values)
    print(n, round(time.perf_counter() - start, 3))
```

Checkpoint 8.

- (a) If merge sort is $O(n \log n)$, by roughly what factor should the time grow from $n = 1,000$ to $2,000$? What factor do you **see**?
- (b) Find the line that costs $O(n)$ and runs $n - 1$ times. (Hint: it is not in `_merge`.)
- (c) Fix it without breaking the animation or the tests.

With the reference code as the lecture shows it, this printed about 0.36, 1.29 and 4.7 seconds — four times as long for twice the input: n^2 , not $n \log n$. The sort is fine; the **snapshots** are not free. Fix it the way the basic sorts of the reference solution do: give the `_steps` form a parameter, `snapshot=list`, write `yield snapshot(a), ...` instead of `yield list(a), ...`, and let the plain form pass a function that returns a itself, uncopied. The animation still gets fresh lists; `merge_sort` gets none, and only its last state is turned into a list. After the fix, the same loop printed about 0.07, 0.13 and 0.27 seconds — twice the time for twice the input, plus a little: the $\log n$. Apply the same fix to `quick_sort` and `heap_sort`, and run the 37 tests again.

Part 9 — Exercises at a glance

Function	Target cost	The trap	-k filter
<code>_merge</code>	$O(hi - lo)$	$\leq$ for stability; both leftover loops; copy back	merge (12)
<code>merge_sort_steps / merge_sort</code>	$O(n \log n)$, $O(n)$ extra	yield from; base case $hi - lo \leq 1$; one scratch Array	merge (12)
<code>_partition</code> <code>_choose_pivot</code>	$O(hi - lo)$ $O(1)$	the final swap; strict < return an index ; median of three without <code>sorted()</code>	quick (13) Part 6
<code>quick_sort_steps / quick_sort</code>	$O(n \log n)$ average, $O(n^2)$ worst, $O(\log n)$ stack	swap the pivot to hi ; while, smaller side first	quick (13)
<code>_sift_down</code>	$O(\log n)$	the larger child; size bounds the heap	heap (12)
<code>heap_sort_steps / heap_sort</code>	$O(n \log n)$, $O(1)$ extra	build down from $n // 2 - 1$; sort with <code>size = end</code>	heap (12)

All 37 at once, and then the whole file with Lab 09's sorts:

```
pytest tests/test_sorting.py -v -k "merge or quick or heap"
pytest tests/test_sorting.py -v
```

Before you show the TA: `git diff --stat tests/` must print nothing, and nothing in `dsa/sorting.py` may call `sorted`, `.sort`, `heapq` or `bisect`.

Part 10 — Take-home practice (not graded)

The question bank for this week is `docs/question-bank/week10-questions.md`, with answers in `week10-answers.md`. Do the questions before opening the answers.

1. **Part G — write the code**, in `practice/week10.py`: `merge_k_sorted`, `count_inversions`, `kth_smallest`, `sort_colours` and `merge_sort_bottom_up` (W10-C1 to W10-C5). Each one is a piece of this week's sorts put to another use:

```
pytest tests/test_practice_week10.py -v
```

Hint for `count_inversions`: it is merge sort, plus one addition inside the merge.

2. **W10-T2** — trace `_partition` on `[5, 8, 1, 9, 3, 7, 2, 6]`: Homework 10, item 2, and Checkpoint 3 of this lab.
3. **W10-S1** — draw the merge tree of `[6, 5, 3, 1, 8, 7, 2, 4]`, every level.
4. **W10-B2** — a heap sort that builds its heap from the top. Find an input on which it fails, without running it.

5. **W10-K2** — the recurrence of quicksort with the first element as pivot, on sorted input. Solve it.

The worked solutions are in `solutions/dsa/sorting.py` and `solutions/practice/week10.py` — for after you have tried. `pytest --solutions tests/test_practice_week10.py` runs the tests on them.

Part 11 — Bridge to Lecture 11: trees

Two things from this lab come back next week.

The pivot story. Insert the values $1, 2, 3, \dots, n$ into a binary search tree, one at a time: each new value is larger than all the others, so it goes to the right of the right of the right... The tree becomes a chain n deep — exactly the shape of quicksort with `pivot="first"` on sorted input, which you drew in Checkpoint 4. Same input, same disaster, different structure.

Sorting for free. A binary search tree keeps every left subtree smaller than its root and every right subtree at least as large. Visit the left subtree, then the root, then the right subtree — an **in-order traversal** — and the values come out sorted. Building the tree is quicksort's partition done one value at a time; reading it in order is the recursion that follows. Lecture 11 makes this precise.

And the heap you drew in Part 5 is a tree stored in an array, with no pointers. Week 11's trees have nodes and pointers; Week 12 returns to the array.

Summary

Idea	The one line to keep
Divide and conquer <code>_merge</code>	Split, solve the parts recursively, combine. Compare fronts, copy the smaller; <code><=</code> takes from the left: stable.
Merge sort	<code>yield</code> from both halves, then merge; $O(n \log n)$ always, $O(n)$ extra.
Lomuto partition	<code>a[lo:boundary] < pivot <= a[boundary:j]</code> ; one final swap.
Pivot	"first" on sorted input: $n(n-1)/2$ comparisons, measured.
Stack depth	Recurse into the smaller part, loop on the larger: $O(\log n)$.
Heap	Children of i at $2i+1$ and $2i+2$; every parent $\geq$ its children.

Idea	The one line to keep
Build-heap	From the last parent down to 0: about $1.9 n$ comparisons — $O(n)$.
Heap sort	Swap the max to the back, sift down with <code>size = end</code> .
Stability	Tuples cannot test it; compare by key only (Card).
Timing	A snapshot costs $O(n)$; the plain form must not copy.

Lab 11 — Lab 11 — Trees and the Four Walks

How to use this lab. The routine from Week 4: read the lecture section named at the top of each part, **draw before you code** — this week that means drawing the tree, with every link, on paper — and predict at each **Checkpoint**. Then write one method, run the tests named in its part, and only then move on. One tree is used all the way through, so the drawings you make in Part 1 are the ones you check everything else against. Parts 1–7 are the session; Parts 8–11 are for home.

Duration	One 2-hour lab session, plus about 3 hours at home
You will write	<code>dsa/tree.py</code> — <code>BinarySearchTree</code> : <code>insert</code> , <code>contains</code> , <code>min</code> , <code>max</code> , <code>height</code> , <code>size</code> , <code>is_valid</code> , <code>in_order</code> , <code>pre_order</code> , <code>post_order</code> , <code>level_order</code> , <code>delete</code>
Graded by	<code>tests/test_tree.py</code> (26 tests)
Connects to	Lecture 11 — Trees; Lab 07's <code>CircularQueue</code> , which <code>level_order</code> uses; and Lecture 12, where a tree lives in an array

What you will be able to do

1. draw the BST that a sequence of inserts builds, and say which insertion order builds a chain;
2. write `insert` and `contains` as loops that walk one path down, and explain why `insert` must stop **at the parent**;
3. write recursive `height` and `size` from a base case and a recursive case, and say why the empty tree has height `-1`;
4. explain why checking each parent against its children does not prove a tree is a BST, and write `is_valid` with a range;
5. write the three recursive traversals, and say what each order is **for**;
6. write `level_order` with your own `CircularQueue`, and trace the queue;
7. write `delete` for all three cases, including the root, and draw the successor case step by step;
8. measure the height of a BST built from sorted and from shuffled input, and meet the recursion limit.

Part 0 — Before you start

0.1 Environment and prerequisite

From the DSA27 folder, with the virtual environment active (the prompt starts with `(.venv)`):

```
git pull
pytest -m "not challenge" -q          # the environment check: must pass
pytest tests/test_stack_queue.py -q  # Weeks 6-7: 18 passed
pytest tests/test_tree.py -q         # 26 failures: nothing yet
```

`level_order`, in Part 6, imports **your** `CircularQueue` from `dsa/queue.py`. It must **grow** when it is full — the challenge of Lab 07, Part 6 — because `test_level_order_on_a_wide_tree` has 16 nodes waiting at once and the default capacity is 8. The provided queue tests never overfill a ring, so check the growth yourself now:

```
>>> from dsa.queue import CircularQueue
>>> q = CircularQueue(2)
>>> for v in range(5):
...     q.enqueue(v)
...
>>> q.dequeue(), list(q)
(0, [1, 2, 3, 4])
```

If that raises “queue is full”, finish Lab 07’s Part 6 before Part 6 of this lab. Everything before Part 6 works without it.

0.2 Read the skeleton: what is given, what is yours

Open `dsa/tree.py` and read all of it — the module docstring first, which warns you about the linked list “wearing a tree costume”.

Given to you	Yours
<code>TreeNode</code> — <code>value</code> , <code>left</code> , <code>right</code>	<code>insert</code> , <code>contains</code> , <code>delete</code>
<code>to_edges(node)</code> — the (<code>parent</code> , <code>child</code>) pairs for <code>draw_tree</code>	<code>min</code> , <code>max</code> , <code>height</code> , <code>size</code> , <code>is_valid</code>
<code>BinarySearchTree.__init__(values=())</code> — sets <code>self.root = None</code> , then calls your <code>level_order</code> insert on each value	<code>in_order</code> , <code>pre_order</code> , <code>post_order</code> , <code>level_order</code>
<code>__len__</code> , <code>__contains__</code> , <code>__iter__</code> , <code>__repr__</code> — they call your methods	

The tree stores exactly one thing: `self.root`, a `TreeNode` or `None`. Every method starts from there.

Because `__init__` calls your `insert`, **every** test fails until `insert` works. That is why it is Part 1.

0.3 The storage rule, this week

A `dsa/` structure keeps its data only in the course `Array`, in node objects, or in another `dsa/` structure (Lecture 02). A BST keeps its data in `TreeNode`s, and nowhere else:

- no `self._values = []` beside the tree, and no sorted list that `in_order` simply returns;
- the four traversals **return** a new Python list each time. That list is **output**, made for the caller — the tree does not keep it (`test_traversals_return_a_fresh_list`);
- `level_order` needs a queue as working storage: your `CircularQueue`, not a Python list with `pop(0)` and not `collections.deque`.

0.4 The tests, and one warning about `-k`

`-k` matches the **file** name as well as the test name, and the file is `test_tree.py`. So `-k tree` selects all 26 tests. Filter by the method instead:

Filter	Selects
<code>-k "insert or contains or dup or struct"</code>	the 5 building tests
<code>-k "min"</code>	the 2 min/max tests
<code>-k "height or empty or degen"</code>	the size-and-height test, the degenerate-tree test, and the three tests with "empty" in their names (5)
<code>-k valid</code>	the broken-tree test for <code>is_valid</code> (1)
<code>-k "order or traversals"</code>	the 8 traversal tests
<code>-k delete</code>	the 6 delete tests

The tree used by most tests is drawn in a comment at the top of `tests/test_tree.py`. It is the lecture's tree, built from `[8, 3, 10, 1, 6, 14, 4, 7, 13]`. This lab uses a **different** tree, so that your drawings are your own work.

Part 1 — insert and contains: one path down

Lecture 11, "Search: one path down" and "Insert: a search that fails".

1.1 The idea

At each node, compare: smaller goes **left**, larger goes **right**, equal is found. A search walks one path from the root until it finds the value or steps onto `None`. An insert is the same walk;

where the search would have stepped onto None, the new node is hung instead. Nothing moves, and the new node is always a leaf.

1.2 Draw it

Take a sheet of paper. This is **the lab tree**, used in every part:

```
T = [15, 6, 20, 3, 9, 18, 24, 7, 12, 22]
```

Insert the values one at a time, left to right, into an empty tree. For each, write the path of comparisons — “9: < 15 left, > 6 right, None: here” — and draw the node where it lands. Draw the links as arrows from parent to child, and leave room: you will add to this drawing all afternoon.

Checkpoint 1. (a) Draw *T*. What is its height? Which nodes are leaves? (b) Now insert 8 and then 19 into your drawing. For each, list the nodes compared on the way down, and say whose left or right changes. (c) Draw `BinarySearchTree([1, 2, 3])` and `BinarySearchTree([2, 1, 3])`. Same values — what is different, and why?

1.3 Write it

contains(value) — a loop, not a recursion:

1. Start with a variable `node` at the root.
2. While `node` is not None: equal → True; smaller → move to `node.left`; larger → move to `node.right`.
3. The loop ended on None: False.

insert(value):

1. Empty tree: the new node becomes `self.root`. Done.
2. Otherwise walk down from the root as in `contains`. If you meet the value, return — duplicates are ignored.
3. Before stepping to a child, **look at it**: if the child you are about to take is None, set that link — `node.left` or `node.right` — to the new node, and return.

Step 3 is the whole trick. Walk until `node` is None and it is too late: you are holding None, not the parent, and `node = TreeNode(value)` changes a local variable that nothing else can see.

1.4 Test it

```
pytest tests/test_tree.py -v -k "insert or contains or dup or struct"
```

Five tests: `test_insert_and_contains` (every value of the sample tree is found; 0, 2, 5, 9 and 15 are not), `test_structure_is_actually_a_bst`, which checks the **links** of [8, 3, 10], not just the values, and `test_to_edges_matches_the_structure`, which reads the same links through the given `to_edges` — these three must pass now. The other

two also call methods you have not written yet, so they fail with `NotImplementedError` until then: `test_duplicates_are_ignored` needs `size` (Part 3) and `in_order` (Part 5), and `test_insert_keeps_the_invariant` needs `is_valid` (Part 4). Expect **3 passed, 2 failed** — that is right for now.

1.5 When it fails

Bug	What you see	Fix
walking until node is None, then node = <code>TreeNode(value)</code>	nearly everything: <code>assert 1 == 2</code> in the duplicates test, <code>AttributeError: 'NoneType' object has no attribute 'value'</code> in the structure test — the tree never grows past its root	stop at the parent: test the child before you step onto it
no equality check: equal values go right	<code>assert 5 == 2</code> in the duplicates test (once size works)	<code>value == node.value</code> → return
forgetting the empty tree	<code>AttributeError: 'NoneType' object has no attribute 'value'</code> on the first insert	if <code>self.root</code> is None, the new node is the root
<code>contains</code> returns inside the loop on the first comparison	<code>assert False</code> in the insert-and-contains test, for a value deeper down	return only when equal; otherwise move and keep going

Part 2 — See your tree; min and max

Lecture 11, “Min and max: no comparisons at all”.

2.1 Draw it — by machine, this time

With `insert` working, let the course tools draw what your code built. In `notebooks/11-trees.ipynb`:

```
from viz.draw import draw_tree
from dsa.tree import BinarySearchTree, to_edges

T = [15, 6, 20, 3, 9, 18, 24, 7, 12, 22]
bst = BinarySearchTree(T)
draw_tree(to_edges(bst.root), highlight={"15"})
```

Compare it with your paper drawing, link by link. If they differ, one of them is wrong — find out which before you go on. (`to_edges` is given, and uses only `value`, `left` and `right`: it shows your links exactly as they are.)

A limitation worth knowing: Graphviz lays out a node with **one** child directly below it, so it does not show whether that child is a left or a right child. For that, read the edges: `to_edges(bst.root)` lists each parent's left child before its right child.

2.2 Write it

`min()`: if the tree is empty, raise `ValueError`. Otherwise start at the root and go **left** while there is a left child; return the value where you stop. **`max()`** is the mirror image.

Checkpoint 2. For the lab tree *T*: which nodes does `min()` visit, and which does `max()`? How many times does either compare two values?

```
pytest tests/test_tree.py -v -k min
```

Two tests. The one that fails most often is `test_min_max_on_empty_raise`: returning `None` for an empty tree gives `Failed: DID NOT RAISE ValueError`. An empty tree has no minimum; `None` would be a lie that the caller might store and use.

Part 3 — height and size: recursion on trees

Lecture 11, “Depth and height” and “Height and size: post-order in one line”.

3.1 The idea

A tree is a root plus two smaller trees. So a question about a tree can often be answered from the answers for its two subtrees:

- **size** of a tree = 1 + size of the left subtree + size of the right;
- **height** of a tree = 1 + the larger of the two subtrees' heights;
- and the empty tree answers directly: size 0, height -1 .

Write each as a public method that calls a private recursive helper on `self.root` — `height()` calls `self._height(self.root)`. The helper takes a **node**, because the recursion is on subtrees, and a subtree is a node.

3.2 Checkpoint

Checkpoint 3. Give `height()` and `size()` for the trees built from: (a) `[]`; (b) `[5]`; (c) `[1, 2, 3, 4, 5]`; (d) `[3, 1, 5, 2, 4]`; (e) `[4, 2, 6, 1, 3, 5, 7]`; (f) `[5, 4, 3, 2, 1, 6]`; (g) the lab tree *T*. Then: if `_height(None)` returned 0 instead of -1 , what would each height become?

3.3 Write it and test it

```
pytest tests/test_tree.py -v -k "height or empty or degen"
```

Five tests. Expect **3 passed, 2 failed**: `test_empty_tree` also checks `in_order`, and `test_traversals_of_an_empty_tree` checks all four walks, so both wait for Parts 5 and 6. (`test_min_max_on_empty_raise` is the third “empty” test; it passed in Part 2.)

Bug	What you see	Fix
<code>_height(None)</code> returns 0	<code>assert 0 == -1</code> in the empty-tree test, <code>assert 4 == 3</code> in the size-and-height test, <code>assert 5 == 4</code> in the degenerate test — every height one too big	the empty tree is <code>-1</code> : height counts edges
<code>1 + max(...)</code> in <code>_size</code>	<code>assert 4 == 9</code> in the size-and-height test: that is the number of nodes on the longest path , not the number of nodes	add both subtrees: <code>1 + left + right</code>
the helper recurses on <code>self.root</code> instead of the node it was given	<code>RecursionError: maximum recursion depth exceeded</code>	pass <code>node.left</code> and <code>node.right</code>

Part 4 — `is_valid`: the whole subtree, not just the parent

Lecture 11, “The BST property” and “`is_valid`: carry the range down”.

4.1 The idea

The tempting check — at each node, `left.value < node.value < right.value` — looks only one level down. The BST property talks about **whole subtrees**. A value deep in the left subtree of 15 must be smaller than 15, even if its own parent is 6.

So pass a range (**low, high**) down the recursion. At the root, there is no limit on either side (use `None`). Going left from a node, the node’s value becomes the new high; going right, the new low. A node whose value is not strictly inside its range breaks the tree.

4.2 Draw it

On your drawing of `T`, write beside each node the range its ancestors allow, as the lecture’s figure does: 15 gets $(-\infty, \infty)$, 6 gets $(-\infty, 15)$, 9 gets $(6, 15)$, 12 gets $(9, 15)$, and so on.

Checkpoint 4. Four hand-built trees. For each, give (i) what `is_valid()` returns and (ii) what the parent-only check returns:

```
A = TreeNode(10, TreeNode(5, None, TreeNode(12)), TreeNode(15))
B = TreeNode(10, TreeNode(5, TreeNode(2), TreeNode(7)),
              TreeNode(15, TreeNode(11), TreeNode(20)))
C = TreeNode(10, TreeNode(5, None, TreeNode(10)), None)
D = TreeNode(10, TreeNode(5, TreeNode(2), TreeNode(7)),
              TreeNode(15, TreeNode(9), TreeNode(20)))
```

4.3 Write it and test it

A helper `_is_valid(node, low, high)`: True for None; False if the value is at or below low (when there is a low) or at or above high (when there is a high); otherwise both subtrees, with the narrowed ranges.

```
pytest tests/test_tree.py -v -k "valid or invariant"
```

Two tests. `test_is_valid_catches_a_hand_built_broken_tree` builds the lecture's broken tree (9 in the left subtree of 8). The parent-only check fails it with `assert not True` — it reports the broken tree as valid.

Two quieter bugs the test cannot see: writing `if low and node.value <= low` treats a bound of **0** as “no bound”, since `0` is falsy (write `low is not None`); and using `<` where `<=` belongs lets a duplicate of an ancestor through, as in tree C.

Part 5 — Three walks, one line apart

Lecture 11, “Recursion on trees”, “In-order: the code” and “What each one is FOR”.

5.1 The idea

The three recursive traversals have the same skeleton: do nothing for None; otherwise handle the node and walk both subtrees, left before right. Where you put “handle the node” — **before**, **between** or **after** the two walks — gives pre-order, in-order and post-order.

Each public method makes one output list, calls a helper with the root and that list, and returns the list. The helper appends to the **same** list at every level; it does not build and return lists of its own.

5.2 Draw it

Checkpoint 5. Write down, by hand, the four walks of the lab tree *T*: pre-order, in-order, post-order and level-order. Then answer: (a) Which one is sorted, and why must it be? (b) Insert the **pre-order** list into a new, empty tree. Is the result the same shape as *T*? What if

you insert the **in-order** list instead? (c) Which node is always first in pre-order, and always last in post-order?

5.3 Write it

Write `in_order` with its helper `_in_order(self, node, out)` exactly as in the lecture, run its test, then write `pre_order` and `post_order` by moving one line.

```
pytest tests/test_tree.py -v -k "order or traversals"
```

Eight tests. Expect **4 passed, 4 failed**: the two level-order tests, and the two that try all four walks (`test_traversals_return_a_fresh_list`, `test_traversals_of_an_empty_tree`), wait for Part 6. The pre-order test rebuilds the tree from its own pre-order list and compares the walks.

5.4 When it fails

Bug	What you see	Fix
the helper starts with its own <code>out = []</code>	<code>assert [] == [1, 3, 4, 6, ...]</code> — every value goes into a list that is thrown away, and the caller's list stays empty	make the list once, in the public method, and pass it down
the list kept on the tree (<code>self._out</code>) and returned every time	<code>AssertionError: in_order handed out a list the tree still uses</code>	a new list per call: a traversal is output, not storage
<code>left_list + [node.value] + right_list</code>	the tests pass	correct, but every level copies its whole subtree: $O(n^2)$ on a chain. Use one list
visiting right before left	<code>assert [8, 10, 14, 13, 3, 6, ...] == [8, 3, 1, 6, 4, 7, ...]</code> in the pre-order test	left always before right, in all three orders

Part 6 — level_order: a queue, not a call stack

Lecture 11, “Level-order needs a queue”.

6.1 The idea

Level-order visits nodes in order of their distance from the root. The call stack cannot do that — it always goes deeper into the most recent node, last in, first out. A **queue** keeps the discovered nodes in the order they were found, first in, first out:

1. empty tree → return an empty list;
2. enqueue the root;
3. while the queue is not empty: dequeue a node, append its value, enqueue its left child and then its right child (only those that are not None).

The queue holds **nodes**, not values: you need a node's children later, and a value does not have any.

6.2 Draw it

Draw the queue as a row of boxes, front on the left, after every dequeue — the format of the lecture's figure.

Checkpoint 6. (a) Trace `level_order` on the lab tree *T*: after each dequeue, which values are in the queue? What is the largest number of nodes in the queue at any one time? (b) The test builds a perfect tree of 31 nodes. How many nodes are in the queue at its fullest? Why does that break a `CircularQueue` that cannot grow?

6.3 Write it and test it

The skeleton already imports it, at the top of `dsa/tree.py`: `from dsa.queue import CircularQueue`. Make one with `CircularQueue()` and let it grow.

```
pytest tests/test_tree.py -v -k level
```

Two tests: `test_level_order` on the sample tree, and `test_level_order_on_a_wide_tree`.

Bug	What you see	Fix
a queue that cannot grow	<code>IndexError: queue is full</code> in the wide-tree test only	finish Lab 07's growth challenge; do not just pass a big capacity to hide it
right child enqueued before the left	<code>assert [8, 10, 3, 14, 6, 1, ...] == [8, 3, 10, 1, 6, 14, ...]</code> , and <code>[16, 24, 8] == [16, 8, 24]</code>	left first
a Python list with <code>pop(0)</code>	the tests pass	the storage rule, and $O(n)$ per dequeue — Lecture 07's <code>SlowQueue</code> . The TA will read your code

Bug	What you see	Fix
no empty-tree check	AttributeError: 'NoneType' object has no attribute 'value' in the empty-traversals test — None was enqueued as if it were the root	return [] before enqueueing anything

Try this. Replace the queue by your Stack from Week 6 — push instead of enqueue, pop instead of dequeue — and push the **right** child before the left. Which of the three recursive orders do you get? (Your TA has the answer — and Week 14 makes this the difference between BFS and DFS.)

Part 7 — delete: three cases

Lecture 11, “Delete: Three Cases”. This is the method of the week. Draw every step on paper before you write a line, and redraw the tree after every delete.

7.1 The idea

1. **Find** the node, and keep a variable for its **parent** as you walk. Not found: return.
2. **Two children?** Find the in-order successor: go right once, then left until there is no left child — keeping *its* parent too. Copy the successor’s value into the node. From now on, the node to remove is the **successor**, with its parent.
3. The node to remove now has **at most one** child. Call it *child* (it may be None). Put *child* where the node was: in `self.root` if the node has no parent; otherwise in whichever of the parent’s links — `left` or `right` — points at the node.

Step 2 always hands step 3 an easy case: the successor was found by going left until there was no left child, so it has no left child.

7.2 Draw it

Start again from a fresh drawing of *T*.

Checkpoint 7. Delete, in this order, redrawing the tree after each: **3**, then **6**, then **15**, then **20**. For each delete, name the case (*leaf*, *one child*, *two children*), and for the two-children case name the successor and the path to it. Give the height of the tree at the end.

Checkpoint 8. In the **original** *T*, what would `delete(9)` do? Which node is the successor, how many times does the “go left” loop run, and which link changes in step 3 — a left or a right?

7.3 Write it

Follow 7.1. Two hints:

- To know which link of the parent to change, ask `parent.left is node` — an identity test, “does this link point at this node?”.
- After step 2, **reassign** your two variables, the node and its parent, to the successor and the successor’s parent. Then step 3 is written once and serves all three cases.

7.4 Test it

```
pytest tests/test_tree.py -v -k delete
```

Six tests: a leaf (1), one child (14), two children (3), the root (8), a missing value (999), and every value of the sample tree deleted in turn, with `is_valid()` checked after each one.

7.5 When it fails

Bug	What you see	Fix
no root case: always <code>parent.left = ...</code> or <code>parent.right = ...</code>	<code>AttributeError:</code> 'NoneType' object has no attribute 'left' in the delete-everything test only	when the parent is <code>None</code> , the node is the root: <code>self.root = child</code>
two children: <code>successor_parent.left</code> <code>= successor.right</code> , always	the root test and the delete-everything test fail at <code>assert bst.is_valid()</code> ; the two-children test passes	when the successor is the node’s own right child, the link to change is the node’s right. Reassigning to the successor and using step 3 handles both
the value is copied, but the successor is never removed	the two-children test, the root test and delete-everything fail at <code>assert bst.is_valid()</code> — the value is now in the tree twice	step 3 must run on the successor
a missing value crashes	<code>AttributeError:</code> 'NoneType' object has no attribute 'left' in the missing-value test	if the search ends on <code>None</code> , return

The first bug is worth a minute. Why does `test_delete_the_root` **pass** with it? Because in the sample tree the root, 8, has two children: its parent is never needed, since step 2 moves the removal down to the successor. The root case only appears when the root has

at most one child — which happens late in `test_delete_everything`, when the tree has shrunk.

Part 8 — Measure it

Lecture 11, “When the Tree Is a Linked List” and “Measured”.

8.1 Height against insertion order

```
import random
from dsa.tree import BinarySearchTree

rng = random.Random(1)
for n in [100, 200, 400, 800]:
    ordered = list(range(n))
    shuffled = ordered[:]
    rng.shuffle(shuffled)
    print(n, BinarySearchTree(ordered).height(),
          BinarySearchTree(shuffled).height())
```

Checkpoint 9. Predict the first height column exactly, and the second roughly. Then run it. What would `BinarySearchTree(range(1000)).height()` do?

8.2 Time a search

```
from viz.complexity import measure, plot_growth

sizes = [100, 200, 400, 800]
def build(n, shuffled):
    values = list(range(n))
    if shuffled:
        random.Random(n).shuffle(values)
    return BinarySearchTree(values), n - 1    # look for the largest

search = lambda a: a[0].contains(a[1])
ordered = measure(search, sizes, lambda n: build(n, False))
random_ = measure(search, sizes, lambda n: build(n, True))
plot_growth({'sorted input': ordered, 'shuffled input': random_},
            reference=['n', 'log n'], loglog=True)
```

`measure` builds each tree outside the timing, so only the search is timed. On sorted input, searching for the largest value walks every node: a line of slope 1. On shuffled input it

walks one short path, and the line is almost flat — and so small that timer noise shows. The lecture's figure averages 2,000 searches per point to smooth that out.

8.3 The recursion limit

Now build `BinarySearchTree(range(1000))` and call `height()`. Read the error, then call `contains(999)` and `level_order()` on the same tree. Two of the three work. Why exactly those two? (Lecture 11, "Measured".)

Part 9 — Exercises at a glance

Method	Target cost	The trap	Tests
<code>insert</code>	$O(h)$	stop at the parent ; duplicates ignored; empty tree	-k "insert or contains or dup or struct"
<code>contains</code>	$O(h)$	a loop; move on, do not return early	as above
<code>min, max</code>	$O(h)$	<code>ValueError</code> when empty	-k min
<code>height, size</code>	$O(n)$	empty tree <code>-1</code> and <code>0</code>	-k "height or empty or degen"
<code>is_valid</code>	$O(n)$	carry (low, high); is not <code>None</code> , not truthiness	-k valid
three recursive walks	$O(n)$	one list, passed down; a new list per call	-k "order or traversals"
<code>level_order</code>	$O(n)$	your growing <code>CircularQueue</code> ; left before right	-k level
<code>delete</code>	$O(h)$	the parent; the root; successor = the right child	-k delete

All 26 at once:

```
pytest tests/test_tree.py -v
```

Before you show the TA: `git diff --stat tests/` must print nothing. And check by eye that nothing in `dsa/tree.py` keeps a Python list, set or dict as an attribute, and that `level_order` uses `CircularQueue`.

Part 10 — Take-home practice (not graded)

The question bank for this week is `docs/question-bank/week11-questions.md`, with answers in `week11-answers.md`. Do the questions before opening the answers.

1. **Part G — write the code**, in `practice/week11.py`: `count_leaves`, `is_balanced`, `range_values`, `build_balanced` and `lowest_common_ancestor` (W11-C1 to W11-C5). They take a `TreeNode` `root`, not a `BinarySearchTree`, and the tests count how many nodes you read — so an answer that walks the whole tree when it needs one path fails:

```
pytest tests/test_practice_week11.py -v
```

Hint for `range_values`: it is your `_in_order`, with two ifs that skip a subtree the BST property says cannot hold an answer.

2. **W11-S1** — draw a BST after each of ten inserts, then after four deletes that cover all three cases.
3. **W11-T3** — rebuild a tree from its pre-order and in-order walks.
4. **W11-B3** — a delete that passes the two-children test and still breaks the tree. Find the input.
5. **W11-K2** — a height that calls itself twice on the taller child. It is correct; count its calls on a chain of 20 nodes.

The worked solutions are in `solutions/dsa/tree.py` and `solutions/practice/week11.py` — for after you have tried. `pytest --solutions tests/test_practice_week11.py` runs the tests on them.

Part 11 — Bridge to Lecture 12: the tree in an array

This week's tree is made of nodes and links, and its shape depends on the order the values arrive in. Next week's is made of **no** links at all.

Number the nodes of a **complete** binary tree level by level, left to right, starting from 0: the root is 0, its children 1 and 2, theirs 3, 4, 5, 6. Then the children of node i are always $2i + 1$ and $2i + 2$, and its parent is $(i - 1) // 2$. So the tree can live in a plain Array, in level order — which is exactly the order your `level_order` produces.

```
from viz.draw import draw_array_as_tree
draw_array_as_tree([1, 3, 2, 7, 4, 5, 9], highlight=[0])
```

Draw it, and check two things by hand: node 1 (value 3) has children at 3 and 4 (values 7 and 4); node 5 (value 5) has its parent at $(5 - 1) // 2 = 2$ (value 2). Every parent in that picture is smaller than its children. That is the **heap property** — weaker than the BST property,

and in exchange the shape is always complete, so the height is always $\lfloor \log_2 n \rfloor$, whatever order the values came in. Lecture 12 builds a priority queue on it in `dsa/heap.py`.

Summary

Idea	The one line to keep
The tree	One root; each other node has one parent; <code>self.root</code> is all the tree stores.
Height	Edges on the longest root-to-leaf path; empty tree -1 , leaf 0 .
BST property	Left subtree smaller, right subtree larger — check it with a range.
Search, insert	One path down, $O(h)$; insert stops at the parent and hangs a new leaf.
Min, max	Left (right) until you cannot; no comparisons.
Recursion	Base case <code>None</code> ; answer from the two subtrees' answers.
Traversals	In-order sorted; pre-order rebuilds; post-order frees and evaluates.
Level-order	A queue of nodes — your growing <code>CircularQueue</code> .
Delete	Leaf: cut. One child: splice. Two: copy the successor, delete it instead.
The root	The one node with no parent: <code>self.root = child</code> .
Measured	Sorted input: height $n - 1$, $O(n)$ search, and a <code>RecursionError</code> at $1,000$.

Lab 12 — Lab 12 — Heaps and Priority Queues

How to use this lab. Read the matching section of Lecture 12 before each part. **Draw before you code:** every heap in this lab has at most ten values, and you should draw each one twice — as a tree and as the array under it. At each **Checkpoint**, write your prediction down first. Build `dsa/heap.py` one method at a time, in the order of the parts, and run the tests named in each part — the order is chosen so that each part's tests can pass before the next part is written. Parts 1–6 are the session; Parts 7–10 are for home.

Duration	One 2-hour lab session, plus about 3 hours at home
You will write	<code>dsa/heap.py</code> — <code>MinHeap.is_valid</code> , <code>peek</code> , <code>_sift_up</code> , <code>push</code> , <code>_sift_down</code> , <code>pop</code> , <code>heapify</code> ; <code>PriorityQueue.enqueue</code> , <code>dequeue</code> , <code>peek</code> ; and, for yourself, a small <code>CountingMinHeap</code> class
Graded by	<code>tests/test_heap.py</code> (28 tests)
Connects to	Lecture 12 — Heaps and priority queues; Lecture 04 (your <code>DynamicArray</code>); Lecture 10 (heap sort); Lecture 13 — Hash tables

What you will be able to do

1. move between a heap drawn as a tree and the same heap as an array, using $2i + 1$, $2i + 2$ and $(i - 1) // 2$;
2. check the heap property on an array, edge by edge, in $O(n)$;
3. write sift-up and sift-down, and trace both as a column of arrays;
4. explain why sift-down must swap with the **better** child, and show an input on which “any child that beats it” goes wrong;
5. write `heapify` bottom-up and say why the loop must run **backwards**;
6. write every comparison through `_beats`, so that `MaxHeap` works with no code of its own — and predict which tests fail when you do not;
7. build a priority queue on top of the heap, with a tie-breaking counter;
8. count comparisons with a subclass, and show `heapify`'s $O(n)$ against the $O(n \log n)$ of `n` pushes.

Part 0 — Before you start

0.1 Environment and prerequisite

From the DSA27 folder, with the virtual environment active (the prompt starts with `(.venv)`):

```
git pull
pytest -m "not challenge" -q          # the environment check: must
↪ pass
pytest tests/test_dynamic_array.py -q # your Week 4 DynamicArray:
↪ must pass
pytest tests/test_heap.py -q          # 28 failures: nothing written
↪ yet
```

The heap stores its values in **your** `DynamicArray` from Week 4, and uses four of its operations: `append` (push), `pop()` with no argument (removing the last item in `pop`), `self._items[i]` for reading and `self._items[i] = x` for writing. If `test_dynamic_array.py` fails, fix that first: a broken `__setitem__` makes every swap in this lab look like a heap bug.

The 28 failures are all `NotImplementedError` — that is the starting line.

0.2 Read the skeleton: what is given, what is yours

Open `dsa/heap.py`. Read all of it before you write anything.

Class	Given to you	Yours
MinHeap	<code>__init__(values=())</code> — copies the values into <code>self._items</code> , a <code>DynamicArray</code> , and calls <code>heapify</code> if there are any; <code>_beats</code> ; <code>is_empty</code> , <code>__len__</code> , <code>__repr__</code>	<code>_sift_up</code> , <code>_sift_down</code> , <code>push</code> , <code>pop</code> , <code>peek</code> , <code>heapify</code> , <code>is_valid</code>
MaxHeap	everything: it overrides <code>_beats</code> only	nothing — if your <code>MinHeap</code> is written right
PriorityQueue	<code>__init__</code> — <code>self._heap</code> , a <code>MinHeap</code> , and <code>self._counter = 0</code> ; <code>is_empty</code> , <code>__len__</code>	<code>enqueue</code> , <code>dequeue</code> , <code>peek</code>

Read `_beats` twice:

```
def _beats(self, child, parent):
    return child < parent
```

It answers one question: **must child move above parent?** In a min-heap, yes when it is smaller; in MaxHeap, whose `_beats` says `child > parent`, yes when it is larger. Every comparison between two values in your code must be a call to `self._beats(...)`, never a `<` or `>` of your own. That is how one class gives you both heaps (Lecture 12, “Order: the heap property”).

Notice also what `__init__` does: `MinHeap([5, 3, 8])` calls **your** `heapify`. Until Part 5, build heaps in the REPL with `MinHeap()` and `push`, not from a list.

0.3 The storage rule, this week

A `dsa/` structure stores its data only in the course `Array`, in node objects, or in another `dsa/` structure — never in a Python `list`, `dict` or `set` (Lecture 02). Here:

- `MinHeap` keeps everything in `self._items`, your `DynamicArray`. There is no second structure: the tree **is** the array.
- `PriorityQueue` keeps everything in `self._heap`, a `MinHeap`. The triples it pushes, `(priority, counter, item)`, are **values** — one small record each — not storage.
- `list(heap._items)` for drawing or printing is fine: it is output, not storage.

0.4 The tests, and one warning about -k

Eleven test functions run twice, once with `[MinHeap]` and once with `[MaxHeap]` in their names; one more compares the two classes; five test the priority queue. That is 28.

`-k` matches the file name too, and the file is `test_heap.py`. So `-k heap` selects **all 28** tests. The filters this lab uses avoid the word:

Filter	Selects
<code>-k "push_then_peek or after_every_push"</code>	4 — Part 3
<code>-k "empty_heap or single_element or interleaved"</code>	6 — Part 4
<code>-k "not priority"</code>	23 — every <code>MinHeap/MaxHeap</code> test, Part 5
<code>-k MaxHeap</code>	the 11 <code>[MaxHeap]</code> cases — Part 6
<code>-k priority</code>	the 5 priority-queue tests — Part 7

Part 1 — The array is the tree

Lecture 12, “The array is the tree”. Nothing to write yet: this part is paper and the REPL.

1.1 The idea

Number a complete binary tree level by level, left to right, from 0. Node `i` lives in slot `i`. Then:

```
left(i)   = 2*i + 1
right(i)  = 2*i + 2
```

```
parent(i) = (i - 1) // 2          for i > 0
```

A child index that is $\geq n$ does not exist. The leaves are the back half of the array, from $n // 2$ on; the last node with a child is $n // 2 - 1$.

1.2 Draw it

Draw `[2, 4, 3, 9, 7, 8, 5, 12, 10]` as a tree on paper, with each node's index beside it. Then check your drawing (paste into `notebooks/12-heaps.ipynb`):

```
from viz.draw import draw_array_as_tree
draw_array_as_tree([2, 4, 3, 9, 7, 8, 5, 12, 10], highlight=1)
```

`draw_array_as_tree` knows nothing about heaps. It draws any list as a tree — which makes it the debugging tool of the week: whenever a test fails, draw `list(h._items)` and look for a parent that a child beats.

1.3 Checkpoint

Checkpoint 1. A heap holds 12 values, at indices 0 to 11.

- (a) What are the children of index 4? Of index 5?
- (b) What is the parent of index 11? Of index 1?
- (c) Which index is the **last parent** — the last node with at least one child? Which indices are leaves?
- (d) What is the height of the tree (the root alone has height 0), and on which level is index 7?
- (e) Index 5 has a left child. Does it have a right child?

Part 2 — is_valid and peek

Lecture 12, “Order: the heap property”.

2.1 The idea

The heap property is a statement about **edges**: for every parent-child pair, the child does not beat the parent. Every node except the root has exactly one parent, so there are exactly $n - 1$ edges — and walking over the **children**, from index 1 to $n - 1$, visits each edge once.

`peek` is the reason a heap exists: the best value is at index 0.

2.2 Draw it

For each array, draw the tree, and put a cross on every edge where a child beats its parent (as a min-heap):

[1, 3, 2, 7, 4] [1, 3, 2, 2, 4] [5, 5, 5] [3, 4, 5, 1]

2.3 Checkpoint

Checkpoint 2. (a) Which of the four arrays above are valid min-heaps? (b) How many calls to `_beats` does a correct `is_valid` make on a **valid** heap of n values? On an invalid one, can it make fewer? (c) A student's `is_valid` checks only the root against its two children. Which of the four arrays does it get wrong?

2.4 Write it

`is_valid`:

1. For every child index from 1 to `len(self._items) - 1`: compute its parent; if `self._beats(child's value, parent's value)`, the property is broken — return `False`.
2. After the loop, return `True`.

peek: raise `IndexError` if the heap is empty; otherwise return the value at index 0.

2.5 Test it — by hand

No test can run `is_valid` yet: they all build their heaps with `push` or `heapify`. So test it at the REPL, by filling `_items` directly:

```
>>> from dsa.heap import MinHeap, MaxHeap
>>> h = MinHeap()
>>> for v in [1, 3, 2, 2, 4]:
...     h._items.append(v)
...
>>> h.is_valid(), h.peek()
(False, 1)
```

Try all four arrays of 2.2, and one of them in a `MaxHeap` as well. Then `MinHeap().peek()` must raise `IndexError`.

2.6 When it fails

Bug	What you see	Fix
loop from 0: <code>range(len(self._items))</code>	index 0's "parent" is $(0 - 1) // 2 = -1$, which your <code>DynamicArray</code> reads as the last item: the root beats it, so <code>is_valid</code> rejects almost every valid heap	start at 1: the root has no parent
<code>items[child] < items[parent]</code> instead of <code>_beats</code>	right for <code>MinHeap</code> ; for <code>MaxHeap</code> , it rejects every valid max-heap — once the rest is written, 5 [MaxHeap] tests fail	<code>self._beats(items[child], items[parent])</code>

Bug	What you see	Fix
only the root's children checked	nothing, for now: all 28 tests pass with this bug once the rest works	check every edge — see below
peek returns None when empty	DID NOT RAISE IndexError in the empty-heap test	raise IndexError

The third row matters most, because no test catches it: every heap the tests check is either valid, or broken at the root (`test_is_valid_rejects_a_broken_array` swaps index 0 with index 2). A validator that only looks at the top is a validator you cannot trust in Parts 3–5, where `is_valid` is how the tests catch **your** sifting bugs.

Part 3 — `_sift_up` and push

Lecture 12, “Sift Up: push”.

3.1 The idea

A new value can only go in one place without breaking the shape: the end of the array. After that, the only edge that can be wrong is the one between the new value and its parent. Swap while it is wrong; each swap moves the problem one level up; stop at the root at the latest.

3.2 Draw it

Push 4, 9, 2, 7, 1, 5 into an empty min-heap. After **each** push, draw the tree, write the array under it, and mark each swap with an arrow.

3.3 Checkpoint

Checkpoint 3. (a) Give the array after each of the six pushes, and the number of swaps each push makes. (b) Push 1 made the most swaps. Could any push into a heap of 5 values make more? What is the most a push can make into a heap of n values? (c) Why does the sift-up stop at index 0 even if you forget to test for it — or does it?

3.4 Write it

`_sift_up(index)`:

- While `index` is not the root:
 - compute the parent's index;
 - if the item at `index` does **not** beat its parent, stop;
 - otherwise swap the two items, and continue from the parent's index.

push(value): append the value to `self._items`, then sift up from the last index.

Hints:

- A swap in Python is one line: `a[i], a[j] = a[j], a[i]`. It works on your `DynamicArray` because it has both `__getitem__` and `__setitem__`.
- Pass the arguments to `_beats` in the right order: the **child** first. For `MinHeap` the order decides the answer: `_beats(3, 5)` is `True`, `_beats(5, 3)` is `False`.

3.5 Test it

```
pytest tests/test_heap.py -v -k "push_then_peek or after_every_push"
```

Four tests: `test_push_then_peek_gives_the_best` pushes 5, 3, 8, 1, 9 and peeks; `test_invariant_holds_after_every_push` pushes 40 random values and calls `is_valid()` after each — for both classes.

3.6 When it fails

Bug	What you see	Fix
<code><</code> in place of <code>_beats</code>	only the <code>[MaxHeap]</code> tests fail — <code>assert 1 == 9</code> in the <code>[MaxHeap]</code> peek test	always <code>self._beats(child, parent)</code>
<code>parent = index // 2</code> (the 1-based formula)	only the <code>[MaxHeap]</code> after-every-push test fails, with <code>assert False</code>	<code>(index - 1) // 2</code> — see below
<code>while index >= 0</code>	<code>(0 - 1) // 2</code> is <code>-1</code> , the last item: the root swaps with a leaf. <code>assert 3 == 1</code> in the <code>[MinHeap]</code> peek test	<code>while index > 0</code>
no swap, only a comparison	<code>assert 5 == 1</code> : nothing ever moves	swap, then move index to the parent

The second bug is the dangerous one. For odd indices `index // 2` happens to be right, and for even ones it names the node just **after** the true parent — so most pushes still work, and three of the four tests pass. On paper: `[1, 5, 2, 6]` plus a push of 3 at index 4 compares 3 with index 2 (value 2), stops, and leaves 3 under 5. Whenever you write an index formula, try it on indices 1, 2, 3 and 4 by hand.

Part 4 — `_sift_down` and `pop`

Lecture 12, “Sift Down: `pop`” and “The trap: swap with the **better** child”.

4.1 The idea

The root is the answer. To remove it without leaving a hole, take the **last** item — the only one whose slot can disappear — and put it at the root. Now the root may be beaten by a child. Swap it with its **better** child, which beats both it and its sibling, and continue below. Stop when no child beats it or it has no children.

4.2 Draw it

Start from the min-heap [1, 4, 2, 7, 5, 3, 6]. Draw it. Pop three times, drawing the tree after each pop: first the last item moved to the root, then each swap.

4.3 Checkpoint

Checkpoint 4. (a) Give each value returned and the array after each of the three pops. (b) A student's sift-down swaps with the **first** child that beats the item — left if the left one does, otherwise right. Run the first pop that way: which array results, and what does the **second** pop return? (c) How many comparisons can one sift-down level make, and why not one?

4.4 Write it

_sift_down(index):

1. Loop:
 - `best = index`; compute left and right;
 - if left exists (it is `< len(self._items)`) and its item beats the item at best, set `best = left`;
 - if right exists and its item beats the item at best, set `best = right`;
 - if `best == index`, stop; otherwise swap the items at index and best, and continue from best.

pop():

1. Empty: raise `IndexError`.
2. Remove the last item with `self._items.pop()`.
3. If the heap is now empty, that item **was** the root: return it.
4. Otherwise save the root, write the last item into index 0, sift down from 0, and return the saved root.

Hints:

- Compare the right child with `items[best]`, not with `items[index]`. That one detail is what makes it the **better** child.
- `self._items.pop()` with no argument removes the **last** item, in $O(1)$. With argument 0 it would shift everything — and give the wrong answer too.

4.5 Test it

```
pytest tests/test_heap.py -v -k "empty_heap or single_element or
↳ interleaved"
```

Six tests: the empty heap (pop and peek raise `IndexError`), a single element pushed and popped, and `test_interleaved_push_and_pop`, which pushes 60 random values, pops every third time, checks each popped value against the true best, and calls `is_valid()` after every step — for both classes. It is the test that makes sift-up and sift-down work **together**.

4.6 When it fails

Bug	What you see	Fix
no one-item case (step 3)	<code>IndexError: 0</code> — raised by your DynamicArray , not by the heap — in the single-element test, and later in 7 more tests	after the removal, if the heap is empty, return the item
<code>right < size</code> check copied as <code>left < size</code> first child that beats, not the better one	<code>IndexError: 2, IndexError: 6, ...</code> from <code>DynamicArray.__getitem__</code> : the right child is read when it does not exist these tests may pass; in Part 5, 6 tests fail, among them the after-every-pop test with <code>assert False</code>	each child needs its own bound compare right with <code>items[best]</code>
one swap, no loop	the heap is repaired one level deep only; 4 tests fail in Part 5	loop until <code>best == index</code>
return <code>self._items[0]</code> after the sift-down	the interleaved test fails with <code>assert 21 == 14</code> : you returned the new root	save the root before overwriting it
<code>self._items.pop(0)</code> as the whole of pop	it returns the right value once, then the heap is broken: <code>assert False</code> from <code>is_valid</code>	the last item moves to the root

Note what is **missing** from the table: a pop that forgets the empty check still passes `test_empty_heap`, because your `DynamicArray.pop` raises `IndexError` on its own. Write the check anyway: the message should say “empty heap”, not something about an array the caller has never heard of.

Part 5 — heapify

Lecture 12, “Build-Heap in $O(n)$ ”.

5.1 The idea

`MinHeap(values)` has the right shape already — every array does — and possibly no order at all. A leaf is a heap of one. Sift down every node that **has** a child, from the last one ($n // 2 - 1$) **back** to the root. When node i is sifted down, both of its subtrees are already heaps, which is exactly the situation sift-down repairs.

5.2 Draw it

Heapify `[8, 7, 6, 5, 4, 3, 2, 1]` on paper. Draw the tree once; then, for each index from the last parent back to 0, sift it down and redraw.

5.3 Checkpoint

Checkpoint 5. (a) Which index do you sift down first, and which last? (b) Give the array after each sift-down, and the total number of swaps. (c) Why must the loop run **backwards**? Give a three-level example of what goes wrong if you sift down index 0 first. (d) The loop could start at $n - 1$ instead of $n // 2 - 1$ and still be correct. Why? What does it waste?

5.4 Write it

One loop: for each index from `len(self._items) // 2 - 1` **down to and including** 0, call `self._sift_down(index)`.

A range with a negative step stops **before** its stop value, so the stop must be `-1` to include 0. That is the entire exercise, and the lecture's heap sort (Week 10) has the same line.

5.5 Test it

```
pytest tests/test_heap.py -v -k "not priority"
```

All 23 `MinHeap` and `MaxHeap` tests — every test that builds a heap from a list now runs, including `test_heapify_from_arbitrary_order` on `[9, 4, 7, 1, 8, 2, 6, 3, 5]`, the lecture's heapify trace.

5.6 When it fails

Bug	What you see	Fix
<code>range(n // 2 - 1, 0, -1)</code> — the root is skipped	5 tests fail; the <code>max-differs-from-min</code> test says <code>assert 5 == 1</code> : the root was never sifted	stop at <code>-1</code>
forward: <code>range(n // 2)</code>	5 tests fail, for example <code>assert [3, 1, 2, 5, 7, 8, ...] == [1, 2, 3, 5, 7, 8, ...]</code> — popping gives a wrong order	backwards: subtrees first

Bug	What you see	Fix
starting one too early, at $n // 2 - 2$	4 tests fail — the last parent is never sifted	start at $n // 2 - 1$
<code>n</code> pushes in disguise: for <code>i</code> in <code>range(1, n)</code> : <code>self._sift_up(i)</code>	nothing: all 28 pass. It is correct, and $O(n \log n)$	Part 8 measures it — the docstring asks for $O(n)$

The last row is the one to think about. A forward `_sift_up` over every index is a correct heap construction — it is exactly n pushes, done in place — and no test can tell it from heapify by the answers alone. Only the **cost** differs. That is what Part 8 is for.

Part 6 — Break it: MaxHeap for free

Lecture 12, “Order: the heap property”. Your MinHeap passes its tests; so, with no code of its own, does MaxHeap — **if** every comparison went through `_beats`. Check it, and then break it on purpose.

```
pytest tests/test_heap.py -v -k MaxHeap # 11 passed
```

Now, in `_sift_down` only, replace the two `self._beats(a, b)` calls by `a < b`. Leave everything else alone.

Checkpoint 6. Before you run anything:

- (a) Which class still passes every test, and why?
- (b) For MaxHeap, what does the broken sift-down do instead of what it should?
- (c) Of the 28 tests, how many fail? Name the kinds of test that fail, and say why `test_invariant_holds_after_every_push[MaxHeap]` still passes.

Now run the whole file and compare with your prediction:

```
pytest tests/test_heap.py -v
```

Then **put `_beats` back** and run it again: 28 passed. `git diff dsa/heap.py` shows what you have changed since your last commit, if you are not sure.

What to take from this: one class, two heaps, and a single point of truth for “better”. A comparison written anywhere else is a comparison MaxHeap does not know about.

Part 7 — PriorityQueue: a heap of triples

Lecture 12, “The Priority Queue on Top”.

7.1 The idea

Push (priority, counter, item) onto self._heap, then add 1 to self._counter. Python compares tuples left to right, so the priority decides; on a tie, the counter decides — and since every counter is different, the item itself is **never** compared. dequeue and peek return only the item: index 2 of the triple.

7.2 Checkpoint

Checkpoint 7. enqueue("b", 2), enqueue("a", 2), enqueue("c", 1).

- (a) Give the heap array, as triples, after each enqueue.
- (b) In what order do three dequeues return the items?
- (c) If enqueue pushed (priority, item) instead, in what order would they come out? And what would happen with enqueue({"task": "a"}, 1) followed by enqueue({"task": "b"}, 1)?

7.3 Write it

- **enqueue(item, priority):** push the triple onto the heap, then add 1 to the counter.
- **dequeue():** empty → IndexError; otherwise pop a triple from self._heap and return its item.
- **peek():** the same with peek.

Build on the heap — call its methods. Do not reach into self._heap._items, and do not sift anything here: that work is done.

7.4 Test it

```
pytest tests/test_heap.py -v -k priority
```

Five tests: serving the lowest number first; test_priority_queue_is_not_a_fifo, which enqueues 0..4 with priorities 9, 7, 5, 3, 1 and expects 4, 3, 2, 1, 0; peek and length; the empty queue; and two dictionaries with equal priority.

7.5 When it fails

Bug	What you see	Fix
(priority, item) pairs, and [2] in dequeue	IndexError: tuple index out of range in 3 tests, and TypeError: '<' not supported between instances of 'dict' and 'dict' in the fourth	push the triple
a counter in the triple, but never incremented	only the unordered-items test fails, with the same TypeError: every counter is 0, so a tie still reaches the items	self._counter += 1 in enqueue

Bug	What you see	Fix
returning the whole triple	<code>assert (1, 1, 'fix the build') == 'fix the build'</code>	return index 2
peek returns None when empty	DID NOT RAISE IndexError in the empty-queue test	raise, as dequeue does

Part 8 — Measure it: heapify against n pushes

Lecture 12, “Measured: heapify against n pushes”. A timing depends on your machine; a **count** of comparisons does not. You will count them with a subclass that looks exactly like a MinHeap to your code.

8.1 Write CountingMinHeap

This is not graded, and it does not go in `dsa/`: write it in the scratch space of `notebooks/12-heaps.ipynb`.

```
import random
from dsa.heap import MinHeap

class CountingMinHeap(MinHeap):
    """A MinHeap that counts every comparison it makes."""

    def __init__(self, values=()):
        self.comparisons = 0      # first: __init__ runs heapify
        super().__init__(values)

    def _beats(self, child, parent):
        self.comparisons += 1
        return super()._beats(child, parent)
```

Because **all** your comparisons go through `_beats` (Part 6), overriding that one method sees every one of them. This is the payoff of the design, a second time.

Checkpoint 8. Predict, for $n = 1,024$, the comparisons **per element** for (a) heapify on descending input, (b) n pushes of descending input into a min-heap, (c) n pushes of random input. Then for $n = 16,384$. Which of the three grows with n ?

8.2 Count

```
def by_heapify(values):
```

```

    return CountingMinHeap(values).comparisons

def by_pushes(values):
    h = CountingMinHeap()
    for v in values:
        h.push(v)
    return h.comparisons

for n in [1024, 16384]:
    descending = list(range(n, 0, -1))
    shuffled = random.Random(n).sample(range(n), n)
    print(f'n = {n}')
    for name, values in [('descending', descending), ('random',
        ↪ shuffled)]:
        a = by_heapify(values) / n
        b = by_pushes(values) / n
        print(f' {name:10} heapify {a:5.2f}   pushes {b:5.2f} '
              ' comparisons per element')

```

With the reference solution:

n = 1024					
descending	heapify	1.98	pushes	8.01	comparisons per element
random	heapify	1.86	pushes	2.25	comparisons per element
n = 16384					
descending	heapify	2.00	pushes	12.00	comparisons per element
random	heapify	1.88	pushes	2.27	comparisons per element

Your counts should match to the last digit if your sifts compare exactly as Part 4.4 says. If your heapify shows **12.00** on descending input at n = 16,384, it is the n-pushes version from the last row of Part 5.6.

8.3 Time it

```

from viz.complexity import measure

def build_by_pushes(values):
    h = MinHeap()
    for v in values:
        h.push(v)

sizes = [2_000, 4_000, 8_000, 16_000, 32_000]
descending = lambda n: list(range(n, 0, -1))

```

```
_, t_heapify = measure(MinHeap, sizes, descending)
_, t_pushes = measure(build_by_pushes, sizes, descending)
for n, a, b in zip(sizes, t_heapify, t_pushes):
    print(f'{n:6} heapify {a * 1e3:7.1f} ms pushes {b * 1e3:7.1f} ms'
          f' ratio {b / a:4.1f}')
```

`measure(MinHeap, ...)` times the constructor, which is your `heapify` plus the copy into the `DynamicArray`. On the instructor's machine the ratio grew from about 4.5 at $n = 2,000$ to about 7 at $n = 32,000$: the pushes grow like $n \log n$, `heapify` like n , so the gap keeps widening. Your milliseconds will differ; the widening will not.

Part 9 — Check your heap against `heapq`

Only now, with your own heap passing its tests, meet the library version (Lecture 12, “Where priority queues are used”). Python's `heapq` works on a plain list with exactly your index arithmetic. Use it as an **oracle** — a second opinion — on many random sequences of operations:

```
import heapq, random
from dsa.heap import MinHeap

rng = random.Random(1)
for trial in range(2_000):
    mine, oracle = MinHeap(), []
    for step in range(rng.randint(0, 40)):
        if oracle and rng.random() < 0.4:
            assert mine.pop() == heapq.heappop(oracle), (trial, step)
        else:
            v = rng.randint(0, 20)
            mine.push(v)
            heapq.heappush(oracle, v)
            assert mine.is_valid() and len(mine) == len(oracle)
    values = [rng.randint(0, 20) for _ in range(rng.randint(0, 30))]
    built = MinHeap(values)
    assert sorted(built._items) == sorted(values) and built.is_valid()
print('2,000 random runs: all agree')
```

Small values (0 to 20) on purpose, so that there are many duplicates, which the graded tests touch only once. The two heaps need not hold the same **array** — there can be more than one valid heap of the same values — but every pop must return the same value. If an assertion fails, the message gives the trial and the step: rerun that trial alone and draw it.

This is also how you use `heapq` from now on: in your own programs, call the library; in `dsa/`, the point is that you can write it.

Part 10 — Exercises at a glance

Method	Target cost	The trap	- k filter
<code>is_valid</code>	$O(n)$	every edge, not just the root's; start at 1	by hand, Part 2
<code>peek</code>	$O(1)$	<code>IndexError</code> when empty	<code>empty_heap</code>
<code>_sift_up,</code> <code>push</code>	$O(\log n)$	$(i - 1) // 2$; stop at the root; <code>_beats(child, parent)</code>	Part 3 (4)
<code>_sift_down,</code> <code>pop</code>	$O(\log n)$	the better child; each child's own bound; the one-item heap	Part 4 (6)
<code>heapify</code>	$O(n)$	backwards from $n // 2 - 1$ to 0 inclusive; not n pushes	"not priority" (23)
<code>MaxHeap</code>	—	every comparison through <code>_beats</code>	<code>MaxHeap</code> (11)
<code>PriorityQueue</code>	$O(\log n)$	the triple; increment the counter; return the item	<code>priority</code> (5)

All 28 at once:

```
pytest tests/test_heap.py -v
```

Before you show the TA: `git diff --stat tests/` must print nothing. And check by eye that no `<` or `>` between two values appears anywhere in `MinHeap` except inside `_beats`, and that `is_valid` looks at every child — the tests cannot see either, but the TA will look.

Part 11 — Take-home practice (not graded)

The question bank for this week is `docs/question-bank/week12-questions.md`, with answers in `week12-answers.md`. Do the questions before opening the answers.

1. **Part G — write the code**, in `practice/week12.py`: `is_min_heap`, `top_k`, `merge_sorted`, `join_ropes` and `running_medians` (W12-C1 to W12-C5). Use your own `MinHeap` and `MaxHeap`; two of the tests count your comparisons, so sorting everything fails:

```
pytest tests/test_practice_week12.py -v
```

Hint for `running_medians`: a `MaxHeap` for the smaller half and a `MinHeap` for the larger half — Part 6 is why you have both for free.

2. **W12-T1** — Homework 12, item 2: push 6, 2, 8, 1, 5, 3; then pop twice.
3. **W12-T2** — `heapify` [3, 9, 2, 1, 4, 5, 8, 7, 6, 0] and count the swaps.
4. **W12-B2** — the first-child sift-down of Checkpoint 4, as a find-the-bug.

5. **W12-K3** — the forward heapify: find an input on which it fails without running it.

The worked solutions are in `solutions/dsa/heap.py` and `solutions/practice/week12.py` — for after you have tried. `pytest --solutions tests/test_practice_week12.py` runs the tests on them.

Part 12 — Bridge to Lecture 13: hash tables

A heap answers one question fast — *what is the best item?* — and every other question slowly. Try this at the REPL, with your finished heap:

```
from dsa.heap import MinHeap
h = MinHeap(range(1, 1001))
print(1000 in list(h._items))    # only by looking at all of it
```

There is no shortcut: 1000 could be in any leaf, and there are 500 of them. So two operations that real priority queues need are $O(n)$ here:

- **“Is this task already queued?”** — a search through the array.
- **“Make this task more urgent.”** — find it ($O(n)$), change its priority, then sift it up ($O(\log n)$). The sift is cheap; the **finding** is not. Dijkstra’s algorithm, which Lecture 12 named as beyond this course, does this millions of times.

The standard fix keeps, next to the heap, a second structure that maps each item to its current **index** in the array, updated on every swap. Finding a task then costs whatever that map costs to look up. With a sorted array and binary search it would be $O(\log n)$. Lecture 13 builds a structure where it is **$O(1)$** on average — the hash table — by computing where a key lives instead of searching for it. Bring one question: what does a hash table have to give up to be that fast?

Summary

Idea	The one line to keep
Shape	A complete binary tree: an array with no gaps, height $\lfloor \log_2 n \rfloor$.
Index arithmetic	Children $2i + 1, 2i + 2$; parent $(i - 1) // 2$; leaves from $n // 2$.
Heap property	Every child against its parent: $n - 1$ checks, starting at index 1.
push	Append, then sift up while the item beats its parent.

Idea	The one line to keep
pop	Save the root, move the last item up, sift down; the one-item heap is special.
The trap	Sift down past the better child — compare the right with <code>items[best]</code> .
heapify	Sift down from $n // 2 - 1$ back to 0 inclusive: $O(n)$. n pushes are $O(n \log n)$.
<code>_beats</code>	One method decides “better”; <code>MaxHeap</code> overrides only it.
PriorityQueue	Push (priority, counter, item); the counter breaks ties, first come first served.
Measuring	Count comparisons by overriding <code>_beats</code> in a subclass.

Lab 13 — Lab 13 — Hash Tables, Twice

How to use this lab. The routine from Week 4: read the lecture section named at the top of each part, **draw before you code** — here that means a row of 8 slots on paper, with the keys in them and the probes as arrows — and predict at each **Checkpoint**. Then write one method, run the tests named in its part, and only then move on. You build the same map twice: first with chains, then with probing and tombstones. Parts 1–7 are the session; Parts 8–10 are for home.

Duration	One 2-hour lab session, plus about 3 hours at home
You will write	<code>dsa/hashmap.py</code> — <code>ChainingHashMap.get</code> , <code>put</code> , <code>delete</code> , <code>_resize</code> ; <code>OpenAddressingHashMap._probe</code> , <code>put</code> , <code>get</code> , <code>delete</code> , <code>_resize</code>
Graded by	<code>tests/test_hashmap.py</code> (17 tests)
Connects to	Lecture 13 — Hash tables; Week 5 (the Entry chain is a linked list); Week 4 (resizing is amortised); Week 14, whose Graph is built on your <code>ChainingHashMap</code>

What you will be able to do

1. compute `hash(key) % capacity` by hand for integer keys, and say which hashes are the same in every run and which are not;
2. draw a chaining table and an open-addressing table after every operation;
3. write `get`, `put` and `delete` for a chaining table, with the head of a chain as the special case;
4. write `_resize` so that every key is **rehashed**, and show what goes wrong when buckets are copied across instead;
5. write linear probing as a generator, and `put/get/delete` that stop at `None` but not at a tombstone;
6. explain, with the failing test in front of you, why an open-addressing delete must leave a tombstone;
7. count probes against the load factor, and make a hash table slow on purpose with keys that all collide.

Part 0 — Before you start

0.1 Environment and prerequisites

From the DSA27 folder, with the virtual environment active (the prompt starts with `(.venv)`):

```
git pull
pytest -m "not challenge" -q      # environment check: must pass
pytest tests/test_hashmap.py -q  # 17 failures: nothing written
```

The 17 failures are all `NotImplementedError` — that is the starting point.

What you need from earlier weeks:

- **The course Array** (Lecture 02) — both tables store everything in it.
- **Linked-list surgery** (Week 5): walking a chain with `entry = entry.next`, pushing on the front, and unlinking with a `prev` reference.
- **Generators** (Lab 03): `_probe` is one — a function with `yield`.
- **The % wrap** of the ring buffer (Week 7).

Nothing from `dsa/` is imported except `Array`: unlike the queues, this week stands on no exercise of yours.

0.2 Read the skeleton: what is given, what is yours

Open `dsa/hashmap.py` and read it from the top. Given to you:

Name	What it is
<code>_MISSING</code>	a private object meaning “no default was passed to <code>get</code> ”
<code>TOMBSTONE</code>	the marker an open-addressing delete leaves behind
<code>Entry</code>	a chain node: <code>key</code> , <code>value</code> , <code>next</code>
<code>both __init__s</code>	the <code>Arrays</code> , <code>_size</code> , <code>max_load</code> (and <code>_tombstones</code> for open addressing)
<code>ChainingHashMap._index</code>	<code>hash(key) % len(self._buckets)</code>
<code>load_factor</code> , <code>__len__</code> , <code>__contains__</code> , <code>__iter__</code>	on both classes

Yours: the four methods of `ChainingHashMap` and the five of `OpenAddressingHashMap` listed at the top of this lab. Read the given `ChainingHashMap.__iter__` now: it walks every chain of every bucket, which is the loop your `_resize` will need.

0.3 The storage rule, this week

A hash table built on a Python `dict` would be a joke with a straight face. The storage rule says a `dsa/` structure keeps its data only in the course `Array`, in node objects, or in another

dsa/ structure. So:

- ChainingHashMap: the buckets are an Array; the chains are Entry nodes.
- OpenAddressingHashMap: two Arrays, `_keys` and `_values`, side by side: slot `i` of one belongs with slot `i` of the other.
- No `list`, `dict` or `set` inside `dsa/hashmap.py` — not even “just for `_resize`”. A `resize` builds new Arrays and walks the old ones.

0.4 The tests, and a -k note

Eight tests are **parametrised** over both classes, so each runs twice; the class name is in the test id. One more test is only for open addressing:

```
# 8 tests, then 9 tests:
pytest tests/test_hashmap.py -v -k Chaining
pytest tests/test_hashmap.py -v -k "OpenAddressing or open_addressing"
```

`-k` matches case-insensitively, but not across the underscore: `-k OpenAddressing` selects 8 tests, and misses `test_open_addressing_delete_preserves_probe_chain` — the one that matters most. Use the second filter above.

Part 1 — Hashing by hand

Lecture 13, “Hash Functions”. Before writing a table, get a feel for the one line that is given to you: `hash(key) % capacity`.

1.1 At the REPL

```
>>> hash(42), hash(-1), hash(3.0), hash(True)
>>> hash((1, 2)) == hash((1, 2))
>>> hash("dsa")
>>> hash([1, 2])
```

Now quit the REPL and run this line **twice**, from PowerShell:

```
python -c "print(hash('dsa'), hash(12345), hash((1, 2)))"
python -c "print(hash('dsa'), hash(12345), hash((1, 2)))"
```

The string’s hash changes from run to run; the other two do not (Lecture 13, “Same run, same hash”). Then fix the seed and run it twice more:

```
$env:PYTHONHASHSEED = "0"
python -c "print(hash('dsa'))"
python -c "print(hash('dsa'))"
Remove-Item Env:PYTHONHASHSEED
```

With a fixed seed the string's hash is the same both times. Remember this command: in Part 6 you will need it.

Checkpoint 1. *Without running anything:*

- (a) *For a table of capacity 8, give the index of the keys 7, -3, 24 and $2^{61} - 1$. (Python's % is never negative here.)*
- (b) *`m = ChainingHashMap()`, then `m.put(3, "a")`, `m.put(3.0, "b")`, `m.put(True, "t")`, `m.put(1, "one")`. What are `len(m)`, `m.get(3)` and `m.get(True)`? Why?*
- (c) *Why does every trace in this lab use integer keys?*

Part 2 — Chaining: get and put

Lecture 13, “A linked list in every bucket” and “The three operations, in words”.

2.1 The idea

Each slot of `self._buckets` is `None` or the first `Entry` of a chain. `get` walks the chain at `self._index(key)`; `put` walks the same chain to see whether the key is already there, and if it is not, pushes a new `Entry` on the front.

2.2 Draw it

Draw 8 boxes in a row, numbered 0 to 7. Put the keys in, one at a time, each at the **front** of its chain (the value does not matter; write only the key):

Checkpoint 2. A `ChainingHashMap()` (capacity 8, `max_load 0.75`) receives `put` for the keys 3, 11, 6, 19, 14, in that order.

- (a) *Draw the buckets. Which chains have more than one entry, and in what order?*
- (b) *What is the load factor? Has the table resized?*
- (c) *`get(27)`: which bucket, how many entries are compared, and what happens?*

2.3 Write get

1. Find the chain: `entry = self._buckets[self._index(key)]`.
2. Walk it: while `entry` is not `None`, if `entry.key == key`, return its value; otherwise move to `entry.next`.
3. At the end of the chain: if `default` is `_MISSING`, **raise `KeyError(key)`**; otherwise return `default`.

Compare keys with `==`, never `is`: two equal strings can be different objects. And compare `default` with `is _MISSING`: `None` is a default a caller is allowed to pass.

2.4 Write put — without resizing, first

1. Find the chain and walk it. If you meet the key, **overwrite** its value and return — the size does not change.
2. Not found: `self._buckets[i] = Entry(key, value, self._buckets[i])` — one line that pushes on the front (Week 5's `push_front`).
3. `self._size += 1`.
4. If `self._size / len(self._buckets) > self.max_load`, call `self._resize(2 * len(self._buckets))`.

Step 4 calls a method you have not written yet. That is deliberate: until the table passes 6 keys, it never runs.

2.5 Test it

```
pytest tests/test_hashmap.py -v -k Chaining
```

With `get` and `put` written, you should see **4 passed** and 4 failed: `put_then_get`, `put_overwrites_without_growing`, `get_missing_key` and `collisions_are_handled` pass. `test_delete` and `test_delete_missing_key_raises` fail with `NotImplementedError` from `delete` (Part 3), and `test_survives_many_keys_and_resizes` and `test_load_factor_stays_bounded` fail with `NotImplementedError` from `_resize` (Part 4).

2.6 When it fails

Bug	What you see	Fix
no “already there?” walk in <code>put</code>	<code>assert 2 == 1</code> in <code>test_put_overwrites_without_growing</code> — <code>len</code> counted a duplicate	walk the chain first; overwrite and return
return <code>None</code> instead of raise <code>KeyError</code> in <code>get</code>	Failed: DID NOT RAISE <code>KeyError</code> in <code>test_get_missing_key</code> (and later in <code>test_delete</code>)	only default is <code>_MISSING</code> raises; anything else is returned
if default: or if default is <code>None</code> :	<code>get(key, None)</code> or <code>get(key, 0)</code> raises instead of returning	the sentinel is <code>_MISSING</code> ; test with <code>is</code>
<code>self._buckets[i] = Entry(key, value)</code> (no third argument)	keys that collide disappear: <code>KeyError</code> in <code>test_collisions_are_handled</code>	the new entry's next is the old head

Part 3 — Chaining: delete

3.1 The idea

Week 5's remove, on a chain whose head lives in an Array slot. Walk with two references, prev (starting at None) and entry. When entry.key == key:

- if prev is None, the match is the **head**: the bucket slot itself becomes entry.next;
- otherwise prev.next = entry.next skips it.

Then self._size -= 1 and return. If the walk ends: raise KeyError(key).

Checkpoint 3. Take the table of Checkpoint 2. Draw it after delete(11), then after delete(19). Which of the two is the head case? What would delete(27) do?

3.2 Write it, then test it

```
pytest tests/test_hashmap.py -v -k "Chaining and delete"
```

Two tests: test_delete and test_delete_missing_key_raises.

3.3 When it fails — and when it does not

Bug	What you see	Fix
no self._size -= 1	assert 1 == 0 in test_delete	the size counts keys, and one just left
self._buckets[i] = entry.next for every match, head or not	nothing — both tests pass	see below

The second bug is the lesson of this part. Unlinking a match in the middle of a chain by writing self._buckets[i] = entry.next throws away every entry **before** it, silently. The tests delete from a table of one key, where the match is always the head, so they cannot see it. Write your own check, at the REPL or in the notebook:

```
from dsa.hashmap import ChainingHashMap
m = ChainingHashMap()
for k in [3, 11, 19]:          # one chain: 19 -> 11 -> 3
    m.put(k, k)
m.delete(11)                  # the middle of the chain
print(len(m), m.get(19), m.get(3))  # must print: 2 19 3
```

A test suite is a set of examples. When the examples cannot reach a line of your code, you write the example that does.

Part 4 — Chaining: `_resize`

Lecture 13, “Resize: every key moves”.

4.1 The idea

`_resize(capacity)` swaps in a new, empty `Array(capacity)` and moves every entry of the old one into it. Each entry must go to its **new** index, `hash(key) % capacity`, which is usually not its old one. Re-link the existing Entry nodes; there is no need to create new ones.

Checkpoint 4. A `ChainingHashMap(capacity=4)` receives put for 1, 5, 2, 6.

- (a) Draw the buckets after each put. After which put does the table resize, and to what capacity?
- (b) Draw the table after the resize.
- (c) Suppose `_resize` had copied `old[i]` into `new[i]` instead. What would `get(5)` do, and why is `len(m)` still right?

4.2 Write it

1. Keep the old array: `old = self._buckets`; then `self._buckets = Array(capacity)`. From here on, `self._index` uses the new capacity — which is exactly what you want.
2. For each chain in `old`, walk it. For each entry:
 - save `nxt = entry.next` **first**;
 - compute `i = self._index(entry.key)`;
 - push the entry on the front of the new chain `i`: `entry.next = self._buckets[i]`, then `self._buckets[i] = entry`;
 - move on to `nxt`.
3. `_size` does not change: the same keys, in more buckets.

4.3 Test it

```
pytest tests/test_hashmap.py -v -k Chaining
```

All 8 must pass. `test_survives_many_keys_and_resizes` puts 500 **string** keys ("key0" to "key499"), so the table resizes seven times, from 8 to 1,024 buckets; `test_load_factor_stays_bounded` puts 500 integers and checks the load factor is at most `max_load + 0.05` at the end.

4.4 When it fails

Bug	What you see	Fix
buckets copied across, <code>new[i] = old[i]</code>	<code>KeyError: 'key0' (or another key) in</code> <code>test_survives_many_keys_and_resizes</code>	recompute every index with the new capacity
<code>entry = entry.next</code> after relinking the entry	<code>KeyError</code> in the same test: the relink overwrote next, so the walk jumps into the new table and the rest of the old chain is lost	save <code>nxt</code> before you relink
never calling <code>_resize</code> in <code>put</code>	<code>assert 62.5 <= (0.75 + 0.05) in</code> <code>test_load_factor_stays_bounded</code> — 500 keys in 8 buckets	the check in step 4 of Part 2.4
<code>self._index</code> computed before replacing <code>self._buckets</code>	the same <code>KeyError</code> as the copy	swap the array first, then compute indices

The exact key in the `KeyError` can change from run to run: the keys are strings, and their hashes are randomised (Part 1). The failure itself does not.

Part 5 — Open addressing: `_probe`, `put`, `get`

Lecture 13, “Linear probing: one key per slot”.

5.1 The idea

No chains: `_keys[i]` holds a key, `None` (never used) or `TOMBSTONE` (a deleted key was here); `_values[i]` holds the value that goes with `_keys[i]`. A key tries slots in the order its **probe sequence** gives:

$$h, h + 1, h + 2, \dots \pmod{\text{capacity}}, \quad h = \text{hash}(\text{key}) \pmod{\text{capacity}}$$

5.2 Draw it

Checkpoint 5. An `OpenAddressingHashMap()` (*capacity* 8, *max_load* 0.66) receives `put` for 3, 11, 19, 4.

- Draw the 8 key slots after each `put`. How many slots does each `put` probe?
- `get(12)`: list the slots probed. Where does it stop, and what does it return?
- Key 4 hashes to 4, yet sits somewhere else. What does that tell you about how `get(4)` must search?

5.3 Write `_probe`

A generator: compute `start = hash(key) % capacity` once, then for `step` in `range(capacity)`: `yield (start + step) % capacity`. Exactly capacity indices, every slot once, wrapping round the end.

5.4 Write `get` (and a helper, if you like)

Loop for `i` in `self._probe(key)`:

- `self._keys[i]` is `None` → a never-used slot: the key is **not** in the table. Stop.
- `self._keys[i]` is `TOMBSTONE` → **skip it**: keep probing.
- `self._keys[i] == key` → found: return `self._values[i]`.

If the loop ends, or stops at `None`: default, or `KeyError`, exactly as in the chaining `get`. `delete` will need the same search, so it is sensible to write it once as a helper — say `_find(key)`, returning the slot index or `-1` — and call it from both.

5.5 Write `put`

1. **Load check first.** If $(\text{self}._\text{size} + \text{self}._\text{tombstones} + 1) / \text{capacity}$ would exceed `max_load`, resize before inserting: to **double** the capacity if $(\text{self}._\text{size} + 1) / \text{capacity}$ alone exceeds it, otherwise to the **same** capacity (which just sweeps away the tombstones). Tombstones must be counted: Lecture 13, “Delete: why not just empty the slot?”, last paragraph.
2. Probe. Remember the **first** tombstone you pass. If you meet the key, overwrite its value and return. If you meet `None`, stop.
3. Insert at the first tombstone you passed if there was one, otherwise at the `None` where you stopped. If you reused a tombstone, `_tombstones -= 1`. Then `_size += 1`.

Until Part 7, `_resize` raises `NotImplementedError`; with the load check first, the fifth distinct key is fine and the sixth calls it.

5.6 Test it

```
pytest tests/test_hashmap.py -v -k "OpenAddressing or open_addressing"
```

With `_probe`, `get` and `put` written, `put_then_get`, `put_overwrites_without_growing`, `get_missing_key` and `collisions_are_handled` pass. The tests that delete or resize wait for Parts 6 and 7.

Part 6 — Tombstones, and breaking them on purpose

Lecture 13, “Delete: why not just empty the slot?”. This is the part of the week that the TA will ask you to explain.

6.1 Write delete

Find the key's slot with the same search as get. Not there: `KeyError`. There: `set_keys[i] = TOMBSTONE` and `_values[i] = None` (so the old value can be freed), `_size -= 1`, `_tombstones += 1`.

Checkpoint 6. Continue the table of Checkpoint 5 (3, 11, 19, 4 in slots 3, 4, 5, 6).

- (a) `delete(11)`. Draw the slots. Then `get(19)`: list the slots probed.
- (b) Now put (12). Does the load check trigger a resize? List the probes and say where 12 goes.
- (c) Go back to (a), but suppose delete had written `None` instead of `TOMBSTONE`. What do `get(19)` and `get(4)` do now?

6.2 Test it

```
pytest tests/test_hashmap.py -v -k "OpenAddressing or open_addressing"
```

`test_delete`, `test_delete_missing_key_raises` and `test_open_addressing_delete_preserves` should now pass too.

6.3 Break it

Change one word in your delete: `self._keys[i] = None` instead of `self._keys[i] = TOMBSTONE`.

Checkpoint 7. Before you run the tests: of the 17, which fail? Write down their names. Why do the other open-addressing tests that delete keys still pass?

Run all 17 and compare:

```
pytest tests/test_hashmap.py -v
```

Read the failing test's docstring and its four lines. Then **put TOMBSTONE back**, and run the tests again. If you are not sure you restored exactly what you had, `git diff dsa/hashmap.py` shows every line you changed.

6.4 When it fails

Bug	What you see	Fix
delete writes None	<code>KeyError: 8 in</code>	<code>TOMBSTONE</code>
get stops at a tombstone as if it were None	<code>test_open_addressing_delete_preserves_probe_chain</code> the same <code>KeyError: 8</code>	a tombstone means "keep going"

Bug	What you see	Fix
<code>_probe</code> yields <code>start + step</code> with no <code>% capacity</code>	<code>IndexError: Array index 8 out of range for length 8 — or 16, 64, 128, 1024, in a different test on a different run</code>	wrap: <code>(start + step) % capacity</code>
<code>put</code> inserts at the first tombstone without probing on	no test fails; the table can hold the same key twice	keep probing to <code>None</code> before inserting

The third row deserves a minute. Which test fails, and with which length, depends on **where the string keys land** — and string hashes change every run. Run the tests three times and you may see three different failures, and even a run where `test_delete` fails for the key "a". When a failure moves around like that, fix the seed to reproduce it:

```
$env:PYTHONHASHSEED = "0"
pytest tests/test_hashmap.py -v -k OpenAddressing
Remove-Item Env:PYTHONHASHSEED
```

For the fourth row, write your own test, as in Part 3.3:

```
from dsa.hashmap import OpenAddressingHashMap
m = OpenAddressingHashMap()
m.put(0, "a"); m.put(8, "b")      # both hash to 0: slots 0 and 1
m.delete(0)                     # slot 0 is now a tombstone
m.put(8, "B")                   # 8 is already in slot 1: overwrite it
print(len(m), m.get(8))         # must print: 1 B
```

Part 7 — Open addressing: `_resize`

7.1 Write it

1. Keep the old `_keys` and `_values`; make two new `Array(capacity)s`.
2. Reset `_size` **and** `_tombstones` to 0 — you are about to rebuild both counts.
3. For every old slot whose key is neither `None` nor `TOMBSTONE`, insert it: probe the new table and put it in the first `None`, then `_size += 1`. The new table has no tombstones and no duplicates, so the first `None` is always right. (Calling `self.put` would also work, but it re-runs the load check and the duplicate search for nothing.)

7.2 Test it

```
pytest tests/test_hashmap.py -v
```

All 17 must pass.

7.3 When it fails

Bug	What you see	Fix
<code>_size</code> not reset before re-inserting	<code>assert 1175 == 500</code> (the number varies) in <code>test_survives_many_keys_and_resizes</code>	reset, then count up again
the load check always resizes to the same capacity	<code>TypeError: 'NoneType' object cannot be interpreted as an integer</code> in two tests — the table filled up, no <code>None</code> was left, and the insert index was never set	double when the live keys alone exceed <code>max_load</code>
tombstones copied into the new table	no test fails, but the table never gets rid of them	skip <code>TOMBSTONE</code> as well as <code>None</code>

Checkpoint 8. In a fresh `OpenAddressingHashMap()` (capacity 8), run `put(k, k)` then `delete(k)` for $k = 0, 1, 2, \dots, 7$.

- (a) With the reference load check, how many tombstones are in the table after the last delete, and in which slots? When were the others swept away?
- (b) Now imagine a load check that counts only **live** keys. What does the table look like after the last delete? How many slots does `get(100)` probe? What would a `while self._keys[i] is not None: loop` do?

Part 8 — Measure it

Lecture 13, “Measured”. Two experiments, in `notebooks/13-hash-tables.ipynb`.

8.1 Probes against load factor

Count probes by **subclassing**: the subclass’s `_probe` passes on every index that yours yields, and counts them. Your code does not change.

```
import random
from dsa.hashmap import OpenAddressingHashMap

class Counting(OpenAddressingHashMap):
    probes = 0
    def _probe(self, key):
```

```

        for i in super()._probe(key):
            Counting.probes += 1
        yield i

capacity = 4096
rng = random.Random(13)
keys = rng.sample(range(10**9), capacity)
misses = rng.sample(range(10**9, 2 * 10**9), 1000) # never in the
↪ table
for alpha in [0.25, 0.5, 0.7, 0.8, 0.9]:
    m = Counting(capacity=capacity, max_load=0.99) # no resizing
    n = int(alpha * capacity)
    for k in keys[:n]:
        m.put(k, k)
    Counting.probes = 0
    for k in keys[:n]:
        m.get(k)
    hit = Counting.probes / n
    Counting.probes = 0
    for k in misses:
        m.get(k, None)
    miss = Counting.probes / 1000
    print(f"alpha:{alpha:4}: hit {hit:5.2f} miss {miss:6.2f}")

```

With the reference solution it prints hit/miss averages of 1.16/1.36, 1.53/2.61, 2.22/6.22, 3.16/12.78 and 5.00/44.83. Yours should match exactly if your `_probe` is linear probing and your `get` counts the same way: the keys are fixed by the seed, and integer hashes do not change between runs. Compare with the lecture's table and with Knuth's formulas, $\frac{1}{2}(1 + \frac{1}{1-\alpha})$ for a hit and $\frac{1}{2}(1 + \frac{1}{(1-\alpha)^2})$ for a miss. With 4,096 slots your numbers will scatter more than the lecture's 16,384 — at 0.9 especially — but the shape must be the same: flat up to about 0.7, then a cliff.

8.2 Make it slow on purpose

```

import time
from dsa.hashmap import ChainingHashMap

def time_gets(keys):
    m = ChainingHashMap()
    for k in keys:
        m.put(k, k)
    start = time.perf_counter()
    for k in keys[:200]:
        m.get(k)
    return (time.perf_counter() - start) / 200 * 1e6 # us per get

```

```

for n in [500, 1000, 2000, 4000]:
    good = random.Random(n).sample(range(10**9), n)
    bad = [i << 20 for i in range(n)]          # i * 2**20
    print(f'n {n:5}: random {time_gets(good):7.2f} us"
          f"    bad {time_gets(bad):8.2f} us")

```

With random keys, a get costs about the same at every n . With the bad keys — every one a multiple of 2^{20} , so key % capacity is 0 for every capacity the table will ever have — each get walks one long chain, and the time doubles when n doubles. On the lecture's machine: about 1.3 μ s against 190 μ s at $n = 4,000$. Building the bad table is itself $O(n^2)$, so do not go much past 4,000.

Then try the same bad keys in a Python dict, and explain why it does not slow down (Lecture 13, "Python's dict and set").

Part 9 — Check against dict, the oracle

Only now, with all 17 tests passing, bring in Python's dict — as a **second opinion** in a test script of your own, never inside `dsa/`. Random operations, the same on both, with small integer keys so that collisions, deletes and re-inserts happen constantly:

```

import random
from dsa.hashmap import ChainingHashMap, OpenAddressingHashMap

for cls in (ChainingHashMap, OpenAddressingHashMap):
    rng = random.Random(1)
    mine, oracle = cls(), {}
    for step in range(20_000):
        key = rng.randrange(64)
        op = rng.random()
        if op < 0.5:
            mine.put(key, step)
            oracle[key] = step
        elif op < 0.8 and key in oracle:
            mine.delete(key)
            del oracle[key]
        else:
            assert mine.get(key, None) == oracle.get(key),
                ↪ (cls.__name__, step, key)
            assert len(mine) == len(oracle), (cls.__name__, step)
    assert sorted(mine) == sorted(oracle)
    print(cls.__name__, "agrees with dict on 20,000 random operations")

```

This exercises exactly what the graded tests do not: deleting from the middle of a chain (Part 3.3), re-inserting over tombstones (Part 6.4), and tables that fill with tombstones and are swept (Checkpoint 8). If an assertion fails, the message gives the step and the key; replay the operations up to that step and draw the table.

Part 10 — Exercises at a glance

Every method is $O(1)$ on average except the two `_resizes`, which are $O(n)$ and make put amortised $O(1)$.

Class	Method	The trap	-k filter
Chaining	get	<code>==</code> not <code>is</code> ; <code>KeyError</code> only when default is <code>_MISSING</code>	Chaining (8)
Chaining	put	overwrite, never duplicate; resize when the load passes <code>max_load</code>	Chaining
Chaining	delete	the head of a chain; <code>size -= 1</code> ; the middle of a chain (untested)	Chaining and delete (2)
Chaining	<code>_resize</code>	rehash every key; save next before relinking	Chaining
Open addr.	<code>_probe</code>	<code>% capacity</code> ; exactly capacity indices	see below (9)
Open addr.	get	stop at <code>None</code> , skip <code>TOMBSTONE</code>	see below
Open addr.	put	count tombstones in the load check; probe on past a tombstone before inserting	see below
Open addr.	delete	leave <code>TOMBSTONE</code> ; update both counts	see below
Open addr.	<code>_resize</code>	reset <code>_size</code> and <code>_tombstones</code> ; skip <code>None</code> and <code>TOMBSTONE</code>	see below

For open addressing the filter is `-k "OpenAddressing or open_addressing"`.

All 17 at once:

```
pytest tests/test_hashmap.py -v
```

Before you show the TA: `git diff --stat tests/` must print nothing, and nothing in `dsa/hashmap.py` may create a list, dict or set.

Part 11 — Take-home practice (not graded)

The question bank for this week is `docs/question-bank/week13-questions.md`, with answers in `week13-answers.md`. Do the questions before opening the answers.

1. **Part G — write the code**, in `practice/week13.py`: `two_sum`, `first_repeated`, `group_anagrams`, `longest_distinct_run` and `count_subarrays_with_sum` (W13-C1 to W13-C5). Each is $O(n)$ with a hash map, and each has a large test on which an $O(n^2)$ answer crawls. Use your `ChainingHashMap` as the working storage:

```
pytest tests/test_practice_week13.py -v
```

2. **W13-S1** — draw a chaining table after every one of eight operations, including a resize.
3. **W13-S2** — the same for open addressing, with tombstones and a sweep.
4. **W13-B2** — a `_resize` that loses entries. Find the one line.
5. **W13-E2** — a hash function that returns 0 for every key: is the table wrong, or only slow? Argue it both ways, then decide.

The worked solutions are in `solutions/dsa/hashmap.py` and `solutions/practice/week13.py` — for after you have tried. `pytest --solutions tests/test_practice_week13.py` runs the tests on them.

Part 12 — Bridge to Lecture 14: graphs

Next week's Graph (in `dsa/graph.py`) stores, for every node, the list of nodes it is connected to — its **neighbours** — and the mapping from node to neighbours is **your** `ChainingHashMap`. Here is the idea in a script of your own (the neighbour lists are ordinary Python lists here; in `dsa/graph.py` they are `DynamicArrays`):

```
from dsa.hashmap import ChainingHashMap

roads = [("Mansoura", "Talkha"), ("Mansoura", "Sherbin"), ("Talkha",
↪ "Tanta"),
         ("Sherbin", "Damietta")]
neighbours = ChainingHashMap()
for a, b in roads:
    for x, y in ((a, b), (b, a)):          # a road goes both ways
        if x not in neighbours:
            neighbours.put(x, [])
            neighbours.get(x).append(y)

print(neighbours.get("Mansoura"))          # ['Talkha', 'Sherbin']
print(len(neighbours))                     # 5 towns
```

Two questions to bring to Lecture 14:

- **Which towns can you reach from Mansoura in at most two roads?** Answer it by hand from the map above. Then think about how a program would answer it for a map of a thousand towns — and why it needs to remember which towns it has **already**

visited. (Hint: in what structure would you keep “visited”, so that asking “have I been here?” costs $O(1)$?)

- **Print every town in neighbours.** Run the loop twice, in two separate runs of Python. Is the order the same? Why not — and why might a graph want its own record of the order in which nodes were added?
-

Summary

Idea	The one line to keep
Hashing	<code>index = hash(key) % capacity</code> ; equal keys must hash equal.
Reproducibility	<code>int</code> hashes are fixed; <code>str</code> hashes change every run — <code>PYTHONHASHSEED</code> pins them.
Chaining	A linked list of <code>Entry</code> nodes per bucket; push on the front after checking for the key.
Chaining delete	<code>prev</code> and <code>entry</code> ; the head of a chain is the bucket slot itself.
Resize	A new <code>Array</code> , and every key rehashed ; save <code>next</code> before relinking.
Linear probing	<code>(h + step) % capacity</code> ; stop at <code>None</code> .
Tombstones	Delete leaves <code>TOMBSTONE</code> ; get skips it; put may reuse it, after probing on.
Load factor	Count tombstones as used; double when the live keys pass <code>max_load</code> , else sweep.
Measuring	Hits and misses stay a few probes below about 0.7, then climb steeply.
Worst case	Keys that share their low bits all collide: $O(n)$ per operation.

Lab 14 — Lab 14 — Graphs and the Two Searches

How to use this lab. The routine from Week 4: read the lecture section named at the top of each part, **draw before you code** — this week that means the graph, its lists, and the queue or stack after every step — and predict at each **Checkpoint**. Then write one method, run the tests named in its part, and only then move on. This is the last structure you build, and it stands on four of the earlier ones: if your hash map, dynamic array, queue or stack is broken, this lab will find it. Parts 1–5 are the session; Parts 6–10 are for home.

Duration	One 2-hour lab session, plus about 3 hours at home
You will write	<code>dsa/graph.py</code> — the <code>Graph</code> and <code>MatrixGraph</code> classes, and <code>bfs</code> , <code>dfs</code> , <code>dfs_iterative</code> , <code>shortest_path_unweighted</code> , <code>connected_components</code> , <code>has_cycle</code> , <code>topological_sort</code>
Graded by	<code>tests/test_graph.py</code> (63 tests)
Connects to	Lecture 14 — Graphs and graph searches; Labs 04, 06, 07 and 13, whose structures this one uses; and Lecture 15, where a parse tree is walked depth first

What you will be able to do

1. store a graph as an adjacency list and as an adjacency matrix, and say what each costs in memory and per operation;
2. write `add_node` so that adding a node twice cannot lose its edges;
3. trace a BFS as a table of the queue, and explain why a vertex is marked when it is enqueued;
4. rebuild a shortest path from a parent map, and say what “shortest” means here;
5. write DFS twice — with the call stack and with your own stack — so that both give the same order, and explain why the obvious iterative version does not;
6. detect cycles in undirected and directed graphs, and say why the two need different rules;
7. produce a topological order with Kahn’s algorithm, and detect when none exists;
8. measure a BFS on both representations and read the slopes.

Part 0 — Before you start

0.1 Environment

From the DSA27 folder, with the virtual environment active (the prompt starts with `(.venv)`):

```
git pull
pytest -m "not challenge" -q          # the environment check: must pass
pytest tests/test_graph.py -q        # 63 failures: nothing written yet
```

The 63 failures are all `NotImplementedError` — that is the starting point.

0.2 Prerequisites

`dsa/graph.py` imports `ChainingHashMap` (Week 13) and `DynamicArray` (Week 4); the searches also use your `CircularQueue` (Week 7) and `Stack` (Week 6). Check all four first:

```
pytest tests/test_dynamic_array.py -q
pytest tests/test_hashmap.py -q -k Chaining
pytest tests/test_stack_queue.py -q
```

Two details matter this week:

- **Your `CircularQueue` must grow** when it is full — the challenge of Lab 07. A BFS cannot know in advance how many vertices will wait at once. If yours raises “queue is full”, finish the challenge first (Lab 07, Part 6).
- **Your hash map’s `in` must work when the stored value is `None`.** The parent map of Part 4 stores `None` as the start’s parent. The `__contains__` given in `dsa/hashmap.py` calls `get` and catches the `KeyError`, so it does; if you changed it, check that `m.put("A", None)` then `"A" in m` gives `True`.

0.3 Read the skeleton

Open `dsa/graph.py`. **Given** to you: `to_networkx`; the `__init__`, `__len__`, `__contains__` and `__repr__` of both classes. **Yours**: every method and function that raises `NotImplementedError`.

Read `Graph.__init__` carefully. It keeps two things:

Attribute	Type	Holds
<code>_adjacent</code>	<code>ChainingHashMap</code>	node $\rightarrow$ <code>DynamicArray</code> of its neighbours, in the order the edges were added
<code>_order</code>	<code>DynamicArray</code>	the nodes, in the order they were added

Why both? A hash map gives its keys back in bucket order, and for strings that order changes every time Python starts (string hashing is randomised). If `nodes()` walked the hash map, `connected_components` and `topological_sort` would give a different — equally correct — answer on every run, and your traces could never match the answers. `nodes()` returns `_order` instead.

`MatrixGraph.__init__` keeps `_index` (node $\rightarrow$ number), `_order` (number $\rightarrow$ node) and `_matrix` (a `DynamicArray` of rows, each a `DynamicArray` of `True/False`).

0.4 The storage rule, this week

No dict, set, list or deque inside `dsa/graph.py` for storage: the visited set is a `ChainingHashMap` with `True` as the value, the queue is your `CircularQueue`, the stack is your `Stack`. The functions **return** Python lists — that is the interface — and `neighbours()` returns a list too, built from the `DynamicArray`. The tests cannot see what you store; the TA will read it.

Part 1 — Graph: the adjacency list

Lecture 14, “Adjacency list and adjacency matrix” and “The trade-off”.

1.1 Draw it

The test file’s sample graph is built by `add_edge` on A-B, A-C, B-C, B-D, in that order:

```
A --- B
|   / |
|  /  |
C     D
```

Checkpoint 1. For this undirected graph, draw `_adjacent` (each node and its `DynamicArray`, in order) and the 4×4 adjacency matrix. How many cells does each hold? What are `degree("B")` and `edges()`?

1.2 Write it

In this order — each one uses the ones before:

1. **`add_node(node)`**: if the node is **not** already a key of `_adjacent`, put it there with an empty `DynamicArray`, and append it to `_order`. If it is already there, do nothing.
2. **`add_edge(source, target)`**: `add_node` both ends; append `target` to the source’s array; if the graph is undirected, also append `source` to the target’s array. (A loop, `add_edge("A", "A")`, needs only one entry — test `source != target` before the second append.)

3. **neighbours(node)**: `self._adjacent.get(node)` raises `KeyError` for a missing node, which is exactly what the contract asks; turn the array into a list with `list(...)`.
4. **has_edge, nodes, degree**: short. `has_edge` on a missing source is `False`, not an error.
5. **edges()**: directed — every (node, neighbour) pair. Undirected — each edge **once**. One way: `walk_order`; keep a `ChainingHashMap` of the nodes already finished; report (node, neighbour) only when the neighbour is not yet finished; mark the node finished after its loop.

Checkpoint 2. A student writes `add_node` with no check: `self._adjacent.put(node, DynamicArray())` and `self._order.append(node)`. After `add_edge("A", "B")` and `add_edge("B", "C")`, what are `neighbours("B")`, `has_edge("B", "A")`, `has_edge("A", "B")` and `nodes()`?

1.3 Test it

```
pytest tests/test_graph.py -v -k "not Matrix and not (bfs or dfs or
↪ search or path or goal or component or cycle or topological or
↪ networkx)"
```

Eleven tests: the structure tests with `[Graph]` in their names. The filter is long because the test names contain words like “node” and “edge” that the search tests share; not `Matrix` drops the `[MatrixGraph]` copies, which wait for Part 2.

1.4 When it fails

Bug	What you see	Fix
<code>add_node</code> with no check (Checkpoint 2)	5 fail, among them <code>test_adding_a_node_twice</code> (assert <code>False</code>) and <code>assert ['C'] == ['B', 'C']</code> in <code>test_neighbours</code>	put and append only when the node is new
no second append in <code>add_edge</code>	<code>assert [] == ['B']</code> in <code>test_neighbours</code> (D has no neighbours), and <code>assert 2 == 3</code> for <code>degree("B")</code>	undirected means both lists
<code>edges()</code> reports both directions	<code>assert 8 == 4</code> in <code>test_undirected_edges_are_reported_once</code>	report each undirected edge once
<code>neighbours</code> returns the <code>DynamicArray</code> itself	<code>assert DynamicArray([]) == []</code> in <code>test_isolated_node_has_no_neighbours</code>	return <code>list(...)</code>
<code>neighbours</code> returns <code>[]</code> for a missing node	Failed: DID NOT RAISE <code>KeyError</code>	let get raise

Part 2 — MatrixGraph: the adjacency matrix

Lecture 14, “The trade-off”. Same contract, different storage. The tests are the same eleven, on MatrixGraph.

2.1 Write it

1. **add_node**: if the node is new, give it the next number (`len(self._order)`) in `_index`, append it to `_order`, then **grow the matrix**: append one `False` to every existing row (the new column), and append a new row of `Falses` as long as the new number of nodes.
2. **add_edge**: `add_node` both; look up both numbers; set the cell, and its mirror when undirected.
3. **neighbours**: `KeyError` for a missing node (from `_index.get`); scan the node’s row and collect `_order[j]` for every `True` cell — that scan is the $O(V)$ the lecture measures.
4. **has_edge**: `False` if either node is missing, else one cell.
5. **edges()**: directed — every `True` cell. Undirected — only the cells with `j >= i`, the upper triangle, so each edge comes once.
6. **degree**: count the `True` cells in the row.

2.2 Test it

```
pytest tests/test_graph.py -v -k "MatrixGraph and not (bfs or dfs or
↪ search or path or goal or component or cycle or topological or
↪ networkx)"
```

2.3 When it fails

Bug	What you see	Fix
a new row, but no new column in the old rows	10 of the 11 fail with an <code>IndexError</code> from your <code>DynamicArray</code>	append a <code>False</code> to every existing row first
no mirror cell when undirected	<code>assert False</code> in <code>test_undirected_edges_go_both_ways</code> , <code>assert [] == ['B']</code> , <code>assert 2 == 3</code>	set <code>[j][i]</code> too
the whole matrix in <code>edges()</code>	<code>assert 8 == 4</code>	only <code>j >= i</code> when undirected

Every search from now on calls only `neighbours()`, so it works on both classes unchanged — the tests run each one twice.

Part 3 — bfs: breadth first, with your queue

Lecture 14, “Breadth-First Search”. The lecture gives the algorithm as pseudocode; turning it into Python with your structures is the exercise.

3.1 Draw it

On paper: three columns — **take, add, queue afterwards** — one row per dequeue, as in the lecture’s trace.

Checkpoint 3. (a) Trace bfs from A on the sample graph of Part 1. (b) On the graph A–B, B–C, A–D (the test that tells the searches apart), what does bfs return from A? (c) A student marks a vertex as visited only when it is **dequeued** (and never when it is enqueued). What does the sample graph give from A, and which vertex appears twice?

3.2 Write it

1. `order` is a Python list (the result); `visited` a ChainingHashMap; `queue` a Circular-Queue.
2. Mark the start and enqueue it.
3. While the queue is not empty: dequeue, append to `order`, and for each neighbour not yet visited — **mark it, then enqueue it**.
4. Return `order`.

Do not check “is this the goal” or build parents here — that is Part 4.

3.3 Test it

```
pytest tests/test_graph.py -v -k "bfs and not dfs"
```

Four tests: `test_bfs_visits_by_distance` and `test_bfs_visits_each_node_once`, on both classes. (`-k bfs` alone also selects `test_dfs_goes_deep_and_bfs_goes_wide`, which needs dfs too.)

3.4 When it fails

Bug	What you see	Fix
mark on dequeue only (Checkpoint 3c)	<code>assert 5 == 4</code> in <code>test_bfs_visits_each_node_once</code> — five visits, four distinct vertices	mark when enqueued
no visited check at all	no message: the run hangs on the first test, going round A-B-C-A for ever	Ctrl+C; the traceback points into your loop
start not marked	<code>assert 5 == 4</code> , and the deep/wide test gets <code>['A', 'B', 'D', 'A', 'C']</code> — A comes back	mark the start before the loop
CircularQueue does not grow	<code>IndexError: queue is full</code> (or your own message)	Lab 07's challenge

Part 4 — `shortest_path_unweighted`: BFS with parents

Lecture 14, “Shortest paths, by edge count”.

4.1 Draw it

`test_shortest_path_is_shortest_not_merely_a_path` builds A-B, B-Z, A-X, X-Y, Y-Z: a short route A-B-Z and a long one A-X-Y-Z.

Checkpoint 4. Run BFS from A on that graph, writing down each parent as it is recorded (“B ← A”, ...). Stop when Z is dequeued. What is the parent map, and what path does walking back from Z give? What does dfs from A return on the same graph — does it reach Z by the short route?

4.2 Write it

1. `parent` is a `ChainingHashMap`: `node` → the node it was discovered from. Put `start` with `None`. It is also your visited set: a node is visited when it has an entry.
2. BFS as in Part 3, but when you take a node off the queue, first check whether it is the goal; and when you discover a neighbour, record its parent.
3. When the goal is taken: start from it, follow `parent.get(...)` until you reach `None`, collecting the nodes; that list runs goal-first — reverse it.
4. If the queue empties, return `None`.

`shortest_path_unweighted(g, "A", "A")` must be `["A"]`: the goal is the first node taken.

4.3 Test it

```
pytest tests/test_graph.py -v -k "path or goal"
```

Eight tests: four cases on both classes.

4.4 When it fails

Bug	What you see	Fix
no reverse	<code>assert ['D', 'B', 'A'] == ['A', 'B', 'D']</code>	reverse the walk
never checks for the goal	<code>assert None == ['A', 'B', 'D']</code> , and <code>assert None == ['A']</code> for the path to itself	check each node as it is dequeued
start not put in parent	<code>KeyError: 'A' in the path-to-itself test</code> , and the others hang — the walk back never meets <code>None</code>	<code>parent.put(start, None)</code> first

Part 5 — dfs and dfs_iterative

Lecture 14, “Depth-First Search”. Two functions that must return the **same** list — `test_dfs_and_dfs_iterative_agree` compares them.

5.1 dfs: recursive

Write a `helper_dfs_visit(graph, node, visited, order)`: mark node, append it, and call itself on each unvisited neighbour, in `neighbours()` order. `dfs` creates the map and the list, calls the helper on `start`, and returns the list. That is all: the call stack remembers where each vertex’s loop had got to.

5.2 Draw the iterative one

Checkpoint 5. (a) Trace `dfs_iterative` from `A` on the sample graph: after each `pop`, the action and the stack (bottom → top). Which vertex is pushed twice? (b) A student marks each vertex when it is **pushed** instead (and marks the start before the loop), still pushing in reverse. What does their version return from `A` on the sample graph, and why does `test_dfs_and_dfs_iterative_agree` fail?

5.3 Write dfs_iterative

1. Push start on your Stack.
2. While the stack is not empty: pop. If the node is already visited, skip it (continue) — it was pushed twice and the first pop won. Otherwise mark it, append it, and push its unvisited neighbours **in reverse order**: for i in range(len(neighbours) - 1, -1, -1).

5.4 Test it

```
pytest tests/test_graph.py -v -k "dfs or search"
```

Ten tests: test_dfs_reaches_everything (both functions, both classes), the agreement test, the deep/wide test, and test_search_from_isolated_node, which runs bfs and dfs from a node with no edges.

5.5 When it fails

Bug	What you see	Fix
mark on push (Checkpoint 5b)	only test_dfs_and_dfs_iterative fails: assert ['A', 'B', 'C', 'D'] == ['A', 'B', 'D', 'C']	mark on pop; skip visited
mark on push, start never marked	also assert 5 == 4 in test_dfs_reaches_everything — A is visited twice	as above
neighbours pushed in order, not reversed	assert ['A', 'B', 'C', 'D'] == ['A', 'C', 'B', 'D']	push in reverse

Read the first row again: every search test passes and one comparison fails. The shortcut is a depth-first search; it is just not the recursive one, and Part 7's cycle check relies on the recursive order.

Part 6 — connected_components and has_cycle

Lecture 14, "Using the Searches".

6.1 connected_components

Loop over graph.nodes(). For each node not yet in a seen map, run bfs from it — that is one component — mark all its members as seen, and append it to the result.

```
pytest tests/test_graph.py -v -k component
```

Four tests. The classic bug is to forget the seen check: every node then gets its own copy of its component — `assert 4 == 1` in `test_one_component_when_connected`.

6.2 has_cycle

Two helpers, both recursive like dfs:

- **undirected**, `_undirected_cycle_from(graph, node, parent, visited)`: for each neighbour — not visited: recurse with node as the parent, and return True if that does; visited **and not the parent**: return True.
- **directed**, `_directed_cycle_from(graph, node, colour)`, with WHITE, GREY, BLACK = 0, 1, 2 and white meaning “not in the map”: colour the node grey; for each neighbour — grey: True; white: recurse, True if that does; black: ignore. Colour the node black before returning False.

`has_cycle` picks by `graph.directed`, and runs its helper from every node not yet visited (white), because a cycle can be in any component.

Checkpoint 6. (a) Without the parent check, what does the undirected `has_cycle` return for the tree $A-B$, $A-C$, $B-D$, $B-E$ — and for the single edge $A-B$? (b) With “visited” (grey or black) instead of “grey” in the directed check, what does it return for the diamond $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow D$, $C \rightarrow D$, and which edge causes it?

```
pytest tests/test_graph.py -v -k "cycle and not topological"
```

Eight tests.

Bug	What you see	Fix
no parent check	<code>assert not True</code> in <code>test_undirected_cycle_detection</code> (the path $A-B-C$) and in <code>test_undirected_tree_has_no_cycle</code>	skip the parent
visited instead of grey	<code>assert not True</code> in <code>test_directed_cycle_detection</code> (the DAG $A \rightarrow B$, $A \rightarrow C$, $B \rightarrow C$) and in the diamond test	only grey is a cycle
never colouring black	the same two failures: a finished node stays grey	black at the end of the helper

Part 7 — topological_sort: Kahn's algorithm

Lecture 14, “Kahn's algorithm: take what nothing waits on”.

1. Undirected graph: raise `ValueError`.
2. A `ChainingHashMap` of in-degrees: 0 for every node, then +1 for every edge's target.
3. A `CircularQueue` of the nodes with in-degree 0, in nodes() order.
4. While it is not empty: dequeue, append to the result, and decrease the in-degree of each neighbour; enqueue any that reach 0.
5. If the result is shorter than `len(graph)`, return `None`.

Checkpoint 7. For the directed graph $A \rightarrow B, B \rightarrow C, C \rightarrow B, A \rightarrow D$, which vertices come out before the queue empties? What does `topological_sort` return, and why is the length check the whole cycle test?

```
pytest tests/test_graph.py -v -k topological
```

Six tests. Forget step 5 and the cycle test gets `assert [] is None`: nothing had in-degree 0, so nothing came out — an empty order, which looks valid and is not.

Part 8 — Measure it

Lecture 14, “Measured”. In `notebooks/14-graphs.ipynb`:

```
import random
from dsa.graph import Graph, MatrixGraph, bfs
from viz.complexity import measure, plot_growth

def sparse(cls, n, seed=14):
    rng = random.Random(seed)
    g = cls()
    for v in range(n):
        g.add_node(v)
    for v in range(1, n):
        # a spanning path:
        # connected
        g.add_edge(v, rng.randrange(v))
    for _ in range(n):
        # about 2n edges in all
        a, b = rng.randrange(n), rng.randrange(n)
        if a != b:
            g.add_edge(a, b)
    return g

sizes = [250, 500, 1000, 2000]
built = {}
def make(cls):
```

```

def build(n):
    if (cls, n) not in built:                # build once;
        ↪ MatrixGraph is  $O(V^2)$  to build
        built[(cls, n)] = sparse(cls, n)
    return built[(cls, n)]
return built

results = {name: measure(lambda g: bfs(g, 0), sizes, make(cls))
           for name, cls in [('adjacency list', Graph), ('adjacency
           ↪ matrix', MatrixGraph)]}
plot_growth(results, reference=['n', 'n^2'], loglog=True)

```

The random extra edges can repeat an existing edge; the graph then has two copies of it in each list, which changes nothing here.

Checkpoint 8. Before running: (a) each time V doubles, by roughly what factor should the list BFS grow, and the matrix BFS? (b) For $V = 1,000$ and $E = 2,000$, how many cells do the two representations store?

Then time has_edge on a **dense** graph — join each node to about half of the others — and see the matrix win. The lecture's figure shows both.

Part 9 — Draw it, and walk a real graph

to_networkx is given; viz.draw.draw_graph draws the result and can highlight nodes and edges. In the notebook:

```

from viz.draw import draw_graph
from dsa.graph import Graph, to_networkx, bfs,
    ↪ shortest_path_unweighted

g = Graph()
for a, b in [("A", "B"), ("A", "D"), ("B", "C"), ("B", "E"),
             ("D", "E"), ("C", "F"), ("E", "F"), ("F", "G")]:
    g.add_edge(a, b)
path = shortest_path_unweighted(g, "A", "G")
draw_graph(to_networkx(g), highlight_nodes=path,
           highlight_edges=list(zip(path, path[1:])), title=" -
    ↪ ".join(path))

```

Then your own programme. The prerequisites are in the course specifications of your by-law (docs/course/regulations/); the lecture's figure used twelve from the Software Engineering bylaw:

```

from viz.draw import draw_graph
from dsa.graph import Graph, to_networkx, topological_sort, has_cycle

courses = Graph(directed=True)
pairs = [("CS012", "IS122"), ("MATH012", "IS122"), ("IS122", "IS123"),
        ("IS123", "IS142"), ("CS012", "IS142"), ("IS122", "SWE141"),
        ("SWE132", "SWE141")]
for before, after in pairs:
    courses.add_edge(before, after)
print(has_cycle(courses), topological_sort(courses))
draw_graph(to_networkx(courses), title="prerequisites")

```

Add ten prerequisites of your own programme, draw the graph, and print a topological order. Which courses does IS122 (or CS2101) unlock — directly and indirectly? That is `bfs(courses, "IS122")`.

Part 10 — Exercises at a glance

Function	Target cost	The trap	-k filter
Graph structure	$O(1)$ / $O(\text{deg})$	<code>add_node</code> twice keeps edges; undirected edges both ways, reported once	see Part 1.3 (11)
MatrixGraph structure	$O(V)$ add, $O(1)$ <code>has_edge</code>	new column and row; mirror cell; upper triangle	see Part 2.2 (11)
<code>bfs</code>	$O(V + E)$	mark on enqueue; mark the start; a growing queue	<code>bfs</code> and not <code>dfs</code> (4)
<code>shortest_path_unweighted</code>	$O(V + E)$	<code>parent[start] = None</code> ; reverse; None	path or goal (8)
<code>dfs</code> , <code>dfs_iterative</code>	$O(V + E)$	mark on pop; push reversed	<code>dfs</code> or search (10)
<code>connected_components</code>	$O(V + E)$	one search per unseen node	component (4)
<code>has_cycle</code>	$O(V + E)$	parent check; grey, not visited	cycle and not topological (8)
<code>topological_sort</code>	$O(V + E)$	length check means None; <code>ValueError</code> if undirected	topological (6)

The eight filters select 62 different tests; the last test, `test_to_networkx_round_trip`, passes once nodes () and edges () work. All 63 at once:

```
pytest tests/test_graph.py -v
```

Before you show the TA: `git diff --stat tests/` must print nothing. And check by eye that nothing in `dsa/graph.py` stores data in a dict, set, list or deque — the tests cannot see that, but the TA will look.

Part 11 — Take-home practice (not graded)

The question bank for this week is `docs/question-bank/week14-questions.md`, with answers in `week14-answers.md`. Do the questions before opening the answers.

1. **Part G — write the code**, in `practice/week14.py`: `maze_distance`, `is_bipartite`, `topological_order_dfs`, `semester_plan` and `count_islands` (W14-C1 to W14-C5):

```
pytest tests/test_practice_week14.py -v
```

Hint for `count_islands`: it is `connected_components` on a grid, and one of the tests has an island 2,000 cells long — deeper than Python's recursion limit.

2. **W14-T1 and W14-T3** — BFS and iterative DFS on the question bank's graph H, as tables.
3. **W14-S2** — the three colours, step by step, on a graph with a cycle and a harmless second route.
4. **W14-B1** — a BFS that marks on deque. What exactly does it return on H?
5. **W14-K2** — components without the seen check: what is wrong, and what does it cost?

The worked solutions are in `solutions/dsa/graph.py` and `solutions/practice/week14.py` — for after you have tried. `pytest --solutions tests/test_practice_week14.py` runs the tests on them.

Part 12 — Bridge to Lecture 15: a tree is a graph you walk depth first

Lecture 15 builds a parser that turns $(2 + 3) * 4$ into a **tree**, then evaluates it. That tree is a directed graph — each operator points to its operands — and evaluating it is a depth-first search that handles a node only after all its children are finished. Try it with this week's Graph, in a script of your own:

```
from dsa.graph import Graph

tree = Graph(directed=True)
```

```

for parent, child in [("*", "+"), ("*", "4"), ("+", "2"), ("+", "3")]:
    tree.add_edge(parent, child)

def postorder(graph, node, out):
    for child in graph.neighbours(node):
        postorder(graph, child, out)
    out.append(node)          # after every child has finished
    return out

print(" ".join(postorder(tree, "*", [])))

```

```
2 3 + 4 *
```

That is the **postfix** form of Week 6 — the order `evaluate_postfix` needs — and it is exactly the DFS **finishing order** of Part 7's alternative topological sort: a node finishes after everything it depends on. (The node names must be distinct here, since the Graph keys on them; Lecture 15 uses real tree nodes instead.) Two questions to bring:

- In $(2 + 3) * 4$, why must $+$ finish before $*$ can be evaluated — and which graph word from this week describes “must finish before”?
- A parse tree never has a cycle. What would a cycle in an expression mean?

Summary

Idea	The one line to keep
Adjacency list	Hash map of DynamicArrays; $V + 2E$ cells; neighbours in $O(\deg)$.
Adjacency matrix	V^2 cells; <code>has_edge</code> in $O(1)$; neighbours in $O(V)$.
<code>add_node</code>	Only if new — otherwise every <code>add_edge</code> wipes edges.
BFS	Queue; mark on enqueue ; layers by distance.
Shortest path	BFS + parent map; walk back and reverse; counts edges, not kilometres.
DFS	Call stack, or your Stack with mark on pop and push reversed .
Components	One search per unseen node; $O(V + E)$ in total.
Cycles	Undirected: not the parent. Directed: grey, not merely visited.

Idea	The one line to keep
Topological sort	Kahn: in-degree 0 first; fewer than V out means a cycle.
Measuring	Sparse: the list's BFS is slope 1, the matrix's slope 2.

Lab 15 — Lab 15 — A Calculator, Stage by Stage

How to use this lab. The routine you know: read the lecture section named at the top of each part, **draw before you code** — this week that means a tree on paper for every expression — and write your prediction at each **Checkpoint** before you check it with your TA. Build `dsa/translation.py` one stage at a time and run the tests of that stage before moving on: a parser tested on top of a broken lexer fails in ways that look like parser bugs. Parts 1–4 are the session; Parts 5–7 are for home. This is the last lab of the course, so it ends with a wrap-up and a plan for the final exam instead of a bridge to next week.

Duration	One 2-hour lab session, plus about 3 hours at home
You will write	<code>dsa/translation.py</code> — <code>tokenize</code> , <code>Parser.parse</code> , <code>Parser.expr</code> , <code>Parser.term</code> , <code>Parser.factor</code> , <code>evaluate</code> , <code>calculate</code> (<code>evaluate_postfix</code> is your Week 6 work)
Graded by	<code>tests/test_translation.py</code> (55 tests)
Connects to	Lecture 15 — The principles of language translation; Lecture 06 (your Stack, postfix, shunting-yard); Lecture 03 (recursion); Lecture 11 (trees and post-order)

What you will be able to do

1. write a lexer as one left-to-right scan, and say why 12 must be one token;
 2. read a grammar in EBNF and draw, by hand, the tree it gives an expression;
 3. explain why `*` binds tighter than `+` using only the **shape** of the grammar, without a precedence table;
 4. turn each grammar rule into one method — a loop for `( ... )*`, a call for a rule, an `if` for `|`;
 5. show why a right-recursive `expr` gets `1 - 2 - 3` wrong and a left-recursive one never stops, and which tests catch each;
 6. evaluate a tree by a post-order walk, and recognise that walk as postfix;
 7. measure the translator's time and its recursion depth, and say which one depends on the **shape** of the input.
-

Part 0 — Before you start

0.1 Environment

From the DSA27 folder, with the virtual environment active (the prompt starts with `(.venv)`):

```
git pull
pytest -m "not challenge" -q          # environment check: must pass
pytest tests/test_stack_queue.py -q   # Weeks 6-7: must pass
pytest tests/test_translation.py -q   # 43 failed, 12 passed
```

The 12 that already pass are your Week 6 `evaluate_postfix` (11 tests) and `test_to_edges_of_a_leaf` which tests code given to you. If `evaluate_postfix` is not passing yet, finish it first (Lab 06): two of this week’s end-to-end tests compare your calculator with it.

0.2 Prerequisites

- **Your Stack and infix_to_postfix** (Week 6). `evaluate_postfix` uses the stack, and `test_both_routes_agree` runs the whole shunting-yard route next to yours.
- **Recursion** (Lecture 03): the parser is three methods that call each other, and one of them calls back to the top.
- **Trees and post-order** (Lecture 11): the parser’s result is a binary tree, and `evaluate` walks it children-first.

0.3 Read the skeleton: what is given, what is yours

Open `dsa/translation.py` and read it top to bottom before writing anything. The grammar is in a comment near the top; copy it onto paper, because every part of this lab refers to it:

```
expr  := term (('+' | '-') term)*
term  := factor (('*' | '/') factor)*
factor := NUMBER | '(' expr ')' | '-' factor
```

Given to you	Yours
<code>Num(value)</code> — a leaf; <code>BinOp(op, left, right)</code> — an operator node; both with <code>==</code> and a readable <code>repr</code>	<code>tokenize(text)</code>
<code>to_edges(node)</code> — pairs for <code>viz.draw.draw_tree</code>	<code>Parser.parse()</code>
<code>Parser.__init__</code> — the tokens in the course Array, and <code>position = 0</code>	<code>Parser.expr()</code> , <code>Parser.term()</code> , <code>Parser.factor()</code>
<code>Parser.peek()</code> — the current token, or <code>None</code> at the end	<code>evaluate(node)</code>

Given to you	Yours
<code>Parser.advance()</code> — return the current token and move past it	<code>calculate(text)</code>

peek and advance are the only way your methods should touch the tokens. Decide by peeking; consume by advancing.

0.4 The storage rule, this week

The Parser is a structure, and it keeps its tokens in the course Array — that is done for you in `__init__`. The tree is made of **node objects** (Num, BinOp), which the rule allows. `tokenize` returns a Python list: lists are the interface for data passed in and out. Two rules follow from the point of the week:

- **No eval**, and no regular expressions (re). Python's `eval("3 + 4 * 2")` would pass most of the end-to-end tests and teach you nothing.
- **Do not build a precedence table** in the parser. Precedence must come from which method handles which operator — that is the lesson.

0.5 The tests, by stage

Every command is `pytest tests/test_translation.py -v -k "<filter>":`

Stage	-k filter	Tests
lexer	<code>tokenize</code>	13
parser	<code>parse or preced or assoc or unary or nested</code>	14
evaluator	<code>evaluate and not postfix</code>	4
end to end	<code>calculate or routes</code>	10
everything	<i>(no filter)</i>	55

The remaining 14 are the 12 postfix tests — 11 for `evaluate_postfix`, and one that pairs it with your `tokenize` — and the 2 for `to_edges`.

Part 1 — tokenize: characters into tokens

Lecture 15, "Stage 1: The Lexer".

1.1 The idea

One pass from left to right with an index `i`. At each position look at **one** character and decide which of four cases it is: a space (skip it), an operator or a parenthesis (a token by itself), the start of a number (read the whole number), or anything else (an error). The lexer

knows nothing about grammar: `tokenize("3 + + 4")` is `['3', '+', '+', '4']`, and it is the parser's job to refuse it.

1.2 Draw it

Write the text in boxes, one character per box, with the index under each — like the lecture's figure — and bracket the characters of each token (`_` marks a space):

text		1	2	_	*	_	(	3	.	5	+	4	)
index		0	1	2	3	4	5	6	7	8	9	10	11
tokens		[12]		]		*	(		[3.5]		+	4	)

When you meet the 1 at index 0, where does the number end? Index 2 — the first character that is neither a digit nor `.`. The token is `text[0:2]`.

Checkpoint 1. Give `tokenize(t)`, or the exception it raises and why, for:

- (a) `" (12+3.25)* 4 "`
- (b) `"7-"`
- (c) `"3,5"`
- (d) `"1..2"`
- (e) `"3 . 5"`
- (f) `"2(3)"`

Which of them will the **parser** later reject, although the lexer accepts them?

1.3 Write it

- Start with an empty result list and `i = 0`; loop while `i < len(text)`.
- Read `ch = text[i]`.
- A space (`ch.isspace()`): move on.
- One of `+ - * / ( )`: append it as a token; move on.
- A digit or `.`: remember where the number starts; move `i` forward **while there is a next character** and it is a digit or `.`, counting the points as you go. Cut the number out with one slice. If it has more than one point, or is just `.`, raise `ValueError`.
- Anything else: raise `ValueError`, and put the character and its position in the message — you will be glad of it later.

Hints:

- Check digits with `ch in "0123456789"`, not `ch.isdigit()`: `isdigit` is also true for Arabic-Indic digits such as `३`, and those are not part of our language.
- In step 5, the while condition must test `i < len(text)` **first**. A number at the very end of the text is the normal case.

1.4 Test it

```
pytest tests/test_translation.py -v -k tokenize
```

Thirteen tests:

- eight cases of `test_tokenize`, from `"` to `"-5"`;
- `test_tokenize_keeps_multi_digit_numbers_whole`, on `"123 + 45"`;
- `test_tokenize_accepts_a_decimal_point`, on `"3.5 + 1"`;
- three cases of `test_tokenize_rejects_unknown_characters`: `$`, a letter, and `&`.

1.5 When it fails

Bug	What you see	Fix
each digit appended as its own token	3 of the 13 fail — At index 0 diff: <code>'1' != '12'</code> for <code>"12*(3+4)"</code> , and <code>['1', '2', '3', '+', '4', '5'] == ['123', '+', '45']</code>	keep reading while the next character is a digit or <code>.</code>
spaces appended as tokens	5 of the 13 fail: the lists contain <code>' '</code> entries	a space separates; it is never a token
no <code>i < len(text)</code> in the inner loop	<code>IndexError: string index out of range</code> in 7 of the 13 — every case that ends with a number	test the bound before reading <code>text[i]</code>
no final <code>else</code> (unknown characters skipped)	Failed: DID NOT RAISE <code><class 'ValueError'></code> in the 3 rejection tests	raise <code>ValueError</code>

Part 2 — The grammar on paper

Lecture 15, “Grammars”. Before any parser code, make sure you can do by hand what the parser will do.

2.1 The idea

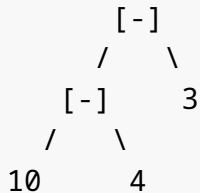
To read an expression with the grammar, split it the way the rules say:

1. an `expr` is split at the `+` and `-` signs that are **not** inside parentheses — the pieces are **terms**, and the operators between them are applied **left to right**;
2. a `term` is split at `*` and `/` in the same way — the pieces are **factors**;
3. a `factor` is a number, a parenthesised `expr` (start again at 1 inside), or `-` followed by a factor, which becomes `0 - factor`.

Each split becomes a node of the tree, with the **last** operator of a chain at the top — so the first one is deepest and is evaluated first.

2.2 Draw it

For $10 - 4 - 3$: the `expr` splits into the terms 10, 4, 3 with `-` between them. Left to right: 10 - 4 first, then that result - 3. The tree, with each operator in brackets:



Checkpoint 2. Draw the tree the grammar gives, and compute the value, for:

- (a) $2 * 3 + 4$
- (b) $2 * (3 + 4)$
- (c) $-2 * 3$
- (d) $8 / 2 / 2 * 3$
- (e) $2 - (3 - 4)$

In (c), is the unary minus inside or outside the multiplication? Does it matter for the value here? Would it matter for $-2 - 3$?

You can check each tree once your parser works (Part 5): `draw_tree(to_edges(...))` draws exactly what your parser built.

Part 3 — The parser: `parse`, `expr`, `term`, `factor`

Lecture 15, “Stage 2: Recursive Descent”. Four short methods; write them all before you test, because they call each other and none of them can be tested alone.

3.1 The idea

One method per rule, translated line by line (the lecture’s table):

In the grammar	In the method
a rule’s name inside another rule (...)*	a call: <code>self.term()</code> a while loop that runs while <code>peek()</code> is one of the operators
<code>a \ b \ c</code> a quoted token such as <code>' '</code>	<code>if/elif</code> on <code>peek()</code> check <code>peek()</code> is that token, then <code>advance()</code>

The lecture shows `expr` in full. `term` is the same shape one level down.

3.2 Draw it

The parser's calls form a tree, and at any moment the Python call stack holds one path of it. Indent one line per call:

```
expr
  term
    factor      reads 3
  term          (after expr reads '+')
    factor      reads 4
  factor        (after term reads '*') reads 2
```

That is $3 + 4 * 2$: two terms for the `expr`, and the second term has two factors.

Checkpoint 3. For $(1 + 2) * 3$:

- Write the indented list of calls, as above.
- How many calls of `expr`, `term` and `factor` are there in total?
- How many of the three methods are on the call stack at the deepest point?
- Which method, and which line of your plan, raises the `ValueError` for `"(3 + 4"`, for `"3 + 4 )"` and for `"()"`?

3.3 Write it

expr — as in the lecture: parse a `term`; while the next token is `+` or `-`, consume it, parse another `term`, and make a `BinOp` with the tree so far on the **left**. Return the tree.

term — the same, with `factor` in place of `term` and `*` / in place of `+` -.

factor — peek at the token:

- None: the input ended where a factor was needed — `ValueError`.
- `(`: consume it, parse an `expr`, then require `)` — if the next token is not `)`, `ValueError`; if it is, **consume it**. Return the inner tree.
- `-`: consume it; return `BinOp("-", Num(0.0), <a factor>)`.
- A number: consume it; return `Num(float(token))`.
- Anything else: `ValueError`.

parse — refuse an empty token list; parse an `expr`; then, if any token is left, `ValueError`. Return the tree.

Hints:

- `self.peek()` in `("+", "-")` is safe at the end: `None` in `(...)` is `False`.
- Test the `None` case in `factor` **first**; the other tests look inside the token, and `None` has nothing to look at.
- A number token is recognisable by its first character: a digit or a `.`

3.4 Test it

```
pytest tests/test_translation.py -k "parse or preced or assoc or unary
↳ or nested"
```

Fourteen tests: a single number and $3 + 4$; `test_precedence_is_the_shape_of_the_tree` ($3 + 4 * 2$); parentheses; the two left-associativity tests ($1 - 2 - 3$ and $8 / 4 / 2$); unary minus (-5); $((7))$; and six malformed inputs that must raise `ValueError`: `"", "3 +", "(3 + 4", "3 + 4 )", "* 3", "()"`.

The assertion messages print whole trees. Read them as nested calls: `BinOp('-', Num(1.0), BinOp('-', Num(2.0), Num(3.0)))` is $1 - (2 - 3)$.

3.5 When it fails

Bug	What you see	Fix
if instead of while in <code>expr</code>	<code>ValueError: unexpected '-' after the expression in the 1 - 2 - 3 test — one operator was read, the second left over</code>	a chain is a loop
<code>expr</code> calls <code>factor</code> (and handles all four operators)	the precedence test fails: $3 + 4 * 2$ is parsed as $(3 + 4) * 2$	<code>expr</code> calls <code>term</code> ; <code>term</code> calls <code>factor</code>
) checked but not consumed	<code>ValueError: unexpected ')' after the expression in the parentheses and ((7)) tests</code>	<code>advance()</code> past the)
no check for leftover tokens in <code>parse</code>	<code>Failed: DID NOT RAISE <class 'ValueError'> for "3 + 4)"</code>	after <code>expr</code> , <code>peek()</code> must be <code>None</code>
no <code>None</code> check in <code>factor</code>	<code>TypeError: 'NoneType' object is not subscriptable for "3 +"</code>	check <code>None</code> first
<code>Num(token)</code> instead of <code>Num(float(token))</code>	<code>assert Num('3') == Num(3.0) — a string in the tree</code>	convert with <code>float</code>

Two bugs pass all 14 of these tests and show up only later, in Part 5: unary minus that calls `self.expr()` instead of `self.factor()`, and a forgotten empty-input check in `parse` (which is harmless, because `factor` then raises on the empty input anyway). Keep reading.

Part 4 — Break it: two grammars that look right

Lecture 15, “Two grammars that look right and are not”. Your parser passes its 14 tests. Save your `expr` somewhere safe (or rely on `git diff`), then try two tempting alternatives. Each is what you get by turning the **recursive** BNF form of the grammar straight into code.

4.1 Right recursion

Replace the body of `expr` with the right-recursive version from the lecture: parse a term; **if** the next token is `+` or `-`, return a `BinOp` whose right child is a recursive call to `self.expr()`.

Checkpoint 4. Before you run anything:

- (a) What tree does it build for `1 - 2 - 3`, and what value does that give?
- (b) Of all 55 tests, which fail? (Hint: which test expressions have two `+` or `-` **at the same level**, one after the other?)
- (c) Why does `8 / 4 / 2` still pass?

Run the whole file and compare:

```
pytest tests/test_translation.py -q
```

Read the failing assertions. A wrong answer with no exception is the worst kind of bug; the only thing that caught it is a test written for exactly this case.

4.2 Left recursion

Now write `expr` as the textbook grammar `expr := expr ('+' | '-') term | term` reads: the first thing `expr` does is call `self.expr()`.

```
pytest tests/test_translation.py -q -x
```

`-x` stops at the first failure. You will see `RecursionError: maximum recursion depth exceeded`, on the very first parser test — “3”, a single number. Why? `expr` calls itself before it has consumed a token, so every call starts from the same position, and nothing ever gets closer to a base case. (Without `-x`, 24 tests fail this way: every test that parses anything.)

Put your loop back and check: 14 passed on the parser filter.

Part 5 — evaluate and calculate

Lecture 15, “Stage 3: Walking the Tree”. (At home, if the session runs out.)

5.1 The idea

`evaluate(node)` is a post-order walk: a `Num` is its own value; a `BinOp` is worth “left value op right value”, which needs both children’s values **first**. Four operators; anything else is a `ValueError`. Division by zero needs no code of yours: Python’s `/` raises `ZeroDivisionError` itself.

`calculate(text)` joins the three stages: tokenize, parse, evaluate.

5.2 Draw it

Number the nodes of the tree in the order evaluate **finishes** them, and write each node’s value beside it — the lecture’s figure for $(3 + 4) * (5 - 2)$.

Checkpoint 5. For $(8 - 2) / (1 + 2)$:

- (a) Draw the tree, and number the nodes in the order evaluate finishes them.
- (b) Write the node labels in that order. What is this sequence called, and where did you meet it before?
- (c) What value does `calculate` return, and what type is it?

5.3 Write it and test it

```
pytest tests/test_translation.py -v -k "evaluate and not postfix"
pytest tests/test_translation.py -v -k "calculate or routes"
```

Four evaluator tests (a leaf, a hand-built tree, division by zero, an unknown operator `^`), then ten end-to-end: nine `calculate` cases and `test_both_routes_agree`, which compares your calculator with `evaluate_postfix(infix_to_postfix(tokens))` on four expressions.

5.4 When it fails

Bug	What you see	Fix
right - left (operands swapped)	the evaluator tests pass — they use only <code>+</code> and <code>*</code> — then <code>calculate("-5 + 2")</code> gives <code>7.0</code> , <code>"1 - 2 - 3"</code> gives <code>2.0</code> ; 4 tests fail	left operand first
no recursion: <code>node.left.value</code>	<code>AttributeError:</code> 'BinOp' object has no attribute 'value' in 8 tests	call <code>evaluate</code> on each child
unknown operator returns <code>None</code>	<code>Failed: DID NOT RAISE</code> <class 'ValueError'>	raise <code>ValueError</code> after the four cases

Bug	What you see	Fix
// instead of /	calculate("10 / 4") gives 2.0, not 2.5	true division
unary minus built with self.expr() in factor	only calculate("-5 + 2") fails: -7.0 instead of -3.0 — the minus swallowed 5 + 2	the operand of unary minus is a factor

The first and last rows are why end-to-end tests exist: each stage's own tests passed, and the calculator was still wrong.

When all 55 pass, run the stages you did not write by hand against each other:

```
from dsa.translation import tokenize, Parser, evaluate
from dsa.translation import evaluate_postfix
from dsa.stack import infix_to_postfix
text = "2 * (3 + (4 - 1))"
print(evaluate(Parser(tokenize(text)).parse()))    # 12.0
print(evaluate_postfix(infix_to_postfix(tokenize(text)))) # 12
```

The tree route gives a float (every Num holds one); the stack route keeps integers when it can. Equal values, different types — which is why `test_both_routes_agree` compares with `pytest.approx`.

Part 6 — See it

6.1 Draw the tree your parser built

In notebooks/15-language-translation.ipynb:

```
from viz.draw import draw_tree
from dsa.translation import Parser, tokenize, to_edges

def show(text):
    return draw_tree(to_edges(Parser(tokenize(text)).parse()),
        ↪ title=text)

show("3 + 4 * 2")
show("(3 + 4) * 2")
show("1 - 2 - 3 + 4")
```

Compare each picture with your answers to Checkpoint 2. A drawing is the fastest way to find a precedence or associativity bug: a wrong tree is obvious at a glance, where a wrong number is not.

6.2 Watch the call stack

Make the parser print its own calls — in a **subclass**, so that `dsa/translation.py` stays clean:

```
from dsa.translation import Parser, tokenize

class Tracing(Parser):
    depth = 0

    def expr(self):
        return self._traced("expr", super().expr)

    def term(self):
        return self._traced("term", super().term)

    def factor(self):
        return self._traced("factor", super().factor)

    def _traced(self, name, method):
        indent = "  " * Tracing.depth
        print(f"{indent}{name} at {self.position} ({self.peek()!r})")
        Tracing.depth += 1
        try:
            return method()
        finally:
            Tracing.depth -= 1

Tracing(tokenize("(1 + 2) * 3")).parse()
```

Check the output against your answer to Checkpoint 3. The indentation is the depth of Python's call stack: the parser has no stack of its own, because the call stack **is** its stack (Lecture 15, "The calls are the parse tree").

Part 7 — Measure it

Lecture 15, "Measured". Two questions: how does the **time** grow with the length of the input, and how does the **recursion depth** grow with its shape?

7.1 Time

```
import random
from viz.complexity import measure, plot_growth
from dsa.translation import calculate
```

```
def balanced(n, rng=random.Random(15)):
    """A balanced, fully parenthesised expression with n numbers."""
    parts = [str(rng.randint(1, 9)) for _ in range(n)]
    while len(parts) > 1:
        merged = [f"({parts[i]} {rng.choice('+-*')} {parts[i + 1]})"
                  for i in range(0, len(parts) - 1, 2)]
        if len(parts) % 2:
            merged.append(parts[-1])
        parts = merged
    return parts[0]

sizes = [500, 1000, 2000, 4000, 8000]
result = measure(calculate, sizes, balanced)
plot_growth({"calculate": result}, reference=["n"], loglog=True)
```

measure builds each input outside the timing and keeps the best of three runs. Expect a straight line of slope 1: double the numbers, double the time — $O(n)$. Your times will differ from anyone else's; the slope will not.

7.2 Depth

```
def deepest(make):
    k = 1
    while True:
        try:
            calculate(make(k))
        except RecursionError:
            return k - 1
        k += 1

print(deepest(lambda k: "(" * k + "1" + ")" * k)) # nested
print(deepest(lambda k: " + ".join(["1"] * k))) # flat
```

Checkpoint 6. Predict before you run:

- How many Python frames does one pair of parentheses cost the parser? Roughly how many pairs will fit under Python's limit of 1,000 frames?
- The flat chain $1 + 1 + \dots + 1$ is parsed with a **loop**. Does it fail at all? If it does, which function runs out of stack, and at about what length?
- How deep does a **balanced** expression with 1,000 numbers go?

The chain takes a few seconds, because it parses every length from 1 upwards. Run from a notebook, both answers come out a little smaller than from a script: Jupyter itself is already using some frames when your code starts.

Part 8 — Exercises at a glance

Method	Target cost	The trap	Tests (Part 0.5)
tokenize	O(n)	numbers stay whole; <code>i < len(text)</code> first; unknown character raises	lexer (13)
Parser.expr	O(n) overall	a loop ; the old tree goes on the left ; calls <code>term</code>	parser (14)
Parser.term		the same shape; calls <code>factor</code>	parser
Parser.factor		None first; consume the <code>)</code> ; unary minus calls <code>factor</code>	parser
Parser.parse		empty input; tokens left over	parser
evaluate	O(n)	children first; swapped operands pass the unit tests; unknown operator	evaluator (4)
calculate	O(n)	three lines	end to end (10)

All 55 at once:

```
pytest tests/test_translation.py -v
```

Before you show the TA: `git diff --stat tests/` must print nothing. And check by eye that nothing in `dsa/translation.py` calls `eval`, imports `re`, or keeps a table of precedences — the tests cannot see those, but the TA will look.

Part 9 — Take-home practice (not graded)

The question bank for this week is `docs/question-bank/week15-questions.md`, with answers in `week15-answers.md`. Do the questions before opening the answers.

1. **Part G — write the code**, in `practice/week15.py`: `to_postfix`, `to_infix`, `to_infix_minimal`, `postfix_to_tree` and `calculate_with_power` (W15-C1 to W15-C5):

```
pytest tests/test_practice_week15.py -v
```

The round-trip tests of C3 parse your output with **your** Parser, so finish this lab first. Hint for `to_infix_minimal`: a child needs parentheses when its operator binds more loosely than its parent's — or equally loosely, if it is the **right** child.

2. **W15-S1** — draw the trees of four expressions, and of the right-recursive parser's mistake.

3. **W15-T2** — the call trace of $2 * (3 - 1) + 4$, with the tokens each call consumes.
4. **W15-B1** — the right-recursive `expr` of Part 4.1, as an exam question: find an input on which it fails without running it.
5. **W15-K2** — the recursion depth of the parser and of `evaluate`, for nested parentheses and for a flat chain. Check against Part 7.2.

The worked solutions are in `solutions/dsa/translation.py` and `solutions/practice/week15.py` — for after you have tried. `pytest --solutions tests/test_practice_week15.py` runs the tests on them.

Part 10 — Course wrap-up and exam preparation

This is the last lab. There is no next week to bridge to — so instead, a look back at what your repository now contains, and a plan for the final.

10.1 What you built

Run the whole suite once:

```
pytest -q
```

Every module that passes is a structure or algorithm you wrote yourself, on the course `Array`, with its cost stated in its docstring and checked by a test. Look at the list of files in `dsa/` and, for each, say aloud one sentence: what it keeps, and what its main operation costs. If you cannot, that is the week to revise first.

This week's calculator used the `Array` (the parser's tokens), recursion (every parser method, and `evaluate`), your `Stack` on your `DynamicArray` (the postfix route), node objects (`Num`, `BinOp`) and a tree walk (post-order). One program, most of the course.

10.2 The final exam

- **Format:** 30 marks multiple choice (one answer of four), 30 marks written — traces, drawings of a structure after each operation, complexity with a justification, find-and-fix, and short code. On paper, with no computer.
- **To pass the course:** at least 60% overall **and** at least 30% of the final exam; attendance of at least 75% to sit it. See the course guide, `docs/course/00-course-guide.md`.
- **Examinable:** every week, 1 to 15. Dynamic programming is enrichment and is not examined.

10.3 How to prepare

1. **The question banks, closed.** For each week, answer the questions on paper with the answers file shut, then mark yourself against it. For every multiple-choice question

you got wrong — or right for the wrong reason — read the explanation of **every** option.

2. **The mock papers**, in docs/question-bank/, under exam conditions: one sitting, closed book, no computer.
3. **Draw**. For each structure — dynamic array, linked list, stack, ring buffer, BST, heap, hash table, graph, and now the expression tree — draw it after each operation of a short sequence. The written paper rewards pictures that are exactly right.
4. **Write from memory**. Pick one function a day from dsa/ — `binary_search`, `sift_down`, `partition`, `bfs`, `Parser.expr` — and write it on paper without looking. Then run it. What you cannot write on paper, you do not yet know.
5. **Complexity, with a reason**. For every function, say its cost **and why**: which loop, which recurrence, which amortised argument. The exam leans on **[why]** questions.

Where it goes from here depends on your program: SWE students meet grammars and parsing again, formally, in **SWE141 Software Construction**; AI students meet algorithm design in **AI3001**. Everyone meets these structures in every program they will ever read.

Summary

Idea	The one line to keep
Three stages	Lexer: text $\rightarrow$ tokens. Parser: tokens $\rightarrow$ tree. Evaluator: tree $\rightarrow$ value.
Lexer	One scan; numbers stay whole; a space separates; unknown characters raise.
Grammar	<code>expr</code> of terms, term of factors; factor recurses to <code>expr</code> through <code>(</code> .
Precedence	The lower the rule, the tighter the operator: the shape , not a table.
Recursive descent	One method per rule; <code>(...)*</code> is a <i>while</i> ; decide by <code>peek()</code> .
Associativity	The old tree becomes the left child.
Right recursion	Builds <code>1 - (2 - 3)</code> : wrong, silently — 3 tests catch it.
Left recursion	Calls itself before consuming a token: <code>RecursionError</code> .
Errors	Empty input, a missing <code>)</code> , a token left over: <code>ValueError</code> .
Evaluate	Post-order: children first. The post-order sequence is postfix.
Measured	Time $O(n)$ in the length; depth grows with the height of the tree.